



Manuela Fischer, Felix Friedrich

Informatik

Vorlesung vom D-INFK der ETH Zürich

Herbst 2023

Willkommen

zur Vorlesung Informatik

am MATH/PHYS Department der ETH Zürich.

Ort und Zeit:

Dienstag 14:15 - 16:00, HG F 7 und HG F 5 (Video).

Pause 15:00 - 15:15, leichte Verschiebung möglich.

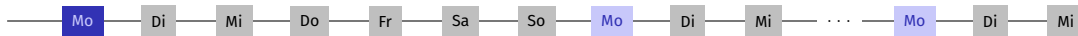
Vorlesungs-Webseite

`http://lec.inf.ethz.ch/ifmp`

Team

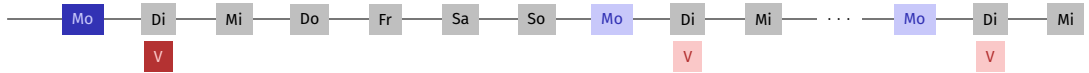
Chefassistent	Manuel Kansy	
Assistierende	Adriana De-Sassi	Alain Flückiger
	Cajetan Tulej	Chen Guo
	Christopher Otto	Dong Kang
	Esther Petat	Fatemeh Zargarbashi
	Fiona Sacher	Flavio De Vincenti
	Hannah Vogel	Johannes Luber
	Katharina Knist	Levin Ceglie
	Maxence Aleksa	Michelle Egger
	Nick Herrmann	Renas Sahin
	Siyuan Shen	Vincent Junker
Dozierende	Dr. Manuela Fischer	Dr. Felix Friedrich

Ablauf



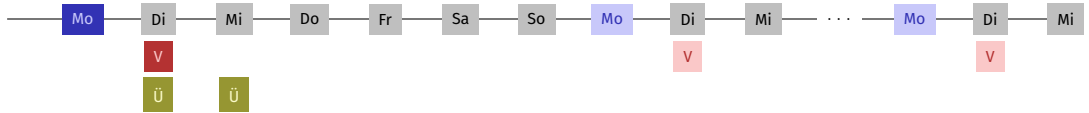
- **Übungsblattausgabe am Montag morgen (online)**
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Ablauf



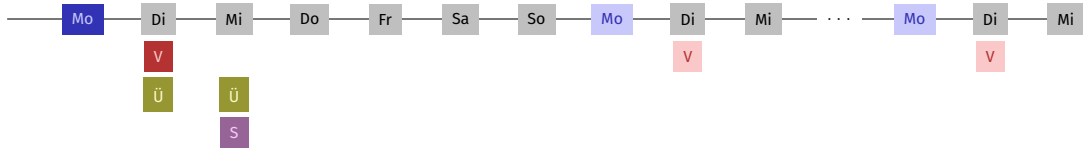
- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Ablauf



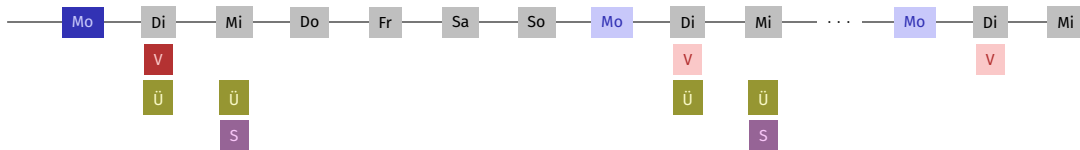
- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Ablauf



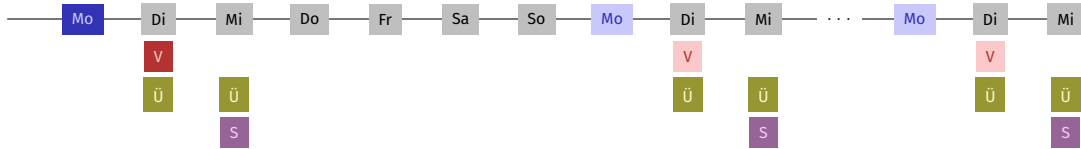
- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Ablauf



- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (`studycenter.ethz.ch`)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Ablauf



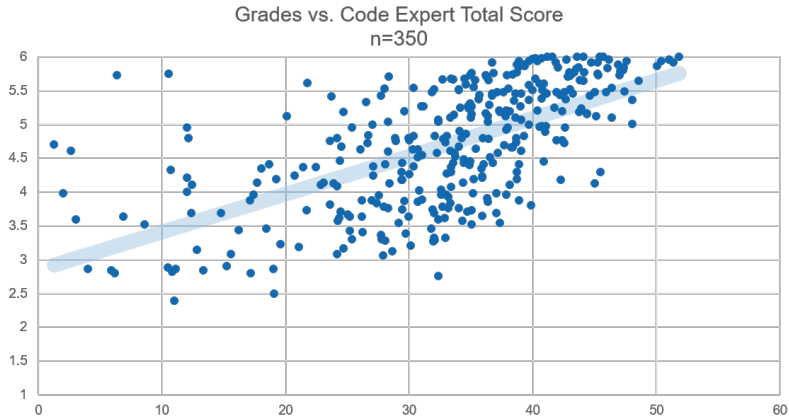
- Übungsblattausgabe am Montag morgen (online)
- Vorlesung am Dienstag
- Vorbesprechung in der folgenden Übung (am Dienstag/Mittwoch)
- StudyCenter (studycenter.ethz.ch)
- Abgabe der Serie spätestens am Montag in der nächste Woche (18:00h)
- Nachbesprechung der Serie in der übernächsten Übung. Feedback zu den Abgaben innerhalb einer Woche nach Abgabe-Termin.

Zu den Übungen

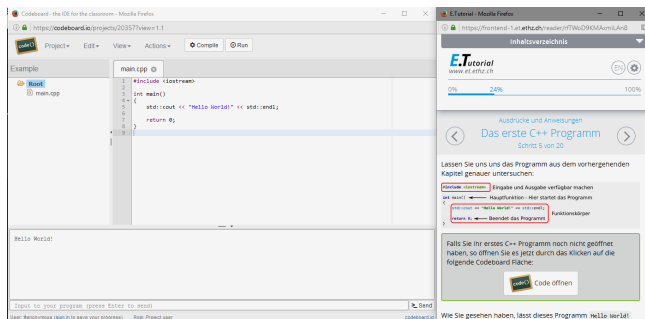
- Bearbeitung der wöchentlichen Übungsserien ist also freiwillig, wird aber dringend empfohlen!

Zu den Übungen

- Bearbeitung der wöchentlichen Übungsserien ist also freiwillig, wird aber dringend empfohlen!

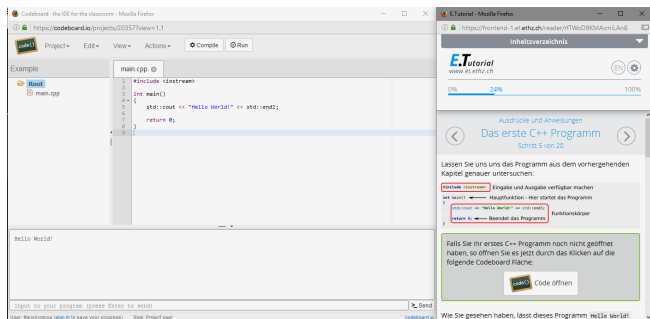


Online Tutorial



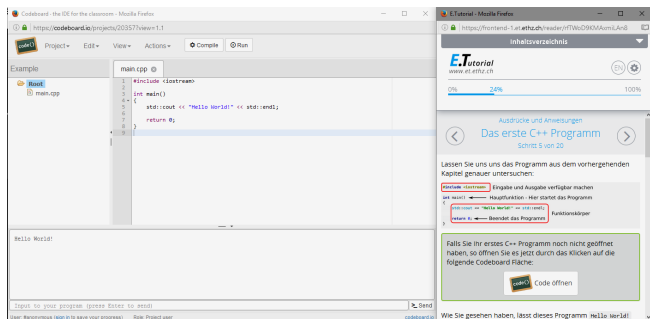
Zum Einstieg stellen wir ein Online-C++ Tutorial zur Verfügung.

Online Tutorial



Zum Einstieg stellen wir ein Online-C++ Tutorial zur Verfügung.
Ziel: Ausgleich der unterschiedlichen Programmierkenntnisse.

Online Tutorial



Zum Einstieg stellen wir ein Online-C++ Tutorial zur Verfügung.
Ziel: Ausgleich der unterschiedlichen Programmierkenntnisse.
⇒ Online Tutorial bis nächste Woche durcharbeiten.

Relevantes für die Prüfung

Prüfungsstoff für die Endprüfung schliesst ein

- Vorlesungsinhalt (Vorlesung, Handout) und
- Übungsinhalte (Übungsstunden, Übungsaufgaben).

Relevantes für die Prüfung

Prüfung ist schriftlich am Computer.

Es wird sowohl praktisches Wissen (Programmierungsfähigkeit) als auch theoretisches Wissen (Hintergründe, Systematik) geprüft.

Relevantes für die Prüfung

Bewertung Prüfung prozentual nach bestandenen Tests, Bonusaufgaben ebenso. Serien verlangen alle Tests um die XP zu erhalten.

(Beim üben:) Moodle zeigt nur eine Antwort als richtig an, hinter den Kulissen werden alle semantisch richtigen akzeptiert.

Unser Angebot (VVZ)

- Ihre Programmierübungen werden (halb)automatisch bewertet. Durch Bearbeitung der wöchentlichen Übungsserien kann ein Bonus von maximal 0.25 Notenpunkten erarbeitet werden, der an die Prüfung mitgenommen wird.
- Der Bonus ist proportional zur erreichten Punktzahl von speziell markierten Bonusaufgaben, wobei volle Punktzahl einem Bonus von 0.25 entspricht. Die Zulassung zu speziell markierten Bonusaufgaben hängt von der erfolgreichen Absolvierung anderer Übungsaufgaben ab. Der erreichte Notenbonus verfällt, sobald die Vorlesung neu gelesen wird.

Unser Angebot (konkret)

Normale Übungen



ca. 2/3

ca. 2/3

ca. 2/3

Bonus Übungen

BonusÜ

BonusÜ

BonusÜ

max 0.125

max 0.125

max 0.125

Σ

$\downarrow$
 x

Notenbonus

$\min(0.25, x)$

Akademische Lauterkeit

Wir ermuntern Sie explizit, Lösungsideen und -ansätze mit Ihren Kollegen zu diskutieren. Teamwork ist auch in der Informatik wichtig. Es ist aber insbesondere wichtig, dass Sie aktiv dazulernen und nicht nur reproduzieren. Daher:

Regeln

Sie geben nur eigene Lösungen ab, welche Sie selbst verfasst und verstanden haben. Copy-Paste ist nicht erlaubt, ebensowenig Teamimplementierungen.

Absichtlich leer

1. Einführung

Informatik: Definition und Geschichte, Algorithmen, Turing Maschine, Höhere Programmiersprachen, Werkzeuge der Programmierung

Was ist Informatik?

Was ist Informatik?

Informatik = Automatisierung geistiger Tätigkeiten

Was ist Informatik?

- Die Wissenschaft der systematischen Verarbeitung von *Informationen*, (...)
- ...insbesondere der automatischen Verarbeitung mit Hilfe von Digitalrechnern.

(Wikipedia, nach dem „Duden Informatik“)

Informatik vs. Computer

Computer science is not about machines, in the same way that astronomy is not about telescopes.

Mike Fellows, US-Informatiker (1991)

Informatik $\neq$ EDV-Kenntnisse

EDV-Kenntnisse: *Anwenderwissen („Computer Literacy“)*

- Umgang mit dem Computer
- Bedienung von Computerprogrammen (für Texterfassung, E-Mail, Präsentationen,...)

Informatik $\neq$ EDV-Kenntnisse

Informatik: *Grundlagenwissen*

- Wie funktioniert ein Computer?
- Wie schreibt man ein Computerprogramm?

Inhalt dieser Vorlesung

- Systematisches Problemlösen mit Algorithmen und der Programmiersprache C++.
- Also: nicht nur,
aber auch Programmierkurs.

Warum Programmieren?

Warum Programmieren?

Na, weil Computer und
Software heutzutage
allgegenwärtig sind!



Warum Programmieren?

- Jedes **Verständnis** moderner Technologie erfordert Wissen über die grundlegende Funktionsweise – und Grenzen – eines Computers
- Wir leben mitten in einer **Revolution**, die nicht nur alle Wissenschaften, sondern die ganze Gesellschaft betrifft: Die Automatisierung der geistigen Tätigkeiten schreitet fort von reinen Berechnungen zur künstlichen Intelligenz. Wer mitwirken will, sollte besser programmieren können.

Warum Programmieren?

- Jedes **Verständnis** moderner Technologie erfordert Wissen über die grundlegende Funktionsweise – und Grenzen – eines Computers
- Wir leben mitten in einer **Revolution**, die nicht nur alle Wissenschaften, sondern die ganze Gesellschaft betrifft: Die Automatisierung der geistigen Tätigkeiten schreitet fort von reinen Berechnungen zur künstlichen Intelligenz. Wer mitwirken will, sollte besser programmieren können.
- Programmieren (mit dem Werkzeug Computer) wird zu einer **Kulturtechnik** wie Lesen und Schreiben (mit den Werkzeugen Papier und Bleistift)

Warum Programmieren?

- Jedes **Verständnis** moderner Technologie erfordert Wissen über die grundlegende Funktionsweise – und Grenzen – eines Computers
- Wir leben mitten in einer **Revolution**, die nicht nur alle Wissenschaften, sondern die ganze Gesellschaft betrifft: Die Automatisierung der geistigen Tätigkeiten schreitet fort von reinen Berechnungen zur künstlichen Intelligenz. Wer mitwirken will, sollte besser programmieren können.
- Programmieren (mit dem Werkzeug Computer) wird zu einer **Kulturtechnik** wie Lesen und Schreiben (mit den Werkzeugen Papier und Bleistift)
- Programmieren bedeutet, etwas zu **erschaffen/erzeugen** – es macht Freude und ist nützlich

Algorithmus: Kernbegriff der Informatik

Algorithmus:

Algorithmus: Kernbegriff der Informatik

Algorithmus:

- Handlungsanweisung zur schrittweisen Lösung eines Problems



“Dixit algorizmi...” (lateinische Übersetzung)

Algorithmus: Kernbegriff der Informatik

Algorithmus:

- Handlungsanweisung zur schrittweisen Lösung eines Problems
- Ausführung erfordert keine Intelligenz, nur Genauigkeit (sogar Computer können es)



“Dixit algorizmi...” (lateinische Übersetzung)

Algorithmus: Kernbegriff der Informatik

Algorithmus:

- Handlungsanweisung zur schrittweisen Lösung eines Problems
- Ausführung erfordert keine Intelligenz, nur Genauigkeit (sogar Computer können es)
- nach *Muhammed al-Chwarizmi*,
Autor eines arabischen
Rechen-Lehrbuchs (um 825)



“Dixit algorizmi...” (lateinische Übersetzung)

Der älteste nichttriviale Algorithmus

Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)

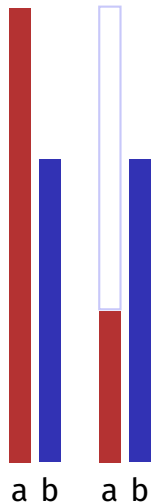


a b

- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Der älteste nichttriviale Algorithmus

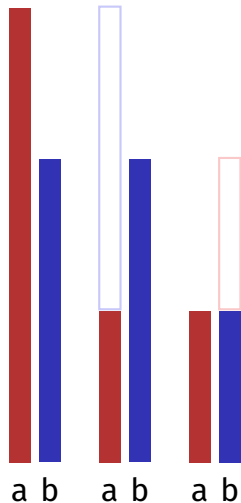
Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)



- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Der älteste nichttriviale Algorithmus

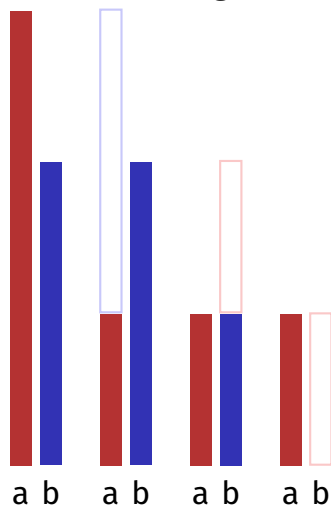
Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)



- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Der älteste nichttriviale Algorithmus

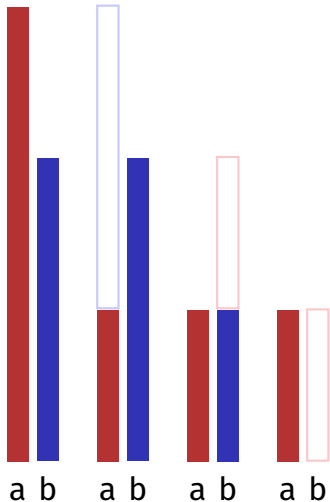
Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)



- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Der älteste nichttriviale Algorithmus

Euklidischer Algorithmus (aus Euklids *Elementen*, 3. Jh. v. Chr.)



- Eingabe: ganze Zahlen $a > 0, b > 0$
- Ausgabe: ggT von a und b

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

Sonst:

$$b \leftarrow b - a$$

Ergebnis: a .

Algorithmen: 3 Abstraktionsstufen

1. **Kernidee** (abstrakt):

Die Essenz eines Algorithmus' („Heureka-Moment“)

Algorithmen: 3 Abstraktionsstufen

1. **Kernidee** (abstrakt):
Die Essenz eines Algorithmus' („Heureka-Moment“)
2. **Pseudocode** (semi-detailliert):
Für Menschen gemacht (Bildung, Korrektheit- und Effizienzdiskussionen, Beweise)

Algorithmen: 3 Abstraktionsstufen

1. **Kernidee** (abstrakt):
Die Essenz eines Algorithmus' („Heureka-Moment“)
2. **Pseudocode** (semi-detailliert):
Für Menschen gemacht (Bildung, Korrektheit- und Effizienzdiskussionen, Beweise)
3. **Implementierung** (sehr detailliert):
Für Mensch & Computer gemacht (les- & ausführbar, bestimmte Programmiersprache, verschiedene Implementierungen möglich)

Algorithmen: 3 Abstraktionsstufen

1. **Kernidee** (abstrakt):
Die Essenz eines Algorithmus' („Heureka-Moment“)
2. **Pseudocode** (semi-detailliert):
Für Menschen gemacht (Bildung, Korrektheit- und Effizienzdiskussionen, Beweise)
3. **Implementierung** (sehr detailliert):
Für Mensch & Computer gemacht (les- & ausführbar, bestimmte Programmiersprache, verschiedene Implementierungen möglich)

Euklid: Kernidee und Pseudocode gesehen, Implementierung noch nicht

Euklid in der Box

Speicher

0	1	2	3	4	5	6	7	8	9

Links

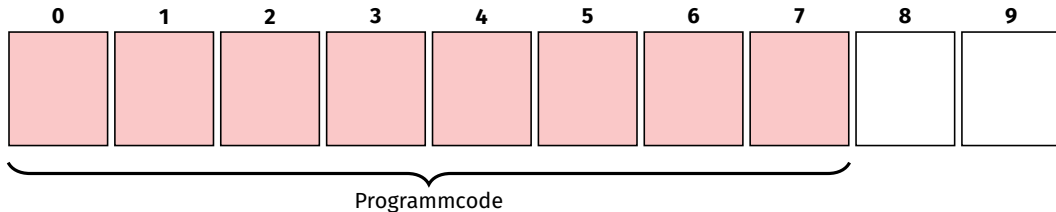
Rechts

--	--

Register

Euklid in der Box

Speicher



Links

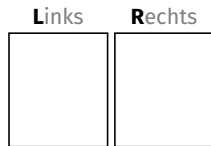
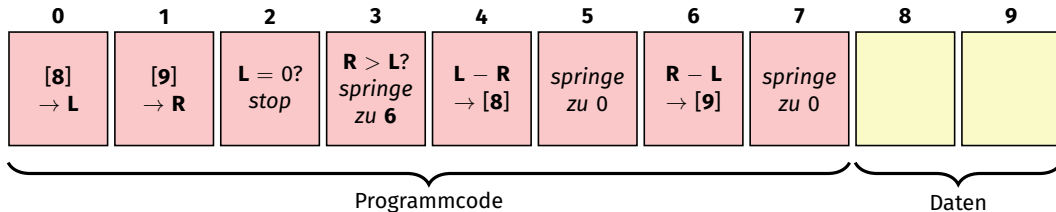
Rechts



Register

Euklid in der Box

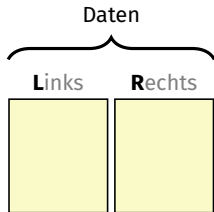
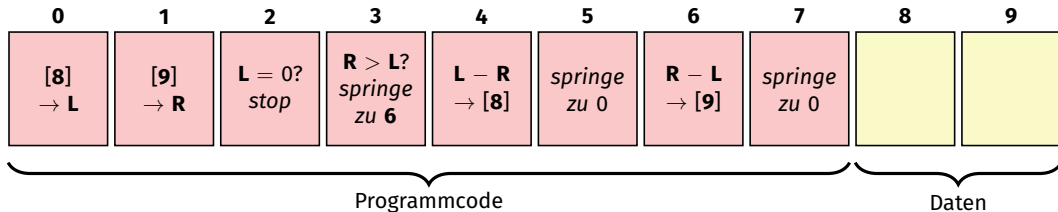
Speicher



Register

Euklid in der Box

Speicher



Register

Euklid in der Box

Speicher

0	1	2	3	4	5	6	7	8	9
[8] → L	[9] → R	L = 0? stop	R > L? springe zu 6	L - R → [8]	springe zu 0	R - L → [9]	springe zu 0		

Links

Rechts



Register

Euklid in der Box

Speicher

0	1	2	3	4	5	6	7	8	9
[8] → L	[9] → R	L = 0? stop	R > L? springe zu 6	L - R → [8]	springe zu 0	R - L → [9]	springe zu 0	<i>b</i>	<i>a</i>

Links	Rechts
<i>b</i>	<i>a</i>

Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

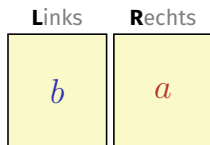
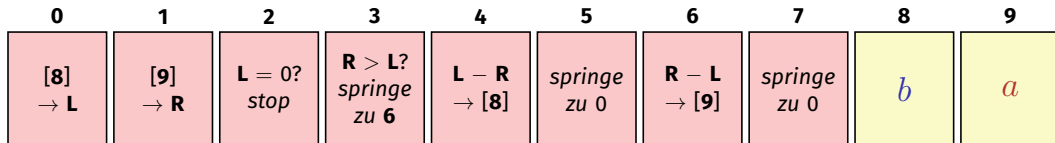
Sonst:

$$b \leftarrow b - a$$

Ergebnis: a .

Euklid in der Box

Speicher



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

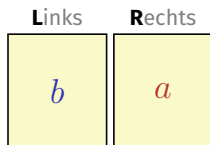
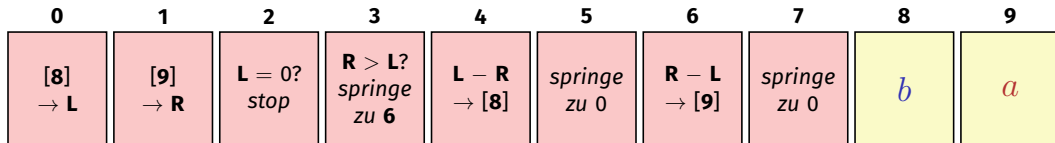
Sonst:

$$b \leftarrow b - a$$

Ergebnis: a .

Euklid in der Box

Speicher



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

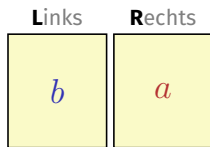
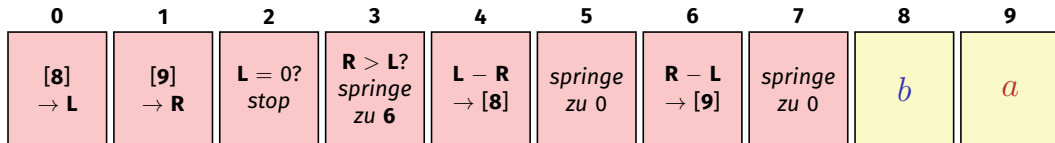
Sonst:

$$b \leftarrow b - a$$

Ergebnis: *a*.

Euklid in der Box

Speicher



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

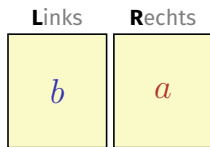
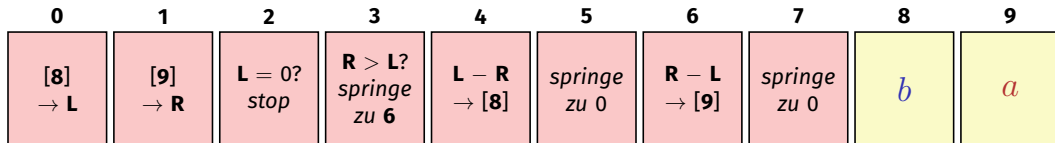
Sonst:

$$b \leftarrow b - a$$

Ergebnis: a .

Euklid in der Box

Speicher



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

Sonst:

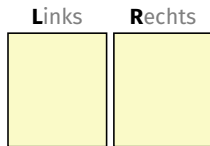
$$b \leftarrow b - a$$

Ergebnis: a .

Euklid in der Box

Speicher

0	1	2	3	4	5	6	7	8	9
[8] → L	[9] → R	L = 0? stop	R > L? springe zu 6	L - R → [8]	springe zu 0	R - L → [9]	springe zu 0	<i>b</i>	<i>a</i>



Register

Solange $b \neq 0$

Wenn $a > b$ dann

$$a \leftarrow a - b$$

Sonst:

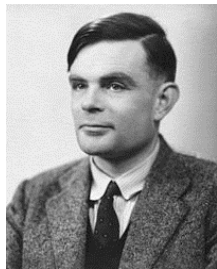
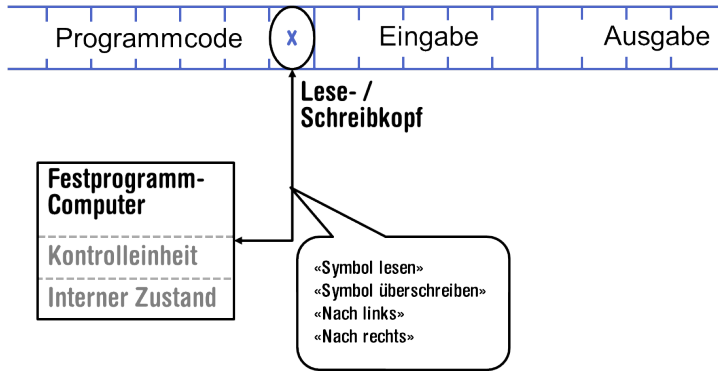
$$b \leftarrow b - a$$

Ergebnis: a .

Computer – Konzept

Eine geniale Idee: Universelle Turingmaschine (Alan Turing, 1936)

Folge von Symbolen auf Ein- und Ausgabeband

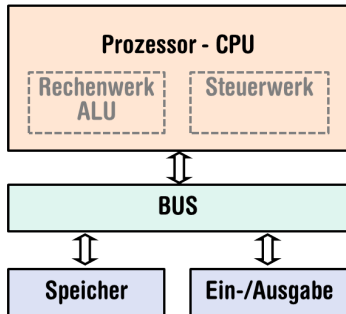


Alan Turing

Computer – Umsetzung

- Z1 – Konrad Zuse (1938)
- ENIAC – John Von Neumann (1945)

Von Neumann Architektur



Konrad Zuse



John von Neumann

Rechengeschwindigkeit

- Computer sind, anderslautenden Berichten zum Trotz, nicht intelligent
- Aber sie führen Befehle unglaublich schnell aus!

Rechengeschwindigkeit

- Computer sind, anderslautenden Berichten zum Trotz, nicht intelligent
- Aber sie führen Befehle unglaublich schnell aus!
- Zum Beispiel (Befehle pro Sekunde (IPS), grob gerundet):
 - Raspberry Pi 2: 5×10^9
 - Standard Intel/AMD: $200 - 800 \times 10^9$
 - AMD Ryzen Threadripper: 2×10^{12}

(Quelle: https://en.wikipedia.org/wiki/Instructions_per_second)

Programmieren

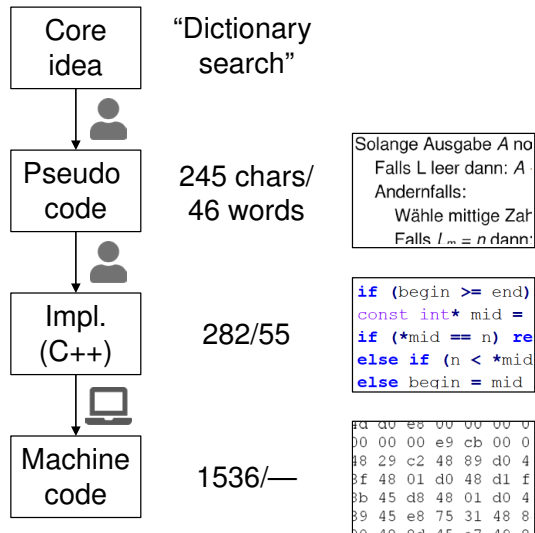
- Mit Hilfe einer Programmiersprache wird dem Computer eine Folge von Befehlen erteilt, damit er genau das macht, was wir wollen.
- Die Folge von Befehlen ist das (Computer)-Programm.



The Harvard Computers, Menschliche Berufsrechner, ca.1890

Programmiersprachen

- Sprache, die der Computer "verstehen", ist sehr primitiv (Maschinensprache).
- Einfache Operationen müssen in (extrem) viele Einzelschritte aufgeteilt werden.
- Sprache variiert von Computer zu Computer.



Warum C++?

Andere populäre Programmiersprachen: Java, C#, Python, JavaScript, Swift, Kotlin, Go, and many more.

Warum C++?

Andere populäre Programmiersprachen: Java, C#, Python, JavaScript, Swift, Kotlin, Go, and many more.

- C++ ist relevant in der Praxis, weit verbreitet und „läuft überall“

Warum C++?

Andere populäre Programmiersprachen: Java, C#, Python, JavaScript, Swift, Kotlin, Go, and many more.

- C++ ist relevant in der Praxis, weit verbreitet und „läuft überall“
- C++ ist eine der „schnellsten“ Programmiersprachen
- C++ eignet sich gut für Systemprogrammierung, da es einen sorgfältigen Umgang mit Ressourcen (Speicher, ...) erlaubt/verlangt

Warum C++?

C++ als Einführungssprache? Warum nicht Python oder JavaScript?

- C++ macht viele Entscheidungen explizit, bzw. zwingt Programmierer dazu, explizit eine Wahl zu treffen
- Dies erleichtert einen späteren Umstieg auf „simplere“ Sprachen
- Analogie: Führerausweis auf Schaltwagen machen, aber später Automatik fahren

Programme müssen, wie unsere Sprache, nach gewissen Regeln geformt werden.

- Syntax: Zusammenfügingsregeln für elementare Zeichen (Buchstaben).
- Semantik: Interpretationsregeln für zusammengefügte Zeichen.

Programme müssen, wie unsere Sprache, nach gewissen Regeln geformt werden.

- Syntax: Zusammenfügingsregeln für elementare Zeichen (Buchstaben).
- Semantik: Interpretationsregeln für zusammengefügte Zeichen.

Entsprechende Regeln für ein Computerprogramm sind einfacher, aber auch strenger, denn Computer sind vergleichsweise dumm.

Deutsch vs. C++

Deutsch

Alleen sind nicht gefährlich, Rasen ist gefährlich!
(Wikipedia: Mehrdeutigkeit)

C++

```
// computation  
int b = a * a; //  $b = a^2$   
b = b * b;     //  $b = a^4$ 
```

Syntax und Semantik von C++

Syntax:

- Wann ist ein Text ein C++Programm?
- D.h. ist es *grammatikalisch* korrekt?
- → Kann vom Computer überprüft werden

Semantik:

- Was *bedeutet* ein Programm?
- Welchen Algorithmus *implementiert* ein Programm?
- → Braucht menschliches Verständnis

2. Ein C++-Programm und seine Bestandteile

Programmiertools, Bestandteile

Was braucht es zum Programmieren?

- **Editor:** Programm zum Ändern, Erfassen und Speichern von C++-Programmtext
- **Compiler:** Programm zum Übersetzen des Programmtexts in Maschinensprache

Was braucht es zum Programmieren?

- **Editor:** Programm zum Ändern, Erfassen und Speichern von C++-Programmtext
- **Compiler:** Programm zum Übersetzen des Programmtexts in Maschinsprache
- **Computer:** Gerät zum Ausführen von Programmen in Maschinsprache
- **Betriebssystem:** Programm zur Organisation all dieser Abläufe (Dateiverwaltung, Editor-, Compiler- und Programmaufruf)

Das erste C++ Programm

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```



```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;  Mache etwas (lies a ein)!
    // computation
    int b = a * a; // b = a^2
    b = b * b;     // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2 ← Berechne einen Wert (a^2)!
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

"Beiwerk": Kommentare

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

"Beiwerk": Kommentare

```
// Program: power8.cpp
// Raise a number to the eighth power. <
#include <iostream>
int main() {
    // input <
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation <
    int b = a * a; // b = a^2
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8 <
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

Kommentare

Dem Compiler ist's egal...

```
#include <iostream>
int main(){std::cout << "Compute a^8 for a =? ";
int a; std::cin >> a; int b = a * a; b = b * b;
std::cout << a << "^8 = " << b*b << "\n";return 0;}
```

Dem Compiler ist's egal...

```
#include <iostream>
int main(){std::cout << "Compute a^8 for a =? ";
int a; std::cin >> a; int b = a * a; b = b * b;
std::cout << a << "^8 = " << b*b << "\n";return 0;}
```

... uns aber nicht!

"Beiwerk": Include und Main-Funktion

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;      // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

"Beiwerk": Include und Main-Funktion

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>  Include-Direktive
int main() {
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;     // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```


"Beiwerk": Include und Main-Funktion

```
// Program: power8.cpp
// Raise a number to the eighth power.
#include <iostream>
int main() { Funktionsdeklaration der main-Funktion
    // input
    std::cout << "Compute a^8 for a =? ";
    int a;
    std::cin >> a;
    // computation
    int b = a * a; // b = a^2
    b = b * b;     // b = a^4
    // output b * b, i.e., a^8
    std::cout << a << "^8 = " << b * b << "\n";
    return 0;
}
```

Anweisungen: Mache etwas!

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a; // b = a^2  
    b = b * b;     // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Anweisungen: Mache etwas!

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a; // b = a^2  
    b = b * b; // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Ausdrucksanweisungen

Anweisungen: Mache etwas!

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a; // b = a^2  
    b = b * b;     // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0; ← Rückgabeanweisung  
}
```

Anweisungen – Effekte

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a;  
    std::cin >> a;  
    // computation  
    int b = a * a;  
    b = b * b;  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Effekt: Ausgabe des Strings Compute ...

Effekt: Eingabe einer Zahl und Speichern in a

Effekt: Speichern des berechneten Wertes von $a \cdot a$ in b

Effekt: Speichern des berechneten Wertes von $b \cdot b$ in b

Effekt: Rückgabe des Wertes 0

Effekt: Ausgabe des Wertes von a und des berechneten Wertes von $b \cdot b$

Anweisungen – Variablendefinitionen

```
int main() {  
    // input  
    std::cout << "Compute a^8 for a =? ";  
    int a; // Deklarationsanweisungen  
    std::cin >> a;  
    // computation  
    int b = a * a; // b = a^2  
    b = b * b;      // b = a^4  
    // output b * b, i.e., a^8  
    std::cout << a << "^8 = " << b * b << "\n";  
    return 0;  
}
```

Typ-
namen

Deklarationsanweisungen

Variablen

- repräsentieren (wechselnde) Werte
- haben
 - Name
 - Typ
 - Wert
 - Adresse

Variablen

- repräsentieren (wechselnde) Werte
- haben
 - Name
 - Typ
 - Wert
 - Adresse

`int a;` definiert Variable mit

- Name: a
- Typ: `int`
- Wert: (vorerst) undefiniert
- Adresse: durch Compiler bestimmt

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)
- oder zusammengesetzt ($b*b$)...

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)
- oder zusammengesetzt ($b*b$)...
- ...aus anderen Ausdrücken, mit Hilfe von Operatoren

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)
- oder zusammengesetzt ($b*b$)...
- ...aus anderen Ausdrücken, mit Hilfe von Operatoren
- haben einen Typ und einen Wert

Ausdrücke: Berechne einen Wert!

Ausdrücke

- repräsentieren *Berechnungen*
- sind entweder primär (b)
- oder zusammengesetzt ($b*b$)...
- ...aus anderen Ausdrücken, mit Hilfe von Operatoren
- haben einen Typ und einen Wert

Analogie: Baukasten

Ausdrücke

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;

// computation
int b = a * a; // b = a^2
b = b * b;     // b = a^4

// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << ".\n";

return 0;
```

Ausdrücke

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;
```

— Variablenname, primärer Ausdruck (+ Name und Adresse)

```
// computation
int b = a * a; // b = a^2
b = b * b; // b = a^4
```

— Variablenname, primärer Ausdruck (+ Name und Adresse)

```
// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << ".\n";
```

```
return 0;
```

— Literal, primärer Ausdruck

`// input`

Zusammengesetzter Ausdruck

```
std::cout << "Compute a^8 for a =? ";
```

```
int a;
```

```
std::cin >> a;
```

`// computation`

```
int b = a * a; // b = a^2
```

```
b = b * b; // b = a^4
```

`// output b * b, i.e., a^8`

```
std::cout << a << "^8 = " << b * b << ".\n";
```

```
return 0;
```

```
// input  
std::cout << "Compute a^8 for a =? ";  
int a;  
std::cin >> a;
```

```
// computation  
int b = a * a; // b = a^2
```

`b = b * b` ← Zweifach zusammengesetzter Ausdruck

~~// output b * b, i.e., a^8~~
`std::cout << a << "^8 = " << b * b << ".\n";`

`return` ↑ Vierfach zusammengesetzter Ausdruck

Literale

- repräsentieren konstante Werte
 - haben festen Typ und Wert
 - sind "syntaktische Werte"
-
- 0 hat Typ `int`, Wert 0.
 - `1.2e5` hat Typ `double`, Wert $1.2 \cdot 10^5$.

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;

// computation
int b = a * a; // b = a^2
b = b * b;     // b = a^4

// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << "\n";
return 0;
```

```
// input
std::cout << "Compute a^8 for a=? ";
int a;
std::cin >> a;

// computation
int b = a * a; // b = a^2
b = b * b;     // b = a^4

// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << "\n";
return 0;
```

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;
// computation
int b = a;
b = b * b;    // b = a^4

// output b * b, i.e., a^8
std::cout << a << "^8 = " << b * b << "\n";
return 0;
```

Rechter Operand (Variablenname)

Eingabe-Operator

Linker Operand (Eingabestrom)

```
// input
std::cout << "Compute a^8 for a =? ";
int a;
std::cin >> a;

// computation
int b = a * a; // b = a^2
b = b * b;     // b = a^4
// ou
std::cout << a << "^8 = " << b * b << "\n";
return 0;
```

Zuweisungsoperator

Multiplikationsoperator

Zuweisungsoperator =

- Linker Operand ist **L**-Wert (hat Adresse),
- Rechter Operand ist **R**-Wert des gleichen Typs.
- Weist linkem Operanden den Wert des rechten Operanden zu und gibt den linken Operanden als L-Wert zurück

Beispiele: $b = b * b$ und $a = b$

Zuweisungsoperator =

- Linker Operand ist **L**-Wert (hat Adresse),
- Rechter Operand ist **R**-Wert des gleichen Typs.
- Weist linkem Operanden den Wert des rechten Operanden zu und gibt den linken Operanden als L-Wert zurück

Beispiele: $b = b * b$ und $a = b$

Vorsicht, Falle!

Der Operator = entspricht dem Zuweisungsoperator in der Mathematik ($:=$), nicht dem Vergleichsoperator ($=$).

Zuweisungsoperator =

- Linker Operand ist **L**-Wert (hat Adresse),
- Rechter Operand ist **R**-Wert des gleichen Typs.
- Weist linkem Operanden den Wert des rechten Operanden zu und gibt den linken Operanden als L-Wert zurück

Beispiele: $b = b * b$ und $a = b$

Vorsicht, Falle!

Der Operator = entspricht dem Zuweisungsoperator in der Mathematik ($:=$), nicht dem Vergleichsoperator ($=$).

Weitere Operatoren und Erklärung zu L- und R-Werten im Handout

Der Compiler als Freund: Konstanten

Konstanten

- sind Variablen mit unveränderbarem Wert

```
const int speed_of_light = 299792458;
```

- Verwendung: `const` vor der Definition

Der Compiler als Freund: Konstanten

Konstanten

- sind Variablen mit unveränderbarem Wert

```
const int speed_of_light = 299792458;
```

- Verwendung: `const` vor der Definition

Der Compiler als Freund: Konstanten

Konstanten

- sind Variablen mit unveränderbarem Wert

```
const int speed_of_light = 299792458;
```

- Verwendung: `const` vor der Definition

Der Compiler als Freund: Konstanten

- Compiler kontrolliert Einhaltung des `const`-Versprechens

```
const int speed_of_light = 299792458;  
...  
speed_of_light = 3000000000;
```

Compilerfehler!



- Hilfsmittel zur Vermeidung von Fehlern: Konstanten erlauben garantierte Einhaltung des Versprechens „Wert ändert sich nicht“

Der Compiler als Freund: Konstanten

- Compiler kontrolliert Einhaltung des `const`-Versprechens

```
const int speed_of_light = 299792458;  
...  
speed_of_light = 3000000000;
```

Compilerfehler!



- Hilfsmittel zur Vermeidung von Fehlern: Konstanten erlauben garantierte Einhaltung des Versprechens „Wert ändert sich nicht“

Der Compiler als Freund: Konstanten

- Compiler kontrolliert Einhaltung des `const`-Versprechens

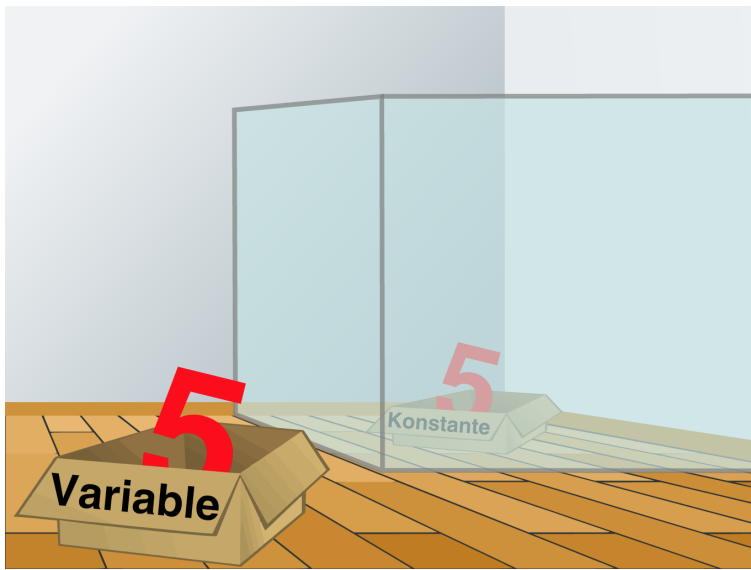
```
const int speed_of_light = 299792458;  
...  
speed_of_light = 3000000000;
```

Compilerfehler!



- Hilfsmittel zur Vermeidung von Fehlern: Konstanten erlauben garantierte Einhaltung des Versprechens „Wert ändert sich nicht“

Konstanten: Variablen hinter Glas



Die `const`-Richtlinie

Const-Richtlinie

Denke bei *jeder Variablen* darüber nach, ob sie im Verlauf des Programmes jemals ihren Wert ändern wird oder nicht. Im letzteren Falle verwende das Schlüsselwort `const`, um die Variable zu einer Konstanten zu machen.

Ein Programm, welches diese Richtlinie befolgt, heisst `const`-korrekt.

3. Ganze Zahlen

Auswertung arithmetischer Ausdrücke, Assoziativität und Präzedenz, arithmetische Operatoren, Wertebereich der Typen **int**, **unsigned int**

Beispiel: power8.cpp

```
int a; // Input
int r; // Result

std::cout << "Compute a^8 for a = ?";
std::cin >> a;

r = a * a; // r = a^2
r = r * r; // r = a^4

std::cout << "a^8 = " << r*r << '\n';
```

Celsius to Fahrenheit

```
// Program: fahrenheit.cpp
// Convert temperatures from Celsius to Fahrenheit.
#include <iostream>

int main() {
    // Input
    std::cout << "Temperature in degrees Celsius =? ";
    int celsius;
    std::cin >> celsius;

    // Computation and output
    std::cout << celsius << " degrees Celsius are "
              << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\n";
    return 0;
}
```

Celsius to Fahrenheit

```
// Program: fahrenheit.cpp
// Convert temperatures from Celsius to Fahrenheit.
#include <iostream>

int main() {
    // Input
    std::cout << "Temperature in degrees Celsius =? ";
    int celsius;
    std::cin >> celsius;

    // Computation and output
    std::cout << celsius << " degrees Celsius are "
              << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\n";
    return 0;
}
```

Arithmetischer Ausdruck

```
9 * celsius / 5 + 32
```

- Arithmetischer Ausdruck,

Arithmetischer Ausdruck

```
9 * celsius / 5 + 32
```

- Arithmetischer Ausdruck,
- **drei Literale**, eine Variable, drei Operatorsymbole

Arithmetischer Ausdruck

9 * celsius / 5 + 32

- Arithmetischer Ausdruck,
- drei Literale, eine Variable, drei Operatorsymbole

Arithmetischer Ausdruck

```
9 * celsius / 5 + 32
```

- Arithmetischer Ausdruck,
- drei Literale, eine Variable, **drei Operatorsymbole**

Arithmetischer Ausdruck

```
9 * celsius / 5 + 32
```

- Arithmetischer Ausdruck,
- drei Literale, eine Variable, drei Operatorsymbole

Wie ist der Ausdruck geklammert?

Punkt vor Strichrechnung

`9 * celsius / 5 + 32`

bedeutet

`(9 * celsius / 5) + 32`

Regel 1: Präzedenz

Multiplikative Operatoren ($*$, $/$, $\%$) haben höhere Präzedenz („binden stärker“) als additive Operatoren ($+$, $-$)

Assoziativität

Von links nach rechts

`9 * celsius / 5 + 32`

bedeutet

`((9 * celsius) / 5) + 32`

Assoziativität

Regel 2: Assoziativität

Arithmetische Operatoren ($*$, $/$, $\%$, $+$, $-$) sind linksassoziativ: bei gleicher Präzedenz erfolgt Auswertung von links nach rechts

Vorzeichen

$-3 - 4$

bedeutet

$(-3) - 4$

Regel 3: Stelligkeit

Unäre Operatoren $+$, $-$ vor binären $+$, $-$.

Jeder Ausdruck kann mit Hilfe der

- Assoziativitäten
- Präzedenzen
- Stelligkeiten

der beteiligten Operatoren eindeutig geklammert werden.

Ausdrucksbäume

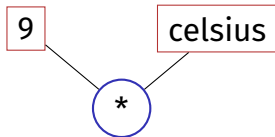
Klammerung ergibt Ausdrucksbaum

`9 * celsius / 5 + 32`

Ausdrucksbäume

Klammerung ergibt Ausdrucksbaum

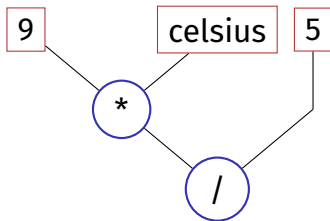
`(9 * celsius) / 5 + 32`



Ausdrucksbäume

Klammerung ergibt Ausdrucksbaum

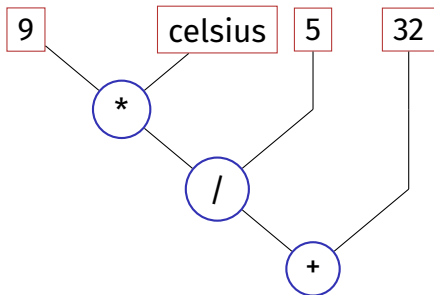
`((9 * celsius) / 5) + 32`



Ausdrucksbäume

Klammerung ergibt Ausdrucksbaum

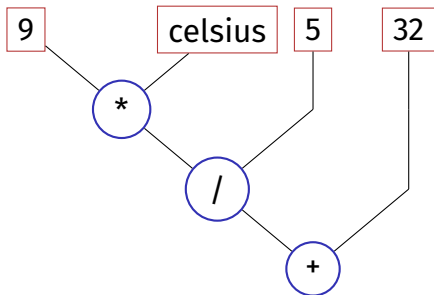
`((9 * celsius) / 5) + 32)`



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

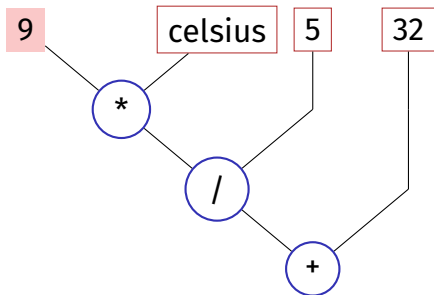
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

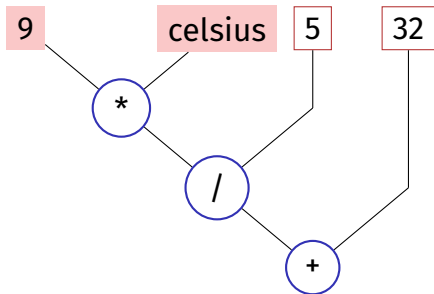
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

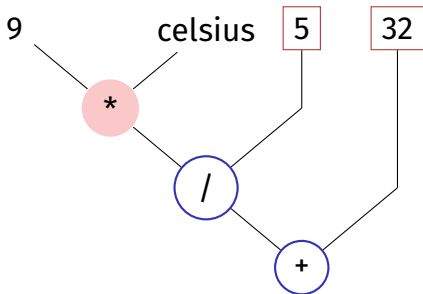
9 * celsius / 5 + 32



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

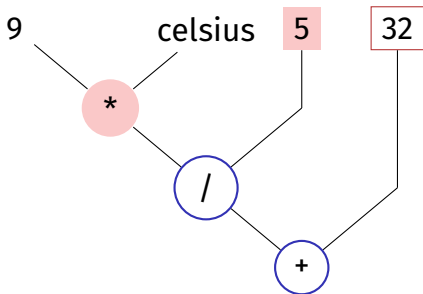
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

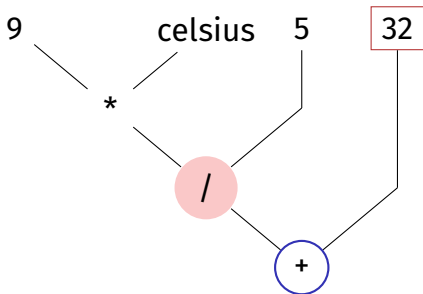
9 * celsius / 5 + 32



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

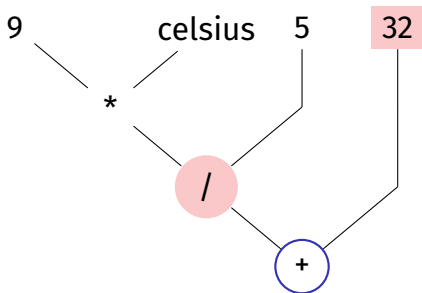
9 * celsius / 5 + 32



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

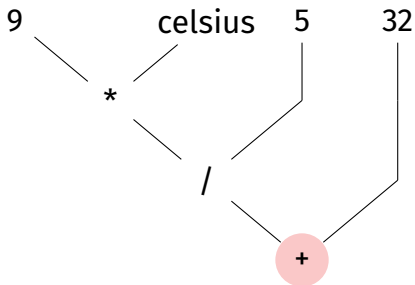
9 * celsius / 5 + 32



Auswertungsreihenfolge

Von oben nach unten im Ausdrucksbaum

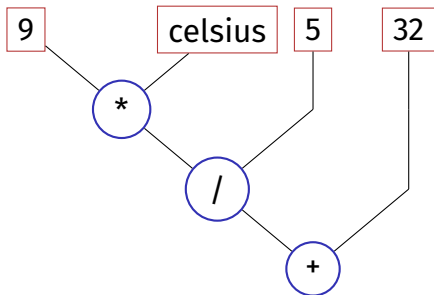
9 * celsius / 5 + 32



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

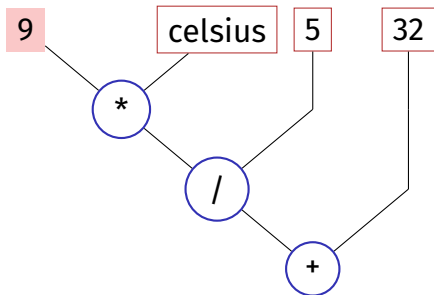
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

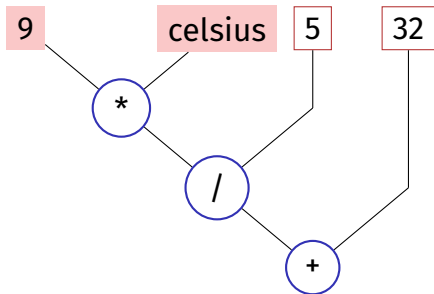
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

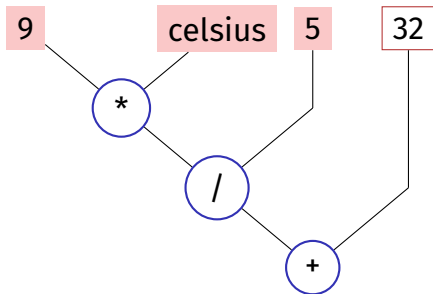
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

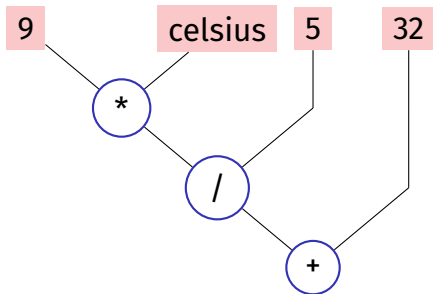
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

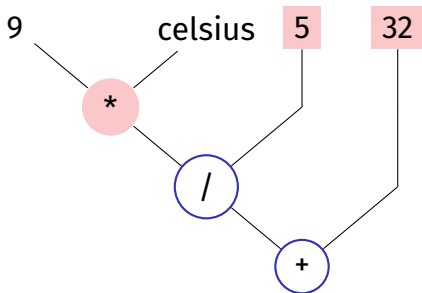
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

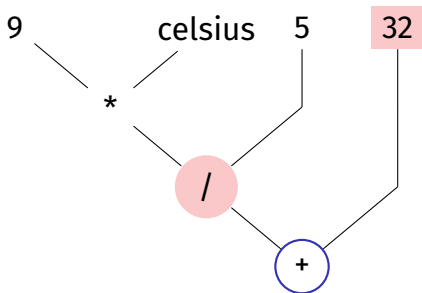
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

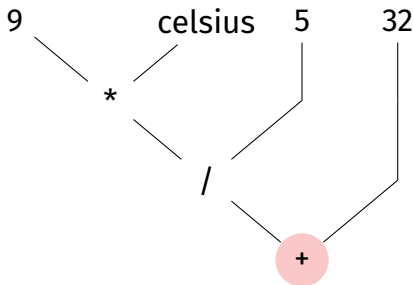
`9 * celsius / 5 + 32`



Auswertungsreihenfolge

Reihenfolge nicht eindeutig bestimmt:

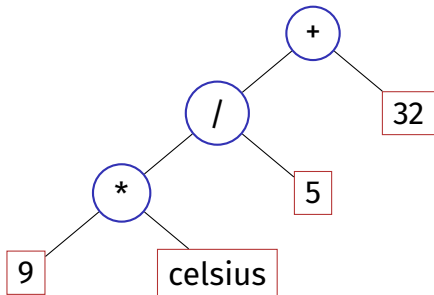
`9 * celsius / 5 + 32`



Ausdrucksbäume – Notation

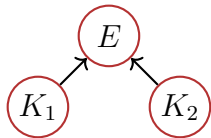
Üblichere Notation: Wurzel oben

`9 * celsius / 5 + 32`



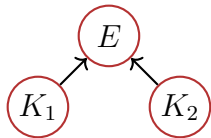
Auswertungsreihenfolge – formaler

Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



Auswertungsreihenfolge – formaler

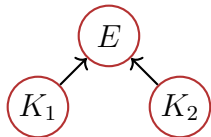
Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

Auswertungsreihenfolge – formaler

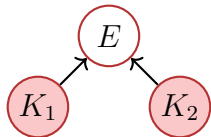
Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

Auswertungsreihenfolge – formaler

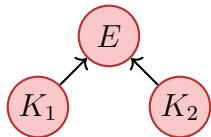
Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

Auswertungsreihenfolge – formaler

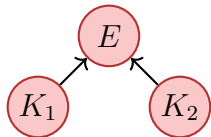
Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

Auswertungsreihenfolge – formaler

Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.

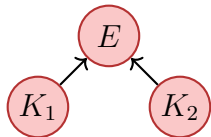


C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

- "Guter Ausdruck": jede gültige Reihenfolge führt zum gleichen Ergebnis.

Auswertungsreihenfolge – formaler

Gültige Reihenfolge: Jeder Knoten wird erst nach seinen Kindern ausgewertet.



C++: anzuwendende gültige Reihenfolge nicht spezifiziert.

- "Guter Ausdruck": jede gültige Reihenfolge führt zum gleichen Ergebnis.
- Beispiel eines "schlechten Ausdrucks": $a * (a=2)$

Auswertungsreihenfolge

Richtlinie

Vermeide das Verändern von Variablen, welche im selben Ausdruck noch einmal verwendet werden!

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl
https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	-a : R-Wert → R-Wert		4	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl

https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl
https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	$a+b : \text{R-Wert} \times \text{R-Wert} \rightarrow \text{R-Wert}$			links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl

https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

-a+b+c

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl
https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

-a+b+c

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl
https://en.cppreference.com/w/cpp/language/operator_precedence

Arithmetische Operatoren

	Symbol	Stelligkeit	Präzedenz	Assoziativität
Unäres +	+	1	16	rechts
Negation	-	1	16	rechts
Multiplikation	*	2	14	links
Division	/	2	14	links
Modulo	%	2	14	links
Addition	+	2	13	links
Subtraktion	-	2	13	links

$$-a+b+c = ((-a) + b) + c$$

R-Wert $\times$ R-Wert $\times$ R-Wert $\rightarrow$ R-Wert

CPP Reference Link, Vorsicht: dort Bindung stärker bei kleinerer Zahl
https://en.cppreference.com/w/cpp/language/operator_precedence

Einschub: Zuweisungsausdruck – nun genauer

- Bereits bekannt: $a = b$ bedeutet Zuweisung von b (R-Wert) an a (L-Wert). Rückgabe: L-Wert.

Einschub: Zuweisungsausdruck – nun genauer

- Bereits bekannt: $a = b$ bedeutet Zuweisung von b (R-Wert) an a (L-Wert). Rückgabe: L-Wert.
- Was bedeutet $a = b = c$?

Einschub: Zuweisungsausdruck – nun genauer

- Bereits bekannt: $a = b$ bedeutet Zuweisung von b (R-Wert) an a (L-Wert). Rückgabe: L-Wert.
- Was bedeutet $a = b = c$?
- Antwort: Zuweisung rechtsassoziativ, also

Einschub: Zuweisungsausdruck – nun genauer

- Bereits bekannt: $a = b$ bedeutet Zuweisung von b (R-Wert) an a (L-Wert). Rückgabe: L-Wert.
- Was bedeutet $a = b = c$?
- Antwort: Zuweisung rechtsassoziativ, also

$$a = b = c \quad \Longleftrightarrow \quad a = (b = c)$$

Mehrfachzuweisung: $a = b = 0 \implies b=0; a=0$

Division

- Operator / realisiert ganzzahlige Division

5 / 2 hat Wert 2

Division

- Operator / realisiert ganzzahlige Division

5 / 2 hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

Division

- Operator / realisiert ganzzahlige Division

5 / 2 hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...

```
9 / 5 * celsius + 32
```

Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...

```
1 * celsius + 32
```

Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...

```
15 + 32
```


Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...

47

Division

- Operator / realisiert ganzzahlige Division

`5 / 2` hat Wert 2

- In `fahrenheit.cpp`

```
9 * celsius / 5 + 32
```

15 degrees Celsius are 59 degrees Fahrenheit

- Mathematisch äquivalent...aber nicht in C++!

```
9 / 5 * celsius + 32
```

15 degrees Celsius are 47 degrees Fahrenheit

Richtlinie

- Auf möglichen Präzisionsverlust achten
- Potentiell verlustbehaftete Operationen möglichst spät durchführen, um „Fehlereskalation“ zu vermeiden

Division und Modulo

- Modulo-Operator berechnet Rest der ganzzahligen Division

$5 / 2$ hat Wert 2, $5 \% 2$ hat Wert 1.

- Es gilt

$$(-a)/b == -(a/b)$$

Division und Modulo

- Modulo-Operator berechnet Rest der ganzzahligen Division

$5 / 2$ hat Wert 2, $5 \% 2$ hat Wert 1.

- Es gilt

$$(-a)/b == -(a/b)$$

- Es gilt auch:

$$(a / b) * b + a \% b \text{ hat den Wert von } a.$$

Division und Modulo

- Modulo-Operator berechnet Rest der ganzzahligen Division

$5 / 2$ hat Wert 2, $5 \% 2$ hat Wert 1.

- Es gilt

$$(-a) / b == -(a / b)$$

- Es gilt auch:

$$(a / b) * b + a \% b \text{ hat den Wert von } a.$$

- Daraus lässt sich herleiten, welche Ergebnisse Division und Modulo mit negativen Zahlen ergeben (müssen)

Inkrement und Dekrement

- Erhöhen / Erniedrigen einer Zahl um 1 ist eine häufige Operation
- geht für einen L-Wert so:

```
expr = expr + 1.
```

Inkrement und Dekrement

```
expr = expr + 1.
```

Nachteile

- relativ lang

Inkrement und Dekrement

```
expr = expr + 1.
```

Nachteile

- relativ lang
- `expr` wird zweimal ausgewertet
 - Später: L-wertige Ausdrücke deren Auswertung „teuer“ ist

Inkrement und Dekrement

```
expr = expr + 1.
```

Nachteile

- relativ lang
- `expr` wird zweimal ausgewertet
 - Später: L-wertige Ausdrücke deren Auswertung „teuer“ ist
 - `expr` könnte einen Effekt haben (aber sollte nicht, siehe Richtlinie)

In-/Dekrement Operatoren

Post-Inkrement

`expr++`

Wert von `expr` wird um 1 erhöht, der alte Wert von `expr` wird (als R-Wert) zurückgegeben

In-/Dekrement Operatoren

Prä-Inkrement

`++expr`

Wert von `expr` wird um 1 erhöht, der neue Wert von `expr` wird (als L-Wert) zurückgegeben

In-/Dekrement Operatoren

Post-Dekrement

`expr--`

Wert von `expr` wird um 1 verringert, der alte Wert von `expr` wird (als R-Wert) zurückgegeben

In-/Dekrement Operatoren

Prä-Dekrement

`--expr`

Wert von `expr` wird um 1 verringert, der neue Wert von `expr` wird (als L-Wert) zurückgegeben

In-/Dekrement Operatoren

```
int a = 7;  
std::cout << ++a << "\n";  
std::cout << a++ << "\n";  
std::cout << a << "\n";
```

In-/Dekrement Operatoren

```
int a = 7;  
std::cout << ++a << "\n"; // 8  
std::cout << a++ << "\n";  
std::cout << a << "\n";
```


In-/Dekrement Operatoren

```
int a = 7;  
std::cout << ++a << "\n"; // 8  
std::cout << a++ << "\n"; // 8  
std::cout << a << "\n";
```

In-/Dekrement Operatoren

```
int a = 7;  
std::cout << ++a << "\n"; // 8  
std::cout << a++ << "\n"; // 8  
std::cout << a << "\n"; // 9
```

Arithmetische Zuweisungen

`a += b`

$\Leftrightarrow$

`a = a + b`

Arithmetische Zuweisungen

a += b

$\Leftrightarrow$

a = a + b

Analog für -, *, / und %

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0$

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

101011

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

101011 entspricht $32+8+2+1$.

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

101011 entspricht 43.

Binäre Zahlendarstellungen

Binäre Darstellung (Bits aus $\{0, 1\}$)

$$b_n b_{n-1} \dots b_1 b_0$$

entspricht der Zahl $b_n \cdot 2^n + \dots + b_1 \cdot 2 + b_0$

101011 entspricht 43.

Niedrigstes Bit, Least Significant Bit (LSB)

Höchstes Bit, Most Significant Bit (MSB)

■ Abschätzung der Grössenordnung von Zweierpotenzen¹:

$$2^{10} =$$

¹Dezimal vs. Binäre Einheiten: MB - Megabyte vs. MiB - Megabibyte (etc.)
kilo (K, Ki) – mega (M, Mi) – giga (G, Gi) – tera(T, Ti) – peta(P, Pi) – exa (E, Ei)

■ Abschätzung der Größenordnung von Zweierpotenzen¹:

$$2^{10} = 1024 = 1\text{Ki} \approx 10^3.$$

$$2^{32} =$$

¹Dezimal vs. Binäre Einheiten: MB - Megabyte vs. MiB - Megabibyte (etc.)
kilo (K, Ki) – mega (M, Mi) – giga (G, Gi) – tera(T, Ti) – peta(P, Pi) – exa (E, Ei)

■ Abschätzung der Grössenordnung von Zweierpotenzen¹:

$$2^{10} = 1024 = 1\text{Ki} \approx 10^3.$$

$$2^{32} = 4 \cdot (1024)^3 = 4\text{Gi} \approx 4 \cdot 10^9.$$

$$2^{64} =$$

¹Dezimal vs. Binäre Einheiten: MB - Megabyte vs. MiB - Megabibyte (etc.)
kilo (K, Ki) – mega (M, Mi) – giga (G, Gi) – tera(T, Ti) – peta(P, Pi) – exa (E, Ei)

■ Abschätzung der Grössenordnung von Zweierpotenzen¹:

$$2^{10} = 1024 = 1\text{Ki} \approx 10^3.$$

$$2^{32} = 4 \cdot (1024)^3 = 4\text{Gi} \approx 4 \cdot 10^9.$$

$$2^{64} = 16\text{Ei} \approx 16 \cdot 10^{18}.$$

¹Dezimal vs. Binäre Einheiten: MB - Megabyte vs. MiB - Megabibyte (etc.)
kilo (K, Ki) – mega (M, Mi) – giga (G, Gi) – tera(T, Ti) – peta(P, Pi) – exa (E, Ei)

Hexadezimale Zahlen

Zahlen zur Basis 16. Darstellung

$$h_n h_{n-1} \dots h_1 h_0$$

entspricht der Zahl

$$h_n \cdot 16^n + \dots + h_1 \cdot 16 + h_0.$$

Schreibweise in C++: vorangestelltes 0x

0xff entspricht 255.

Hex Nibbles

hex	bin	dec
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
a	1010	10
b	1011	11
c	1100	12
d	1101	13
e	1110	14
f	1111	15

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.
- Kompakte Darstellung von Binärzahlen.

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.
- Kompakte Darstellung von Binärzahlen.

32-bit Zahlen: 0x00000000 -- 0xffffffff .

0x80000000: höchstes Bit einer 32-bit Zahl gesetzt.

0xffffffff: alle Bits einer 32-bit Zahl gesetzt.

“0x8a20aaf0 ist eine Adresse in den oberen 2G des 32-bit Addressraumes”

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.
- Kompakte Darstellung von Binärzahlen.

32-bit Zahlen: 0x00000000 -- 0xffffffff .

0x80000000: höchstes Bit einer 32-bit Zahl gesetzt.

0xffffffff: alle Bits einer 32-bit Zahl gesetzt.

“0x8a20aaf0 ist eine Adresse in den oberen 2G des 32-bit Addressraumes”

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.
- Kompakte Darstellung von Binärzahlen.

32-bit Zahlen: 0x00000000 -- 0xffffffff .

0x80000000: höchstes Bit einer 32-bit Zahl gesetzt.

0xffffffff: alle Bits einer 32-bit Zahl gesetzt.

“0x8a20aaf0 ist eine Adresse in den oberen 2G des 32-bit Addressraumes”

Wozu Hexadezimalzahlen?

- Ein Hex-Nibble entspricht genau 4 Bits.
- Kompakte Darstellung von Binärzahlen.

32-bit Zahlen: 0x00000000 -- 0xffffffff .

0x80000000: höchstes Bit einer 32-bit Zahl gesetzt.

0xffffffff: alle Bits einer 32-bit Zahl gesetzt.

“0x8a20aaf0 ist eine Adresse in den oberen 2G des 32-bit Addressraumes”

Beispiel: Hex-Farben

#00FF00



r g b

Beispiel: Hex-Farben

#FFFFFF00



r g b

Beispiel: Hex-Farben

#808080



r g b

Beispiel: Hex-Farben

#FF0050



r g b

Wertebereich des Typs int

```
// Output the smallest and the largest value of type int.
#include <iostream>
#include <limits>

int main() {
    std::cout << "Minimum int value is "
               << std::numeric_limits<int>::min() << "...\n"
               << "Maximum int value is "
               << std::numeric_limits<int>::max() << "...\n";
    return 0;
}
```

Wertebereich des Typs int

```
// Output the smallest and the largest value of type int.
#include <iostream>
#include <limits>

int main() {
    std::cout << "Minimum int value is "
               << std::numeric_limits<int>::min() << "...\n"
               << "Maximum int value is "
               << std::numeric_limits<int>::max() << "...\n";
    return 0;
}
```

```
Minimum int value is -2147483648.
Maximum int value is 2147483647.
```

Wertebereich des Typs int

```
// Output the smallest and the largest value of type int.
#include <iostream>
#include <limits>

int main() {
    std::cout << "Minimum int value is "
               << std::numeric_limits<int>::min() << "...\n"
               << "Maximum int value is "
               << std::numeric_limits<int>::max() << "...\n";
    return 0;
}
```

Minimum int value is -2147483648.
Maximum int value is 2147483647.
Woher kommen diese Zahlen?

Wertebereich des Typs `int`

- Repräsentation mit B Bits. Wertebereich

$$\{-2^{B-1}, \dots, -1, 0, 1, \dots, 2^{B-1} - 2, 2^{B-1} - 1\}$$

Wertebereich des Typs `int`

- Repräsentation mit B Bits. Wertebereich

$$\{-2^{B-1}, \dots, -1, 0, 1, \dots, 2^{B-1} - 2, 2^{B-1} - 1\}$$

- Auf den meisten Plattformen $B = 32$

Wertebereich des Typs `int`

- Repräsentation mit B Bits. Wertebereich

$$\{-2^{B-1}, \dots, -1, 0, 1, \dots, 2^{B-1} - 2, 2^{B-1} - 1\}$$

- Für den Typ `int` garantiert C++ $B \geq 16$

Überlauf und Unterlauf

- Arithmetische Operationen (+, -, *) können aus dem Wertebereich herausführen.
- Ergebnisse können inkorrekt sein.

power8.cpp: $15^8 = -1732076671$

- In C++, Über-/Unterlauf auf `int` sind C++ **undefiniertes Verhalten**, d.h. **keine Fehlermeldungen**, keine Garantien für das Ergebnis

Der Typ `unsigned int`

- Wertebereich

$$\{0, 1, \dots, 2^B - 1\}$$

- Alle arithmetischen Operationen gibt es auch für `unsigned int`.
- Literale: `1u`, `17u` ...

int VS. unsigned int

- Für `int` sind Überlauf und Unterlauf undefiniertes Verhalten.
- Für `unsigned int` sind Überlauf und Unterlauf klar definiert, durch Rechnung modulo $2^B - 1$.

https://en.cppreference.com/w/cpp/language/operator_arithmetic#Overflows

https://en.wikipedia.org/wiki/Integer_overflow#Methods_to_address_integer_overflow_problems

Gemischte Ausdrücke

- Operatoren können Operanden verschiedener Typen haben (z.B. `int` und `unsigned int`).

```
17 + 17u
```

- Solche gemischten Ausdrücke sind vom "allgemeineren" Typ `unsigned int`.
- `int`-Operanden werden konvertiert nach `unsigned int`.

Konversion

int Wert	Vorzeichen	unsigned int Wert
x	≥ 0	x
x	< 0	$x + 2^B$

Zahlen mit Vorzeichen

Hinweis: Die verbleibenden Folien zur vorzeichenbehafteten Zahlendarstellung, dem Rechnen mit Binärzahlen sowie der Zweierkomplementdarstellung sind *nicht* klausurrelevant

Vorzeichenbehaftete Zahlendarstellung

- Soweit klar (hoffentlich): Binäre Zahlendarstellung ohne Vorzeichen, z.B.

$$[b_{31}b_{30} \dots b_0]_u \quad \hat{=} \quad b_{31} \cdot 2^{31} + b_{30} \cdot 2^{30} + \dots + b_0$$

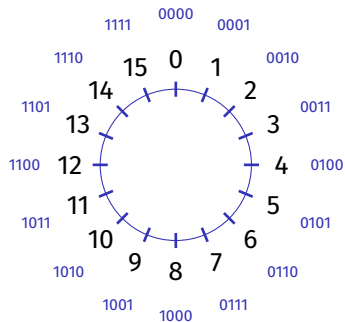
- Suche möglichst konsistente Lösung

Die Darstellung mit Vorzeichen sollte möglichst viel mit der vorzeichenlosen Lösung „gemein haben“. Positive Zahlen sollten sich in beiden Systemen algorithmisch möglichst gleich verhalten.

Rechnen mit Binärzahlen (4 Stellen)

Einfache Addition

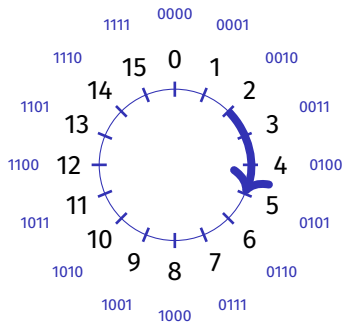
$$\begin{array}{r} 2 \qquad 0010 \\ +3 \quad +0011 \\ \hline \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Einfache Addition

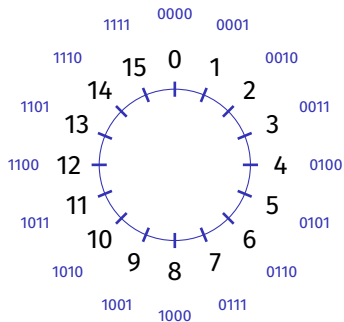
$$\begin{array}{r} 2 \\ +3 \\ \hline 5 \end{array} \quad \begin{array}{r} 0010 \\ +0011 \\ \hline 0101_2 = 5_{10} \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Einfache Subtraktion

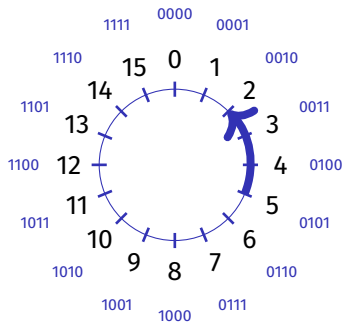
$$\begin{array}{r} 5 \qquad 0101 \\ -3 \quad -0011 \\ \hline \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Einfache Subtraktion

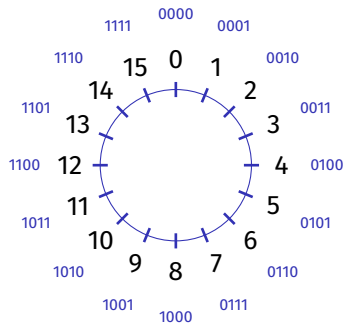
$$\begin{array}{r} 5 \qquad 0101 \\ -3 \qquad -0011 \\ \hline 2 \qquad 0010_2 = 2_{10} \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Addition mit Überlauf

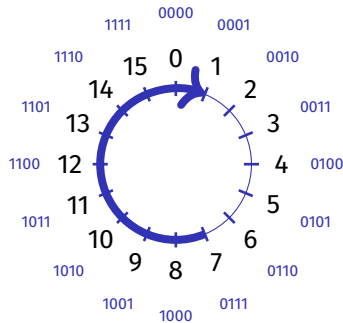
$$\begin{array}{r} 7 \qquad 0111 \\ +10 \quad +1010 \\ \hline \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Addition mit Überlauf

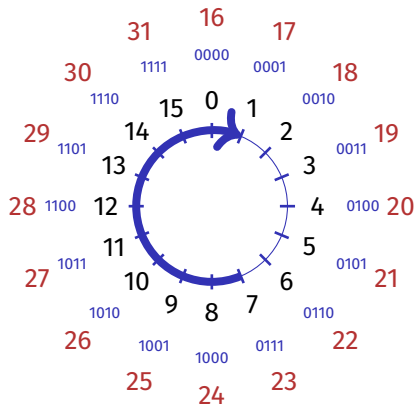
$$\begin{array}{rcl} 7 & 0111 & \\ +10 & +1010 & \\ \hline 17 & (1)0001_2 & = 1_{10}(= 17 \bmod 16) \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Addition mit Überlauf

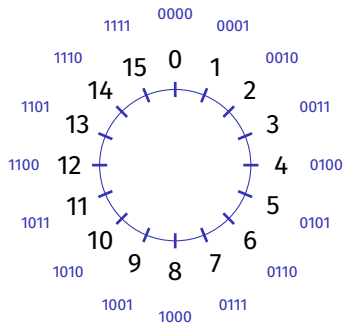
$$\begin{array}{rcl} 7 & 0111 & \\ +10 & +1010 & \\ \hline 17 & (1)0001_2 & = 1_{10}(= 17 \bmod 16) \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Subtraktion mit Unterlauf

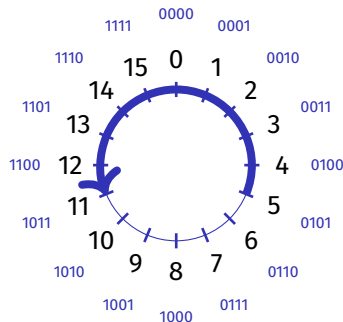
$$\begin{array}{r} 5 \qquad 0101 \\ -10 \quad -1010 \\ \hline \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

Subtraktion mit Unterlauf

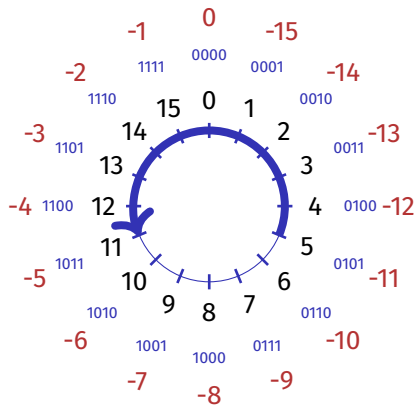
$$\begin{array}{r} 5 \qquad 0101 \\ -10 \quad -1010 \\ \hline -5 \quad (\dots 11)1011_2 = 11_{10} (= -5 \bmod 16) \end{array}$$



Rechnen mit Binärzahlen (4 Stellen)

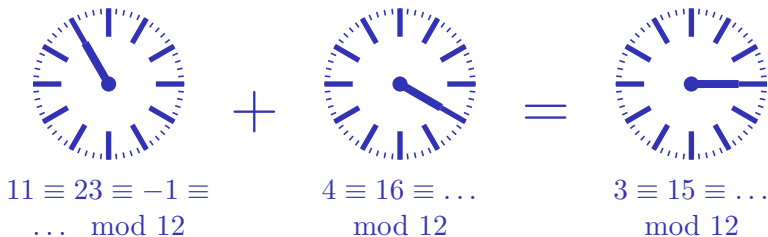
Subtraktion mit Unterlauf

$$\begin{array}{r} 5 \qquad 0101 \\ -10 \quad -1010 \\ \hline -5 \quad (\dots 11)1011_2 = 11_{10} (= -5 \bmod 16) \end{array}$$



Analogie

Modulo-Arithmetik: Rechnen im Kreis mit der Uhr²



²Die Arithmetik funktioniert auch für die Multiplikation

Negative Zahlen (3 Stellen)

	a	$-a$
0	000	
1	001	
2	010	
3	011	
4	100	
5	101	
6	110	
7	111	

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001		
2	010		
3	011		
4	100		
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010		
3	011		
4	100		
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011		
4	100		
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011	101	-3
4	100		
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011	101	-3
4	100	100	-4
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011	101	-3
4	100	100	-4
5	101		
6	110		
7	111		

Negative Zahlen (3 Stellen)

	a	$-a$	
0	000	000	0
1	001	111	-1
2	010	110	-2
3	011	101	-3
4	100	100	-4
5	101		
6	110		
7	111		

Das höchste Bit entscheidet über das Vorzeichen *und* es trägt zum Zahlwert bei.

4. Wahrheitswerte

Boolesche Funktionen; der Typ **bool**; logische und relationale Operatoren; Kurzschlussauswertung

Wo wollen wir hin?

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Wo wollen wir hin?

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Verhalten hängt ab vom Wert eines *Booleschen Ausdrucks*

Wo wollen wir hin?

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Verhalten hängt ab vom Wert eines Booleschen Ausdrucks

Boolesche Werte in der Mathematik

Boolesche Ausdrücke können zwei mögliche Werte annehmen:

0 oder 1

Boolesche Werte in der Mathematik

Boolesche Ausdrücke können zwei mögliche Werte annehmen:

0 oder 1

- 0 entspricht „falsch“
- 1 entspricht „wahr“

Der Typ `bool` in C++

- Repräsentiert Wahrheitswerte

Der Typ `bool` in C++

- Repräsentiert Wahrheitswerte
- Literale `false` und `true`

Der Typ `bool` in C++

- Repräsentiert Wahrheitswerte
- Literale `false` und `true`
- Wertebereich `{false, true}`

```
bool b = true; // Variable mit Wert true (wahr)
```

Relationale Operatoren

$a < b$ (kleiner als)

$\text{Zahlentyp} \times \text{Zahlentyp} \rightarrow \text{bool}$

$\text{R-Wert} \times \text{R-Wert} \rightarrow \text{R-Wert}$

Relationale Operatoren

`a < b` (kleiner als)

```
bool b = (1 < 3); // b =
```

Relationale Operatoren

$a < b$ (kleiner als)

```
bool b = (1 < 3); // b = true (wahr)
```

Relationale Operatoren

`a >= b` (grösser gleich)

```
int a = 0;  
bool b = (a >= 3); // b =
```

Relationale Operatoren

`a >= b` (grösser gleich)

```
int a = 0;  
bool b = (a >= 3); // b = false (falsch)
```

Relationale Operatoren

a == b (gleich)

```
int a = 4;  
bool b = (a % 3 == 1); // b =
```


Relationale Operatoren

`a == b` (gleich)

```
int a = 4;  
bool b = (a % 3 == 1); // b = true (wahr)
```

Relationale Operatoren

`a != b` (ungleich)

```
int a = 1;  
bool b = (a != 2*a-1); // b =
```

Relationale Operatoren

`a != b` (ungleich)

```
int a = 1;  
bool b = (a != 2*a-1); // b = false (falsch)
```

Boolesche Funktionen in der Mathematik

- Boolesche Funktion

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

- 0 entspricht „falsch“.
- 1 entspricht „wahr“.

$$x \wedge y$$

■ „Logisches Und“

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

■ 0 entspricht „falsch“.

■ 1 entspricht „wahr“.

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Logischer Operator &&

a && b (logisches Und)

```
int n = -1;  
int p = 3;  
bool b = (n < 0) && (0 < p); //
```

Logischer Operator &&

`a && b` (logisches Und)

```
int n = -1;  
int p = 3;  
bool b = (n < 0) && (0 < p); // b = true (wahr)
```

■ „Logisches Oder“

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}$$

■ 0 entspricht „falsch“.

■ 1 entspricht „wahr“.

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

Logischer Operator ||

a || b (logisches Oder)

```
int n = 1;  
int p = 0;  
bool b = (n < 0) || (0 < p); //
```

Logischer Operator ||

a || b (logisches Oder)

```
int n = 1;  
int p = 0;  
bool b = (n < 0) || (0 < p); // b = false (falsch)
```

- „Logisches Nicht“

$$f : \{0, 1\} \rightarrow \{0, 1\}$$

- 0 entspricht „falsch“.
- 1 entspricht „wahr“.

x	$\neg x$
0	1
1	0

Logischer Operator !

`!b` (logisches Nicht)

`bool` $\rightarrow$ `bool`

R-Wert $\rightarrow$ R-Wert

Logischer Operator !

!b (logisches Nicht)

```
int n = 1;  
bool b = !(n < 0); //
```

Logischer Operator !

!b (logisches Nicht)

```
int n = 1;  
bool b = !(n < 0); // b = true (wahr)
```

`!b && a`

`!b && a`
⇕
`(!b) && a`

Präzedenzen

`a && b || c && d`

Präzedenzen

$$\begin{array}{c} a \ \&\& \ b \ || \ c \ \&\& \ d \\ \Updownarrow \\ (a \ \&\& \ b) \ || \ (c \ \&\& \ d) \end{array}$$

Präzedenzen

a || b && c || d

Präzedenzen

$a \mid\mid b \ \&\& \ c \mid\mid d$
 $\Updownarrow$
 $a \mid\mid (b \ \&\& \ c) \mid\mid d$

Präzedenzen

```
7 + x < y && y != 3 * z || ! b
```

Präzedenzen

Der unäre logische Operator !
bindet stärker als

```
7 + x < y && y != 3 * z || (!b)
```

Präzedenzen

Der unäre logische Operator !
bindet stärker als
binäre arithmetische Operatoren. Diese
binden stärker als

```
(7 + x) < y && y != (3 * z) || (!b)
```

Präzedenzen

Der unäre logische Operator !
bindet stärker als
binäre arithmetische Operatoren. Diese
binden stärker als
relationale Operatoren,
und diese binden stärker als

```
((7 + x) < y) && (y != (3 * z)) || (!b)
```


Präzedenzen

Der unäre logische Operator !
bindet stärker als
binäre arithmetische Operatoren. Diese
binden stärker als
relationale Operatoren,
und diese binden stärker als
binäre logische Operatoren.

```
((7 + x) < y) && (y != (3 * z)) || (!b)
```

Einige Klammern auf den vorher gezeigten Folien waren unnötig.

Vollständigkeit

- $\wedge$, $\vee$ und $\neg$ sind die in C++ verfügbaren Booleschen Funktionen.

Vollständigkeit: $x \oplus y$

- $\wedge$, $\vee$ und $\neg$ sind die in C++ verfügbaren Booleschen Funktionen.
- Alle anderen *binären* Booleschen Funktionen sind daraus erzeugbar.
- Beispiel: Exclusives Oder $\oplus$.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Vollständigkeit: $x \oplus y$

$$x \oplus y = (x \vee y) \wedge \neg(x \wedge y).$$

Vollständigkeit: $x \oplus y$

$$x \oplus y = (x \vee y) \wedge \neg(x \wedge y).$$

$$(x \ || \ y) \ \&\& \ !(x \ \&\& \ y)$$

Vollständigkeit Beweis

- Identifiziere binäre Boolesche Funktionen mit ihrem charakteristischen Vektor.

Vollständigkeit Beweis

- Identifiziere binäre Boolesche Funktionen mit ihrem charakteristischen Vektor.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Vollständigkeit Beweis

- Identifiziere binäre Boolesche Funktionen mit ihrem charakteristischen Vektor.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Charakteristischer Vektor: 0110

Vollständigkeit Beweis

- Identifiziere binäre Boolesche Funktionen mit ihrem charakteristischen Vektor.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Charakteristischer Vektor: 0110

$$x \oplus y = f_{0110}(x, y)$$

Vollständigkeit Beweis

- Schritt 1: erzeuge die *elementaren* Funktionen f_{0001} , f_{0010} , f_{0100} , f_{1000}

$$f_{0001}(x, y) = x \wedge y$$

$$f_{0010}(x, y) = x \wedge \neg y$$

$$f_{0100}(x, y) = \neg x \wedge y$$

$$f_{1000}(x, y) = \neg(x \vee y)$$

Vollständigkeit Beweis

- Schritt 2: erzeuge alle Funktionen durch „Veroderung“ elementarer Funktionen

$$f_{1101}(x, y) = f_{1000}(x, y) \vee f_{0100}(x, y) \vee f_{0001}(x, y).$$

Vollständigkeit Beweis

- Schritt 2: erzeuge alle Funktionen durch „Veroderung“ elementarer Funktionen

$$f_{\textcolor{red}{1}1\textcolor{red}{0}1}(x, y) = f_{\textcolor{red}{1}000}(x, y) \vee f_{01\textcolor{red}{1}00}(x, y) \vee f_{000\textcolor{red}{1}}(x, y).$$

- Schritt 3: erzeuge f_{0000}

$$f_{0000} = 0.$$

bool vs int: Konversion

- `bool` kann überall dort verwendet werden, wo `int` gefordert ist

bool vs int: Konversion

- bool kann überall dort verwendet werden, wo int gefordert ist

bool	→	int
<i>true</i>	→	1
<i>false</i>	→	0

bool vs int: Konversion

- bool kann überall dort verwendet werden, wo int gefordert ist – und umgekehrt.

bool	→	int
<i>true</i>	→	1
<i>false</i>	→	0

int	→	bool
$\neq 0$	→	<i>true</i>
0	→	<i>false</i>

bool vs int: Konversion

- bool kann überall dort verwendet werden, wo int gefordert ist – und umgekehrt.

bool	→	int
<i>true</i>	→	1
<i>false</i>	→	0
int	→	bool
$\neq 0$	→	<i>true</i>
0	→	<i>false</i>

```
bool b = 3; // b=true
```


bool vs int: Konversion

- bool kann überall dort verwendet werden, wo int gefordert ist – und umgekehrt.
- Viele existierende Programme verwenden statt bool den Typ int.

Das ist schlechter Stil, der noch auf die Sprache C zurückgeht.

bool	→	int
<i>true</i>	→	1
<i>false</i>	→	0
int	→	bool
$\neq 0$	→	<i>true</i>
0	→	<i>false</i>

```
bool b = 3; // b=true
```

DeMorgansche Regeln

■ $\neg(a \wedge b) \iff (\neg a \vee \neg b)$

DeMorgansche Regeln

■ $!(a \ \&\& \ b) == (!a \ || \ !b)$

$! (\text{reich } \textit{und} \ \text{schön}) == (\text{arm } \textit{oder} \ \text{hässlich})$

DeMorgansche Regeln

■ $\neg(a \wedge b) \iff (\neg a \vee \neg b)$

■ $\neg(a \vee b) \iff (\neg a \wedge \neg b)$

$\neg(\text{reich und schön}) \iff (\text{arm oder hässlich})$

Anwendung: Entweder ... Oder (XOR)

`(x || y) && !(x && y)`

Anwendung: Entweder ... Oder (XOR)

`(x || y)      && !(x && y)` x oder y, und nicht beide

Anwendung: Entweder ... Oder (XOR)

`(x || y)      && !(x && y)` x oder y, und nicht beide

`(x || y)      && (!x || !y)`

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

$\neg(\neg x \&\& \neg y) \quad \&\& \neg(x \&\& y)$

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

$\neg(\neg x \&\& \neg y) \quad \&\& \neg(x \&\& y)$ nicht keines, und nicht beide

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

$\neg(\neg x \&\& \neg y) \quad \&\& \neg(x \&\& y)$ nicht keines, und nicht beide

$\neg(\neg x \&\& \neg y \mid\mid x \&\& y)$

Anwendung: Entweder ... Oder (XOR)

$(x \mid\mid y) \quad \&\& \neg(x \&\& y)$ x oder y, und nicht beide

$(x \mid\mid y) \quad \&\& (\neg x \mid\mid \neg y)$ x oder y, und eines nicht

$\neg(\neg x \&\& \neg y) \quad \&\& \neg(x \&\& y)$ nicht keines, und nicht beide

$\neg(\neg x \&\& \neg y \mid\mid x \&\& y)$ nicht: keines oder beide

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 6 $\Rightarrow$

```
x != 0 && z / x > y
```

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 6 $\Rightarrow$

```
true && z / x > y
```

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 6 $\Rightarrow$

```
true && z / x > y
```


Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 0 $\Rightarrow$

```
x != 0 && z / x > y
```

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 0 $\Rightarrow$

```
false && z / x > y
```

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 0 $\Rightarrow$

`false` (falsch)

Kurzschlussauswertung

- Logische Operatoren `&&` und `||` werten den *linken Operanden zuerst* aus.
- Falls das Ergebnis dann schon feststeht, wird der rechte Operand *nicht mehr* ausgewertet.

x hat Wert 0 $\Rightarrow$

```
x != 0 && z / x > y
```

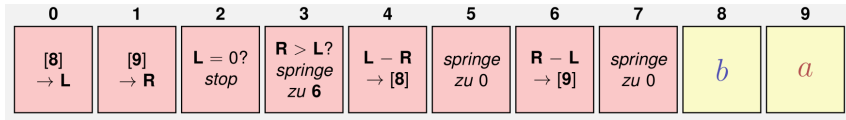
$\Rightarrow$ Keine Division durch 0

5. Kontrollanweisungen I

Auswahanweisungen, Iterationsanweisungen, Terminierung, Blöcke

Kontrollfluss

- Bisher: *linear* (von oben nach unten)
- Interessante Programme nutzen „Verzweigungen“ und „Sprünge“



Auswahanweisungen

realisieren Verzweigungen

- `if` Anweisung
- `if-else` Anweisung

if-Anweisung

```
if ( condition )  
    statement
```


if-Anweisung

```
if ( condition )  
    statement
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";
```

if-Anweisung

```
if ( condition )  
    statement
```

Ist *condition* wahr, dann wird *statement* ausgeführt.

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";
```

if-Anweisung

```
if ( condition )  
    statement
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";
```

Ist *condition* wahr, dann wird *statement* ausgeführt.

- *statement*: beliebige Anweisung (*Rumpf* der if-Anweisung)
- *condition*: konvertierbar nach `bool`

if-else-Anweisung

```
if ( condition )  
    statement1  
else  
    statement2
```

if-else-Anweisung

```
if ( condition )  
    statement1  
else  
    statement2
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

if-else-Anweisung

```
if ( condition )  
    statement1  
else  
    statement2
```

Ist *condition* wahr, so wird *statement1* ausgeführt, andernfalls wird *statement2* ausgeführt.

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

if-else-Anweisung

```
if ( condition )  
    statement1  
else  
    statement2
```

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```

Ist *condition* wahr, so wird *statement1* ausgeführt, andernfalls wird *statement2* ausgeführt.

- *condition*: konvertierbar nach bool.
- *statement1*: Rumpf des if-Zweiges
- *statement2*: Rumpf des else-Zweiges

Layout!

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```


Layout!

```
int a;  
std::cin >> a;  
if (a % 2 == 0)  
    std::cout << "even";  
else  
    std::cout << "odd";
```



The diagram illustrates indentation in the code. Two blue arrows point from the right side of the code to blue rectangular boxes labeled "Einrückung" (Indentation). The first arrow points from the line `std::cout << "even";` to the first "Einrückung" box. The second arrow points from the line `std::cout << "odd";` to the second "Einrückung" box.

Iterationsanweisungen

realisieren Schleifen:

- `for`-Anweisung
- `while`-Anweisung
- `do`-Anweisung

Berechne $1 + 2 + \dots + n$

```
// input
std::cout << "Compute the sum 1+...+n for n=?";
unsigned int n;
std::cin >> n;

// computation of sum_{i=1}^n i
unsigned int s = 0;
for (unsigned int i = 1; i <= n; ++i)
    s += i;

// output
std::cout << "1+...+" << n << " = " << s << ".\n";
```

Berechne $1 + 2 + \dots + n$

```
// input
std::cout << "Compute the sum 1+...+n for n=?";
unsigned int n;
std::cin >> n;

// computation of sum_{i=1}^n i
unsigned int s = 0;
for (unsigned int i = 1; i <= n; ++i)
    s += i;

// output
std::cout << "1+...+" << n << " = " << s << ".\n";
```

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i

s

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i	s
i==1	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i	s
i==1	i <= 2?

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1
i==2		

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1
i==2	i <= 2?	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2$, $s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3
i==3		

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2, s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3
i==3	i <= 2?	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2$, $s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3
i==3	falsch	

for-Anweisung am Beispiel

```
for ( unsigned int i=1; i <= n ; ++i )  
    s += i;
```

Annahmen: $n == 2$, $s == 0$

i		s
i==1	wahr	s == 1
i==2	wahr	s == 3
i==3	falsch	
		s == 3

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

- *init statement*: Ausdrucksanweisung, Deklarationsanweisung, Nullanweisung

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

- *init statement*: Ausdrucksanweisung, Deklarationsanweisung, Nullanweisung
- *condition*: konvertierbar nach `bool`

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

- *init statement*: Ausdrucksanweisung, Deklarationsanweisung, Nullanweisung
- *condition*: konvertierbar nach `bool`
- *expression*: beliebiger Ausdruck

for-Anweisung: Syntax

```
for (init statement; condition; expression)  
    body statement
```

- *init statement*: Ausdrucksanweisung, Deklarationsanweisung, Nullanweisung
- *condition*: konvertierbar nach `bool`
- *expression*: beliebiger Ausdruck
- *body statement*: beliebige Anweisung (*Rumpf* der for-Anweisung)

for-Anweisung: Terminierung

```
for (unsigned int i = 1; i <= n; ++i)  
    s += i;
```

Hier und meistens:

- *expression* ändert einen Wert, der in *condition* vorkommt.

for-Anweisung: Terminierung

```
for (unsigned int i = 1; i <= n; ++i)  
    s += i;
```

Hier und meistens:

- Nach endlich vielen Iterationen wird *condition* falsch: Terminierung.

Endlosschleifen

- Endlosschleifen sind leicht zu produzieren:

```
for ( ; ; ) ;
```

- Die *leere condition* ist wahr.
- Die *leere expression* hat keinen Effekt.
- Die *Nullanweisung* hat keinen Effekt.

Endlosschleifen

- Endlosschleifen sind leicht zu produzieren:

```
for ( ; ; ) ;
```

- Die *leere condition* ist wahr.
 - Die *leere expression* hat keinen Effekt.
 - Die *Nullanweisung* hat keinen Effekt.
- ... aber nicht automatisch zu erkennen.

```
for (init; cond; expr) stmt;
```

Halteproblem

Unentscheidbarkeit des Halteproblems

Es gibt kein C++ Programm, das für jedes C++- Programm P und jede Eingabe I korrekt feststellen kann, ob das Programm P bei Eingabe von I terminiert.

³Alan Turing, 1936. Theoretische Fragestellungen dieser Art waren für Alan Turing die Hauptmotivation für die Konstruktion seiner Rechenmaschine.

Halteproblem

Unentscheidbarkeit des Halteproblems

Es gibt kein C++ Programm, das für jedes C++- Programm P und jede Eingabe I korrekt feststellen kann, ob das Programm P bei Eingabe von I terminiert.

Das heisst, die Korrektheit von Programmen kann *nicht* automatisch überprüft werden.³

³Alan Turing, 1936. Theoretische Fragestellungen dieser Art waren für Alan Turing die Hauptmotivation für die Konstruktion seiner Rechenmaschine.

Beispiel: Primzahltest

Def.: Eine natürliche Zahl $n \geq 2$ ist eine Primzahl, wenn kein $d \in \{2, \dots, n-1\}$ ein Teiler von n ist.

Beispiel: Primzahltest

Def.: Eine natürliche Zahl $n \geq 2$ ist eine Primzahl, wenn kein $d \in \{2, \dots, n-1\}$ ein Teiler von n ist.

Eine Schleife, die das testet:

```
unsigned int d;  
for (d=2; n%d != 0; ++d);
```

Beispiel: Primzahltest

Def.: Eine natürliche Zahl $n \geq 2$ ist eine Primzahl, wenn kein $d \in \{2, \dots, n-1\}$ ein Teiler von n ist.

Eine Schleife, die das testet:

```
unsigned int d;  
for (d=2; n%d != 0; ++d);
```

(Rumpf ist die Null-Anweisung)

Primzahltest: Terminierung

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

- Fortschritt: Startwert $d=2$, dann in jeder Iteration plus 1 ($++d$)

Primzahltest: Terminierung

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

- Fortschritt: Startwert $d=2$, dann in jeder Iteration plus 1 ($++d$)
- Abbruch: $n\%d \neq 0$ evaluiert zu `false` sobald ein Teiler erreicht wurde
— spätestes, wenn $d == n$

Primzahltest: Terminierung

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

- Fortschritt: Startwert $d=2$, dann in jeder Iteration plus 1 ($++d$)
- Abbruch: $n\%d \neq 0$ evaluiert zu `false` sobald ein Teiler erreicht wurde
— spätestens, wenn $d == n$
- Fortschritt garantiert, dass Abbruchbedingung erreicht wird

Primzahltest: Korrektheit

```
unsigned int d;  
for (d=2; n%d != 0; ++d); // for n >= 2
```

Jeder mögliche Teiler $2 \leq d \leq n$ wird ausprobiert. Falls die Schleife mit $d == n$ terminiert, dann und genau dann ist n prim.

- Blöcke gruppieren mehrere Anweisungen zu einer neuen Anweisung

```
{statement1 statement2 ... statementN}
```

Blöcke

- Blöcke gruppieren mehrere Anweisungen zu einer neuen Anweisung
- Beispiel: Rumpf der main Funktion

```
int main() {  
    ...  
}
```

- Blöcke gruppieren mehrere Anweisungen zu einer neuen Anweisung
- Beispiel: Schleifenrumpf

```
for (unsigned int i = 1; i <= n; ++i) {  
    s += i;  
    std::cout << "partial sum is " << s << "\n";  
}
```

- Blöcke gruppieren mehrere Anweisungen zu einer neuen Anweisung
- Beispiel: if / else

```
if (d < n) // d is a divisor of n in {2,...,n-1}
    std::cout << n << " = " << d << " * " << n / d << ".\n";
else {
    assert (d == n);
    std::cout << n << " is prime.\n";
}
```

6. Kontrollanweisungen II

Sichtbarkeit, Lokale Variablen, While-Anweisung, Do-Anweisung, Sprunganweisungen

Sichtbarkeit

Deklaration in einem Block ist ausserhalb des Blocks nicht *sichtbar*.

```
int main()
{
    {
        int i = 2;
    }
    std::cout << i; // Fehler: undeklarerter Name
    return 0;
}
```

Sichtbarkeit

Deklaration in einem Block ist ausserhalb des Blocks nicht *sichtbar*.

```
int main()
{
  {
    int i = 2;
  }
  std::cout << i; // Fehler: undeklariierter Name
  return 0;
}
```

main block

block

„Blickrichtung“

Kontrollanweisung definiert Block

Kontrollanweisungen verhalten sich in diesem Zusammenhang wie Blöcke.

```
int main()
{
    int s = 0;
    for (unsigned int i = 0; i < 10; ++i)
        s += i;
    std::cout << i; // Fehler: undeklarer Name
    return 0;
}
```

Kontrollanweisung definiert Block

Kontrollanweisungen verhalten sich in diesem Zusammenhang wie Blöcke.

```
int main()
{
    int s = 0;
    block | for (unsigned int i = 0; i < 10; ++i)
           s += i;
    std::cout << i; // Fehler: undeklariertes Name
    return 0;
}
```

Potentieller Gültigkeitsbereich

Im Block

```
{  
    ...  
    int i = 2;  
    ...  
}
```

Im Funktionsrumpf

```
int main() {  
    ...  
    int i = 2;  
    ...  
    return 0;  
}
```

In Kontrollanweisung

```
for (int i = 0; i < 10; ++i) {s += i; ... }
```

Potentieller Gültigkeitsbereich

Im Block

```
{  
    ...  
    int i = 2;  
    ...  
}
```

scope

Im Funktionsrumpf

```
int main() {  
    ...  
    int i = 2;  
    ...  
    return 0;  
}
```

scope

In Kontrollanweisung

```
for (int i = 0; i < 10; ++i) {s += i; ... }
```

scope

Gültigkeitsbereich

```
int main()
{
    int i = 2;
    for (int i = 0; i < 5; ++i)
        // outputs 0,1,2,3,4
        std::cout << i;
    // outputs 2
    std::cout << i;
    return 0;
}
```

Potentieller Gültigkeitsbereich

```
int main()
{
    int i = 2;
    for (int i = 0; i < 5; ++i)
        // outputs 0,1,2,3,4
        std::cout << i;
    // outputs 2
    std::cout << i;
    return 0;
}
```


Wirklicher Gültigkeitsbereich

```
int main()
{
    int i = 2;
    for (int i = 0; i < 5; ++i)
        // outputs 0,1,2,3,4
        std::cout << i;
    // outputs 2
    std::cout << i;
    return 0;
}
```

Lokale Variablen

```
int main()
{
    int i = 5;
    for (int j = 0; j < 5; ++j) {
        std::cout << ++i; // outputs
        int k = 2;
        std::cout << --k; // outputs
    }
}
```

Lokale Variablen

```
int main()
{
    int i = 5;
    for (int j = 0; j < 5; ++j) {
        std::cout << ++i; // outputs 6, 7, 8, 9, 10
        int k = 2;
        std::cout << --k; // outputs 1, 1, 1, 1, 1
    }
}
```

Lokale Variablen

```
int main()
{
    int i = 5;
    for (int j = 0; j < 5; ++j) {
        std::cout << ++i; // outputs
        int k = 2;
        std::cout << --k; // outputs
    }
}
```

Lokale Variablen (Deklaration in einem Block) haben *automatische Speicherdauer*.

while Anweisung

```
while (condition)  
    statement
```

while Anweisung

```
while (condition)  
    statement
```

ist äquivalent zu

```
for (; condition; )  
    statement
```

while-Anweisung: Warum?

- Bei for-Anweisung ist oft expression allein für den Fortschritt zuständig („Zählschleife“)

```
for (unsigned int i = 1; i <= n; ++i)  
    s += i;
```

while-Anweisung: Warum?

- Bei `for`-Anweisung ist oft expression allein für den Fortschritt zuständig („Zählschleife“)

```
for (unsigned int i = 1; i <= n; ++i)
    s += i;
```

- Falls der Fortschritt nicht so einfach ist, kann `while` besser lesbar sein.

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1 , 4

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases}, i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1 , 4 , 2

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases} , i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1 , 4 , 2 , 1

Die Collatz-Folge

- $n_0 = n$
- $n_i = \begin{cases} \frac{n_{i-1}}{2} & , \text{ falls } n_{i-1} \text{ gerade} \\ 3n_{i-1} + 1 & , \text{ falls } n_{i-1} \text{ ungerade} \end{cases} , i \geq 1.$

n=5: 5 , 16 , 8 , 4 , 2 , 1 , 4 , 2 , 1 , ... (Repetition bei 1)

Die Collatz-Folge in C++

```
// Input
std::cout << "Compute Collatz sequence, n =? ";
unsigned int n;
std::cin >> n;

// Iteration
while (n != 1) { // stop when 1 reached
    if (n % 2 == 0) n = n / 2; // n is even
    else n = 3 * n + 1; // n is odd
    std::cout << n << " ";
}
std::cout << n; // output 1
```

Die Collatz-Folge in C++

```
// Input
std::cout << "Compute Collatz sequence, n =? ";
unsigned int n;
std::cin >> n;

// Iteration
while (n != 1) { // stop when 1 reached
    if (n % 2 == 0) n = n / 2; // n is even
    else n = 3 * n + 1; // n is odd
    std::cout << n << " ";
}
std::cout << n; // output 1
```

Die Collatz-Folge in C++

n = 27:

82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242,
121, 364, 182, 91, 274, 137, 412, 206, 103, 310, 155, 466, 233, 700,
350, 175, 526, 263, 790, 395, 1186, 593, 1780, 890, 445, 1336, 668,
334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638,
319, 958, 479, 1438, 719, 2158, 1079, 3238, 1619, 4858, 2429, 7288,
3644, 1822, 911, 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616,
2308, 1154, 577, 1732, 866, 433, 1300, 650, 325, 976, 488, 244, 122,
61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8,
4, 2, 1

do-Anweisung

```
do  
    statement  
while (condition);
```

do-Anweisung

```
do  
    statement  
while (condition);
```

ist äquivalent zu

```
statement  
while (condition)  
    statement
```


Beispiel Taschenrechner

Summiere ganze Zahlen (bei 0 ist Schluss):

```
int a;    // next input value
int s = 0; // sum of values so far
do {
    std::cout << "next number =? ";
    std::cin >> a;
    s += a;
    std::cout << "sum = " << s << "\n";
} while (a != 0);
```

Sprunganweisungen

- `break;`
- `continue;`

Taschenrechner mit break

Summiere ganze Zahlen (bei 0 ist Schluss):

```
int a;  
int s = 0;  
do {  
    std::cout << "next number =? ";  
    std::cin >> a;  
    s += a; /* irrelevant in last iteration */  
    std::cout << "sum = " << s << "\n";  
} while (a != 0);
```

Taschenrechner mit break

Unterdrücke irrelevante Addition von 0:

```
int a;
int s = 0;
do {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a == 0) break; // Abbruch in der Mitte
    s += a;
    std::cout << "sum = " << s << "\n";
} while (a != 0)
```

Taschenrechner mit break

Äquivalent und noch etwas einfacher:

```
int a;
int s = 0;
while (true) {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a == 0) break; // Abbruch in der Mitte
    s += a;
    std::cout << "sum = " << s << "\n";
}
```

Taschenrechner mit `continue`

Ignoriere alle negativen Eingaben:

```
int a;
int s = 0;
while (true) {
    std::cout << "next number =? ";
    std::cin >> a;
    if (a < 0) continue; // springe zu }
    if (a == 0) break;
    s += a;
    std::cout << "sum = " << s << "\n";
}
```

break und continue in der Praxis

- Vorteil: Können verschachtelte `if-else`-Blöcke (oder komplexe Disjunktionen) vermeiden

break und continue in der Praxis

- Vorteil: Können verschachtelte `if-else`-Blöcke (oder komplexe Disjunktionen) vermeiden
- Aber führen zu mehr Sprüngen und somit zu potentiell komplexerem Kontrollfluss

break und continue in der Praxis

- Vorteil: Können verschachtelte `if-else`-Blöcke (oder komplexe Disjunktionen) vermeiden
- Aber führen zu mehr Sprüngen und somit zu potentiell komplexerem Kontrollfluss
- Ihr Einsatz ist daher umstritten und sollte mit Vorsicht geschehen

Kontrollfluss

Reihenfolge der (wiederholten) Ausführung von Anweisungen

- Grundsätzlich von oben nach unten...



Kontrollfluss

Reihenfolge der (wiederholten) Ausführung von Anweisungen

- Grundsätzlich von oben nach unten...
- ...ausser in Auswahl- und Kontrollanweisungen

condition



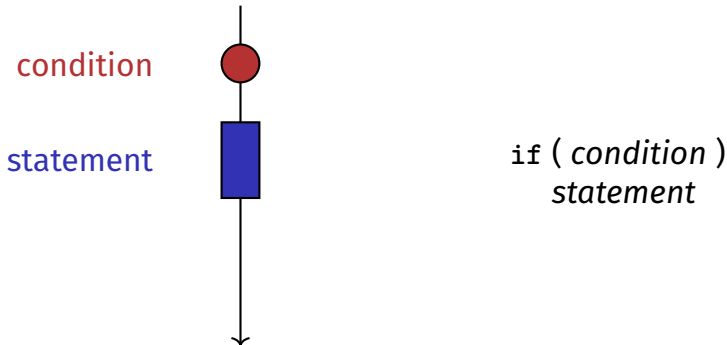
if (*condition*)
 statement



Kontrollfluss

Reihenfolge der (wiederholten) Ausführung von Anweisungen

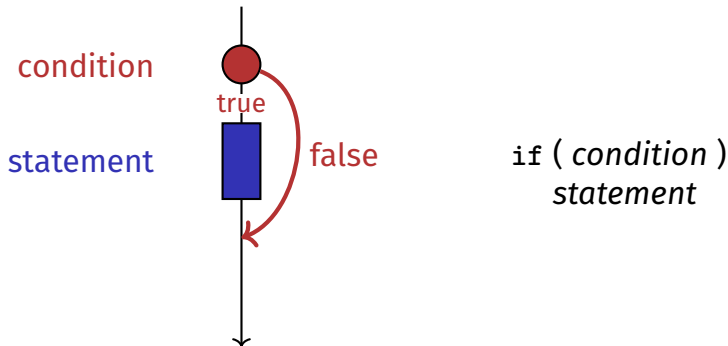
- Grundsätzlich von oben nach unten...
- ...ausser in Auswahl- und Kontrollanweisungen



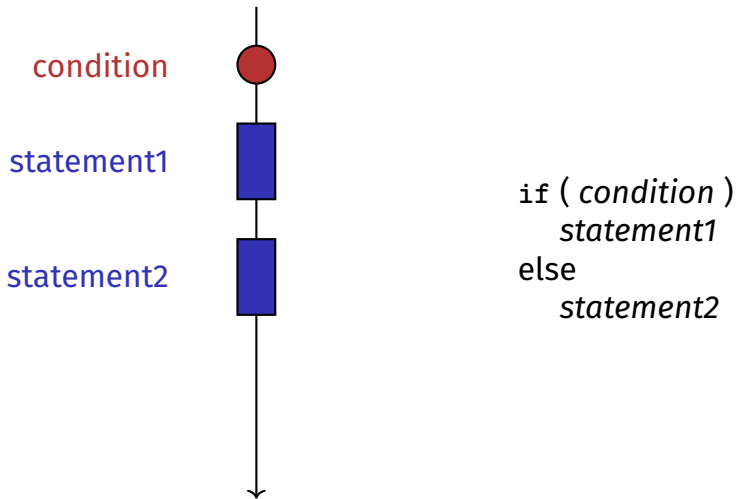
Kontrollfluss

Reihenfolge der (wiederholten) Ausführung von Anweisungen

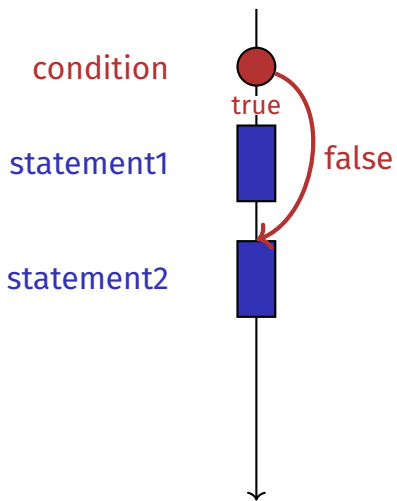
- Grundsätzlich von oben nach unten...
- ...ausser in Auswahl- und Kontrollanweisungen



Kontrollfluss if else

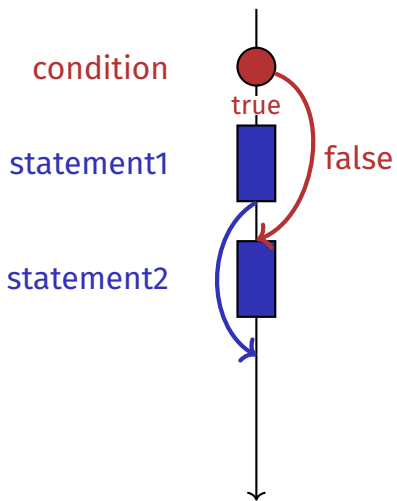


Kontrollfluss if else



```
if ( condition )  
    statement1  
else  
    statement2
```

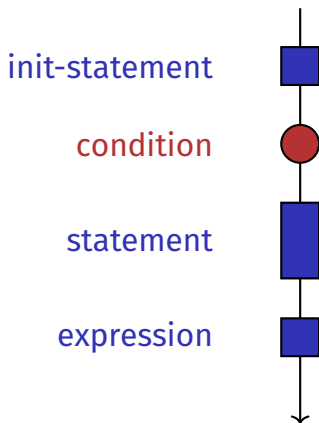
Kontrollfluss if else



```
if ( condition )  
    statement1  
else  
    statement2
```

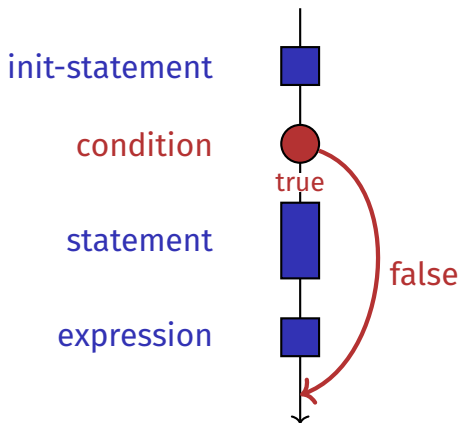

Kontrollfluss for

`for (init statement ; condition ; expression)
statement`



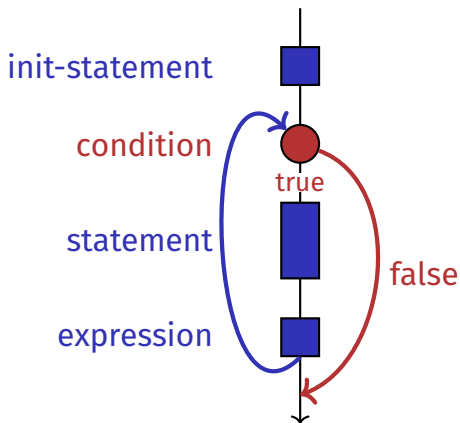
Kontrollfluss for

`for (init statement ; condition ; expression)
statement`

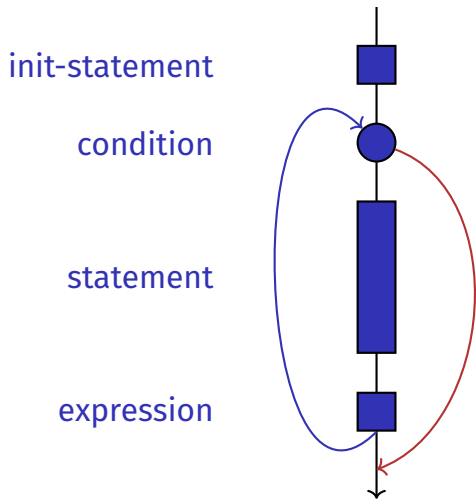


Kontrollfluss for

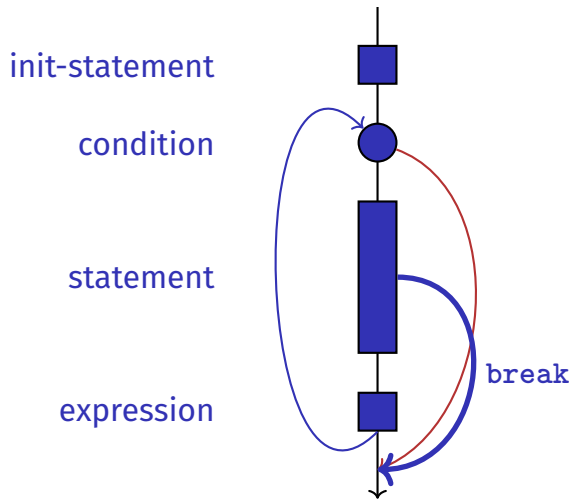
`for ( init statement ; condition ; expression )`
`statement`



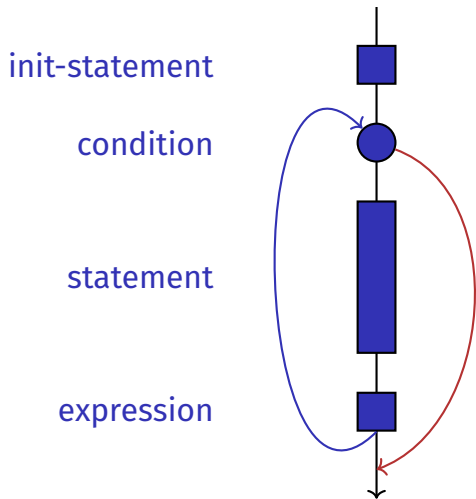
Kontrollfluss `break` und `continue` in `for`



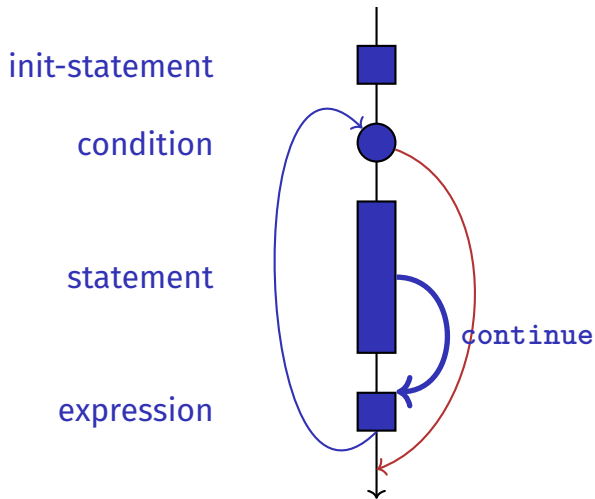
Kontrollfluss *break* und *continue* in for



Kontrollfluss `break` und `continue` in `for`



Kontrollfluss `break` und `continue` in `for`



Kontrollfluss: Die guten alten Zeiten?

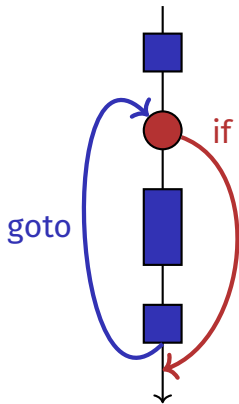
Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).

Kontrollfluss: Die guten alten Zeiten?

Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).



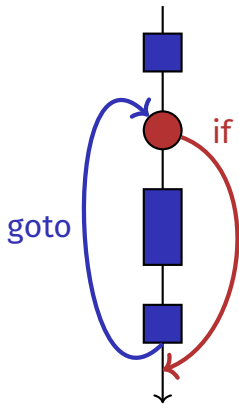
Kontrollfluss: Die guten alten Zeiten?

Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).

Sprachen, die darauf basieren:

- Maschinensprache



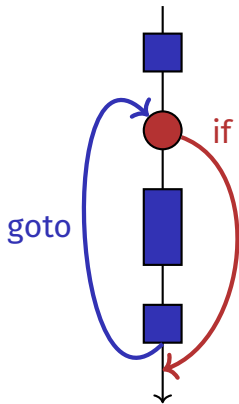
Kontrollfluss: Die guten alten Zeiten?

Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).

Sprachen, die darauf basieren:

- Maschinensprache
- Assembler („höhere“ Maschinensprache)



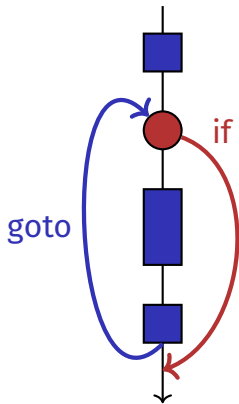
Kontrollfluss: Die guten alten Zeiten?

Beobachtung

Wir brauchen eigentlich nur `ifs` und Sprünge an beliebige Stellen im Programm (`goto`).

Sprachen, die darauf basieren:

- Maschinensprache
- Assembler („höhere“ Maschinensprache)
- BASIC, die erste Programmiersprache für ein allgemeines Publikum (1964)



BASIC und die Home-Computer...

...ermöglichten einer ganzen Generation von Jugendlichen das Programmieren.



Home-Computer Commodore C64 (1982)

Spaghetti-Code mit goto

Ausgabe von ???????????

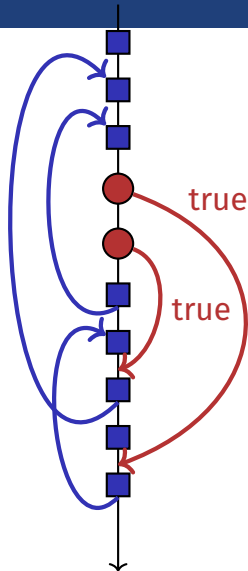
mit der Programmiersprache BASIC:

```
10 N=2
20 D=1
30 D=D+1
40 IF N=D GOTO 100
50 IF N/D = INT(N/D) GOTO 70
60 GOTO 30
70 N=N+1
80 GOTO 20
100 PRINT N
110 GOTO 70
```

Spaghetti-Code mit goto

Ausgabe aller Primzahlen
mit der Programmiersprache BASIC:

```
10 N=2
20 D=1
30 D=D+1
40 IF N=D GOTO 100
50 IF N/D = INT(N/D) GOTO 70
60 GOTO 30
70 N=N+1
80 GOTO 20
100 PRINT N
110 GOTO 70
```



Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen

Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen
- Wenige Zeilen Code

Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen
- Wenige Zeilen Code
- Einfacher Kontrollfluss

Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen
- Wenige Zeilen Code
- Einfacher Kontrollfluss
- Einfache Ausdrücke

Die „richtige“ Iterationsanweisung

Ziele: Lesbarkeit, Prägnanz. Insbesondere

- Wenige Anweisungen
- Wenige Zeilen Code
- Einfacher Kontrollfluss
- Einfache Ausdrücke

Ziele sind oft nicht gleichzeitig erreichbar.

Ungerade Zahlen in $\{0, \dots, 100\}$

Erster (korrekter) Versuch:

```
for (unsigned int i = 0; i < 100; ++i) {  
    if (i % 2 == 0)  
        continue;  
    std::cout << i << "\n";  
}
```

Ungerade Zahlen in $\{0, \dots, 100\}$

Weniger Anweisungen, weniger Zeilen:

```
for (unsigned int i = 0; i < 100; ++i) {  
    if (i % 2 != 0)  
        std::cout << i << "\n";  
}
```

Ungerade Zahlen in $\{0, \dots, 100\}$

Weniger Anweisungen, einfacherer Kontrollfluss:

```
for (unsigned int i = 1; i < 100; i += 2)
    std::cout << i << "\n";
```


Ungerade Zahlen in $\{0, \dots, 100\}$

Weniger Anweisungen, einfacherer Kontrollfluss:

```
for (unsigned int i = 1; i < 100; i += 2)  
    std::cout << i << "\n";
```

Das ist hier die „richtige“ Iterationsanweisung

1. Funktionale Anforderung:

6 → "Excellent ... You passed!"

5,4 → "You passed!"

3 → "Close, but ... You failed!"

2,1 → "You failed!"

sonst → "Error!"

1. Funktionale Anforderung:

```
6 → "Excellent ... You passed!"  
5,4 → "You passed!"  
3 → "Close, but ... You failed!"  
2,1 → "You failed!"  
sonst → "Error!"
```

2. Ausserdem: Text- und Codeduplikation vermeiden

Notenausgabe mit if-Anweisungen

```
int grade;
...
if (grade == 6) std::cout << "Excellent ... ";
if (4 <= grade && grade <= 6) {
    std::cout << "You passed!";
} else if (1 <= grade && grade < 4) {
    if (grade == 3) std::cout << "Close, but ... ";
    std::cout << "You failed!";
} else std::cout << "Error!";
```

Notenausgabe mit if-Anweisungen

```
int grade;
...
if (grade == 6) std::cout << "Excellent ... ";
if (4 <= grade && grade <= 6) {
    std::cout << "You passed!";
} else if (1 <= grade && grade < 4) {
    if (grade == 3) std::cout << "Close, but ... ";
    std::cout << "You failed!";
} else std::cout << "Error!";
```

Nachteil: Kontrollfluss – und somit Programmverhalten – nicht gerade offensichtlich

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```

Notenausgabe mit switch-Anweisung

```
switch (grade) {  Springe zu passendem case  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```



Durchfallen

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;   
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}  

```

Durchfallen

Verlasse switch

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```



Durchfallen

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;   
    default: std::cout << "Error!";  
}
```

Durchfallen

Verlasse switch

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  In allen anderen Fällen  
}
```

Notenausgabe mit switch-Anweisung

```
switch (grade) {  
    case 6: std::cout << "Excellent ... ";  
    case 5:  
    case 4: std::cout << "You passed!";  
        break;  
    case 3: std::cout << "Close, but ... ";  
    case 2:  
    case 1: std::cout << "You failed!";  
        break;  
    default: std::cout << "Error!";  
}
```

Vorteil: Kontrollfluss klar erkennbar

Die switch-Anweisung

```
switch (expression)  
statement
```

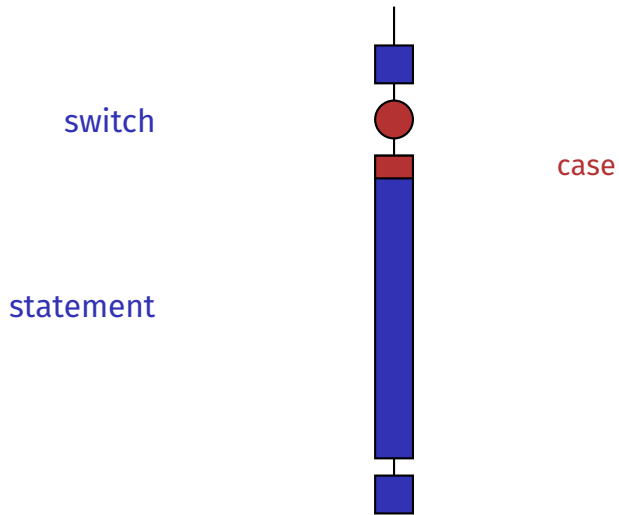
- *expression*: Ausdruck, konvertierbar in einen integralen Typ
- *statement* : beliebige Anweisung, in welcher `case` und `default`-Marken erlaubt sind, `break` hat eine spezielle Bedeutung.

Die `switch`-Anweisung

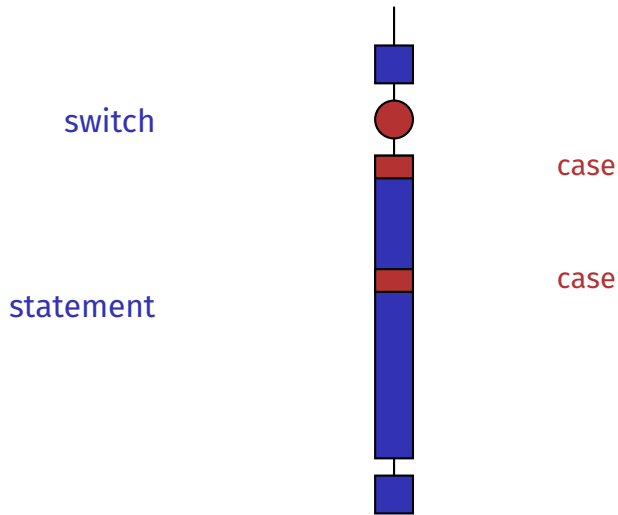
```
switch (expression)  
    statement
```

- *expression*: Ausdruck, konvertierbar in einen integralen Typ
- *statement* : beliebige Anweisung, in welcher `case` und `default`-Marken erlaubt sind, `break` hat eine spezielle Bedeutung.
- Benutzung des Durchfallens in der Praxis umstritten, Einsatz gut abwägen (entsprechende Compilerwarnung kann aktiviert werden)

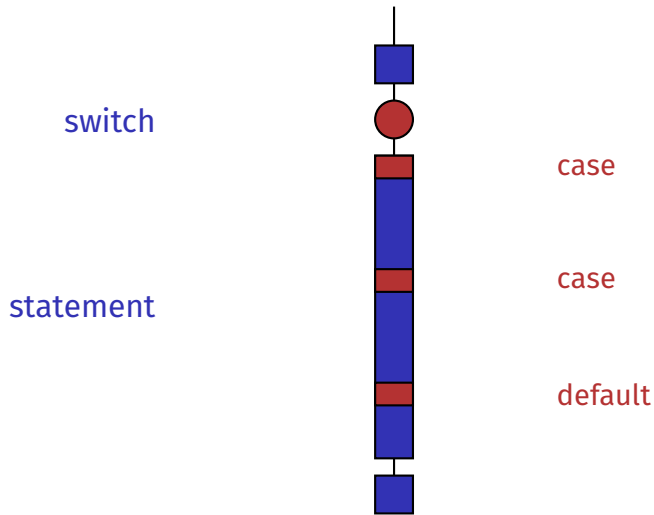
Kontrollfluss switch



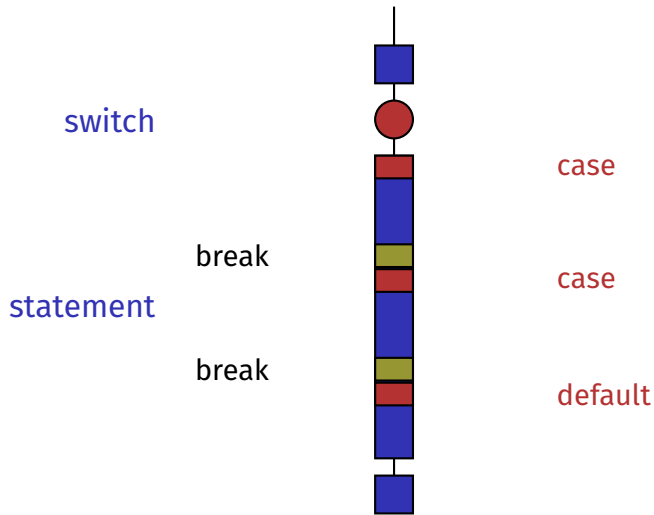
Kontrollfluss switch



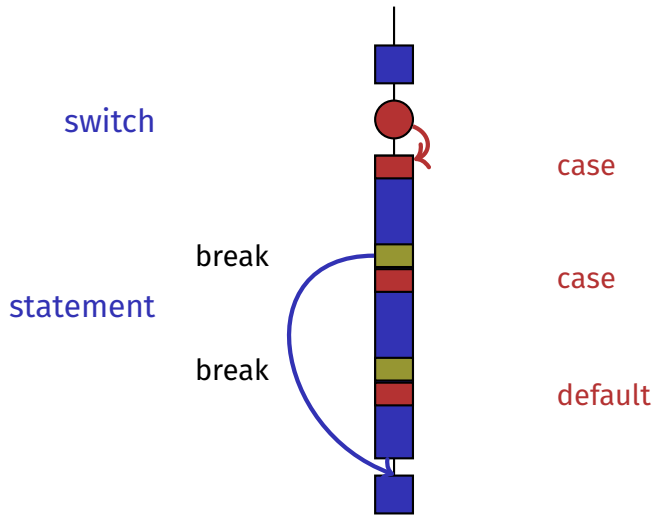
Kontrollfluss switch



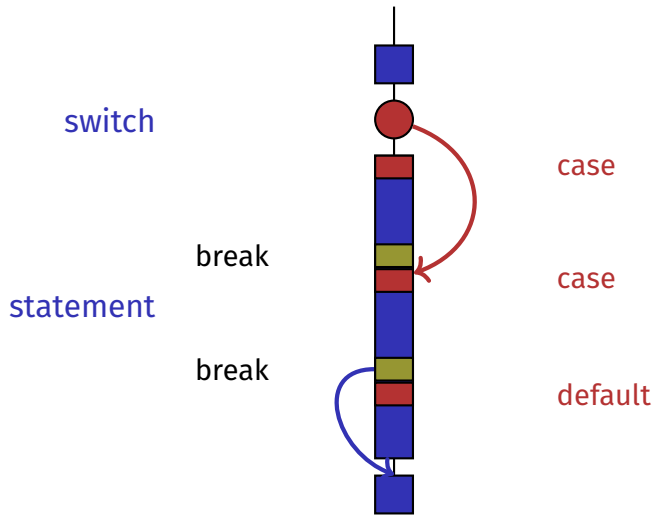
Kontrollfluss switch



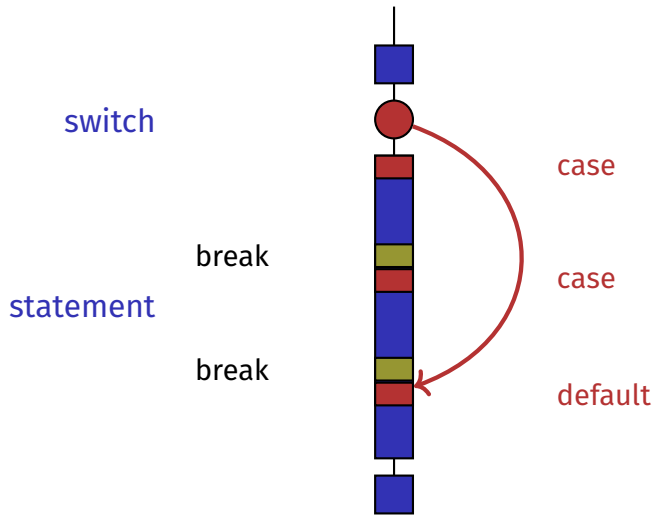
Kontrollfluss switch



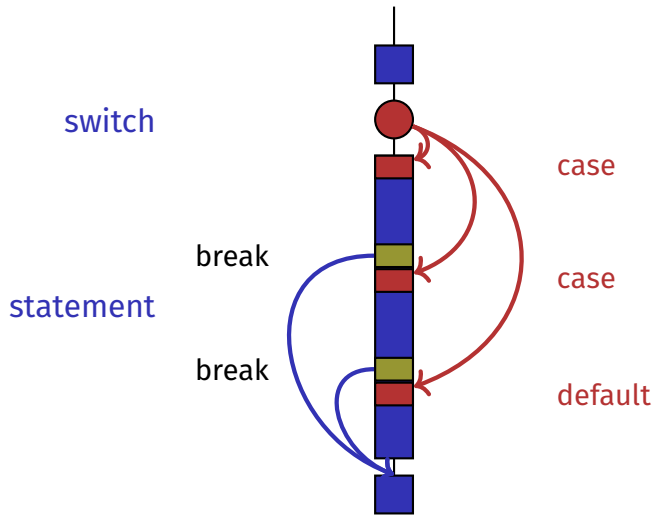
Kontrollfluss switch



Kontrollfluss switch



Kontrollfluss switch



7. Fließkommazahlen I

Typen **float** und **double**; Gemischte Ausdrücke und Konversionen; Lücken im Wertebereich

„Richtig“ Rechnen

```
// Input
std::cout << "Temperature in degrees Celsius =? ";
int celsius;
std::cin >> celsius;

// Computation and output
std::cout << celsius << " degrees Celsius are "
          << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\\n";
```

28 degrees Celsius are 82 degrees Fahrenheit.

„Richtig“ Rechnen

```
// Input
std::cout << "Temperature in degrees Celsius =? ";
int celsius;
std::cin >> celsius;

// Computation and output
std::cout << celsius << " degrees Celsius are "
          << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\\n";
```

28 degrees Celsius are 82 degrees Fahrenheit.



richtig wäre 82.4

„Richtig“ Rechnen

```
// Input
std::cout << "Temperature in degrees Celsius =? ";
float celsius; // Enable fractional numbers
std::cin >> celsius;

// Computation and output
std::cout << celsius << " degrees Celsius are "
          << 9 * celsius / 5 + 32 << " degrees Fahrenheit.\\n";
```

28 degrees Celsius are 82.4 degrees Fahrenheit.

Fixkommazahlen

- feste Anzahl Vorkommastellen (z.B. 7)
- feste Anzahl Nachkommastellen (z.B. 3)

Fixkommazahlen

- feste Anzahl Vorkommastellen (z.B. 7)
- feste Anzahl Nachkommastellen (z.B. 3)

82.4 = 0000082.400

Fixkommazahlen

- feste Anzahl Vorkommastellen (z.B. 7)
- feste Anzahl Nachkommastellen (z.B. 3)

82.4 = 0000082.400

Nachteile

- Wertebereich wird *noch* kleiner als bei ganzen Zahlen.

Fixkommazahlen

- feste Anzahl Vorkommastellen (z.B. 7)
- feste Anzahl Nachkommastellen (z.B. 3)

0.0824 = 0000000.082 ← dritte Stelle abgeschnitten

Nachteile

- Repräsentierbarkeit hängt von der Stelle des Kommas ab.

Fließkommazahlen

- Beobachtung: Unterschiedlich „effiziente“ Darstellungen einer Zahl, z.B.

$$\begin{aligned} 0.0824 &= 0.00824 \cdot 10^1 = 0.824 \cdot 10^{-1} \\ &= 8.24 \cdot 10^{-2} = 824 \cdot 10^{-4} \end{aligned}$$

Anzahl *signifikanter Stellen* bleibt konstant

Fließkommazahlen

- Beobachtung: Unterschiedlich „effiziente“ Darstellungen einer Zahl, z.B.

$$\begin{aligned}0.0824 &= 0.00824 \cdot 10^1 = 0.824 \cdot 10^{-1} \\ &= 8.24 \cdot 10^{-2} = 824 \cdot 10^{-4}\end{aligned}$$

Anzahl *signifikanter Stellen* bleibt konstant

- Fließkommarepräsentation daher:
 - Feste Anzahl signifikanter Stellen (z.B. 10),
 - Plus Position des Kommas mittels Exponenten
 - Zahl ist *Signifikand* $\times 10^{\text{Exponent}}$

Typen `float` und `double`

- sind die fundamentalen C++ Typen für Fließkommazahlen
- approximieren den Körper der reellen Zahlen $(\mathbb{R}, +, \times)$ in der Mathematik

Typen `float` und `double`

- sind die fundamentalen C++ Typen für Fließkommazahlen
- approximieren den Körper der reellen Zahlen $(\mathbb{R}, +, \times)$ in der Mathematik
- haben grossen Wertebereich, ausreichend für viele Anwendungen:
 - `float`: ca. 7 Stellen, Exponent bis ± 38
 - `double`: ca. 15 Stellen, Exponent bis ± 308

Typen `float` und `double`

- sind die fundamentalen C++ Typen für Fließkommazahlen
- approximieren den Körper der reellen Zahlen $(\mathbb{R}, +, \times)$ in der Mathematik
- haben grossen Wertebereich, ausreichend für viele Anwendungen:
 - `float`: ca. 7 Stellen, Exponent bis ± 38
 - `double`: ca. 15 Stellen, Exponent bis ± 308
- sind auf den meisten Rechnern sehr schnell (Hardwareunterstützung)

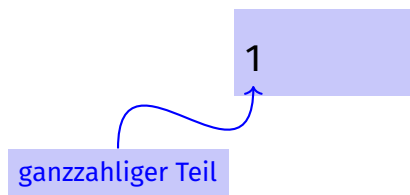
Arithmetische Operatoren

Wie bei `int`, aber ...

- Divisionsoperator `/` modelliert „echte“ (reelle, nicht ganzzahlige) Division
- Kein Modulo-Operator, d.h. kein `%`

Literale

unterscheiden sich von Ganzzahlliteralen



Literale

unterscheiden sich von Ganzzahlliteralen durch Angabe von

■ Dezimalkomma

1.0 : Typ `double`, Wert 1

1.23

ganzzahliger Teil

fraktionaler Teil

Literale

unterscheiden sich von Ganzzahlliteralen durch Angabe von

■ Dezimalkomma

`1.0` : Typ `double`, Wert 1

ganzzahliger Teil

1

e-7

Exponent

■ oder Exponent.

`1e3` : Typ `double`, Wert 1000

Literale

unterscheiden sich von Ganzzahlliteralen durch Angabe von

■ Dezimalkomma

`1.0` : Typ `double`, Wert 1

ganzzahliger Teil

`1.23e-7`

Exponent

fraktionaler Teil

■ und / oder Exponent.

`1e3` : Typ `double`, Wert 1000

`1.23e-7` : Typ `double`, Wert $1.23 \cdot 10^{-7}$

Literale

unterscheiden sich von Ganzzahlliteralen durch Angabe von

■ Dezimalkomma

`1.0` : Typ `double`, Wert 1

`1.27f` : Typ `float`, Wert 1.27

■ und / oder Exponent.

`1e3` : Typ `double`, Wert 1000

`1.23e-7` : Typ `double`, Wert $1.23 \cdot 10^{-7}$

`1.23e-7f` : Typ `float`, Wert $1.23 \cdot 10^{-7}$

1.23e-7f

ganzzahliger Teil

Exponent

fraktionaler Teil

Rechnen mit `float`: Beispiel

Approximation der Euler-Zahl

$$e = \sum_{i=0}^{\infty} \frac{1}{i!} \approx 2.71828 \dots$$

mittels der ersten 10 Terme.

Rechnen mit float: Eulersche Zahl

```
std::cout << "Approximating the Euler number... \n";

// values for i-th iteration, initialized for i = 0
float t = 1.0f; // term 1/i!
float e = 1.0f; // i-th approximation of e

// iteration 1, ..., n
for (unsigned int i = 1; i < 10; ++i) {
    t /= i;    // 1/(i-1)! -> 1/i!
    e += t;
    std::cout << "Value after term " << i << ": "
                << e << "\n";
}
```

Rechnen mit float: Eulersche Zahl

```
Value after term 1: 2
Value after term 2: 2.5
Value after term 3: 2.66667
Value after term 4: 2.70833
Value after term 5: 2.71667
Value after term 6: 2.71806
Value after term 7: 2.71825
Value after term 8: 2.71828
Value after term 9: 2.71828
```

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

```
9 * celsius / 5 + 32
```


Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

9 * celsius / 5 + 32

↑
Typ float, Wert 28

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

9 * 28.0f / 5 + 32

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

9 * 28.0f / 5 + 32

wird zu float konvertiert: 9.0f

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

252.0f / 5 + 32

wird zu float konvertiert: 5.0f

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

50.4f + 32

wird zu float konvertiert: 32.0f

Gemischte Ausdrücke, Konversion

- Fließkommazahlen sind allgemeiner als ganzzahlige Typen.
- In gemischten Ausdrücken werden ganze Zahlen zu Fließkommazahlen konvertiert.

82.4f

Löcher im Wertebereich

```
float n1;  
std::cout << "First number  =? ";  
std::cin >> n1;
```

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Löcher im Wertebereich

```
float n1;  
std::cout << "First number =? ";  
std::cin >> n1;
```

Eingabe 1.5

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.5

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```


Löcher im Wertebereich

```
float n1;  
std::cout << "First number =? ";  
std::cin >> n1;
```

Eingabe 1.5

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.5

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Ausgabe 0

Löcher im Wertebereich

```
float n1;  
std::cout << "First number  =? ";  
std::cin >> n1;
```

Eingabe 1.1

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.1

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Löcher im Wertebereich

```
float n1;  
std::cout << "First number =? ";  
std::cin >> n1;
```

Eingabe 1.1

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.1

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Ausgabe 2.23517e-8

Löcher im Wertebereich

```
float n1;  
std::cout << "First number =? ";  
std::cin >> n1;
```

Eingabe 1.1

```
float n2;  
std::cout << "Second number =? ";  
std::cin >> n2;
```

Eingabe 1.0

```
float d;  
std::cout << "Their difference =? ";  
std::cin >> d;
```

Eingabe 0.1

```
std::cout << "Computed difference - input difference = "  
          << n1 - n2 - d << "\n";
```

Ausgabe 2.23517e-8

Was ist denn hier los?

Ganzzahlige Typen:

- Über- und Unterlauf häufig, aber ...
- Wertebereich ist zusammenhängend (keine Löcher): $\mathbb{Z}$ ist „diskret“.

Wertebereich

Ganzzahlige Typen:

- Über- und Unterlauf häufig, aber ...
- Wertebereich ist zusammenhängend (keine Löcher): $\mathbb{Z}$ ist „diskret“.

Fliesskommatypen:

- Über- und Unterlauf selten, aber ...
- es gibt Löcher: $\mathbb{R}$ ist „kontinuierlich“.

8. Fließkommazahlen II

Fließkommazahlensysteme, IEEE Standard, Grenzen der Fließkommaarithmetik, Fließkomma-Richtlinien, Harmonische Zahlen

Fließkommazahlensysteme

Ein Fließkommazahlensystem ist durch vier natürliche Zahlen definiert:

- $\beta \geq 2$, die Basis,
- $p \geq 1$, die Präzision (Stellenzahl),
- $e_{\min}$, der kleinste Exponent,
- $e_{\max}$, der grösste Exponent.

Fließkommazahlensysteme

Ein Fließkommazahlensystem ist durch vier natürliche Zahlen definiert:

- $\beta \geq 2$, die Basis,
- $p \geq 1$, die Präzision (Stellenzahl),
- $e_{\min}$, der kleinste Exponent,
- $e_{\max}$, der grösste Exponent.

Bezeichnung:

$$F(\beta, p, e_{\min}, e_{\max})$$

Fließkommazahlensysteme

$F(\beta, p, e_{\min}, e_{\max})$ enthält die Zahlen

$$\pm \sum_{i=0}^{p-1} d_i \beta^{-i} \cdot \beta^e,$$

$$d_i \in \{0, \dots, \beta - 1\}, \quad e \in \{e_{\min}, \dots, e_{\max}\}.$$

Fließkommazahlensysteme

$F(\beta, p, e_{\min}, e_{\max})$ enthält die Zahlen

$$\pm \sum_{i=0}^{p-1} d_i \beta^{-i} \cdot \beta^e,$$

$$d_i \in \{0, \dots, \beta - 1\}, \quad e \in \{e_{\min}, \dots, e_{\max}\}.$$

In Basis- β -Darstellung:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e$$

Darstellungen der Dezimalzahl 0.1 (mit $\beta = 10$):

$$1.0 \cdot 10^{-1}, \quad 0.1 \cdot 10^0, \quad 0.01 \cdot 10^1, \quad \dots$$

Unterschiedliche Darstellungsmöglichkeiten durch Wahl des Exponenten

Normalisierte Darstellung

Normalisierte Zahl:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e, \quad d_0 \neq 0$$

Normalisierte Darstellung

Normalisierte Zahl:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e, \quad d_0 \neq 0$$

Bemerkung 1

Die normalisierte Darstellung ist eindeutig und deshalb zu bevorzugen.

Normalisierte Darstellung

Normalisierte Zahl:

$$\pm d_0 \bullet d_1 \dots d_{p-1} \times \beta^e, \quad d_0 \neq 0$$

Bemerkung 2

Die Zahl 0, sowie alle Zahlen kleiner als $\beta^{e_{\min}}$, haben keine normalisierte Darstellung (greifen wir später wieder auf).

Menge der normalisierten Zahlen

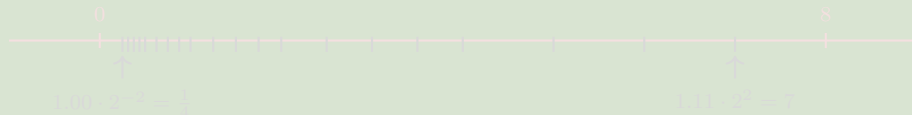
$$F^*(\beta, p, e_{\min}, e_{\max})$$

Normalisierte Darstellung

Beispiel $F^*(2, 3, -2, 2)$

(nur positive Zahlen)

$d_0 \bullet d_1 d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.00_2	0.25	0.5	1	2	4
1.01_2	0.3125	0.625	1.25	2.5	5
1.10_2	0.375	0.75	1.5	3	6
1.11_2	0.4375	0.875	1.75	3.5	7

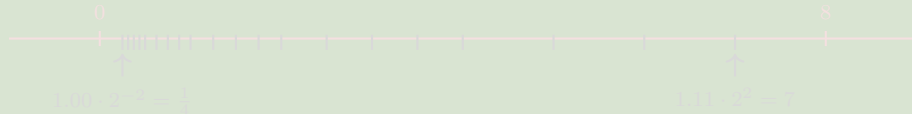


Normalisierte Darstellung

Beispiel $F^*(2, \textcolor{red}{3}, -2, 2)$

(nur positive Zahlen)

$d_0 \bullet d_1 d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
$\textcolor{red}{1.00}_2$	0.25	0.5	1	2	4
$\textcolor{red}{1.01}_2$	0.3125	0.625	1.25	2.5	5
$\textcolor{red}{1.10}_2$	0.375	0.75	1.5	3	6
$\textcolor{red}{1.11}_2$	0.4375	0.875	1.75	3.5	7

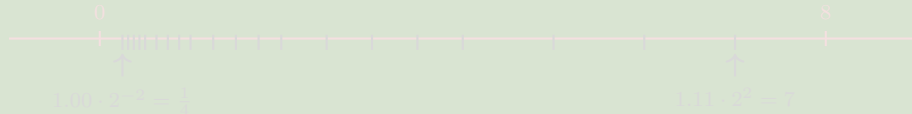


Normalisierte Darstellung

Beispiel $F^*(2, 3, -2, 2)$

(nur positive Zahlen)

$d_0 \bullet d_1 d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.00_2	0.25	0.5	1	2	4
1.01_2	0.3125	0.625	1.25	2.5	5
1.10_2	0.375	0.75	1.5	3	6
1.11_2	0.4375	0.875	1.75	3.5	7

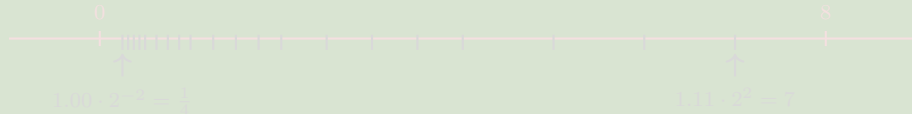


Normalisierte Darstellung

Beispiel $F^*(2, 3, -2, 2)$

(nur positive Zahlen)

$d_0.d_1d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.00_2	0.25	0.5	1	2	4
1.01_2	0.3125	0.625	1.25	2.5	5
1.10_2	0.375	0.75	1.5	3	6
1.11_2	0.4375	0.875	1.75	3.5	7

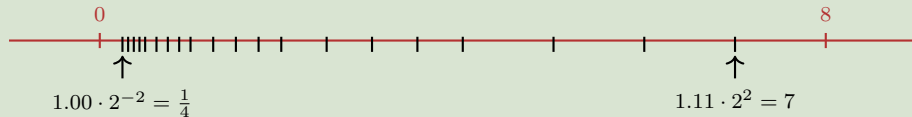


Normalisierte Darstellung

Beispiel $F^*(2, 3, -2, 2)$

(nur positive Zahlen)

$d_0.d_1d_2$	$e = -2$	$e = -1$	$e = 0$	$e = 1$	$e = 2$
1.00_2	0.25	0.5	1	2	4
1.01_2	0.3125	0.625	1.25	2.5	5
1.10_2	0.375	0.75	1.5	3	6
1.11_2	0.4375	0.875	1.75	3.5	7



Binäre und dezimale Systeme

- Intern rechnet der Computer mit $\beta = 2$
(binäres System)

Binäre und dezimale Systeme

- Intern rechnet der Computer mit $\beta = 2$
(binäres System)
- Literale und Eingaben haben $\beta = 10$
(dezimales System)

Berechnung der *Binärdarstellung*:

$$x = \sum_{i=0}^{\infty} b_i 2^{-i}$$

Berechnung der *Binärdarstellung*:

$$\begin{aligned} x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\ &= b_0 \bullet b_1 b_2 b_3 \dots \end{aligned}$$

Berechnung der *Binärdarstellung*:

$$\begin{aligned}x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\&= b_0 \bullet b_1 b_2 b_3 \dots \\&= b_0 + 0 \bullet b_1 b_2 b_3 \dots\end{aligned}$$

Berechnung der *Binärdarstellung*:

$$\begin{aligned}x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\&= b_0 \bullet b_1 b_2 b_3 \dots \\&= b_0 + 0 \bullet b_1 b_2 b_3 \dots \\&\implies\end{aligned}$$

Berechnung der *Binärdarstellung*:

$$\begin{aligned}x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\&= b_0 \bullet b_1 b_2 b_3 \dots \\&= b_0 + 0 \bullet b_1 b_2 b_3 \dots \\&\implies\end{aligned}$$

$$(x - b_0) = 0 \bullet b_1 b_2 b_3 b_4 \dots$$

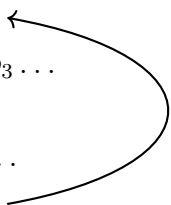
Berechnung der *Binärdarstellung*:

$$\begin{aligned}x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\&= b_0 \bullet b_1 b_2 b_3 \dots \\&= b_0 + 0 \bullet b_1 b_2 b_3 \dots \\&\implies\end{aligned}$$

$$(x - b_0) = 0 \bullet b_1 b_2 b_3 b_4 \dots$$

$$2 \cdot (x - b_0) = b_1 \bullet b_2 b_3 b_4 \dots$$

Berechnung der *Binärdarstellung*:

$$\begin{aligned}x &= \sum_{i=0}^{\infty} b_i 2^{-i} \\&= b_0 \bullet b_1 b_2 b_3 \dots \leftarrow \\&= b_0 + 0 \bullet b_1 b_2 b_3 \dots \\&\Rightarrow \\(x - b_0) &= 0 \bullet b_1 b_2 b_3 b_4 \dots \\2 \cdot (x - b_0) &= b_1 \bullet b_2 b_3 b_4 \dots\end{aligned}$$


Berechnung der *Binärdarstellung*:

$$x = \sum_{i=0}^{\infty} b_i 2^{-i}$$

$$= b_0 \bullet b_1 b_2 b_3 \dots \leftarrow$$

$$= b_0 + 0 \bullet b_1 b_2 b_3 \dots$$

$$\implies$$

$$(x - b_0) = 0 \bullet b_1 b_2 b_3 b_4 \dots$$

$$2 \cdot (x - b_0) = b_1 \bullet b_2 b_3 b_4 \dots$$

```
for (int b_0; x != 0; x = 2 * (x - b_0)) {  
    if (x >= 1) b_0 = 1;  
    else b_0 = 0;  
    std::cout << b_0;  
}
```

Beispiel (binär)

$$\begin{aligned}x &= \textcolor{red}{1}\bullet 01011 \\ &= \textcolor{red}{1} + 0\bullet 01011\end{aligned}$$

$\implies$

$$2 \cdot (x - \textcolor{red}{1}) = 0\bullet 1011$$

Beispiel (binär)

$$\begin{aligned}x &= 1.\textcolor{blue}{01011} \\ &= 1 + 0.\textcolor{blue}{01011}\end{aligned}$$

$\implies$

$$2 \cdot (x - 1) = \textcolor{blue}{0.1011}$$

Beispiel (binär)

$$\begin{aligned}x &= 0.1011 \\ &= 0 + 0.1011\end{aligned}$$

$\implies$

$$2 \cdot (x - 0) = 1.011$$

Beispiel (binär)

$$x = 0.\textcolor{blue}{1011}$$

$$= 0 + 0.\textcolor{blue}{1011}$$

$$\implies$$

$$2 \cdot (x - 0) = \textcolor{blue}{1.011}$$

Beispiel (binär)

$$\begin{aligned}x &= \textcolor{red}{1}.\textbullet 011 \\&= \textcolor{red}{1} + 0.\textbullet 011 \\&\implies\end{aligned}$$

$$2 \cdot (x - \textcolor{red}{1}) = 0.\textbullet 11$$

Beispiel (binär)

$$x = 1.\textcolor{blue}{011}$$

$$= 1 + 0.\textcolor{blue}{011}$$

$$\implies$$

$$2 \cdot (x - 1) = \textcolor{blue}{0.11}$$

Beispiel (binär)

$$\begin{aligned}x &= 0_{\bullet}11 \\ &= 0 + 0_{\bullet}11\end{aligned}$$

$\implies$

$$2 \cdot (x - 0) = 1_{\bullet}1$$

Beispiel (binär)

$$\begin{aligned}x &= 0.\textcolor{blue}{11} \\ &= 0 + 0.\textcolor{blue}{11}\end{aligned}$$

$\implies$

$$2 \cdot (x - 0) = \textcolor{blue}{1}.\textcolor{blue}{1}$$

Beispiel (binär)

$$x = \textcolor{red}{1}.\textcolor{black}{1}$$

$$= \textcolor{red}{1} + 0.\textcolor{black}{1}$$

$$\implies$$

$$2 \cdot (x - \textcolor{red}{1}) = 1$$

Beispiel (binär)

$$x = 1.\textcolor{blue}{1}$$

$$= 1 + 0.\textcolor{blue}{1}$$

$$\implies$$

$$2 \cdot (x - 1) = \textcolor{blue}{1}$$

Beispiel (binär)

$$x = 1$$

$$= 1 + 0$$

$$\implies$$

$$2 \cdot (x - 1) = 0$$

Beispiel (binär)

$$x = 1$$

$$= 1 + 0$$

$$\implies$$

$$2 \cdot (x - 1) = 0$$

Binärdarstellung von 1.1_{10}

$$\begin{array}{cccc} x & b_i & x - b_i & 2(x - b_i) \\ \hline 1.1 & b_0 = 1 & & \end{array}$$

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$		

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$		

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$		

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$		

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2

Binärdarstellung von 1.1_{10}

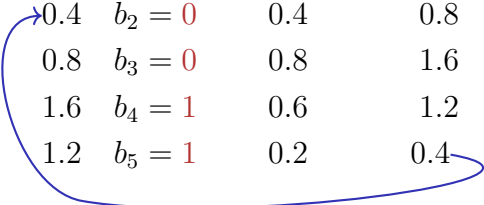
x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2
1.2	$b_5 = 1$		

Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2
1.2	$b_5 = 1$	0.2	0.4

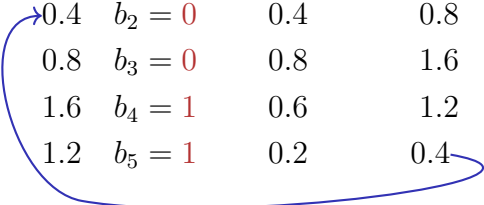
Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2
1.2	$b_5 = 1$	0.2	0.4



Binärdarstellung von 1.1_{10}

x	b_i	$x - b_i$	$2(x - b_i)$
1.1	$b_0 = 1$	0.1	0.2
0.2	$b_1 = 0$	0.2	0.4
0.4	$b_2 = 0$	0.4	0.8
0.8	$b_3 = 0$	0.8	1.6
1.6	$b_4 = 1$	0.6	1.2
1.2	$b_5 = 1$	0.2	0.4



$\Rightarrow 1.\overline{00011}$, periodisch, *nicht* endlich

Binärdarstellungen von 1.1 und 0.1

- sind nicht endlich $\Rightarrow$ Fehler bei der Konversion

Binärdarstellungen von 1.1 und 0.1

- sind nicht endlich $\Rightarrow$ Fehler bei der Konversion
- 1.1f und 0.1f: *Approximationen* von 1.1 und 0.1

Binärdarstellungen von 1.1 und 0.1

- sind nicht endlich $\Rightarrow$ Fehler bei der Konversion
- `1.1f` und `0.1f`: *Approximationen* von 1.1 und 0.1
- In `diff.cpp`: $1.1 - 1.0 \neq 0.1$

Binärdarstellungen von 1.1 und 0.1

auf meinem Computer:

$$\begin{aligned} 1.1 &= \underline{1.100000000000000000}888178\dots \\ 1.1f &= \underline{1.1000000}238418\dots \end{aligned}$$

Rechnen mit Fließkommazahlen

ist fast so einfach wie mit ganzen Zahlen.

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + \quad 1.011 \cdot 2^{-1} \end{array}$$

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + \quad 1.011 \cdot 2^{-1} \end{array}$$

1. Exponenten anpassen durch Denormalisieren einer Zahl

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + \quad 10.110 \cdot 2^{-2} \end{array} \checkmark$$

1. Exponenten anpassen durch Denormalisieren einer Zahl

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline \end{array}$$

2. Binäre Addition der Signifikanden

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline = 100.101 \cdot 2^{-2} \checkmark \end{array}$$

2. Binäre Addition der Signifikanden

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline = 100.101 \cdot 2^{-2} \end{array}$$

3. Renormalisierung

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline = 1.00101 \cdot 2^0 \checkmark \end{array}$$

3. Renormalisierung

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline = 1.00101 \cdot 2^0 \end{array}$$

4. Runden auf p signifikante Stellen, falls nötig

Rechnen mit Fließkommazahlen

Beispiel ($\beta = 2, p = 4$):

$$\begin{array}{r} 1.111 \cdot 2^{-2} \\ + 10.110 \cdot 2^{-2} \\ \hline = 1.001 \cdot 2^0 \checkmark \end{array}$$

4. Runden auf p signifikante Stellen, falls nötig

Der IEEE Standard 754

legt Fließkommazahlensysteme und deren Rundungsverhalten fest und wird fast überall benutzt

■ Single precision (float) Zahlen:

$$F^*(2, 24, -126, 127) \text{ (32 bit)}$$

Der IEEE Standard 754

legt Fließkommazahlensysteme und deren Rundungsverhalten fest und wird fast überall benutzt

- Single precision (`float`) Zahlen:

$$F^*(2, 24, -126, 127) \text{ (32 bit)}$$

- Double precision (`double`) Zahlen:

$$F^*(2, 53, -1022, 1023) \text{ (64 bit)}$$

Der IEEE Standard 754

legt Fließkommazahlensysteme und deren Rundungsverhalten fest und wird fast überall benutzt

- Single precision (`float`) Zahlen:

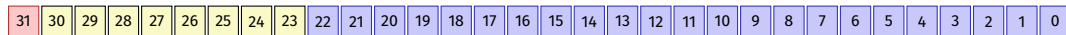
$F^*(2, 24, -126, 127)$ (32 bit) plus 0, ∞ , ...

- Double precision (`double`) Zahlen:

$F^*(2, 53, -1022, 1023)$ (64 bit) plus 0, ∞ , ...

- Alle arithmetischen Operationen runden das *exakte* Ergebnis auf die nächste darstellbare Zahl

Beispiel: 32-bit Darstellung einer Fließkommazahl



± Exponent

Mantisse

± $2^{-126}, \dots, 2^{127}$
 $0, \infty, \dots$

1.000000000000000000000000
...
1.111111111111111111111111

Regel 1

Teste keine gerundeten Fliesskommazahlen auf Gleichheit!

Regel 1

Teste keine gerundeten Fließkommazahlen auf Gleichheit!

```
for (float i = 0.1; i != 1.0; i += 0.1)
    std::cout << i << "\n";
```

Regel 1

Teste keine gerundeten Fließkommazahlen auf Gleichheit!

```
for (float i = 0.1; i != 1.0; i += 0.1)
    std::cout << i << "\n";
```

Endlosschleife, weil i niemals exakt 1 ist!

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

$$\begin{array}{r} 1.000 \cdot 2^5 \\ + 1.000 \cdot 2^0 \end{array}$$

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

$$\begin{aligned} &1.000 \cdot 2^5 \\ &+ 1.000 \cdot 2^0 \\ &= 1.00001 \cdot 2^5 \end{aligned}$$

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

$$\begin{aligned} & 1.000 \cdot 2^5 \\ & + 1.000 \cdot 2^0 \\ & = 1.00001 \cdot 2^5 \\ & \text{„}=\text{“ } 1.000 \cdot 2^5 \text{ (Rundung auf 4 Stellen)} \end{aligned}$$

Regel 2

Addiere keine zwei Zahlen sehr unterschiedlicher Grösse!

$$\begin{aligned} & 1.000 \cdot 2^5 \\ & + 1.000 \cdot 2^0 \\ & = 1.00001 \cdot 2^5 \\ & \text{„}=\text{“ } 1.000 \cdot 2^5 \text{ (Rundung auf 4 Stellen)} \end{aligned}$$

Addition von 1 hat keinen Effekt!

- Die n -te Harmonische Zahl ist

$$H_n = \sum_{i=1}^n \frac{1}{i}$$

- Die n -te Harmonische Zahl ist

$$H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n.$$

- Die n -te Harmonische Zahl ist

$$H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n.$$

- Diese Summe kann vorwärts oder rückwärts berechnet werden, was mathematisch gesehen natürlich äquivalent ist.

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.
- Die Rückwärtssumme approximiert H_n gut.

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.
- Die Rückwärtssumme approximiert H_n gut.

Erklärung:

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.
- Die Rückwärtssumme approximiert H_n gut.

Erklärung:

- Bei $1 + 1/2 + 1/3 + \dots$ sind späte Terme zu klein, um noch beizutragen.

Beobachtung:

- Die Vorwärtssumme wächst irgendwann nicht mehr und ist „richtig“ falsch.
- Die Rückwärtssumme approximiert H_n gut.

Erklärung:

- Bei $1 + 1/2 + 1/3 + \dots$ sind späte Terme zu klein, um noch beizutragen.
- Problematik wie bei $2^5 + 1$ „=“ 2^5

```
std::cout << "Compute H_n for n =? ";
unsigned int n;
std::cin >> n;

float fs = 0;
for (unsigned int i = 1; i <= n; ++i)
    fs += 1.0f / i;
std::cout << "Forward sum = " << fs << "\n";

float bs = 0;
for (unsigned int i = n; i >= 1; --i)
    bs += 1.0f / i;
std::cout << "Backward sum = " << bs << "\n";
```

```
std::cout << "Compute H_n for n =? ";  
unsigned int n;  
std::cin >> n;
```

Eingabe: 1000000

```
float fs = 0;  
for (unsigned int i = 1; i <= n; ++i)  
    fs += 1.0f / i;  
std::cout << "Forward sum = " << fs << "\n";
```

Vorwärts: 15.4037

```
float bs = 0;  
for (unsigned int i = n; i >= 1; --i)  
    bs += 1.0f / i;  
std::cout << "Backward sum = " << bs << "\n";
```

Rückwärts: 16.686

```
std::cout << "Compute H_n for n =? ";  
unsigned int n;  
std::cin >> n;
```

Eingabe: 10000000

```
float fs = 0;  
for (unsigned int i = 1; i <= n; ++i)  
    fs += 1.0f / i;  
std::cout << "Forward sum = " << fs << "\n";
```

Vorwärts: 15.4037

```
float bs = 0;  
for (unsigned int i = n; i >= 1; --i)  
    bs += 1.0f / i;  
std::cout << "Backward sum = " << bs << "\n";
```

Rückwärts: 18.8079

Regel 3

Subtrahiere keine zwei Zahlen sehr ähnlicher Grösse!

Auslöschungsproblematik, nicht vertieft.

David Goldberg: What Every Computer Scientist Should Know About Floating-Point Arithmetic (1991)



Randy Glasbergen, 1996

9. Funktionen I

Funktionsdefinitionen- und Aufrufe, Auswertung von Funktionsaufrufen,
Der Typ **void**

Potenzberechnung

```
double a;
int n;
std::cin >> a; // Eingabe a
std::cin >> n; // Eingabe n

double result = 1.0;
if (n < 0) { //  $a^n = (1/a)^{-n}$ 
    a = 1.0/a;
    n = -n;
}
for (int i = 0; i < n; ++i)
    result *= a;

std::cout << a << "^" << n << " = " << result << ".\n";
```

Potenzberechnung

```
double a;  
int n;  
std::cin >> a; // Eingabe a  
std::cin >> n; // Eingabe n
```

```
double result = 1.0;  
if (n < 0) { //  $a^n = (1/a)^{-n}$   
    a = 1.0/a;  
    n = -n;  
}  
for (int i = 0; i < n; ++i)  
    result *= a;
```

```
std::cout << a << "^" << n << " = " << result << ".\n";
```

Potenzberechnung

```
double a;  
int n;  
std::cin >> a; // Eingabe a  
std::cin >> n; // Eingabe n
```

```
double result = 1.0;  
if (n < 0) { //  $a^n = (1/a)^{-n}$   
    a = 1.0/a;  
    n = -n;  
}  
for (int i = 0; i < n; ++i)  
    result *= a;
```



"Funktion pow"

```
std::cout << a << "^" << n << " = " << pow(a,n) << ".\n";
```

Funktion zur Potenzberechnung

```
// assumes e >= 0 || b != 0.0
// returns the value b^e
double pow(double b, int e)
{
    double result = 1.0;
    if (e < 0) { // b^e = (1/b)^(-e)
        b = 1.0/b;
        e = -e;
    }
    for (int i = 0; i < e; ++i)
        result *= b;
    return result;
}
```

Funktion zur Potenzberechnung

```
double pow(double b, int e){...}
```

Funktion zur Potenzberechnung

```
// Prog: callpow.cpp
// Define and call a function for computing powers.
#include <iostream>
```

```
double pow(double b, int e){...}
```

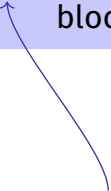
```
int main()
{
    std::cout << pow( 2.0, -2) << "\n"; // outputs 0.25
    std::cout << pow( 1.5, 2) << "\n"; // outputs 2.25
    std::cout << pow(-2.0, 9) << "\n"; // outputs -512

    return 0;
}
```

Funktionsdefinitionen

T fname (T_1 pname₁, T_2 pname₂, ..., T_N pname_N)
block

Funktionsname



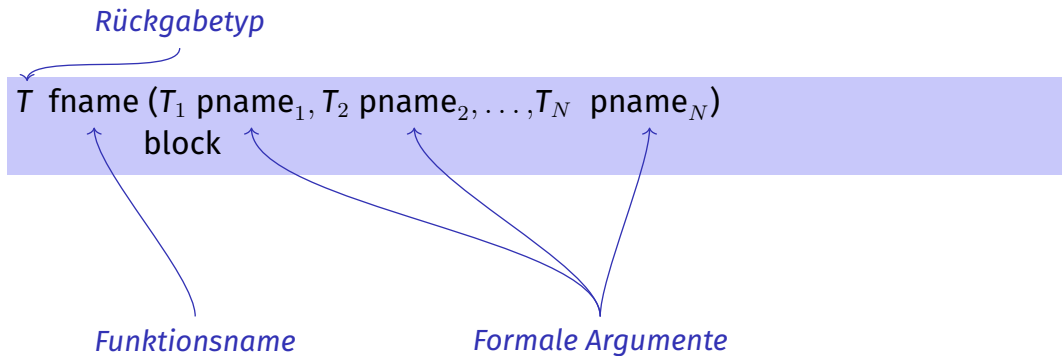
Funktionsdefinitionen

Rückgabebetyp

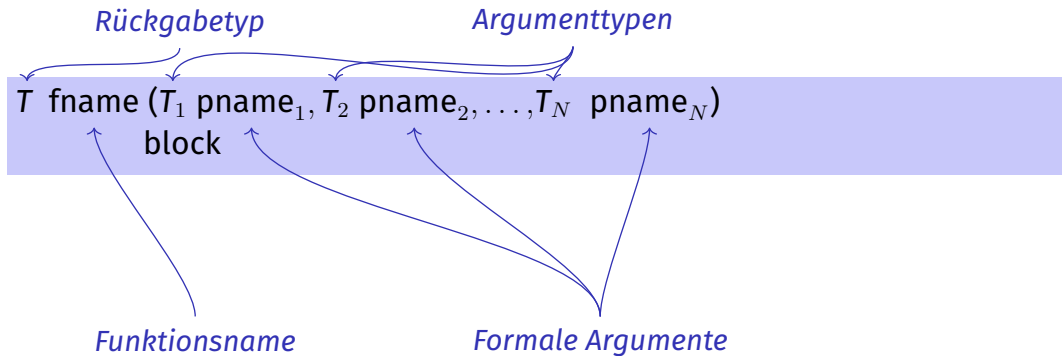
T fname (T_1 pname₁, T_2 pname₂, ..., T_N pname_N)
block

Funktionsname

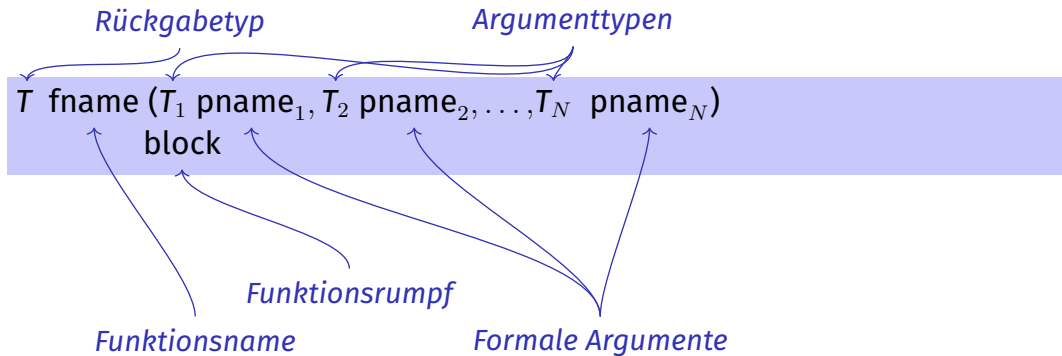
Funktionsdefinitionen



Funktionsdefinitionen



Funktionsdefinitionen



Xor

```
// returns l XOR r
bool Xor(bool l, bool r)
{
    return l && !r || !l && r;
}
```

Harmonic

```
// assumes n >= 0
// returns nth harmonic number
// computed with backward sum
float Harmonic(int n)
{
    float res = 0;
    for (unsigned int i = n; i >= 1; --i)
        res += 1.0f / i;
    return res;
}
```

min

```
// returns the minimum of a and b
int min(int a, int b)
{
    if (a<b)
        return a;
    else
        return b;
}
```

Funktionsaufrufe

`fname ( expression1, expression2, ..., expressionN )`

- Alle Aufrufargumente müssen konvertierbar sein in die entsprechenden Argumenttypen.

Funktionsaufrufe

`fname ( expression1, expression2, ..., expressionN )`

- Alle Aufrufargumente müssen konvertierbar sein in die entsprechenden Argumenttypen.
- Der Funktionsaufruf selbst ist ein Ausdruck vom Rückgabetyt.

Funktionsaufrufe

$fname (expression_1, expression_2, \dots, expression_N)$

- Alle Aufrufargumente müssen konvertierbar sein in die entsprechenden Argumenttypen.
- Der Funktionsaufruf selbst ist ein Ausdruck vom Rückgabotyp.

Beispiel: `pow(a,n)`: Ausdruck vom Typ `double`

Funktionsaufrufe

Für die Typen, die wir bisher kennen, gilt:

- Aufrufargumente sind R-Werte
 $\hookrightarrow$ *call-by-value* (auch *pass-by-value*), dazu gleich mehr
- Funktionsaufruf selbst ist R-Wert.

Funktionsaufrufe

Für die Typen, die wir bisher kennen, gilt:

- Aufrufargumente sind R-Werte
 $\hookrightarrow$ *call-by-value* (auch *pass-by-value*), dazu gleich mehr
- Funktionsaufruf selbst ist R-Wert.

$fname: \text{R-Wert} \times \text{R-Wert} \times \dots \times \text{R-Wert} \longrightarrow \text{R-Wert}$

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

```
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

Aufruf von pow

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```



```
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

b=2.0,e=-2

result=1.0

```
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```



e == -2

```
...  
pow (2.0, -2)
```


Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```



b=0.5

```
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  e=2  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i) → i=0  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

...

pow (2.0, -2)

i=0

result=0.5

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i) → i=1  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

...

pow (2.0, -2)

i=1

result=0.25

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i) → i=2  
        result * = b;  
    return result;  
}  
  
...  
pow (2.0, -2)
```

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

→ result=0.25

```
...  
pow (2.0, -2)
```


Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

...

pow (2.0, -2)

result=0.25

Rückgabe

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

...

pow (2.0, -2)

Rückgabe

Wert: 0.25

Auswertung Funktionsaufruf

```
double pow(double b, int e){  
    double result = 1.0;  
    if (e<0) {  
        //  $b^e = (1/b)^{-e}$   
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        result * = b;  
    return result;  
}
```

...

pow (2.0, -2)

→ Wert: 0.25

Gültigkeit formaler Argumente

```
int main(){  
    double b = 2.0;  
    int e = -2;  
    double z = pow(b, e);  
  
    std::cout << z; // 0.25  
    std::cout << b; // 2  
    std::cout << e; // -2  
    return 0;  
}
```

Gültigkeit formaler Argumente

```
double pow(double b, int e){
    double r = 1.0;
    if (e<0) {
        b = 1.0/b;
        e = -e;
    }
    for (int i = 0; i < e ; ++i)
        r * = b;
    return r;
}
```

```
int main(){
    double b = 2.0;
    int e = -2;
    double z = pow(b, e);

    std::cout << z; // 0.25
    std::cout << b; // 2
    std::cout << e; // -2
    return 0;
}
```

Gültigkeit formaler Argumente

```
double pow(double b, int e){  
    double r = 1.0;  
    if (e<0) {  
        b = 1.0/b;  
        e = -e;  
    }  
    for (int i = 0; i < e ; ++i)  
        r * = b;  
    return r;  
}
```

```
int main(){  
    double b = 2.0;  
    int e = -2;  
    double z = pow(b, e);  
  
    std::cout << z; // 0.25  
    std::cout << b; // 2  
    std::cout << e; // -2  
    return 0;  
}
```

Nicht die formalen Argumente `b` und `e` von `pow`, sondern die hier definierten Variablen lokal zum Rumpf von `main`

Der Typ void

```
// writes "(i, j)" to standard output
???? print_pair(int i, int j) {
    std::cout << "(" << i << ", " << j << ")\n";
}

int main() {
    print_pair(3,4); // outputs (3, 4)
    return 0;
}
```

Der Typ void

```
// writes "(i, j)" to standard output
void print_pair(int i, int j) {
    std::cout << "(" << i << ", " << j << ")\n";
}

int main() {
    print_pair(3,4); // outputs (3, 4)
    return 0;
}
```


Der Typ `void`

- Fundamentaler Typ mit leerem Wertebereich

Der Typ `void`

- Fundamentaler Typ mit leerem Wertebereich
- Verwendung als Rückgabebetyp für Funktionen, die *nur* einen Effekt haben

void-Funktionen

- benötigen kein `return`.
- Ausführung endet, wenn
 - Ende des Funktionsrumpfes erreicht wird oder
 - `return;` erreicht wird

Funktionen und return

Falsch:

```
bool compare(float x, float y) {  
    float delta = x - y;  
    if (delta*delta < 0.001f) return true;  
}
```

Funktionen und return

Das Verhalten einer Funktion mit Rückgabetyp ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Falsch:

```
bool compare(float x, float y) {  
    float delta = x - y;  
    if (delta*delta < 0.001f) return true;  
}
```

Der Wert von `compare(10,20)` ist hier undefiniert

Funktionen und return

Das Verhalten einer Funktion mit Rückgabetyt ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Besser:

```
bool compare(float x, float y) {  
    float delta = x - y;  
    if (delta*delta < 0.001f)  
        return true;  
    else  
        return false;  
}
```

Alle Ausführungspfade erreichen ein `return`

Funktionen und return

Das Verhalten einer Funktion mit Rückgabotyp ungleich `void` ist **undefiniert** wenn das Ende des Funktionsrumpfes ohne `return` Anweisung erreicht wird.

Noch besser (weil einfacher)

```
bool compare(float x, float y) {  
    float delta = x - y;  
    return delta*delta < 0.001f;  
}
```

10. Funktionen II

Vor- und Nachbedingungen, Stepwise Refinement, Gültigkeitsbereich, Bibliotheken, Standardfunktionen

Vorbedingungen

Vorbedingung (precondition):

- Was muss bei Funktionsaufruf gelten?
- Spezifiziert *Definitionsereich* der Funktion.

Vorbedingungen

Vorbedingung (precondition):

- Was muss bei Funktionsaufruf gelten?
- Spezifiziert *Definitionsereich* der Funktion.

⇒ Wert 0^e ist für $e < 0$ undefiniert

Vorbedingungen

Vorbedingung (precondition):

- Was muss bei Funktionsaufruf gelten?
- Spezifiziert *Definitionsereich* der Funktion.

⇒ Wert 0^e ist für $e < 0$ undefiniert

```
// PRE: e >= 0 || b != 0.0
double pow(double b, int e){
    ...
}
```

Nachbedingungen

Nachbedingung (postcondition):

- Was gilt nach Funktionsaufruf?
- Spezifiziert *Wert* und *Effekt* des Funktionsaufrufes.

Nachbedingungen

Nachbedingung (postcondition):

- Was gilt nach Funktionsaufruf?
- Spezifiziert *Wert* und *Effekt* des Funktionsaufrufes.

```
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e){
    ...
}
```

Hier nur Wert, kein Effekt.

Vor- und Nachbedingungen

- sollten korrekt sein:

Vor- und Nachbedingungen

- sollten korrekt sein:
- *Wenn* die Vorbedingung beim Funktionsaufruf gilt, *dann* gilt auch die Nachbedingung nach dem Funktionsaufruf.

Vor- und Nachbedingungen

- sollten korrekt sein:
- *Wenn* die Vorbedingung beim Funktionsaufruf gilt, *dann* gilt auch die Nachbedingung nach dem Funktionsaufruf.

Funktion `pow`: funktioniert für alle Basen $b \neq 0$

Fromme Lügen...

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is  $b^e$ 
```

ist formal inkorrekt

Fromme Lügen...

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is  $b^e$ 
```

ist formal inkorrekt :

- Überlauf, falls e oder b zu gross sind
- b^e vielleicht nicht als double Wert darstellbar (Löcher im Wertebereich)

Fromme Lügen...sind erlaubt.

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is be
```

Mathematische Bedingungen als Kompromiss zwischen formaler Korrektheit und lascher Praxis.

Prüfen von Vorbedingungen...

- Vorbedingungen sind nur Kommentare.

Prüfen von Vorbedingungen...

- Vorbedingungen sind nur Kommentare.
- Wie können wir *sicherstellen*, dass sie beim Funktionsaufruf gelten?

...mit Assertions

```
#include <cassert>

...
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e) {
    assert (e >= 0 || b != 0);
    double result = 1.0;
    ...
}
```

Assertions: Syntax und Semantik

```
assert ( expression )
```

- *expression*: Ausdruck, dessen Wert nach `bool` konvertierbar ist.
- Semantik:
 - Falls *expression* == `true`: kein Effekt.
 - Falls *expression* == `false`: Programm wird mit entsprechender Fehlermeldung abgebrochen.

Assertions: Syntax und Semantik

```
assert ( expression )
```

- *expression*: Ausdruck, dessen Wert nach `bool` konvertierbar ist.
- Semantik:
 - Falls *expression* == `true`: kein Effekt.
 - Falls *expression* == `false`: Programm wird mit entsprechender Fehlermeldung abgebrochen.
- benötigt `#include <cassert>`

Nachbedingungen mit Assertions

- Das Ergebnis „komplizierter“ Berechnungen ist oft einfach zu prüfen.

Nachbedingungen mit Assertions

- Das Ergebnis „komplizierter“ Berechnungen ist oft einfach zu prüfen.
- Dann lohnt sich der Einsatz von assert für die Nachbedingung

Nachbedingungen mit Assertions

- Das Ergebnis „komplizierter“ Berechnungen ist oft einfach zu prüfen.
- Dann lohnt sich der Einsatz von assert für die Nachbedingung

```
// PRE: the discriminant p*p/4 - q is nonnegative
// POST: returns larger root of the polynomial x^2 + p x + q
double root(double p, double q)
{
    assert(p*p/4 >= q); // precondition
    double x1 = - p/2 + sqrt(p*p/4 - q);
    assert(equals(x1*x1+p*x1+q,0)); // postcondition
    return x1;
}
```

Assertions für den $ggT(x, y)$

Überprüfe, ob das Programm auf dem richtigen Weg ist ...

```
// Input x and y
std::cout << "x =? ";
std::cin >> x;
std::cout << "y =? ";
std::cin >> y;
```

Eingabe der Argumente für
die Berechnung

```
// Check validity of inputs
assert(x > 0 && y > 0);
```

```
... // Compute gcd(x,y), store result in variable a
```

Assertions für den $ggT(x, y)$

Überprüfe, ob das Programm auf dem richtigen Weg ist ...

```
// Input x and y
```

```
std::cout << "x =? ";
```

```
std::cin >> x;
```

```
std::cout << "y =? ";
```

```
std::cin >> y;
```

```
// Check validity of inputs
```

```
assert(x > 0 && y > 0);
```

← Vorbedingung für die weitere Berechnung

```
... // Compute gcd(x,y), store result in variable a
```

Assertions für den $ggT(x, y)$

... und hinterfrage das Offensichtliche! ...

...

`assert(x > 0 && y > 0);`  Vorbedingung für die weitere Berechnung

... `// Compute gcd(x,y), store result in variable a`

`assert (a >= 1);`

`assert (x % a == 0 && y % a == 0);`

`for (int i = a+1; i <= x && i <= y; ++i)`

`assert(!(x % i == 0 && y % i == 0));`

Assertions für den $ggT(x, y)$

... und hinterfrage das Offensichtliche! ...

...

```
assert(x > 0 && y > 0);
```

```
... // Compute gcd(x,y), store result in variable a
```

```
assert (a >= 1);
```

```
assert (x % a == 0 && y % a == 0);
```

```
for (int i = a+1; i <= x && i <= y; ++i)  
    assert(!(x % i == 0 && y % i == 0));
```

Verschiedene
Eigenschaften
des ggT
überprüfen

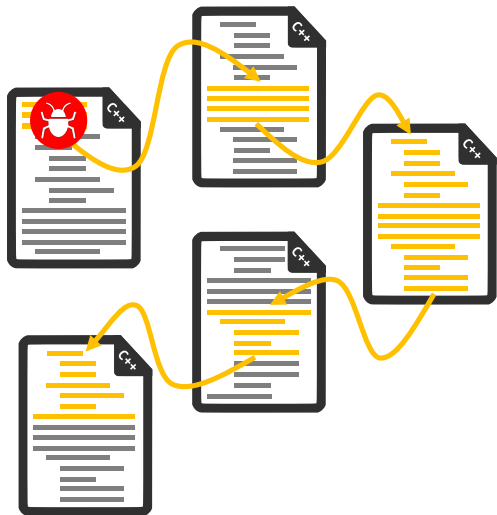
Fail-Fast mit Assertions

- Reale Software: viele C++-Dateien, komplexer Kontrollfluss



Fail-Fast mit Assertions

- Reale Software: viele C++-Dateien, komplexer Kontrollfluss



Fail-Fast mit Assertions

- Reale Software: viele C++-Dateien, komplexer Kontrollfluss
- Fehler machen sich erst spät(er) bemerkbar → Fehlersuche erschwert



Fail-Fast mit Assertions

- Reale Software: viele C++-Dateien, komplexer Kontrollfluss
- Fehler machen sich erst spät(er) bemerkbar → Fehlersuche erschwert
- Assertions: Fehler frühzeitig bemerken



Assertions – Empfehlung

- Assertions sind ein wichtiges Werkzeug zur Fehlervermeidung während der Programmentwicklung, wenn man sie *konsequent* und *oft* einsetzt.
- Der Laufzeit-Overhead ist vernachlässigbar. Darüber hinaus kann der Compiler angewiesen werden, die Assertions „hinauszukompilieren“, also zu ignorieren.

Assertions abschalten

```
#define NDEBUG // To ignore assertions  
#include<cassert>
```

```
...
```

```
assert(x > 0 && y > 0); // Ignored
```

```
... // Compute gcd(x,y), store result in variable a
```

```
assert(a >= 1); // Ignored
```

```
...
```

Stepwise Refinement

Einfache Programmieretechnik zum Lösen komplexer Probleme

Niklaus Wirth. Program development by
stepwise refinement. Commun. ACM 14,
4, 1971

Education

P. Wegner
Editor

Program Development by Stepwise Refinement

Niklaus Wirth
Eidgenössische Technische Hochschule
Zürich, Switzerland

The creative activity of programming—to be distinguished from coding—is usually taught by examples serving to exhibit certain techniques. It is here considered as a sequence of design decisions concerning the decomposition of tasks into subtasks and of data into data structures. The process of successive refinement of specifications is illustrated by a short but nontrivial example, from which a number of conclusions are drawn regarding the art and the instruction of programming.

Key Words and Phrases: education in programming, programming techniques, stepwise program construction
CR Categories: 1.50, 4.0

1. Introduction

Programming is usually taught by examples. Experience shows that the success of a programming course critically depends on the choice of these examples. Unfortunately, they are too often selected with the prime intent to demonstrate what a computer can do. Instead, a main criterion for selection should be their suitability to exhibit certain widely applicable techniques. Furthermore, examples of programs are commonly presented as finished “products” followed by explanations of their purpose and their linguistic details. But active programming consists of the design of new programs, rather than contemplation of old programs. As a consequence of these teaching methods, the student obtains the impression that programming consists mainly of mastering a language (with all the peculiarities and intricacies so abundant in modern PL’s) and relying on one’s intuition to somehow transform ideas into finished programs. Clearly, programming courses should teach methods of design and construction, and the selected examples should be such that a gradual development can be nicely demonstrated.

This paper deals with a single example chosen with

these two purposes in mind. Some well-known techniques are briefly demonstrated and motivated (strategy of preselection, stepwise construction of trial solutions, introduction of auxiliary data, recursion), and the program is gradually developed in a sequence of refinement steps.

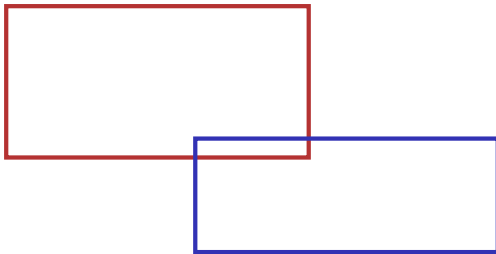
In each step, one or several instructions of the given program are decomposed into more detailed instructions. This successive decomposition or refinement of specifications terminates when all instructions are expressed in terms of an underlying computer or programming language, and must therefore be guided by the facilities available on that computer or language. The result of the execution of a program is expressed in terms of data, and it may be necessary to introduce further data for communication between the obtained subtasks or instructions. As tasks are refined, so the data may have to be refined, decomposed, or structured, and it is natural to refine program and data specifications in parallel.

Every refinement step implies some design decisions. It is important that these decisions be made explicit, and that the programmer be aware of the underlying criteria and of the existence of alternative solutions. The possible solutions to a given problem emerge as the leaves of a tree, each node representing a point of deliberation and decision. Subtrees may be considered as families of solutions with certain common characteristics and structures. The notion of such a tree may be particularly helpful in the situation of changing purpose and environment to which a program may sometime have to be adapted.

A guideline in the process of stepwise refinement should be the principle to decompose decisions as much as possible, to untangle aspects which are only seemingly interdependent, and to defer those decisions which concern details of representation as long as possible. This

Beispielproblem

Finde heraus, ob zwei Rechtecke überlappen!



Top-Down Ansatz

- Formulierung einer groben Lösung mit Hilfe von
 - Kommentaren
 - fiktiven Funktionen

Top-Down Ansatz

- Formulierung einer groben Lösung mit Hilfe von
 - Kommentaren
 - fiktiven Funktionen
- Wiederholte Verfeinerung:
 - Kommentare $\longrightarrow$ Programmtext
 - fiktive Funktionen $\longrightarrow$ Funktionsdefinitionen

Grobe Lösung

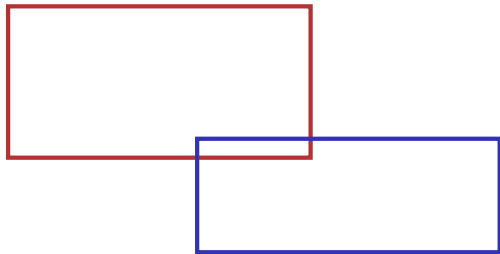
```
int main()
{
    // Eingabe Rechtecke

    // Überlappung?

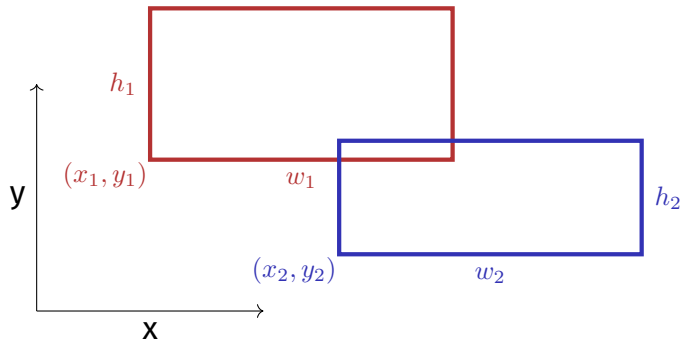
    // Ausgabe der Loesung

    return 0;
}
```

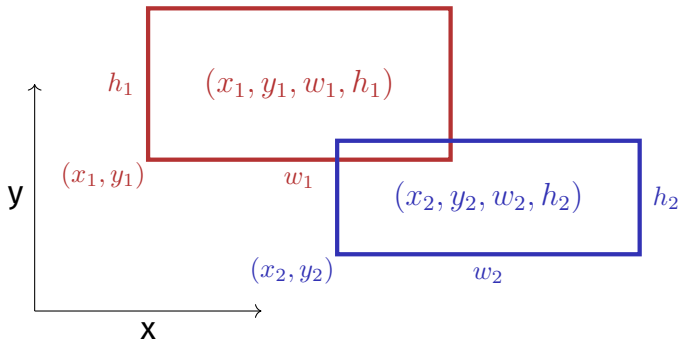
Verfeinerung 1: Eingabe Rechtecke



Verfeinerung 1: Eingabe Rechtecke

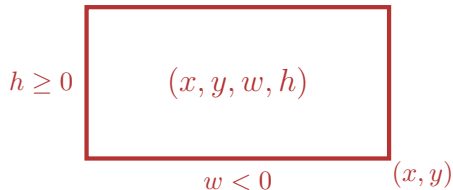


Verfeinerung 1: Eingabe Rechtecke



Verfeinerung 1: Eingabe Rechtecke

Breite w und/oder Höhe h dürfen negativ sein!



Verfeinerung 1: Eingabe Rechtecke

```
int main()
{
    std::cout << "Enter two rectangles [x y w h each] \n";
    int x1, y1, w1, h1;
    std::cin >> x1 >> y1 >> w1 >> h1;
    int x2, y2, w2, h2;
    std::cin >> x2 >> y2 >> w2 >> h2;

    // Ueberlappung?

    // Ausgabe der Loesung

    return 0;
}
```

Verfeinerung 2: Überlappung? und Ausgabe

```
int main()
```

```
{
```

Eingabe Rechtecke ✓

```
bool overlap = rectangles_overlap(x1,y1,w1,h1,x2,y2,w2,h2);
```

```
if (overlap)
```

```
    std::cout << "overlap!\n";
```

```
else
```

```
    std::cout << "no overlap!\n";
```

```
return 0;
```

```
}
```


Verfeinerung 3: Überlappungsfunktion...

```
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

```
int main() {
```

Eingabe Rechtecke ✓

Überlappung? ✓

Ausgabe der Lösung ✓

```
    return 0;
```

```
}
```

Verfeinerung 3: Überlappungsfunktion...

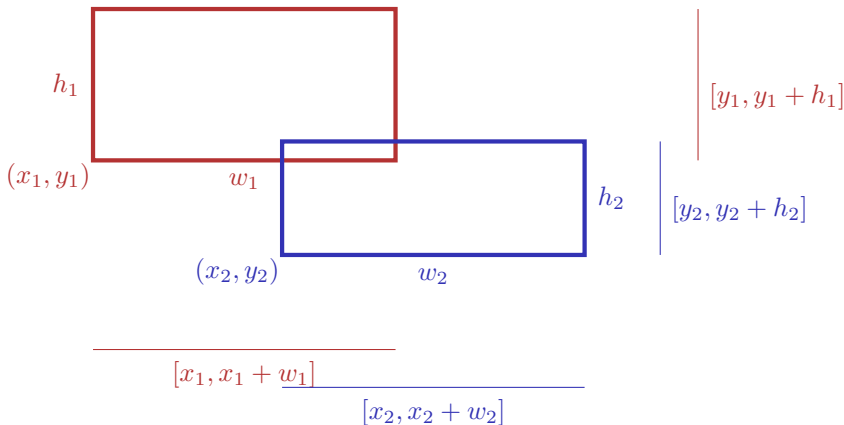
```
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

Funktion main ✓

```
// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles,  
//       where w1, h1, w2, h2 may be negative.  
// POST: returns true if (x1, y1, w1, h1) and  
//       (x2, y2, w2, h2) overlap  
bool rectangles_overlap(int x1, int y1, int w1, int h1,  
                        int x2, int y2, int w2, int h2)  
{  
    return false; // todo  
}
```

Verfeinerung 4: Intervallüberlappung

Zwei Rechtecke überlappen genau dann, wenn ihre x - **und** y -Intervalle überlappen.



Verfeinerung 4: Intervallüberlappung

```
// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles, where
//       w1, h1, w2, h2 may be negative.
// POST: returns true if (x1, y1, w1, h1), (x2, y2, w2, h2) overlap
bool rectangles_overlap(int x1, int y1, int w1, int h1,
                        int x2, int y2, int w2, int h2)
{
    return intervals_overlap(x1, x1 + w1, x2, x2 + w2)
        && intervals_overlap(y1, y1 + h1, y2, y2 + h2);
}
```

Verfeinerung 4: Intervallüberlappung

```
// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles, where
//       w1, h1, w2, h2 may be negative.
// POST: returns true if (x1, y1, w1, h1), (x2, y2, w2, h2) overlap
bool rectangles_overlap(int x1, int y1, int w1, int h1,
                        int x2, int y2, int w2, int h2)
{
    return intervals_overlap(x1, x1 + w1, x2, x2 + w2)
        && intervals_overlap(y1, y1 + h1, y2, y2 + h2); ✓
}
```

Verfeinerung 4: Intervallüberlappung

```
// PRE: [a1, b1], [a2, b2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return false; // todo  
}
```

Funktion rectangles_overlap ✓

Funktion main ✓

Verfeinerung 5: Min und Max

```
// PRE: [a1, b1], [a2, b2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return max(a1, b1) >= min(a2, b2)  
        && min(a1, b1) <= max(a2, b2);  
}
```


Verfeinerung 5: Min und Max

```
// PRE: [a1, b1], [a2, b2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return max(a1, b1) >= min(a2, b2)  
        && min(a1, b1) <= max(a2, b2); ✓  
}
```

Verfeinerung 5: Min und Max

```
// POST: the maximum of x and y is returned
int max(int x, int y){
    if (x>y) return x; else return y;
}
```

```
// POST: the minimum of x and y is returned
int min(int x, int y){
    if (x<y) return x; else return y;
}
```

Funktion intervals_ioverlap ✓

Funktion rectangles_overlap ✓

Funktion main ✓

Verfeinerung 5: Min und Max

// POST: the maximum of x and y is returned

```
int max(int x, int y){  
    if (x>y) return x; else return y;  
}
```

gibt es schon in der Standardbibliothek

// POST: the minimum of x and y is returned

```
int min(int x, int y){  
    if (x<y) return x; else return y;  
}
```

Funktion intervals_ioverlap ✓

Funktion rectangles_overlap ✓

Funktion main ✓

Nochmal zurück zu Intervallen

```
// PRE: [a1, b1], [a2, h2] are (generalized) intervals,  
//       with [a,b] := [b,a] if a>b  
// POST: returns true if [a1, b1],[a2, b2] overlap  
bool intervals_overlap(int a1, int b1, int a2, int b2)  
{  
    return std::max(a1, b1) >= std::min(a2, b2)  
        && std::min(a1, b1) <= std::max(a2, b2); ✓  
}
```

Das haben wir schrittweise erreicht!

```
#include <iostream>
#include <algorithm>

// PRE: [a1, b1], [a2, h2] are (generalized) intervals,
//      with [a,b] := [b,a] if a>b
// POST: returns true if [a1, b1],[a2, b2] overlap
bool intervals_overlap(int a1, int b1, int a2, int b2)
{
    return std::max(a1, b1) >= std::min(a2, b2)
        && std::min(a1, b1) <= std::max(a2, b2);
}

// PRE: (x1, y1, w1, h1), (x2, y2, w2, h2) are rectangles, where
//      w1, h1, w2, h2 may be negative.
// POST: returns true if (x1, y1, w1, h1),(x2, y2, w2, h2) overlap
bool rectangles_overlap(int x1, int y1, int w1, int h1,
                       int x2, int y2, int w2, int h2)
{
    return intervals_overlap(x1, x1 + w1, x2, x2 + w2)
        && intervals_overlap(y1, y1 + h1, y2, y2 + h2);
}
```

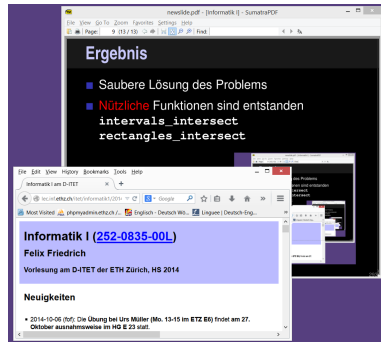
```
int main ()
{
    std::cout << "Enter two rectangles [x y w h] each\n";
    int x1, y1, w1, h1;
    std::cin >> x1 >> y1 >> w1 >> h1;
    int x2, y2, w2, h2;
    std::cin >> x2 >> y2 >> w2 >> h2;
    bool clash = rectangles_overlap(x1,y1,w1,h1,x2,y2,w2,h2);
    if (clash)
        std::cout << "overlap!\n";
    else
        std::cout << "no overlap!\n";
    return 0;
}
```

Ergebnis

- Saubere Lösung des Problems
- Nützliche Funktionen sind entstanden
`intervals_overlap`
`rectangles_overlap`

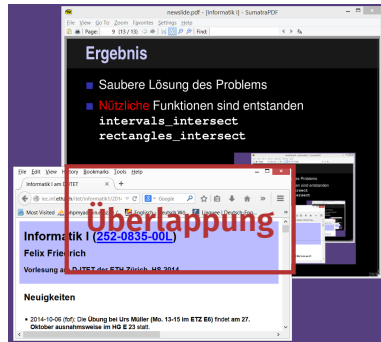
Ergebnis

- Saubere Lösung des Problems
- **Nützliche** Funktionen sind entstanden
`intervals_overlap`
`rectangles_overlap`



Ergebnis

- Saubere Lösung des Problems
- **Nützliche** Funktionen sind entstanden
`intervals_overlap`
`rectangles_overlap`



Wo darf man eine Funktion benutzen?

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    std::cout << f(1); // Fehler: f undeklariert
```

```
    return 0;
```

```
}
```

```
int f(int i) // Gültigkeitsbereich von f ab hier
```

```
{
```

```
    return i;
```

```
}
```

Gültigkeit f



Gültigkeitsbereich einer Funktion

- ist der Teil des Programmes, in dem die Funktion aufgerufen werden kann

Gültigkeitsbereich einer Funktion

- ist der Teil des Programmes, in dem die Funktion aufgerufen werden kann

Erweiterung durch Deklaration einer Funktion: wie Definition aber ohne {...}.

```
double pow(double b, int e);
```

So geht's also nicht...

```
#include <iostream>
```

```
int main()
{
    std::cout << f(1); // Fehler: f undeklariert
    return 0;
}
```

Gültigkeit f

```
int f(int i) // Gültigkeitsbereich von f ab hier
{
    return i;
}
```



...aber so!

```
#include <iostream>

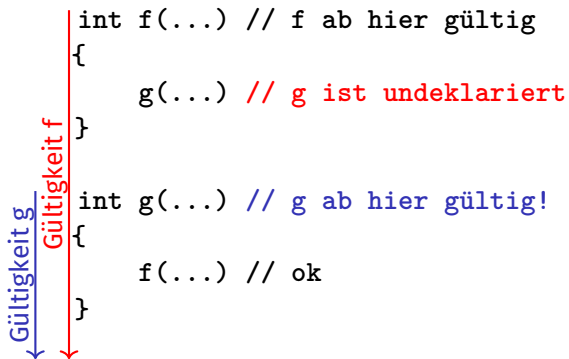
int f(int i); // Gueltigkeitsbereich von f ab hier


int main()
{
    std::cout << f(1);
    return 0;
}

int f(int i)
{
    return i;
}
```

Forward Declarations, wozu?

Funktionen, die sich gegenseitig aufrufen:



```
int f(...) // f ab hier gültig
{
    g(...) // g ist undeklariert
}

int g(...) // g ab hier gültig!
{
    f(...) // ok
}
```

Forward Declarations, wozu?

Funktionen, die sich gegenseitig aufrufen:

```
int g(...); // g ab hier gültig

int f(...) // f ab hier gültig
{
    g(...) // ok
}

int g(...)
{
    f(...) // ok
}
```

Wiederverwendbarkeit

- Funktionen wie `rectangles_overlap` und `pow` sind in vielen Programmen nützlich.

Wiederverwendbarkeit

- Funktionen wie `rectangles_overlap` und `pow` sind in vielen Programmen nützlich.
- „Lösung:“ Funktion einfach ins Hauptprogramm hineinkopieren, wenn wir sie brauchen!

Level 1: Auslagern der Funktion

```
// PRE: e >= 0 || b != 0.0
// POST: return value is b^e
double pow(double b, int e)
{
    double result = 1.0;
    if (e < 0) { // b^e = (1/b)^(-e)
        b = 1.0/b;
        e = -e;
    }
    for (int i = 0; i < e; ++i)
        result *= b;
    return result;
}
```

Level 1: Auslagern der Funktion

```
double pow(double b, int e); in  
separater Datei mymath.cpp
```

Level 1: Inkludieren der Funktion

```
// Prog: callpow2.cpp  
// Call a function for computing powers.
```

```
#include <iostream>  
#include "mymath.cpp"
```

```
int main()  
{  
    std::cout << pow( 2.0, -2) << "\n";  
    std::cout << pow( 1.5, 2) << "\n";  
    std::cout << pow( 5.0, 1) << "\n";  
    std::cout << pow(-2.0, 9) << "\n";  
  
    return 0;  
}
```

Level 1: Inkludieren der Funktion

```
// Prog: callpow2.cpp  
// Call a function for computing powers.
```

```
#include <iostream>  
#include "mymath.cpp" ← im Arbeitsverzeichnis
```

```
int main()  
{  
    std::cout << pow( 2.0, -2) << "\n";  
    std::cout << pow( 1.5, 2) << "\n";  
    std::cout << pow( 5.0, 1) << "\n";  
    std::cout << pow(-2.0, 9) << "\n";  
  
    return 0;  
}
```

Nachteil des Inkludierens

- `#include` kopiert die Datei (`mymath.cpp`) in das Hauptprogramm (`callpow2.cpp`).

Nachteil des Inkludierens

- `#include` kopiert die Datei (`mymath.cpp`) in das Hauptprogramm (`callpow2.cpp`).
- Der Compiler muss die Funktionsdefinition für jedes Programm neu übersetzen.



```
Shabdas-iMac:~ admin$ sudo port install amarok
--> Fetching pkgconfig
--> Attempting to fetch pkg-config-0.25.tar.gz from http://aarnet.au.distfiles
      .macports.org/pub/macports/mpdistfiles/pkgconfig
--> Verifying checksum(s) for pkgconfig
--> Extracting pkgconfig
--> Applying patches to pkgconfig
--> Configuring pkgconfig
--> Building pkgconfig
--> Staging pkgconfig into destroot
--> Installing pkgconfig @0.25_1
--> Deactivating pkgconfig @0.23_1
--> Activating pkgconfig @0.25_1
--> Cleaning pkgconfig
--> Computing dependencies for openssl
--> Fetching openssl
--> Attempting to fetch openssl-1.0.0c.tar.gz from http://aarnet.au.distfiles.
      macports.org/pub/macports/mpdistfiles/openssl
--> Verifying checksum(s) for openssl
--> Extracting openssl
--> Applying patches to openssl
--> Configuring openssl
--> Building openssl
--> Staging openssl into destroot
```

Level 2: Getrennte Übersetzung

```
double pow(double b,  
           int e)  
{  
    ...  
}
```

mymath.cpp

g++ -c mymath.cpp

001110101100101010
000101110101000111
000101110101000111
Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010

mymath.o

Level 2: Getrennte Übersetzung

```
// PRE: e >= 0 || b != 0.0  
// POST: return value is  $b^e$   
double pow(double b, int e);
```

mymath.h

Level 2: Getrennte Übersetzung

```
#include <iostream>
#include "mymath.h"
int main()
{
    std::cout << pow(2,-2) << "\n";
    return 0;
}
```

callpow3.cpp



```
001110101100101010
000101110101000111
000101110101000111
Funktion main
111100001101010001
010101101011010001
101010110101010101
rufe pow auf!
111111101000111010
```

callpow3.o

Der Linker vereint...

```
001110101100101010
000101110101000111
000101110101000111
111100001101010001
111111101000111010
010101101011010001
100101111100101010
```

mymath.o

+

```
001110101100101010
000101110101000111
000101110101000111
111100001101010001
010101101011010001
100101111100101010
111111101000111010
```

callpow3.o

... was zusammengehört

```
001110101100101010
000101110101000111
000101 Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010
```

mymath.o

+

```
001110101100101010
000101110101000111
000101 Funktion main
111100001101010001
010101101011010001
100101111100101010
111111101000111010
```

callpow3.o

=

```
001110101100101010
000101110101000111
000101 Funktion pow
111100001101010001
111111101000111010
010101101011010001
100101111100101010
001110101100101010
000101110101000111
000101 Funktion main
111100001101010001
010101101011010001
100101111100101010
111111101000111010
```

Ausführbare Datei callpow3

Verfügbarkeit von Quellcode?

Beobachtung

`mymath.cpp` (Quellcode) wird nach dem Erzeugen von `mymath.o` (Object Code) nicht mehr gebraucht.

Verfügbarkeit von Quellcode?

Beobachtung

`mymath.cpp` (Quellcode) wird nach dem Erzeugen von `mymath.o` (Object Code) nicht mehr gebraucht.

Viele Anbieter von Funktionsbibliotheken liefern dem Benutzer keinen Quellcode.

Verfügbarkeit von Quellcode?

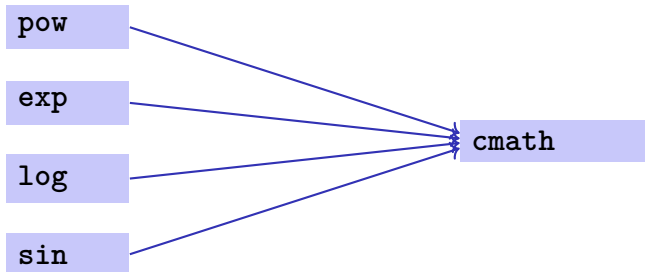
Beobachtung

`mymath.cpp` (Quellcode) wird nach dem Erzeugen von `mymath.o` (Object Code) nicht mehr gebraucht.

Viele Anbieter von Funktionsbibliotheken liefern dem Benutzer keinen Quellcode.

Header-Dateien sind dann die *einzigsten* lesbaren Informationen.

■ Logische Gruppierung ähnlicher Funktionen



Namensräume...

```
// cmath
namespace std {

    double pow(double b, int e);

    ....
    double exp(double x);
    ...
}

// informatik
namespace informatik {
    double pow(double b, int e);
}
```

...vermeiden Namenskonflikte

```
#include <cmath>
#include "mymath.h"

int main()
{
    double x = std::pow(2.0, -2); // <cmath>
    double y = informatik::pow(2.0, -2); // mymath.h
}
```

Funktionen aus der Standardbibliothek

- vermeiden die Neuerfindung des Rades (wie bei `std::pow`);
- führen auf einfache Weise zu interessanten und effizienten Programmen;

Funktionen aus der Standardbibliothek

- vermeiden die Neuerfindung des Rades (wie bei `std::pow`);
- führen auf einfache Weise zu interessanten und effizienten Programmen;
- garantieren einen Qualitäts-Standard, der mit selbstgeschriebenen Funktionen kaum erreicht werden kann.

Funktionen aus der Standardbibliothek

- vermeiden die Neuerfindung des Rades (wie bei `std::pow`);
- führen auf einfache Weise zu interessanten und effizienten Programmen;
- garantieren einen Qualitäts-Standard, der mit selbstgeschriebenen Funktionen kaum erreicht werden kann.
- Übersicht auf <https://cppreference.com>

Beispiel: Primzahltest mit sqrt

$n \geq 2$ ist Primzahl genau dann, wenn kein d in $\{2, \dots, n-1\}$ ein Teiler von n ist.

```
unsigned int d;  
for (d=2; n % d != 0; ++d);
```

Primzahltest mit sqrt

$n \geq 2$ ist Primzahl genau dann, wenn kein d in $\{2, \dots, \lfloor \sqrt{n} \rfloor\}$ ein Teiler von n ist.

```
unsigned int bound = std::sqrt(n);  
unsigned int d;  
for (d = 2; d <= bound && n % d != 0; ++d);
```

Primzahltest mit sqrt

$n \geq 2$ ist Primzahl genau dann, wenn kein d in $\{2, \dots, \lfloor \sqrt{n} \rfloor\}$ ein Teiler von n ist.

```
unsigned int bound = std::sqrt(n);  
unsigned int d;  
for (d = 2; d <= bound && n % d != 0; ++d);
```

- Das funktioniert, weil `std::sqrt` auf die nächste darstellbare double-Zahl rundet (IEEE Standard 754).


```
void swap(int x, int y) {  
    int t = x;  
    x = y;  
    y = t;  
}  
  
int main(){  
    int a = 2;  
    int b = 1;  
    swap(a, b);  
    assert(a==1 && b==2);  
}
```

```
void swap(int x, int y) {  
    int t = x;  
    x = y;  
    y = t;  
}  
  
int main(){  
    int a = 2;  
    int b = 1;  
    swap(a, b);  
    assert(a==1 && b==2); // fail! 😞  
}
```

```
// POST: values of x and y are exchanged
void swap(int& x, int& y) {
    int t = x;
    x = y;
    y = t;
}

int main(){
    int a = 2;
    int b = 1;
    swap(a, b);
    assert(a==1 && b==2);
}
```

// POST: values of x and y are exchanged

```
void swap(int& x, int& y) {
```

```
    int t = x;
```

```
    x = y;
```

```
    y = t;
```

```
}
```

```
int main(){
```

```
    int a = 2;
```

```
    int b = 1;
```

```
    swap(a, b);
```

```
    assert(a==1 && b==2); // ok!
```



```
}
```

Sneak Preview: Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern!

Sneak Preview: Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern!
- Kein neues Konzept auf der Funktionenseite, sondern eine neue Klasse von Typen

Sneak Preview: Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern!
- Kein neues Konzept auf der Funktionenseite, sondern eine neue Klasse von Typen



Referenztypen (z.B. `int&`)

11. Vektoren I

Vektoren, Sieb des Eratosthenes, Speicherlayout, Iteration

Vektoren: Motivation

- Wir können jetzt über Zahlen iterieren

```
for (int i=0; i<n; ++i) {...}
```

Vektoren: Motivation

- Wir können jetzt über Zahlen iterieren

```
for (int i=0; i<n; ++i) {...}
```

- .. aber noch nicht über Daten!

Vektoren: Motivation

- Wir können jetzt über Zahlen iterieren

```
for (int i=0; i<n; ++i) {...}
```

- .. aber noch nicht über Daten!
- Vektoren speichern *gleichartige* Daten.

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Streiche alle echten Vielfachen von 2 ...

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	---	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----

Streiche alle echten Vielfachen von 2 ...

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	---	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----

... und gehe zur nächsten Zahl

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	---	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----	---------------	----

Streiche alle echten Vielfachen von 3 ...

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

Streiche alle echten Vielfachen von 3 ...

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

... und gehe zur nächsten Zahl

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

Am Ende des Streichungsprozesses bleiben nur die Primzahlen übrig.

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

- Frage: wie streichen wir Zahlen aus?

Vektoren: erste Anwendung

Das Sieb des Erathostenes

- berechnet alle Primzahlen $< n$
- Methode: Ausstreichen der Nicht-Primzahlen

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------	---------------	---------------	----

- Frage: wie streichen wir Zahlen aus?
- Antwort: mit einem *Vektor*.

Eratosthenes mit Vektoren: Initialisierung

...

```
#include <vector>
```

...

```
std::vector<bool> crossed_out = std::vector<bool>(n, false);
```

Initialisierung mit `n` Elementen
Initialwert `false`.



Elementtyp, in spitzen Klammern

Eratosthenes mit Vektoren: Berechnung

```
for (unsigned int i = 2; i < crossed_out.size(); ++i)
    if (!crossed_out[i]) { // i is prime
        std::cout << i << " ";

        // cross out all proper multiples of i
        for (unsigned int m = 2*i; m < crossed_out.size(); m += i)
            crossed_out[m] = true;
    }
```


Speicherlayout eines Vektors

Ein Vektor belegt einen *zusammenhängenden* Speicherbereich

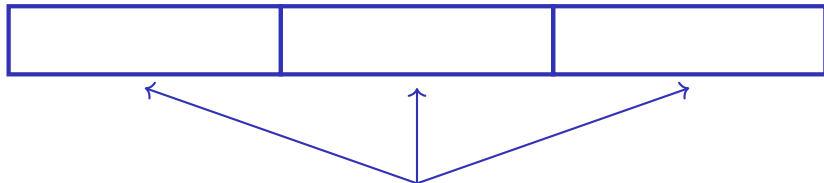
Beispiel: Ein Vektor mit 3 Elementen vom Type T



Speicherlayout eines Vektors

Ein Vektor belegt einen *zusammenhängenden* Speicherbereich

Beispiel: Ein Vektor mit 3 Elementen vom Type T

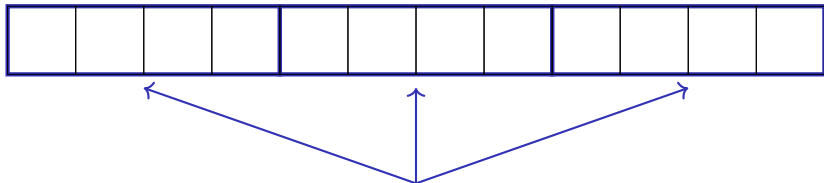


Speichersegmente für jeweils einen Wert vom Typ T

Speicherlayout eines Vektors

Ein Vektor belegt einen *zusammenhängenden* Speicherbereich

Beispiel: Ein Vektor mit 3 Elementen vom Type T



Speichersegmente für jeweils einen Wert vom Typ T
(T belegt z.B. 4 Bytes)

Wahlfreier Zugriff (Random Access)

Gegeben

- Vektor `vec` mit `T`-Elementen
- `int`-Ausdruck `exp` mit Wert $i \geq 0$

Wahlfreier Zugriff (Random Access)

Gegeben

- Vektor `vec` mit `T`-Elementen
- `int`-Ausdruck `exp` mit Wert $i \geq 0$

Dann ist der Ausdruck

`vec [ exp ]`

- ein *L-Wert* vom Type `T`

Wahlfreier Zugriff (Random Access)

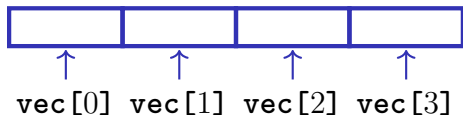
Gegeben

- Vektor `vec` mit `T`-Elementen
- `int`-Ausdruck `exp` mit Wert $i \geq 0$

Dann ist der Ausdruck

`vec [ exp ]`

- ein `L`-Wert vom Type `T`
- der sich auf das i -te Element von `vec` bezieht (Zählung ab 0!)



Wahlfreier Zugriff (Random Access)

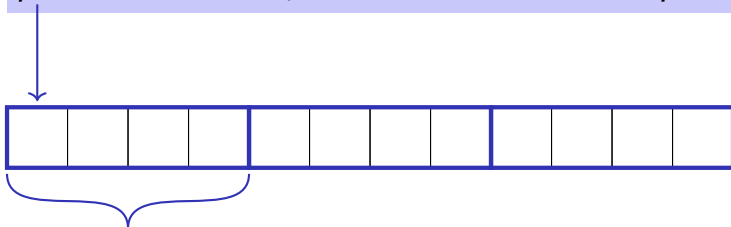
`vec [ exp ]`

- Der Wert i von `exp` heisst *Index*
- `[]` ist der *Index-Operator* (auch *Subskript-Operator*)

Wahlfreier Zugriff (Random Access)

Wahlfreier Zugriff ist sehr effizient:

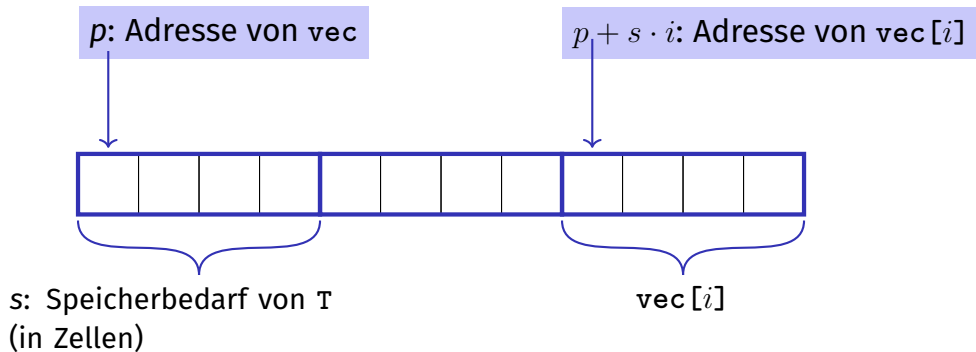
p : Adresse von vec , d.h. Adresse der ersten Speicherzelle



s : Speicherbedarf von T
(in Zellen)

Wahlfreier Zugriff (Random Access)

Wahlfreier Zugriff ist sehr effizient:



Vektorinitialisierung

■ `vector<int> fivezeros = vector<int>(5);`

Die fünf Elemente von `fivezeros` werden mit 0 initialisiert

Namensraumpräfix `std::` aus Platzgründen weggelassen

Vektorinitialisierung

- `vector<int> fivezeros = vector<int>(5);`
Die fünf Elemente von `fivezeros` werden mit 0 initialisiert
- `vector<int> fivetwos = vector<int>(5, 2);`
Die fünf Elemente von `fivetwos` werden mit 2 initialisiert

Namensraumpräfix `std::` aus Platzgründen weggelassen

Vektorinitialisierung

- `vector<int> fivezeros = vector<int>(5);`
Die fünf Elemente von `fivezeros` werden mit 0 initialisiert
- `vector<int> fivetwos = vector<int>(5, 2);`
Die fünf Elemente von `fivetwos` werden mit 2 initialisiert
- `vector<int> countdown = {5, 4, 3, 2, 1};`
Der Vektor wird direkt mit einer Liste an Werten initialisiert

Namensraumpräfix `std::` aus Platzgründen weggelassen

Vektorinitialisierung

- `vector<int> fivezeros = vector<int>(5);`
Die fünf Elemente von `fivezeros` werden mit 0 initialisiert
- `vector<int> fivetwos = vector<int>(5, 2);`
Die fünf Elemente von `fivetwos` werden mit 2 initialisiert
- `vector<int> countdown = {5, 4, 3, 2, 1};`
Der Vektor wird direkt mit einer Liste an Werten initialisiert
- `vector<int> empty;`
Deklariert *und initialisiert* einen leeren Vektor

Namensraumpräfix `std::` aus Platzgründen weggelassen

Achtung

Der Zugriff auf Elemente ausserhalb der gültigen Grenzen eines Vektors führt zu *undefiniertem Verhalten*

```
std::vector<int> vec = std::vector<int>(10);  
for (unsigned int i = 0; i <= 10; ++i)  
    vec[i] = 30;
```

Achtung

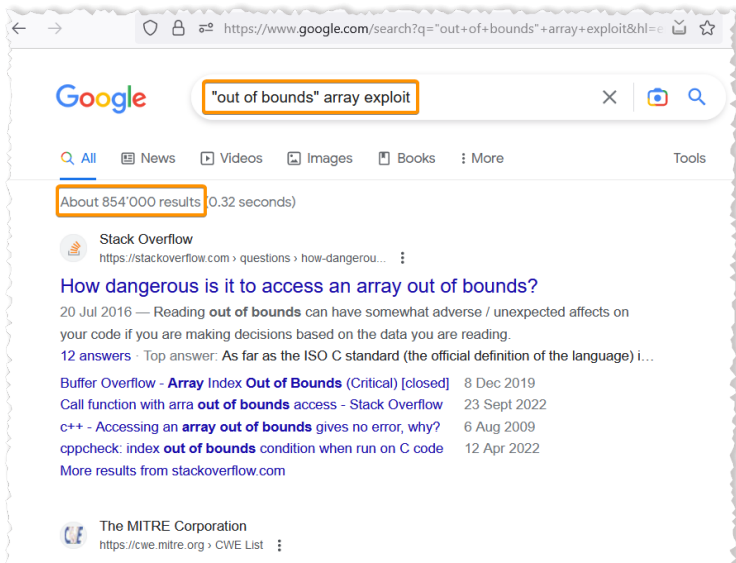
Der Zugriff auf Elemente ausserhalb der gültigen Grenzen eines Vektors führt zu *undefiniertem Verhalten*

```
std::vector<int> vec = std::vector<int>(10);  
for (unsigned int i = 0; i <= 10; ++i)  
    vec[i] = 30; // undefiniertes Verhalten: Zugriff auf vec[10]
```

Prüfung der Indexgrenzen

Bei Verwendung des Indexoperators auf einem Vektor ist es die alleinige Verantwortung des Programmierers, die Gültigkeit aller Elementzugriffe zu prüfen.

Konsequenzen illegaler Index-Zugriffe



Konsequenzen illegaler Index-Zugriffe

The screenshot shows the Rapid7 Vulnerability & Exploit Database interface. The search bar contains 'buffer overflow' and the results are displayed as a list of vulnerabilities. The first result is highlighted with an orange box around the 'Results' count.

Results 01 - 20 of 21,562 in total

Vulnerability Title	Published	Severity	Action
Google Chrome Vulnerability: CVE-2023-1222 Heap buffer overflow in Web Audio API	March 08, 2023	4	EXPLORE
Google Chrome Vulnerability: CVE-2023-1217 Stack buffer overflow in Crash reporting	March 08, 2023	4	EXPLORE
Microsoft Edge Chromium : CVE-2023-1220 Heap buffer overflow in UMA	March 07, 2023	4	EXPLORE
SUSE: CVE-2023-1220: SUSE Linux Security Advisory	March 07, 2023	4	EXPLORE
FreeBSD: VID-D357F6BB-0AF4-4AC9-B096-EEEC183AD829 (CVE-2023-1220): chromium -- multiple vulnerabilities	March 07, 2023	4	EXPLORE
Debian: CVE-2023-1220: chromium -- security update			EXPLORE

Vektoren bieten viel Funktionalität

Hier ein paar Beispielfunktionen, weitere folgen später in der Vorlesung.

Vektoren bieten viel Funktionalität

Hier ein paar Beispielfunktionen, weitere folgen später in der Vorlesung.

```
std::vector<int> v = std::vector<int>(10);  
std::cout << v.at(10);  
    // Zugriff mit Index-Check → Laufzeitfehler  
    // Ideal für Hausaufgaben
```

Vektoren bieten viel Funktionalität

Hier ein paar Beispielfunktionen, weitere folgen später in der Vorlesung.

```
std::vector<int> v = std::vector<int>(10);  
std::cout << v.at(10);  
    // Zugriff mit Index-Check → Laufzeitfehler  
    // Ideal für Hausaufgaben
```

```
v.push_back(-1); // -1 is appended (added at end)  
std::cout << v.size(); // outputs 11  
std::cout << v.at(10); // outputs -1
```

12. Referenztypen

Referenztypen: Definition und Initialisierung, Pass by Value, Pass by Reference, temporäre Objekte, Const-Referenzen

Swap!

// POST: values of x and y have been exchanged

```
void swap(int& x, int& y) {  
    int t = x;  
    x = y;  
    y = t;  
}
```

```
int main() {  
    int a = 2;  
    int b = 1;  
    swap(a, b);  
    assert(a == 1 && b == 2); // ok! 😊  
}
```

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern

Referenztypen

- Wir können Funktionen in die Lage versetzen, die Werte ihrer Aufrufargumente zu ändern
- Kein neues Konzept auf der Funktionenseite, sondern eine neue Klasse von Typen: *Referenztypen*

Referenztypen: Definition



Referenztypen: Definition



- $T\&$ hat den gleichen Wertebereich und gleiche Funktionalität wie T ...

Referenztypen: Definition



- *T&* hat den gleichen Wertebereich und gleiche Funktionalität wie *T* ...
- ...aber Initialisierung und Zuweisung funktionieren anders

Anakin Skywalker alias Darth Vader



Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;  
  
std::cout << anakin_skywalker; // 22
```

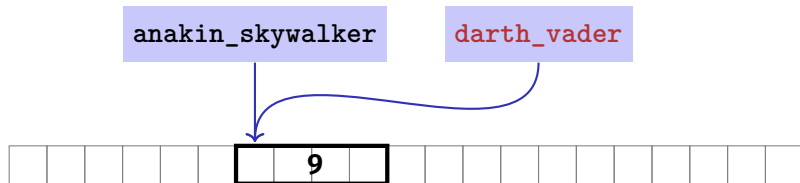
Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;  
  
std::cout << anakin_skywalker; // 22
```



Anakin Skywalker alias Darth Vader

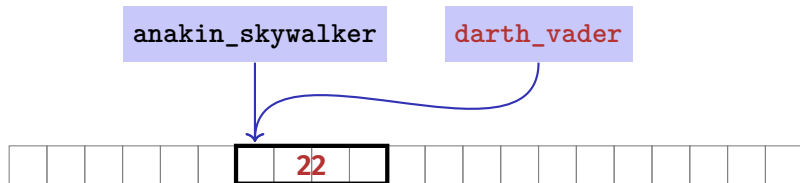
```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;  
  
std::cout << anakin_skywalker; // 22
```



Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;
```

```
std::cout << anakin_skywalker; // 22
```



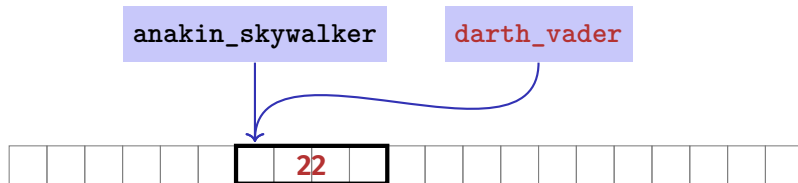
Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias
```

```
darth_vader = 22;
```

Zuweisung an den L-Wert hinter dem Alias

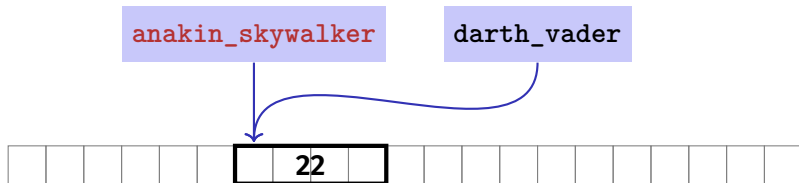
```
std::cout << anakin_skywalker; // 22
```



Anakin Skywalker alias Darth Vader

```
int anakin_skywalker = 9;  
int& darth_vader = anakin_skywalker; // Alias  
darth_vader = 22;
```

```
std::cout << anakin_skywalker; // 22
```



Referenztypen: Initialisierung & Zuweisung

```
int& darth_vader = anakin_skywalker;
```

- Eine Variable mit **Referenztyp** (eine *Referenz*) muss mit einem **L-Wert** initialisiert werden

Referenztypen: Initialisierung & Zuweisung

```
int& darth_vader = anakin_skywalker;
```

- Eine Variable mit **Referenztyp** (eine *Referenz*) muss mit einem **L-Wert** initialisiert werden
- Die Variable wird dabei ein *Alias* des **L-Werts** (ein anderer Name für das referenzierte Objekt)

Referenztypen: Initialisierung & Zuweisung

```
int& darth_vader = anakin_skywalker;  
darth_vader = 22; // Effekt: anakin_skywalker = 22
```

- Eine Variable mit **Referenztyp** (eine *Referenz*) muss mit einem **L-Wert** initialisiert werden
- Die Variable wird dabei ein *Alias* des **L-Werts** (ein anderer Name für das referenzierte Objekt)
- Zuweisung an die Referenz erfolgt an das Objekt *hinter* dem Alias

Referenztypen: Realisierung

Intern wird ein Wert vom Typ $T\&$ durch die Adresse eines Objekts vom Typ T repräsentiert.

```
int& j; // Fehler: j muss Alias von irgendetwas sein
```

Referenztypen: Realisierung

Intern wird ein Wert vom Typ $T\&$ durch die Adresse eines Objekts vom Typ T repräsentiert.

```
int& j; // Fehler: j muss Alias von irgendetwas sein
```

```
int& k = 5; // Fehler: Literal 5 hat keine Adresse
```


Pass by Reference

```
void increment(int& i) {  
    ++i;  
}  
...  
int j = 5;  
increment(j);  
std::cout << j;
```

Pass by Reference

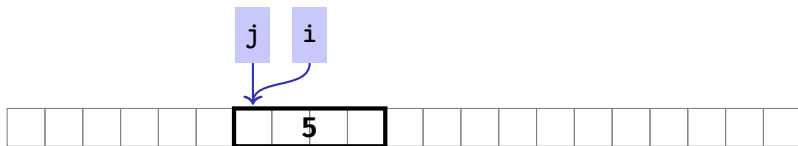
```
void increment(int& i) {  
    ++i;  
}  
...  
int j = 5;  
increment(j);  
std::cout << j;
```



Pass by Reference

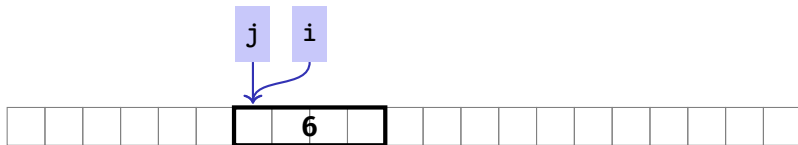
```
void increment(int& i) ← {  
    ++i;  
}  
...  
int j = 5;  
increment(j);  
std::cout << j;
```

Initialisierung der formalen Argumente: i
wird Alias des Aufrufarguments j



Pass by Reference

```
void increment(int& i) {  
    ++i;  
}  
...  
int j = 5;  
increment(j);  
std::cout << j;
```



Pass by Reference

```
void increment(int& i) {  
    ++i;  
}  
...  
int j = 5;  
increment(j);  
std::cout << j; // 6
```



Pass by Reference

Formales Argument *hat* Referenztyp:

⇒ *Pass by Reference*

Formales Argument wird (intern) mit der *Adresse* des Aufrufarguments (L-Wert) initialisiert und wird damit zu einem *Alias*.

Pass by Value

Formales Argument *hat keinen* Referenztyp:

⇒ *Pass by Value*

Formales Argument wird mit dem *Wert* des Aufrufarguments (R-Wert) initialisiert und wird damit zu einer *Kopie*.

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```


Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error
- Funktion muss L-Wert zurückgeben, gibt aber R-Wert zurück

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error
- Funktion muss L-Wert zurückgeben, gibt aber R-Wert zurück

```
int& inc(int& i) {  
    return ++i;  
}
```

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error
- Funktion muss L-Wert zurückgeben, gibt aber R-Wert zurück

```
int& inc(int& i) {  
    return ++i;  
}
```

- Auch der Rückgabetyt ist Referenztyp: *Return by Reference*
- Funktionsaufruf selbst ist nun ein L-Wert

Return by Reference (Beispiel 1)

```
int increment(int& i) {  
    return ++i;  
}
```

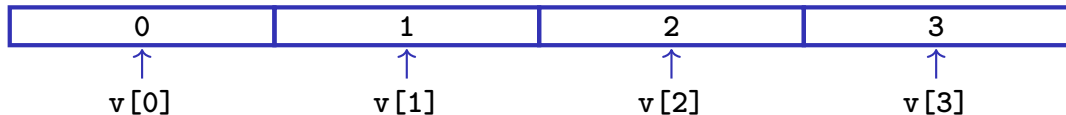
- Wir möchten wiederholte Anwendung: `increment(increment(x))`
Compiler Error
- Funktion muss L-Wert zurückgeben, gibt aber R-Wert zurück

```
int& inc(int& i) {  
    return ++i;  
}
```

- Auch der Rückgabetypp ist Referenztyp: *Return by Reference*
- Funktionsaufruf *selbst* ist nun ein L-Wert
- Aufruf `inc(x)` hat exakt die Semantik des Prä-Inkrementes `++x`

Return by Reference (Beispiel 2)

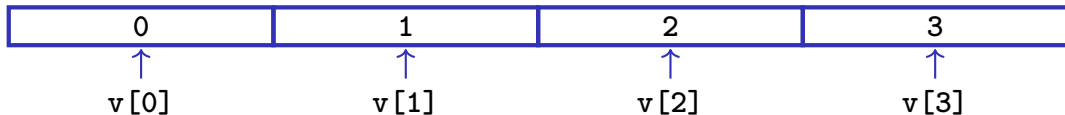
```
std::vector<int> v = {0, 1, 2, 3};
```



Return by Reference (Beispiel 2)

```
std::vector<int> v = {0, 1, 2, 3};
```

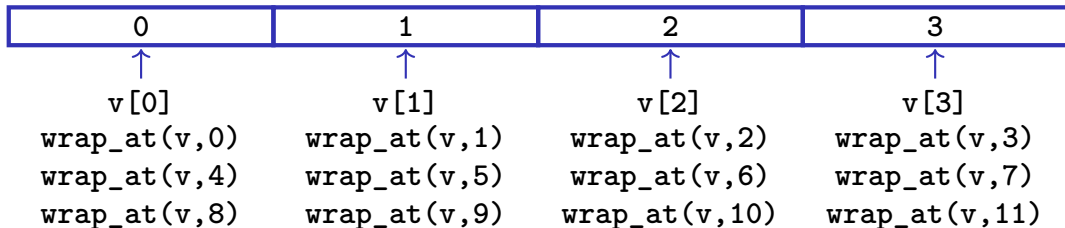
Ziel: Indexieren auf Vektor v mit Wrap-Around beim Index $i \geq 4$.



Return by Reference (Beispiel 2)

```
std::vector<int> v = {0, 1, 2, 3};
```

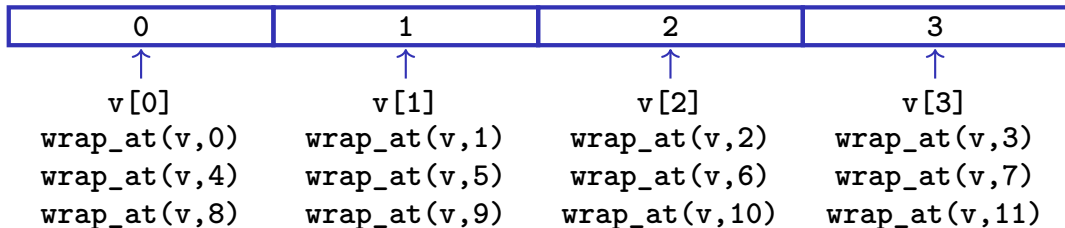
Ziel: Indexieren auf Vektor v mit Wrap-Around beim Index $i \geq 4$.



Return by Reference (Beispiel 2)

```
std::vector<int> v = {0, 1, 2, 3};
```

Ziel: Indexieren auf Vektor v mit Wrap-Around beim Index $i \geq 4$.



$\text{wrap_at}(v, i)$ greift auf Vektor v bei Index $i \% v.size()$ zu

wrap_at-Funktion: Erster Versuch

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2);  
std::cout << wrap_at(v, 6);
```

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6);
```

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6;
```


wrap_at-Funktion: Erster Versuch

```
int wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6; // compiler error
```

Compilerfehler: lvalue required as left operand of assignment

⇒ wrap_at gibt einen R-Wert zurück, nicht einen L-Wert

wrap_at-Funktion

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6;  
std::cout << wrap_at(v, 2);
```

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6;  
std::cout << wrap_at(v, 2); // outputs 6
```

wrap_at-Funktion

```
int& wrap_at(std::vector<int>& vec, unsigned int index){  
    return vec[index % vec.size()];  
}
```

```
std::vector<int> v = {0, 1, 2, 3};  
std::cout << wrap_at(v, 2); // outputs 2  
std::cout << wrap_at(v, 6); // outputs 2
```

```
wrap_at(v, 6) = 6;  
std::cout << wrap_at(v, 2); // outputs 6
```

Return by Reference: Rückgabe von L-Wert

Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;  
}
```


Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;  
}
```

```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;  
}
```



// main()

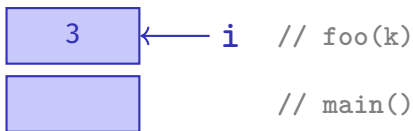
```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Temporäre Objekte

Was ist hier falsch?

Wert des Aufrufarguments kommt auf
den *Aufrufstapel*

```
int& foo(int i) {  
    return i;  
}
```



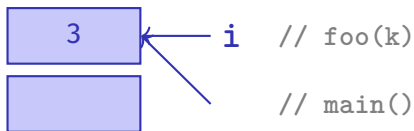
```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Temporäre Objekte

Was ist hier falsch?

i wird als Referenz zurückgegeben

```
int& foo(int i) {  
    return i;  
}
```



```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Temporäre Objekte

Was ist hier falsch?

...und verschwindet vom Aufrufstapel

```
int& foo(int i) {  
    return i;  
}
```

Speicher
freigegeben



// main()

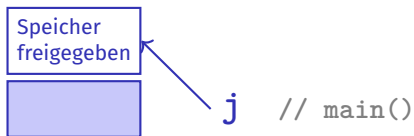
```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;  
}
```

j wird Alias des freigegebenen Speichers



```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

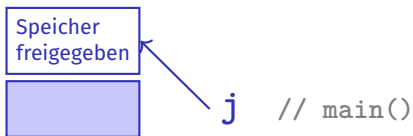
Temporäre Objekte

Was ist hier falsch?

```
int& foo(int i) {  
    return i;  
}
```

```
int k = 3;  
int& j = foo(k); // j ist Alias einer "Leiche"  
std::cout << j; // undefined behavior
```

Zugriff auf j ist undefiniertes Verhalten!



Die Referenz-Richtlinie

Referenz-Richtlinie

Wenn man eine Referenz erzeugt, muss das Objekt, auf das sie verweist, mindestens so lange „leben“ wie die Referenz selbst.

Return by Reference – Warum?

- Nutzen von *Pass by Reference*:
 - Kopieren von Daten beim Aufruf vermeiden
 - Effekt auf Argumente/Aufruferseite gewünscht:
`swap(...)`

Return by Reference – Warum?

- Nutzen von *Pass by Reference*:

- Kopieren von Daten beim Aufruf vermeiden
- Effekt auf Argumente/Aufruferseite gewünscht:
`swap(...)`

- Nutzen von *Return by Reference*
wenn Rückgabewert L-Wert sein soll

- Vektoren:
`vec[i] = x`
`wrap_at(vec, i) = x`
- mehr Anwendungen später im Kapitel zu dynamischen Datenstrukturen

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double> values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double> values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

- Aufruf

```
std::vector<double> temperatures =  
{0.9,1.3,5.1,9.1,13.2,17.1,18.8,18.6,14.7,10.6,5.2,1.8};  
std::cout << average(temperatures); // outputs 9.7
```

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

pass by reference, da Kopieren von Vektor vermieden werden soll!

- Aufruf

```
std::vector<double> temperatures =  
{0.9,1.3,5.1,9.1,13.2,17.1,18.8,18.6,14.7,10.6,5.2,1.8};  
std::cout << average(temperatures); // outputs 9.7
```

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

pass by reference, da Kopieren von Vektor vermieden werden soll!

- Aufruf

```
const std::vector<double> temperatures =  
{0.9,1.3,5.1,9.1,13.2,17.1,18.8,18.6,14.7,10.6,5.2,1.8};  
std::cout << average(temperatures);
```

const-Vektor, da die Werte nicht verändert werden (sollen)

Durchschnitt: Erste Versuche

- Funktion, die den Durchschnitt eines Vektors berechnet

```
double average(std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

pass by reference, da Kopieren von Vektor vermieden werden soll!

- Aufruf

```
const std::vector<double> temperatures =  
{0.9,1.3,5.1,9.1,13.2,17.1,18.8,18.6,14.7,10.6,5.2,1.8};  
std::cout << average(temperatures); // compiler error
```

const-Vektor, da die Werte nicht verändert werden (sollen)

Durchschnitt: Lösung mit Pass by Const Reference

```
double average(const std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```


Durchschnitt: Lösung mit Pass by Const Reference

```
double average(const std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

```
const std::vector<double> temperatures =  
{0.9, 1.3, 5.1, 9.1, 13.2, 17.1, 18.8, 18.6, 14.7, 10.6, 5.2, 1.8};  
std::cout << average(temperatures);
```

```
std::vector<double> numbers = {2.0, 3.0, 4.0};  
std::cout << average(numbers);
```

Durchschnitt: Lösung mit Pass by Const Reference

```
double average(const std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

```
const std::vector<double> temperatures =  
{0.9, 1.3, 5.1, 9.1, 13.2, 17.1, 18.8, 18.6, 14.7, 10.6, 5.2, 1.8};  
std::cout << average(temperatures); // outputs 9.7
```

```
std::vector<double> numbers = {2.0, 3.0, 4.0};  
std::cout << average(numbers);
```

Durchschnitt: Lösung mit Pass by Const Reference

```
double average(const std::vector<double>& values){  
    double sum = 0;  
    for (unsigned int i = 0; i < values.size(); ++i)  
        sum += values[i];  
    return sum / values.size();  
}
```

```
const std::vector<double> temperatures =  
{0.9, 1.3, 5.1, 9.1, 13.2, 17.1, 18.8, 18.6, 14.7, 10.6, 5.2, 1.8};  
std::cout << average(temperatures); // outputs 9.7
```

```
std::vector<double> numbers = {2.0, 3.0, 4.0};  
std::cout << average(numbers); // outputs 3.0
```

Const-Referenzen

- haben Typ `const T &`
- Typ kann verstanden werden als „`(const T) &`“

Const-Referenzen

- haben Typ `const T &`
- Typ kann verstanden werden als „`(const T) &`“

```
const T& r = lvalue;
```

`r` wird mit der Adresse von *lvalue* initialisiert (effizient)

Const-Referenzen

- haben Typ `const T &`
- Typ kann verstanden werden als „`(const T) &`“

```
const T& r = lvalue;
```

`r` wird mit der Adresse von `lvalue` initialisiert (effizient)

- können auch mit R-Werten initialisiert werden (Compiler erzeugt temporäres Objekt ausreichender Lebensdauer)

Const-Referenzen

- haben Typ `const T &`
- Typ kann verstanden werden als „`(const T) &`“

```
const T& r = lvalue;
```

`r` wird mit der Adresse von `lvalue` initialisiert (effizient)

- können auch mit R-Werten initialisiert werden (Compiler erzeugt temporäres Objekt ausreichender Lebensdauer)

```
const T& r = rvalue;
```

`r` wird mit der Adresse eines temporären Objektes vom Wert des `rvalue` initialisiert (pragmatisch)

13. Zeichen und Texte

Zeichen, ASCII, UTF-8, Caesar-Code, Texte als Strings, String-Operationen

Zeichen und Texte

- Texte haben wir schon gesehen:

```
std::cout << "Prime numbers in {2,...,999}:\n";
```

String-Literal

Zeichen und Texte

- Texte haben wir schon gesehen:

```
std::cout << "Prime numbers in {2,...,999}:\n";
```

String-Literal

- Können wir auch „richtig“ mit Texten arbeiten?

Zeichen und Texte

- Texte haben wir schon gesehen:

```
std::cout << "Prime numbers in {2,...,999}:\n";
```

String-Literal

- Können wir auch „richtig“ mit Texten arbeiten? Ja!

Zeichen: Wert des fundamentalen Typs `char`

Text: `std::string` $\approx$ Vektor von `char` Elementen

Der Typ `char` („character“)

Repräsentiert druckbare Zeichen (z.B. `'a'`) und *Steuerzeichen* (z.B. `'\n'`)

Der Typ char („character“)

Repräsentiert druckbare Zeichen (z.B. 'a') und *Steuerzeichen* (z.B. '\n')

```
char c = 'a';
```

Deklariert und initialisiert
Variable c vom Typ char
mit Wert 'a'

Der Typ char („character“)

Repräsentiert druckbare Zeichen (z.B. 'a') und *Steuerzeichen* (z.B. '\n')

```
char c = 'a';
```

Deklariert und initialisiert
Variable c vom Typ char
mit Wert 'a'

Literal vom Typ char

Der Typ `char` („character“)

Ist formal ein ganzzahliger Typ

- Werte konvertierbar nach `int` / `unsigned int`

Der Typ `char` („character“)

Ist formal ein ganzzahliger Typ

- Werte konvertierbar nach `int` / `unsigned int`
- Werte belegen meistens 8 Bit

Wertebereich:

$\{-128, \dots, 127\}$ oder $\{0, \dots, 255\}$

Der ASCII-Code

- Definiert konkrete Konversionsregeln `char` $\longrightarrow$ (unsigned) `int`

Zeichen $\longrightarrow \{0, \dots, 127\}$

'A', 'B', ... , 'Z' $\longrightarrow$ 65, 66, ..., 90

'a', 'b', ... , 'z' $\longrightarrow$ 97, 98, ..., 122

'0', '1', ... , '9' $\longrightarrow$ 48, 49, ..., 57

Der ASCII-Code

- Definiert konkrete Konversionsregeln `char` $\longrightarrow$ (unsigned) `int`

Zeichen $\longrightarrow \{0, \dots, 127\}$

'A', 'B', ... , 'Z' $\longrightarrow$ 65, 66, ..., 90

'a', 'b', ... , 'z' $\longrightarrow$ 97, 98, ..., 122

'0', '1', ... , '9' $\longrightarrow$ 48, 49, ..., 57

- Wird von allen gängigen Computersystemen unterstützt

Der ASCII-Code

- Definiert konkrete Konversionsregeln `char` $\longrightarrow$ (unsigned) `int`

Zeichen $\longrightarrow \{0, \dots, 127\}$

'A', 'B', ... , 'Z' $\longrightarrow$ 65, 66, ..., 90

'a', 'b', ... , 'z' $\longrightarrow$ 97, 98, ..., 122

'0', '1', ... , '9' $\longrightarrow$ 48, 49, ..., 57

- Wird von allen gängigen Computersystemen unterstützt
- Erlaubt Arithmetik über Zeichen

```
for (char c = 'a'; c <= 'z'; ++c)
    std::cout << c; // abcdefghijklmnopqrstuvwxyz
```

Erweiterung von ASCII: Unicode, UTF-8

- Internationalisierung von Software $\Rightarrow$ grosse Zeichensätze nötig. Heute daher üblich:
 - Zeichensatz *Unicode*: 150 Schriftsysteme, ca. 137'000 Zeichen
 - Codierungsstandard *UTF-8*: Abbildung Zeichen $\leftrightarrow$ Zahlen

Erweiterung von ASCII: Unicode, UTF-8

- Internationalisierung von Software $\Rightarrow$ grosse Zeichensätze nötig. Heute daher üblich:
 - Zeichensatz *Unicode*: 150 Schriftsysteme, ca. 137'000 Zeichen
 - Codierungsstandard *UTF-8*: Abbildung Zeichen $\leftrightarrow$ Zahlen
- UTF-8 ist eine *Multibyte-Codierung*: Häufig genutzte Zeichen (z.B. lat. Alphabet) belegen nur ein Byte, andere Zeichen bis zu vier

Erweiterung von ASCII: Unicode, UTF-8

- Internationalisierung von Software $\Rightarrow$ grosse Zeichensätze nötig. Heute daher üblich:
 - Zeichensatz *Unicode*: 150 Schriftsysteme, ca. 137'000 Zeichen
 - Codierungsstandard *UTF-8*: Abbildung Zeichen $\leftrightarrow$ Zahlen
- UTF-8 ist eine *Multibyte-Codierung*: Häufig genutzte Zeichen (z.B. lat. Alphabet) belegen nur ein Byte, andere Zeichen bis zu vier
- Länge einer Zeichen-Bytefolge wird dabei durch Bitmuster codiert

Nutzbare Bits	Bitmuster
7	0xxxxxxx
11	110xxxxx 10xxxxxx
16	1110xxxx 10xxxxxx 10xxxxxx
21	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Einige Unicode-Zeichen in UTF-8

Symbol	Codierung (jeweils 16 Bit)
ن	11101111 10101111 10111001

Einige Unicode-Zeichen in UTF-8

Symbol	Codierung (jeweils 16 Bit)
ع	11101111 10101111 10111001
☠	11100010 10011000 10100000
☎	11100010 10011000 10000011
☎	11100010 10011000 10011001

Einige Unicode-Zeichen in UTF-8

Symbol	Codierung (jeweils 16 Bit)
ع	11101111 10101111 10111001
☠	11100010 10011000 10100000
☎	11100010 10011000 10000011
☎	11100010 10011000 10011001
A	01000001

P.S.: Suchen Sie mal nach apple "unicode of death" P.S.: Unicode & UTF-8 sind nicht klausurrelevant

Caesar-Code

Ersetze jedes druckbare Zeichen in einem Text durch seinen Vor-Vor-Vorgänger.

' ' (32) → '|' (124)

'!' (33) → '}' (125)

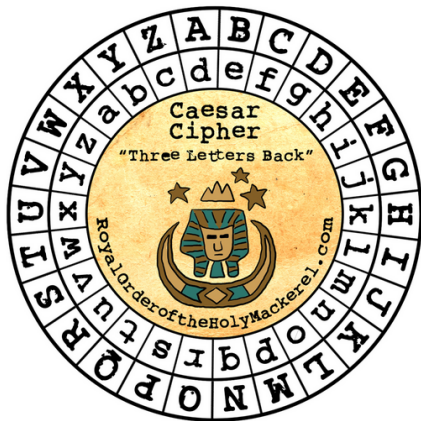
...

'D' (68) → 'A' (65)

'E' (69) → 'B' (66)

...

~ (126) → '{' (123)



```
// PRE:  divisor > 0
// POST: return the remainder of dividend / divisor
//        with 0 <= result < divisor
int mod(int dividend, int divisor);

// POST: if c is one of the 95 printable ASCII characters, c is
//        cyclically shifted s printable characters to the right
char shift(char c, int s) {
    if (c >= 32 && c <= 126) { // c is printable
        c = 32 + mod(c - 32 + s, 95);
    }

    return c;
}
```

```
// PRE:  divisor > 0
// POST: return the remainder of dividend / divisor
//        with 0 <= result < divisor
int mod(int dividend, int divisor);

// POST: if c is one of the 95 printable ASCII characters, c is
//        cyclically shifted s printable characters to the right
char shift(char c, int s) {
    if (c >= 32 && c <= 126) { // c is printable
        c = 32 + mod(c - 32 + s, 95);
    }

    return c;
}
```

"- 32" transforms interval [32, 126] to

[0, 94]

"mod(x , 95)" computes $x \bmod 95$ in [0, 94]

"32 +" transforms [0, 94] back to [32, 126]

```
// POST: Each character read from std::cin was shifted cyclically
//       by s characters and afterwards written to std::cout
void caesar(int s) {
    std::cin >> std::noskipws; // #include <ios>

    char next;
    while (std::cin >> next) {
        std::cout << shift(next, s);
    }
}
```

Leerzeichen und Zeilenumbrüche
sollen *nicht* ignoriert werden

```
// POST: Each character read from std::cin was shifted cyclically
//       by s characters and afterwards written to std::cout
void caesar(int s) {
    std::cin >> std::noskipws; // #include <ios>

    char next;
    while (std::cin >> next) {
        std::cout << shift(next, s);
    }
}
```

Konversion nach bool: liefert *false*
genau dann, wenn die Eingabe leer ist

```
// POST: Each character read from std::cin was shifted cyclically
//       by s characters and afterwards written to std::cout
void caesar(int s) {
    std::cin >> std::noskipws; // #include <ios>

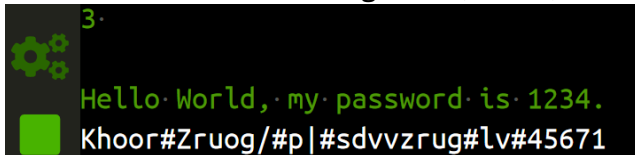
    char next;
    while (std::cin >> next) {
        std::cout << shift(next, s);
    }
}
```



Verschiebung druckbarer Zeichen um s

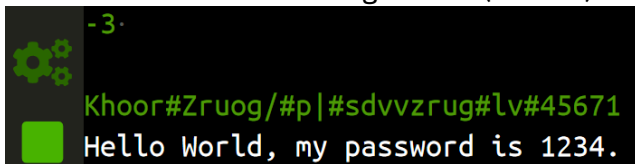
```
int main() {  
    int s;  
    std::cin >> s;  
  
    // Shift input by s  
    caesar(s);  
  
    return 0;  
}
```

Verschlüsseln: Verschiebung um n (hier: 3)



A terminal window with a dark background. The prompt is a green gear icon. The user enters '3'. The prompt is a green square icon. The user enters 'Hello World, my password is 1234.'. The output is 'Khoor#Zruog/#p|#sdvvzrug#lv#45671'.

Entschlüsseln: Verschiebung um $-n$ (hier: -3)



A terminal window with a dark background. The prompt is a green gear icon. The user enters '-3'. The prompt is a green square icon. The user enters 'Khoor#Zruog/#p|#sdvvzrug#lv#45671'. The output is 'Hello World, my password is 1234.'.

- Text „Sein oder nicht sein“ könnte als `vector<char>` repräsentiert werden

- Text „Sein oder nicht sein“ könnte als `vector<char>` repräsentiert werden
- Texte sind jedoch allgegenwärtig, daher existiert in der Standardbibliothek ein eigener Typ für sie: `std::string` (Zeichenkette)
- Benutzung benötigt `#include <string>`

Benutzung von `std::string`

- Deklaration und Initialisierung mittels Literal:

```
std::string text = "Essen ist fertig!"
```

Benutzung von `std::string`

- Deklaration und Initialisierung mittels Literal:

```
std::string text = "Essen ist fertig!"
```

- Mit variabler Länge initialisieren:

```
std::string text = std::string(n, 'a')
```

Benutzung von `std::string`

- Deklaration und Initialisierung mittels Literal:

```
std::string text = "Essen ist fertig!"
```

- Mit variabler Länge initialisieren:

```
std::string text = std::string(n, 'a')
```

- Texte vergleichen:

```
if (text1 == text2) ...
```

Benutzung von `std::string`

■ Grösse auslesen:

```
for (unsigned int i = 0; i < text.size(); ++i) ...
```

Benutzung von `std::string`

- Grösse auslesen:

```
for (unsigned int i = 0; i < text.size(); ++i) ...
```

- Einzelne Zeichen lesen:

```
if (text[0] == 'a') ... // or text.at(0)
```

Benutzung von `std::string`

- Grösse auslesen:

```
for (unsigned int i = 0; i < text.size(); ++i) ...
```

- Einzelne Zeichen lesen:

```
if (text[0] == 'a') ... // or text.at(0)
```

- Einzelne Zeichen schreiben:

```
text[0] = 'b'; // or text.at(0)
```


Benutzung von `std::string`

- Strings konkatenieren (zusammensetzen):

```
text = ":-";  
text += ")";  
assert(text == ":-)");
```

- Viele weitere Operationen, bei Interesse siehe <https://en.cppreference.com/w/cpp/string>

14. Vektoren II

Mehrdimensionale Vektoren/Vektoren von Vektoren

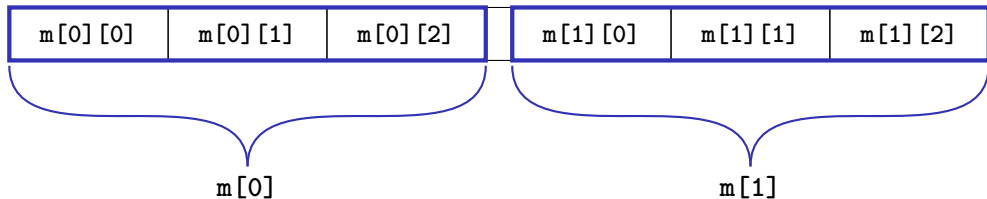
Mehrdimensionale Vektoren

- Zum Speichern von mehrdimensionalen Strukturen wie Tabellen, Matrizen, ...
- ...können *Vektoren von Vektoren* verwendet werden:

```
std::vector<std::vector<int>> m; // An empty matrix
```

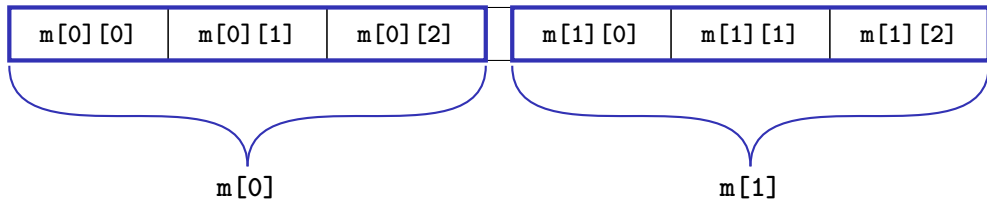
Mehrdimensionale Vektoren

Im Speicher: flach



Mehrdimensionale Vektoren

Im Speicher: flach



Im Kopf: Matrix

Spalten				
	0	1	2	
Zeilen	0	m[0][0]	m[0][1]	m[0][2]
	1	m[1][0]	m[1][1]	m[1][2]

Mehrdimensionale Vektoren: Initialisierung

Mittels Initialisierungslisten:

```
// the 4-by-4 identity matrix
std::vector<std::vector<std::int>> matrix = {
    {1, 0, 0, 0},
    {0, 1, 0, 0},
    {0, 0, 1, 0},
    {0, 0, 0, 1}
};

for (unsigned int i = 0; i < 4; ++i) assert(matrix[i][i] == 1);
```

Mehrdimensionale Vektoren: Initialisierung

Auf bestimmte Grösse füllen:

```
unsigned int rows = ...;  
unsigned int cols = ...;  
std::vector<std::vector<int>> m;  
m = std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

Mehrdimensionale Vektoren: Initialisierung

Auf bestimmte Grösse füllen:

```
unsigned int rows = ...;  
unsigned int cols = ...;  
std::vector<std::vector<int>> m;  
m = std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

`m` (Typ `std::vector<std::vector<int>>`) ist ein Vektor der Länge `rows`, dessen Elemente (Typ `std::vector<int>`) Vektoren der Länge `cols` sind, deren Elemente (Typ `int`) alles Einsen sind

Mehrdimensionale Vektoren: Initialisierung

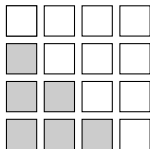
Auf bestimmte Grösse füllen:

```
unsigned int rows = ...;  
unsigned int cols = ...;  
std::vector<std::vector<int>> m;  
m = std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

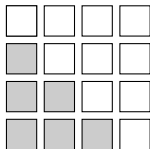
`m` (Typ `std::vector<std::vector<int>>`) ist ein Vektor der Länge `rows`, dessen Elemente (Typ `std::vector<int>`) Vektoren der Länge `cols` sind, deren Elemente (Typ `int`) alles Einsen sind

(Es gibt noch viele weitere Wege, Vektoren zu initialisieren)

Matrix: Summe des unteren Dreiecks



Matrix: Summe des unteren Dreiecks



```
int lower_triangle_sum(const std::vector<std::vector<int>>& matrix) {  
    int sum = 0;  
    for (unsigned int row = 0; row < matrix.size(); ++row) {  
        for (unsigned int column = 0; column < row; ++column) {  
            sum += matrix[row][column];  
        }  
    }  
    return sum;  
}
```

Mehrdimensionale Vektoren

nicht-quadratische und sogar nicht-rechteckige 2-dimensionale Vektoren sind möglich:

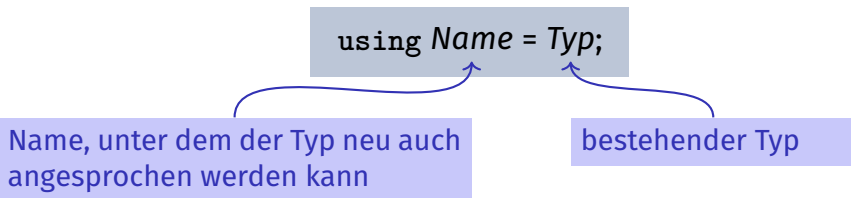
```
std::vector<std::vector<std::string>> cantons = {  
    {"ZH", "BE", "SG", "GE", "LU", "SO", "NE", "ZG"},  
    {"FR", "VD", "VS", "JU"},  
    {"AI", "OW", "NW", "AR", "BS", "BL"}  
};  
  
assert(cantons[1][2] == "VS");  
assert(cantons[1].size() == 4);
```

Mehrdimensionale Vektoren und Typ-Aliasse

- Auch möglich: Vektoren von Vektoren von Vektoren von ...:
`std::vector<std::vector<std::vector<...>>>`
- Typnamen können offensichtlich laaaaaaaang werden

Mehrdimensionale Vektoren und Typ-Alias

- Auch möglich: Vektoren von Vektoren von Vektoren von ...:
`std::vector<std::vector<std::vector<...>>>`
- Typnamen können offensichtlich laaaaaaang werden
- Dann hilft die Deklaration eines *Typ-Alias*:



Typ-Alias: Beispiel

```
#include <iostream>
#include <vector>
using imatrix = std::vector<std::vector<int>>>;

// POST: Matrix 'm' was output to standard output
void print(const imatrix& m);

int main() {
    imatrix m = ...;
    print(m);
}
```

Erinnerung: `const`-Referenz für Effizienz (keine Kopie) und Sicherheit (unveränderlich)

Keyword auto

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

```
auto x = 2 + 3;
```

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

```
auto x = 2 + 3; // type of x is int
```

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

```
auto x = 2 + 3; // type of x is int  
std::cout << x[2];
```

Keyword auto

```
std::vector<std::vector<int>> all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

stattdessen auch möglich:

```
auto all_ones_matrix =  
    std::vector<std::vector<int>>(rows, std::vector<int>(cols, 1));
```

keyword auto erlaubt kompaktere Schreibweise:
Platzhalter für den Typ der deklarierten Variable
⇒ Compiler leitet Typ vom Initialisierungswert ab

```
auto x = 2 + 3; // type of x is int  
std::cout << x[2]; // compiler error
```

15. Rekursion I

Mathematische Rekursion, Terminierung, der Aufrufstapel, Beispiele

Mathematische Rekursion

- Viele mathematische Funktionen sind sehr natürlich *rekursiv* definierbar

Mathematische Rekursion

- Viele mathematische Funktionen sind sehr natürlich *rekursiv* definierbar
- Das heisst, die Funktion erscheint in ihrer eigenen Definition

$$n! = \begin{cases} 1, & \text{falls } n \leq 1 \\ n \cdot (n-1)!, & \text{andernfalls} \end{cases}$$

Rekursion in C++: Genauso!

$$n! = \begin{cases} 1, & \text{falls } n \leq 1 \\ n \cdot (n-1)!, & \text{andernfalls} \end{cases}$$

```
// POST: return value is n!  
unsigned int fac(unsigned int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

Unendliche Rekursion

- ist so schlecht wie eine Endlosschleife ...

Unendliche Rekursion

- ist so schlecht wie eine Endlosschleife ...
- ...nur noch schlechter: „verbrennt“ Zeit *und* Speicher

Unendliche Rekursion

- ist so schlecht wie eine Endlosschleife ...
- ...nur noch schlechter: „verbrennt“ Zeit *und* Speicher

```
void f() {  
    f() // f() → f() → ... → stack overflow  
}
```

Unendliche Rekursion

- ist so schlecht wie eine Endlosschleife ...
- ...nur noch schlechter: „verbrennt“ Zeit *und* Speicher

```
void f() {  
    f() // f() → f() → ... → stack overflow  
}
```

Ein Euro ist ein Euro.

Wim Duisenberg, erster Präsident der EZB

Rekursive Funktionen: Terminierung

Wie bei Schleifen brauchen wir garantierten Fortschritt Richtung einer Abbruchbedingung ($\approx$ Basisfall)

Beispiel `fac(n)`:

- Rekursion endet falls $n \leq 1$
- Rekursiver Aufruf mit neuem Argument $< n$
- Abbruchbedingung wird daher garantiert erreicht

```
unsigned int fac(  
    unsigned int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```


Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}  
  
...  
std::cout << fac(4);
```

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

```
...  
std::cout << fac(4);
```

fac(4)

Aufruf von fac(4)

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}  
  
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$

Aufruf von $\text{fac}(4) \rightsquigarrow$ Initialisierung des formalen Arguments n

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}  
  
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$

Auswertung des Rückgabedruckausdrucks

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}  
  
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$
 $\hookrightarrow \text{fac}(n - 1)$

Rekursiver Aufruf mit Argument $n - 1 = 4 - 1 = 3$

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}  
  
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$

$\hookrightarrow \text{fac}(n - 1) \rightsquigarrow \text{int } n = 3$

Initialisierung des formalen Arguments n

Rekursive Funktionen: Auswertung

```
int fac(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * fac(n-1);  
}
```

```
...  
std::cout << fac(4);
```

$\text{fac}(4) \rightsquigarrow \text{int } n = 4$

$\hookrightarrow \text{fac}(n - 1) \rightsquigarrow \text{int } n = 3$

$\vdots$

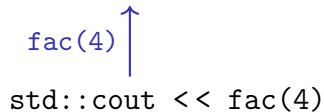
Jeder Aufruf von fac arbeitet mit seinem eigenen n

Der Aufrufstapel

```
std::cout << fac(4)
```


Der Aufrufstapel

Bei jedem Funktionsaufruf:

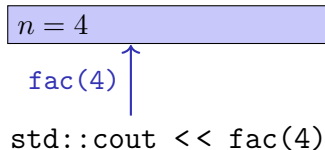


```
fac(4)  
std::cout << fac(4)
```

Der Aufrufstapel

Bei jedem Funktionsaufruf:

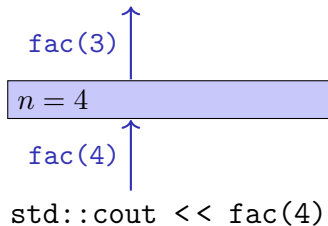
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

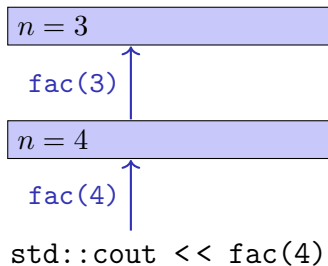
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

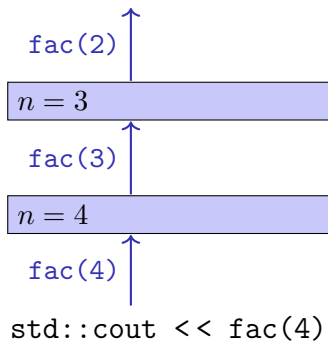
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

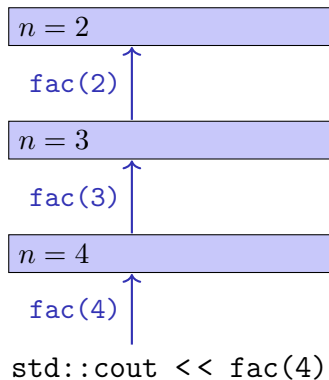
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

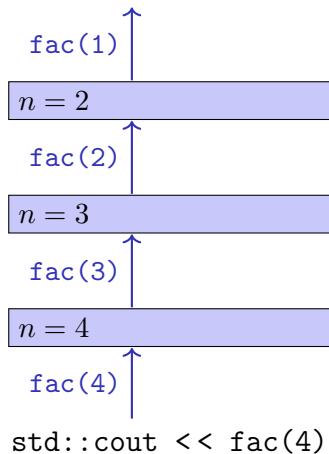
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

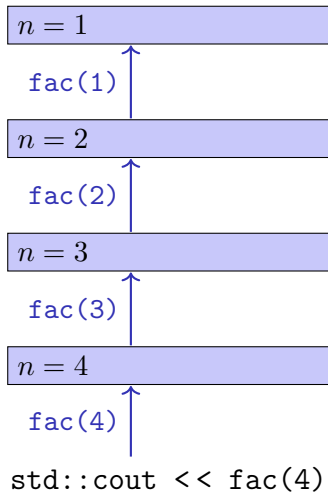
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

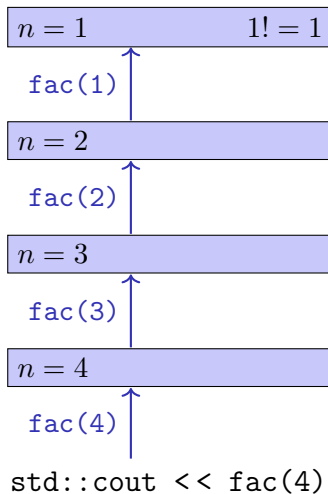
- Wert des Aufrufarguments kommt auf einen Stapel



Der Aufrufstapel

Bei jedem Funktionsaufruf:

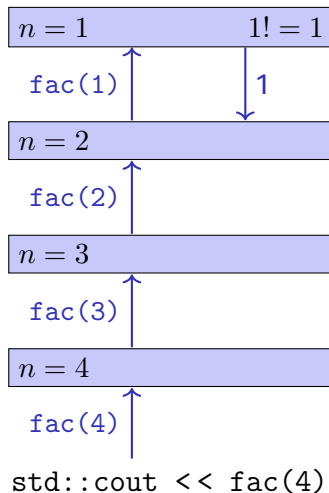
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet



Der Aufrufstapel

Bei jedem Funktionsaufruf:

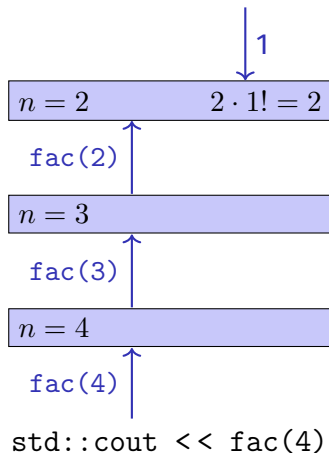
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

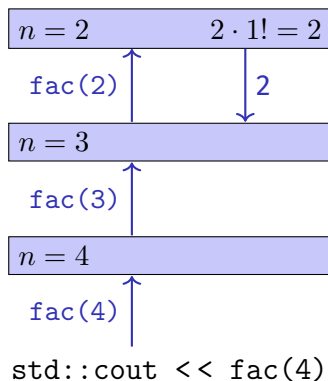
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

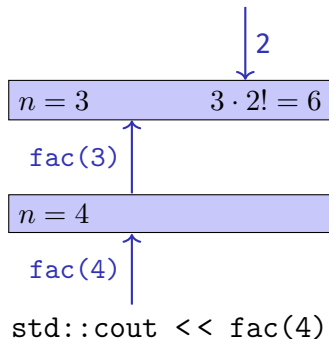
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

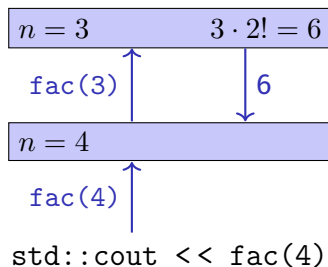
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

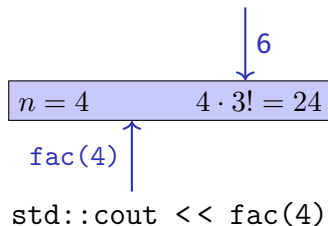
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

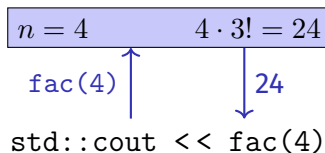
- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht



Der Aufrufstapel

Bei jedem Funktionsaufruf:

- Wert des Aufrufarguments kommt auf einen Stapel
- Es wird immer mit dem obersten Wert gearbeitet
- Am Ende des Aufrufs wird der oberste Wert wieder vom Stapel gelöscht

 24
`std::cout << fac(4)`

Euklidischer Algorithmus

- findet den grössten gemeinsamen Teiler $\gcd(a, b)$ zweier natürlicher Zahlen a und b

Euklidischer Algorithmus

- findet den grössten gemeinsamen Teiler $\gcd(a, b)$ zweier natürlicher Zahlen a und b
- basiert auf folgender mathematischen Rekursion:

$$\gcd(a, b) = \begin{cases} a, & \text{falls } b = 0 \\ \gcd(b, a \bmod b), & \text{andernfalls} \end{cases}$$

Euklidischer Algorithmus in C++

$$\text{gcd}(a, b) = \begin{cases} a, & \text{falls } b = 0 \\ \text{gcd}(b, a \bmod b), & \text{andernfalls} \end{cases}$$

```
unsigned int gcd(unsigned int a, unsigned int b) {  
    if (b == 0)  
        return a;  
    else  
        return gcd(b, a % b);  
}
```

Euklidischer Algorithmus in C++

$$\text{gcd}(a, b) = \begin{cases} a, & \text{falls } b = 0 \\ \text{gcd}(b, a \bmod b), & \text{andernfalls} \end{cases}$$

```
unsigned int gcd(unsigned int a, unsigned int b) {  
    if (b == 0)  
        return a;  
    else  
        return gcd(b, a % b);  
}
```

Terminierung: $a \bmod b < b$, also wird b in jedem rekursiven Aufruf kleiner

Bitstrings generieren

alle Bitstrings der Länge n generieren

Bitstrings generieren

alle Bitstrings der Länge n generieren

Beispiel für $n = 6$:

000000

000001

000010

000011

...

111100

111101

111110

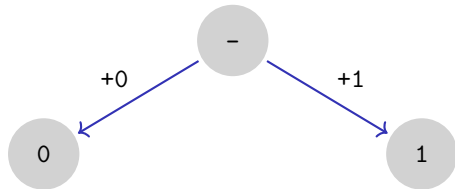
111111

Bitstrings generieren: Idee

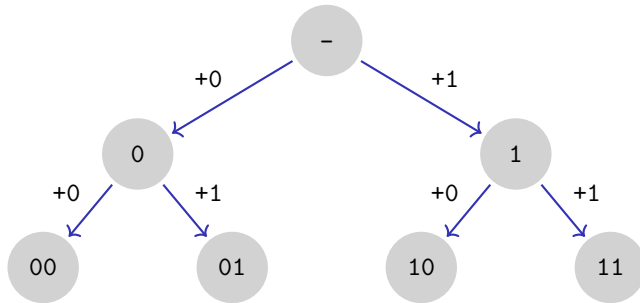
Bitstrings generieren: Idee



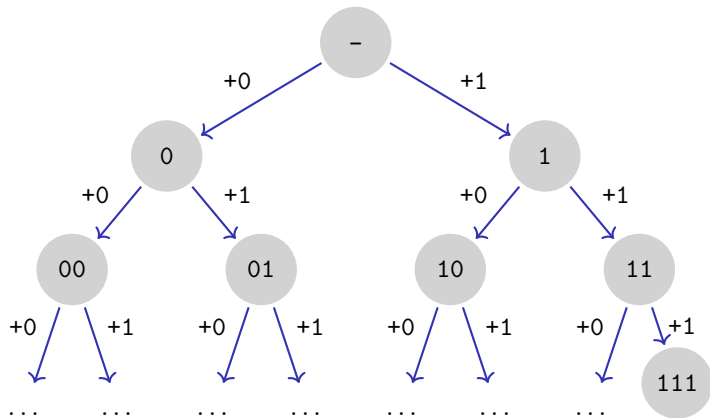
Bitstrings generieren: Idee



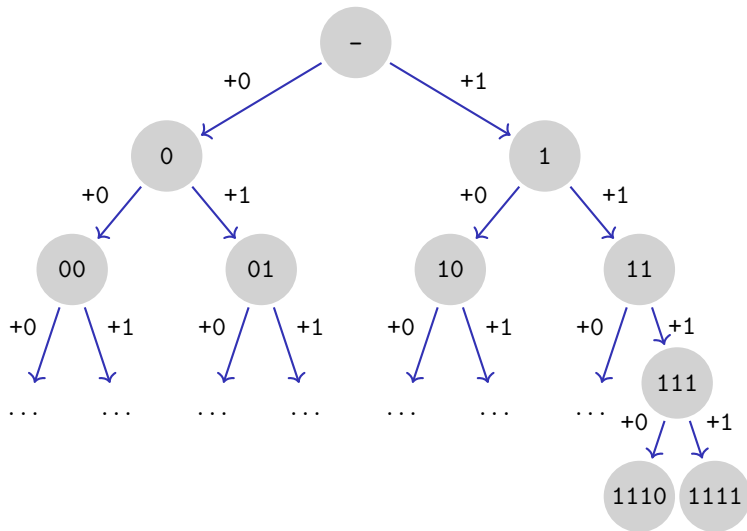
Bitstrings generieren: Idee



Bitstrings generieren: Idee



Bitstrings generieren: Idee



Bitstrings generieren: 1. Version

```
void all_bitstrings(unsigned int n, std::string bitstring) {  
    if (bitstring.size() == n) {  
        std::cout << bitstring << "\n";  
    } else {  
        all_bitstrings(n, bitstring + "0");  
        all_bitstrings(n, bitstring + "1");  
    }  
}
```

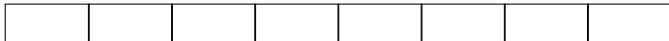
Bitstrings generieren: 1. Version

```
void all_bitstrings(unsigned int n, std::string bitstring) {  
    if (bitstring.size() == n) {  
        std::cout << bitstring << "\n";  
    } else {  
        all_bitstrings(n, bitstring + "0");  
        all_bitstrings(n, bitstring + "1");  
    }  
}
```

Aufruf mit `all_bitstrings(6, "")`;

Bitstrings: Aufrufstapel und Speicher

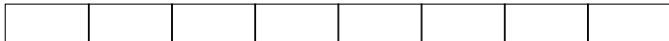
```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



all_bs(3, "")

Bitstrings: Aufrufstapel und Speicher

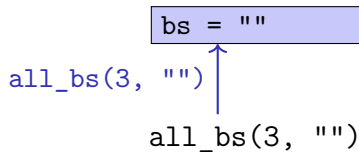
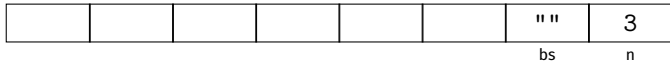
```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



all_bs(3, "")
 ↑
all_bs(3, "")

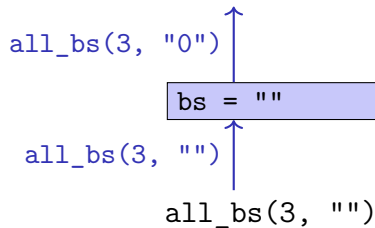
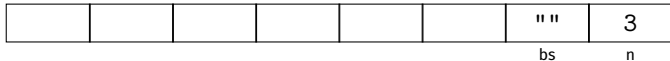
Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



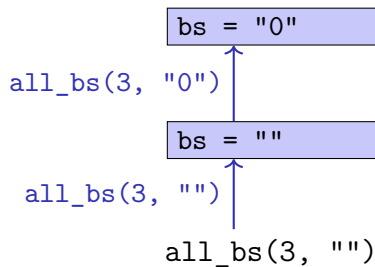
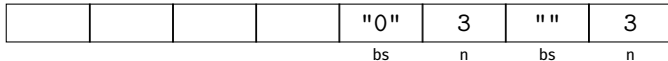
Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



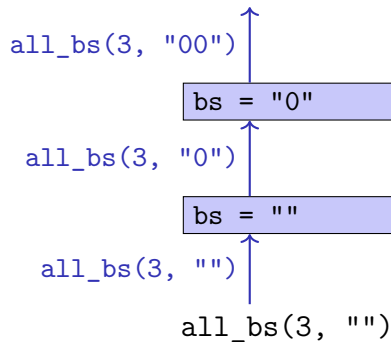
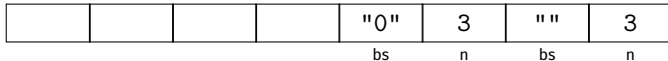
Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



Bitstrings: Aufrufstapel und Speicher

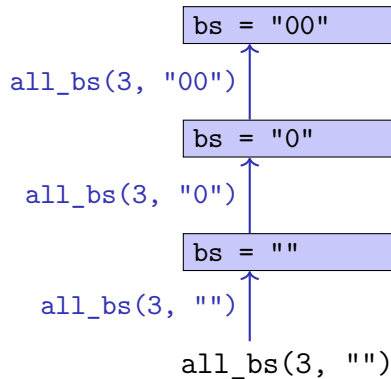
```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```



Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```

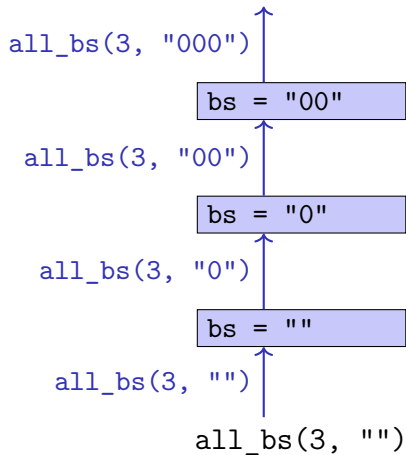
		"00"	3	"0"	3	" "	3
		bs	n	bs	n	bs	n



Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```

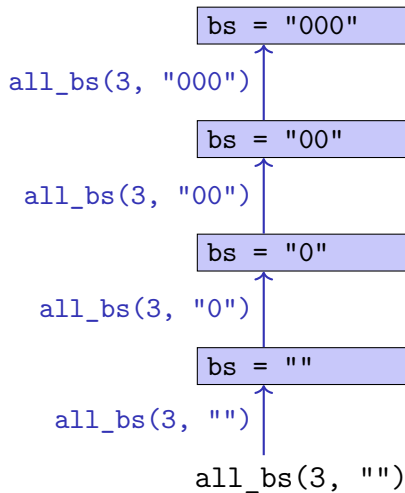
		"00"	3	"0"	3	" "	3
		bs	n	bs	n	bs	n



Bitstrings: Aufrufstapel und Speicher

```
void all_bs(unsigned int n, std::string bs) {  
    if (bs.size() == n) {  
        std::cout << bs << "\n";  
    } else {  
        all_bs(n, bs + "0");  
        all_bs(n, bs + "1");  
    }  
}
```

"000"	3	"00"	3	"0"	3	" "	3
bs	n	bs	n	bs	n	bs	n



Bitstrings generieren: 2. Version

```
void all_bitstrings(std::string& bitstring, unsigned int i) {
    if (i == bitstring.size()) {
        std::cout << bitstring << "\n";
    } else {
        bitstring[i] = '0';
        all_bitstrings(bitstring, i+1);
        bitstring[i] = '1';
        all_bitstrings(bitstring, i+1);
    }
}

...
std::string bitstring = std::string(3, '*');
all_bitstrings(bitstring, 0);
```

Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



all_bs(bs, 0)

Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



bs

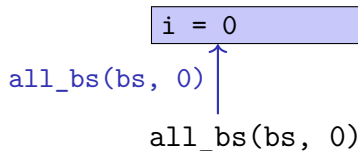
all_bs(bs, 0)

Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```

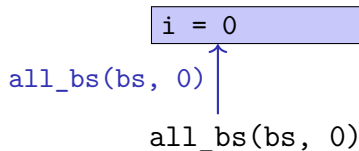


bs



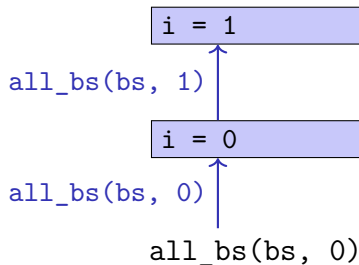
Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



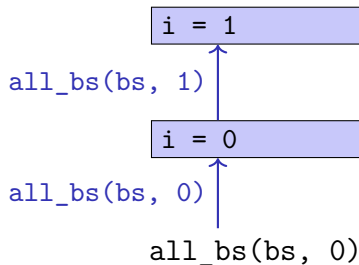
Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



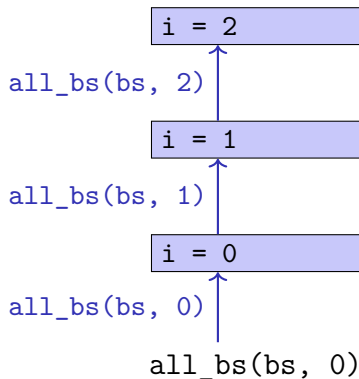
Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



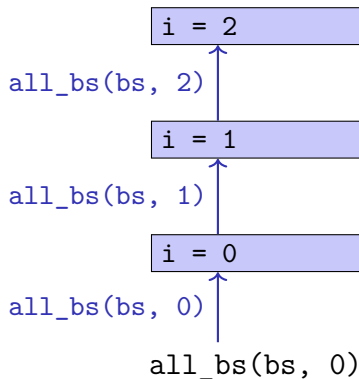
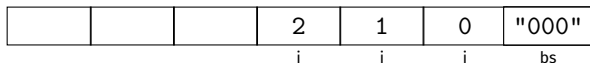
Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



Bitstrings: Aufrufstapel und Speicher

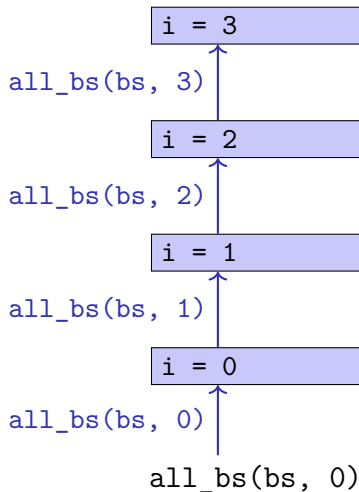
```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```



Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```

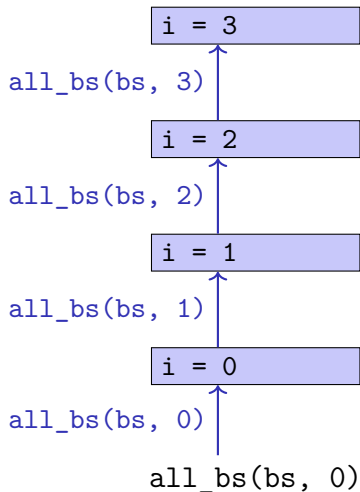
			2	1	0	"000"
			i	i	i	bs



Bitstrings: Aufrufstapel und Speicher

```
void bs(std::string& bs, unsigned int i) {  
    if (i == bs.size()) {  
        std::cout << bs << "\n";  
    } else {  
        bs[i] = '0';  
        all_bs(bs, i+1);  
        bs[i] = '1';  
        all_bs(bs, i+1);  
    }  
}  
...  
std::string bs = std::string(3, '*');  
all_bs(bs, 0);
```

		3	2	1	0	"000"
		i	i	i	i	bs



Die Macht der Rekursion

Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.

- n -Damen-Problem
- Die Türme von Hanoi
- Sudoku-Löser
- Parsen von Ausdrücken
- Umgekehrte Aus- oder Eingabe
- Suchen in Bäumen
- Divide-And-Conquer (z.B. Sortieren)

16. Rekursion II

Decrease and Conquer vs. Divide and Conquer, Rekursion vs. Iteration

Vektorsumme

altbekanntes Problem:

Summe aller Einträge eines Vektors `std::vector<int> v`

Vektorsumme

altbekanntes Problem:

Summe aller Einträge eines Vektors `std::vector<int> v`

einfache iterative Lösung:

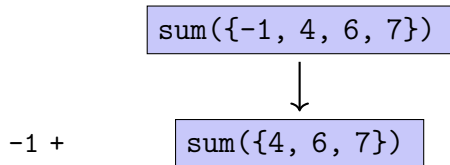
```
int sum(const std::vector<int>& v) {  
    int s = 0;  
    for (unsigned int i = 0; i < v.size(); ++i) {  
        s += v.at(i);  
    }  
    return s;  
}
```

Vektorsumme: Variante 1

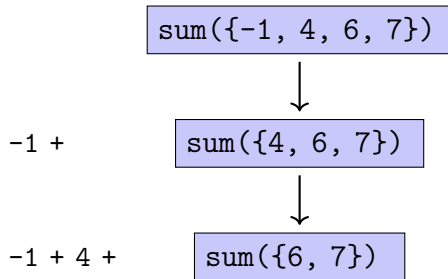
Vektorsumme: Variante 1

```
sum({-1, 4, 6, 7})
```

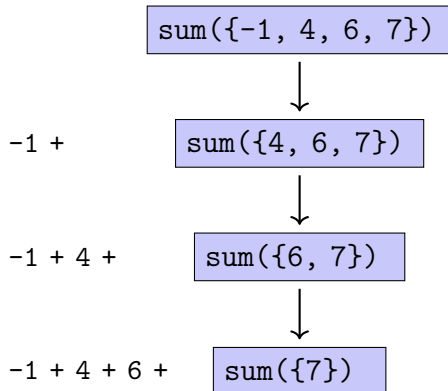
Vektorsumme: Variante 1



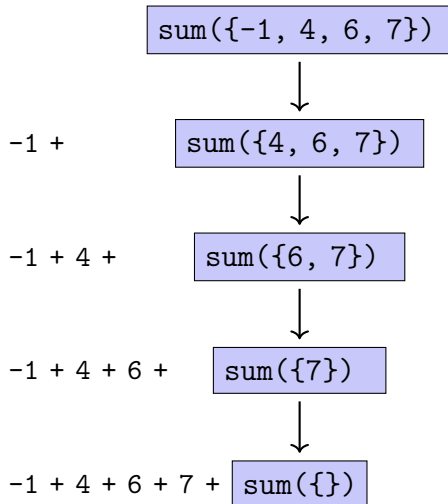
Vektorsumme: Variante 1



Vektorsumme: Variante 1



Vektorsumme: Variante 1



Vektorsumme: Variante 1

```
int sum(const std::vector<int>& v, unsigned int from) {  
    if (from >= v.size())  
        return 0;  
    else  
        return v.at(from) + sum(v, from + 1);  
}
```

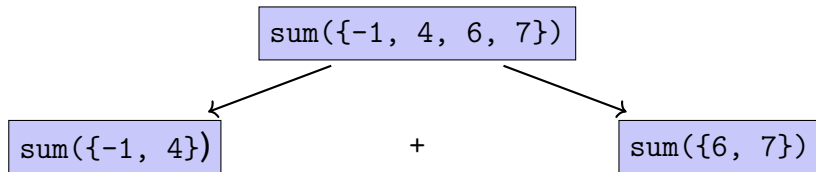
Aufruf mit z.B. `sum({-1, 4, 6, 7}, 0);`

Vektorsumme: Variante 2

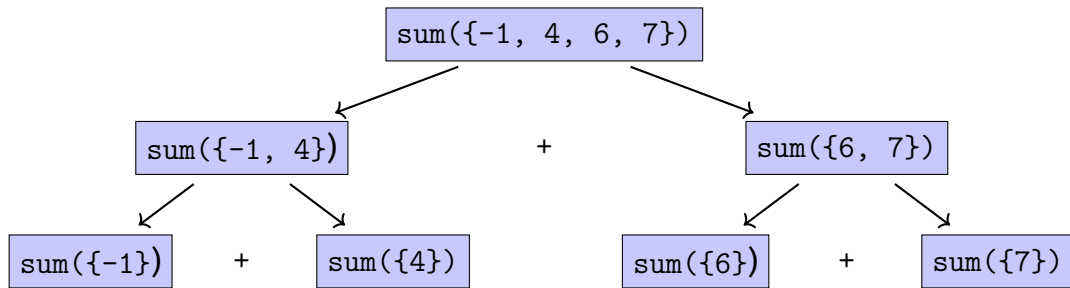
Vektorsumme: Variante 2

```
sum({-1, 4, 6, 7})
```


Vektorsumme: Variante 2



Vektorsumme: Variante 2

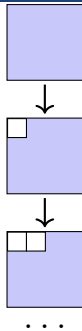


Vektorsumme: Variante 2

```
using uint = unsigned int;
int sum(const std::vector<int>& v, uint from, uint to) {
    if (from > to)
        return 0;
    else if (from == to)
        return v.at(from);
    else {
        uint middle = (from + to) / 2;
        return sum(v, from, middle) + sum(v, middle + 1, to);
    }
}
```

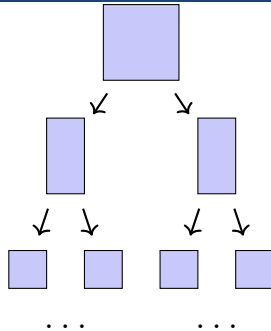
Aufruf mit z.B. `sum({-1, 4, 6, 7}, 0, 3);`

Rekursive Strategien



Decrease and Conquer

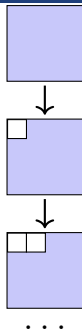
- ein Element entfernen,
ein rekursiver Aufruf auf Rest



Divide and Conquer

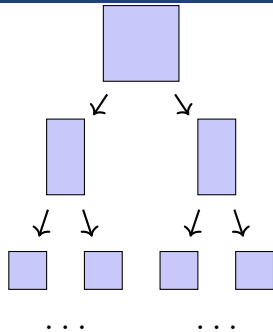
- halbieren der Elemente,
ein rekursiver Aufruf pro Hälfte

Rekursive Strategien



Decrease and Conquer

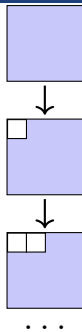
- ein Element entfernen,
ein rekursiver Aufruf auf Rest
- begrenzt parallelisierbar



Divide and Conquer

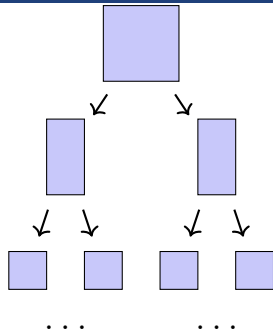
- halbieren der Elemente,
ein rekursiver Aufruf pro Hälfte
- oft gut parallelisierbar

Rekursive Strategien



Decrease and Conquer

- ein Element entfernen,
ein rekursiver Aufruf auf Rest
- begrenzt parallelisierbar
- grosse Rekursionstiefe



Divide and Conquer

- halbieren der Elemente,
ein rekursiver Aufruf pro Hälfte
- oft gut parallelisierbar
- geringere Rekursionstiefe

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Oft sind rekursive Formulierungen einfacher, aber meistens auch weniger effizient.

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Oft sind rekursive Formulierungen einfacher, aber meistens auch weniger effizient.

- Overhead durch Funktionsaufrufe

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Oft sind rekursive Formulierungen einfacher, aber meistens auch weniger effizient.

- Overhead durch Funktionsaufrufe
- Stack Overflow bei zu grosser Rekursionstiefe
(undefined behavior, typischerweise Segmentation Fault)

Rekursion und Iteration

Rekursion kann *immer* simuliert werden durch

- Iteration (Schleifen)
- expliziten „Aufrufstapel“ (z.B. mittels eines Vektors)

Oft sind rekursive Formulierungen einfacher, aber meistens auch weniger effizient.

- Overhead durch Funktionsaufrufe
- Stack Overflow bei zu grosser Rekursionstiefe
(undefined behavior, typischerweise Segmentation Fault)
- potentielle Mehrfachberechnung von Zwischenresultaten

Fibonacci-Zahlen

$$F_n := \begin{cases} 0, & \text{falls } n = 0 \\ 1, & \text{falls } n = 1 \\ F_{n-1} + F_{n-2}, & \text{falls } n > 1 \end{cases}$$

Fibonacci-Zahlen

$$F_n := \begin{cases} 0, & \text{falls } n = 0 \\ 1, & \text{falls } n = 1 \\ F_{n-1} + F_{n-2}, & \text{falls } n > 1 \end{cases}$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 ...

Fibonacci-Zahlen in Zürich



Fibonacci-Zahlen in C++

$$F_n := \begin{cases} 0, & \text{falls } n = 0 \\ 1, & \text{falls } n = 1 \\ F_{n-1} + F_{n-2}, & \text{falls } n > 1 \end{cases}$$

```
unsigned int fib(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n-1) + fib(n-2); // n > 1  
}
```

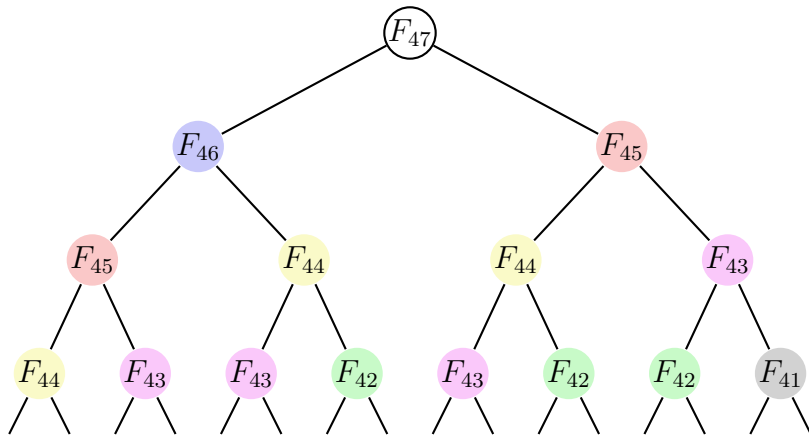
Fibonacci-Zahlen in C++

Laufzeit

`fib(50)` dauert „ewig“, denn es berechnet
 F_{48} 2-mal, F_{47} 3-mal, F_{46} 5-mal, F_{45} 8-mal, F_{44} 13-mal,
 F_{43} 21-mal ... F_1 ca. 10^9 mal (!)

```
unsigned int fib(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n-1) + fib(n-2); // n > 1  
}
```


Fibonacci visualisiert



Mehrfachberechnung selber Werte = Mehrfachlösung selber Teilprobleme

Schnelle (iterative) Fibonacci-Zahlen

Idee:

- Berechne jede Fibonacci-Zahl nur einmal, in der Reihenfolge $F_0, F_1, F_2, \dots, F_n$

Schnelle (iterative) Fibonacci-Zahlen

Idee:

- Berechne jede Fibonacci-Zahl nur einmal, in der Reihenfolge $F_0, F_1, F_2, \dots, F_n$
- Speichere jeweils die zwei letzten berechneten Fibonacci-Zahlen (Variablen `fib1` und `fib2`)

Schnelle (iterative) Fibonacci-Zahlen

Idee:

- Berechne jede Fibonacci-Zahl nur einmal, in der Reihenfolge $F_0, F_1, F_2, \dots, F_n$
- Speichere jeweils die zwei letzten berechneten Fibonacci-Zahlen (Variablen `fib1` und `fib2`)
- Berechne die nächste Zahl als Summe von `fib1` und `fib2`

Schnelle (iterative) Fibonacci-Zahlen

Idee:

- Berechne jede Fibonacci-Zahl nur einmal, in der Reihenfolge $F_0, F_1, F_2, \dots, F_n$
- Speichere jeweils die zwei letzten berechneten Fibonacci-Zahlen (Variablen `fib1` und `fib2`)
- Berechne die nächste Zahl als Summe von `fib1` und `fib2`

Kann rekursiv und iterativ implementiert werden, letzteres ist einfacher/direkter

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

$$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$$

fib2

fib1

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

$$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$$

fib2

fib1

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

$$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$$

fib2

fib1

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

$$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$$

fib2

fib1

Schnelle Fibonacci-Zahlen in C++

```
unsigned int fibonacci(unsigned int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    unsigned int fib2 = 0; // value two before  
    unsigned int fib1 = 1; // value one before  
    unsigned int fib;      // current value  
  
    for (unsigned int i = 2; i <= n; ++i) {  
        fib = fib2 + fib1; // i-th fibonacci number  
        fib2 = fib1;  
        fib1 = fib;  
    }  
    return fib;  
}
```

sehr schnell auch bei fib(50)

$(F_{i-2}, F_{i-1}) \longrightarrow (F_{i-1}, F_i)$

fib2

fib1

- Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.

- Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.
- Rekursive Lösungen sind oftmals einfacher und eleganter...

- Einige Probleme scheinen ohne Rekursion kaum lösbar zu sein. Mit Rekursion werden sie plötzlich deutlich einfacher lösbar.
- Rekursive Lösungen sind oftmals einfacher und eleganter...
- ...aber iterative Lösungen meist effizienter.

Überladen von Funktionen

```
// iterative solution
```

```
int sum(const std::vector<int>& v) {  
    ...  
}
```

```
// recursive solution: decrease and conquer
```

```
int sum(const std::vector<int>&v, uint from) {  
    ...  
}
```

```
// recursive solution: divide and conquer
```

```
int sum(const std::vector<int>&v, uint from, uint to) {  
    ...  
}
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

```
std::cout << sq(3);
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

```
std::cout << sq(3);    // Compiler wählt f2
std::cout << sq(1.414);
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

```
std::cout << sq(3);    // Compiler wählt f2
std::cout << sq(1.414); // Compiler wählt f1
std::cout << pow(2);
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

```
std::cout << sq(3);    // Compiler wählt f2
std::cout << sq(1.414); // Compiler wählt f1
std::cout << pow(2);   // Compiler wählt f4
std::cout << pow(3,3);
```

Funktionsüberladung

- Eine Funktion ist bestimmt durch Namen, Typen, Anzahl und Reihenfolge der Argumente

```
double sq(double x) { ... }           // f1
int sq(int x) { ... }                 // f2
int pow(int b, int e) { ... }         // f3
int pow(int e) { return pow (2,e); } // f4
```

- Der Compiler wählt bei einem Funktionsaufruf automatisch die Funktion, welche „am besten passt“

```
std::cout << sq(3);    // Compiler wählt f2
std::cout << sq(1.414); // Compiler wählt f1
std::cout << pow(2);    // Compiler wählt f4
std::cout << pow(3,3);  // Compiler wählt f3
```

17. Custom Data Types I

Rationale Zahlen, Struct-Definition, Operatorüberladung

Rechnen mit rationalen Zahlen

- Rationale Zahlen ($\mathbb{Q}$) sind von der Form $\frac{n}{d}$ mit n und d in $\mathbb{Z}$
- C++ hat keinen „eingebauten“ Typ für rationale Zahlen

Rechnen mit rationalen Zahlen

- Rationale Zahlen ($\mathbb{Q}$) sind von der Form $\frac{n}{d}$ mit n und d in $\mathbb{Z}$
- C++ hat keinen „eingebauten“ Typ für rationale Zahlen

Ziel

Wir bauen uns selbst einen C++-Typ für rationale Zahlen!



Vision

```
// input
std::cout << "Rational number r =? ";
rational r;
std::cin >> r;
std::cout << "Rational number s =? ";
rational s;
std::cin >> s;

// computation and output
std::cout << "Sum is " << r + s << ".\n";
```

Ein erstes Struct

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Ein erstes Struct

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Member-Variable (**n**umerator)



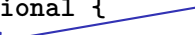
Member-Variable (**d**enominator)



Ein erstes Struct

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Member-Variable



Member-Variable

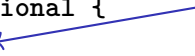


- struct definiert einen neuen Typ

Ein erstes Struct

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Member-Variable



Member-Variable



- struct definiert einen neuen Typ
- Formaler Wertebereich: *kartesisches Produkt* der Wertebereiche existierender Typen

Ein erstes Struct

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Member-Variable

Member-Variable

- struct definiert einen neuen Typ
- Formaler Wertebereich: *kartesisches Produkt* der Wertebereiche existierender Typen
- Echter Wertebereich: $\text{rational} \subsetneq \text{int} \times \text{int}$.

Struct-Definitionen: Beispiele

```
struct rational_vector_3 {  
    rational x;  
    rational y;  
    rational z;  
};
```

Zugrundeliegende Typen können fundamentale aber auch *benutzerdefinierte* Typen sein.

Struct-Definitionen: Beispiele

```
struct extended_int {  
    // represents value if is_positive==true  
    // and -value otherwise  
    unsigned int value;  
    bool is_positive;  
};
```

Die zugrundeliegenden Typen können natürlich auch *verschieden* sein.

Eingabe

```
// Input r
rational r;
std::cout << "Rational number r is: \n";
std::cout << " numerator =? ";
std::cin >> r.n;
std::cout << " denominator =? ";
std::cin >> r.d;

// Input s the same way
rational s;
...
```

Addition

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};  
  
rational add(rational a, rational b){  
    rational result;  
    result.n = a.n * b.d + a.d * b.n;  
    result.d = a.d * b.d;  
    return result;  
}
```

$$\frac{r_n}{r_d} := \frac{a_n}{a_d} + \frac{b_n}{b_d} = \frac{a_n \cdot b_d + a_d \cdot b_n}{a_d \cdot b_d}$$

Structs: Initialisierung und Zuweisung

`rational s;` ← Member-Variablen uninitialisiert (wird sich bald ändern)

`rational t = {1, 5};`

`rational u = t;`

`t = u;`

`rational v = add(u,t);`

Structs: Initialisierung und Zuweisung

```
rational s;
```

```
rational t = {1, 5};
```

← *Memberweise Initialisierung:*
t.n = 1, t.d = 5

```
rational u = t;
```

```
t = u;
```

```
rational v = add(u,t);
```

Structs: Initialisierung und Zuweisung

```
rational s;
```

```
rational t = {1, 5};
```

```
rational u = t; ← Memberweise Kopie
```

```
t = u;
```

```
rational v = add(u,t);
```

Structs: Initialisierung und Zuweisung

```
rational s;
```

```
rational t = {1, 5};
```

```
rational u = t;
```

```
t = u; ← Memberweise Kopie
```

```
rational v = add(u,t);
```

Structs: Initialisierung und Zuweisung

```
rational s;
```

```
rational t = {1, 5};
```

```
rational u = t;
```

```
t = u;
```

```
rational v = add(u,t); ← Memberweise Kopie
```

Vision in Reichweite ...

```
// computation
const rational t = add(r, s);

// output
std::cout << "Sum is " << t.n << "/" << t.d << ".\n";
```


Benutzerdefinierte Operatoren

Statt

```
rational t = add(r, s);
```

würden wir lieber

```
rational t = r + s;
```

schreiben.

Benutzerdefinierte Operatoren

Statt

```
rational t = add(r, s);
```

würden wir lieber

```
rational t = r + s;
```

schreiben.

Das geht mit *Überladen von Operatoren*.

Operator-Überladung

- Operatoren sind spezielle Funktionen und können auch überladen werden
- Name des Operators *op*:

```
operatorop
```

rational addieren, bisher

```
// POST: return value is the sum of a and b
rational add(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}

...
const rational t = add(r, s);
```

rational addieren, neu

```
// POST: return value is the sum of a and b
rational operator+(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}
...
const rational t = r + s;
```

rational addieren, neu

```
// POST: return value is the sum of a and b
rational operator+(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}
...
const rational t = r + s;
```



Infix-Notation

rational addieren, neu

```
// POST: return value is the sum of a and b
rational operator+(rational a, rational b)
{
    rational result;
    result.n = a.n * b.d + a.d * b.n;
    result.d = a.d * b.d;
    return result;
}
...
const rational t = operator+(r, s);
```



Äquivalent, aber unpraktisch: funktionale Notation

Unäres Minus

Nur ein Argument:

```
// POST: return value is -a
rational operator-(rational a)
{
    a.n = -a.n;
    return a;
}
```


Arithmetische Zuweisungen

Wir wollen z.B. schreiben

```
rational r;  
r.n = 1; r.d = 2;           // 1/2
```

```
rational s;  
s.n = 1; s.d = 3;           // 1/3
```

```
r += s;  
std::cout << r.n << "/" << r.d;    // 5/6
```

Operator +=

```
rational& operator+=(rational& a, rational b)
{
    a.n = a.n * b.d + a.d * b.n;
    a.d *= b.d;
    return a;
}
```

Operator +=

```
rational& operator+=(rational& a, rational b)
{
    a.n = a.n * b.d + a.d * b.n;
    a.d *= b.d;
    return a;
}
```

- Der L-Wert a wird um den Wert von b erhöht und als L-Wert zurückgegeben.

Structs vergleichen?

Für jeden fundamentalen Typ (`int`, `double`, ...) gibt es die Vergleichsoperatoren `==` und `!=`, aber nicht für Structs! Warum?

Structs vergleichen?

Für jeden fundamentalen Typ (`int`, `double`, ...) gibt es die Vergleichsoperatoren `==` und `!=`, aber nicht für Structs! Warum?

- Memberweiser Vergleich ergibt im allgemeinen keinen Sinn,...

Structs vergleichen?

Für jeden fundamentalen Typ (`int`, `double`, ...) gibt es die Vergleichsoperatoren `==` und `!=`, aber nicht für Structs! Warum?

- Memberweiser Vergleich ergibt im allgemeinen keinen Sinn,...

- ...denn dann wäre z.B. $\frac{2}{3} \neq \frac{4}{6}$

Vergleichsoperatoren

können auch definiert werden, so dass sie das „Richtige“ machen:

Vergleichsoperatoren

können auch definiert werden, so dass sie das „Richtige“ machen:

```
// POST: returns true iff a == b
bool operator==(rational a, rational b)
{
    return a.n * b.d == a.d * b.n;
}
```


Vergleichsoperatoren

können auch definiert werden, so dass sie das „Richtige“ machen:

```
// POST: returns true iff a == b
bool operator==(rational a, rational b)
{
    return a.n * b.d == a.d * b.n;
}
```

$$\frac{2}{3} = \frac{4}{6} \quad \checkmark$$

Ein-/Ausgabeoperatoren

können auch überladen werden.

■ Bisher:

```
std::cout << "Sum is " << t.n << "/" << t.d << "\n";
```

■ Neu (gewünscht):

```
std::cout << "Sum is " << t << "\n";
```

Ein-/Ausgabeoperatoren

können auch überladen werden wie folgt:

```
// POST: r has been written to out
std::ostream& operator<<(std::ostream& out, rational r)
{
    return out << r.n << "/" << r.d;
}
```

Ein-/Ausgabeoperatoren

können auch überladen werden wie folgt:

```
// POST: r has been written to out
std::ostream& operator<<(std::ostream& out, rational r)
{
    return out << r.n << "/" << r.d;
}
```

schreibt `r` auf den Ausgabestrom
und gibt diesen als L-Wert zurück

`std::cout` ist vom Typ `std::ostream`

Eingabe

```
// PRE: in starts with a rational number of the form "n/d"
// POST: r has been read from in
std::istream& operator>>(std::istream& in, rational& r){
    char c; // separating character '/'
    return in >> r.n >> c >> r.d;
}
```

liest `r` aus dem Eingabestrom
und gibt diesen als L-Wert zurück.

`std::cin` ist vom Typ `std::istream`

Ziel erreicht!

```
// input
std::cout << "Rational number r =? ";
rational r;
std::cin >> r;

std::cout << "Rational number s =? ";
rational s;
std::cin >> s;

// computation and output
std::cout << "Sum is " << r + s << ".\n";
```

Ziel erreicht!

```
// input
std::cout << "Rational number r =? ";
rational r;
std::cin >> r;

std::cout << "Rational number s =? ";
rational s;
std::cin >> s;

// computation and output
std::cout << "Sum is " << r + s << ".\n";
```

operator >>

operator +

operator <<

Invarianten garantieren?

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```


Invarianten garantieren?

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};
```

...

```
rational r;  
r.n = 1;  
r.d = 1;
```

...

```
r.d--;
```

18. Custom Data Types II

Datenkapselung, Klassen, Memberfunktionen, Konstruktoren

Ein neuer Typ mit Funktionalität...

```
struct rational {  
    int n;  
    int d; // INV: d != 0  
};  
  
// POST: return value is the sum of a and b  
rational operator+ (rational a, rational b)  
{  
    rational result;  
    result.n = a.n * b.d + a.d * b.n;  
    result.d = a.d * b.d;  
    return result;  
}  
...
```

...gehört in eine Bibliothek!

rational.h

- Definition des Structs `rational`
- Funktionsdeklarationen

rational.cpp

- Arithmetische Operatoren (`operator+`, `operator+=`, ...)
- Relationale Operatoren (`operator==`, `operator>`, ...)
- Ein-/Ausgabe (`operator >>`, `operator <<`, ...)

Invarianten und Repräsentation

- Wollen für eine Datenstruktur Invarianten garantieren, ohne den Benutzercode mit Asserts zu übersäen.

Beispiel 1: Für jedes `Rational r` gilt stets $r.d \neq 0$.

Beispiel 2: Jedes `Rational r` ist stets gekürzt.

Invarianten und Repräsentation

- Wollen für eine Datenstruktur Invarianten garantieren, ohne den Benutzercode mit Asserts zu übersäen.

Beispiel 1: Für jedes `Rational r` gilt stets $r.d \neq 0$.

Beispiel 2: Jedes `Rational r` ist stets gekürzt.

- Liefern dem Benutzer das „was“ und nicht das „wie“.

```
Vector v: v.size(), v[3], v.at(3)
```

Invarianten und Repräsentation

- Wollen für eine Datenstruktur Invarianten garantieren, ohne den Benutzercode mit Asserts zu übersäen.

Beispiel 1: Für jedes `Rational r` gilt stets $r.d \neq 0$.

Beispiel 2: Jedes `Rational r` ist stets gekürzt.

- Liefern dem Benutzer das „was“ und nicht das „wie“.

`Vector v: v.size(), v[3], v.at(3)`

- Die interne Darstellung sollte einfach anzupassen sein, ohne dass Benutzercode neu geschrieben werden muss.

`Complex`: Polarkoordinaten vs. kartesische Koordinaten

Idee der Datenkapselung (Information Hiding)

- Ein Typ ist durch *Wertebereich* und *Funktionalität* eindeutig definiert.

Idee der Datenkapselung (Information Hiding)

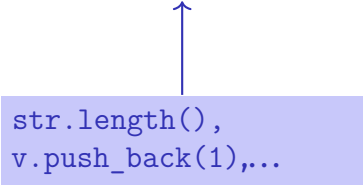
- Ein Typ ist durch *Wertebereich* und *Funktionalität* eindeutig definiert.
- Die *Repräsentation* soll *nicht sichtbar* sein.

Idee der Datenkapselung (Information Hiding)

- Ein Typ ist durch *Wertebereich* und *Funktionalität* eindeutig definiert.
- Die *Repräsentation* soll *nicht sichtbar* sein.
- $\Rightarrow$ Es wird keine *Repräsentation*, sondern Funktionalität angeboten.

Idee der Datenkapselung (Information Hiding)

- Ein Typ ist durch *Wertebereich* und *Funktionalität* eindeutig definiert.
- Die *Repräsentation* soll *nicht sichtbar* sein.
- $\Rightarrow$ Es wird keine *Repräsentation*, sondern Funktionalität angeboten.



```
str.length(),  
v.push_back(1),...
```

- sind das Konzept zur Datenkapselung in C++

Klassen

- sind das Konzept zur Datenkapselung in C++
- sind eine Variante von Structs

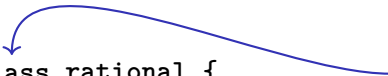
- sind das Konzept zur Datenkapselung in C++
- sind eine Variante von Structs
- gibt es in vielen *objektorientierten Programmiersprachen*

Datenkapselung: public / private

```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Wird statt struct verwendet, wenn überhaupt etwas "versteckt" werden soll.

Datenkapselung: public / private



```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Wird statt struct verwendet, wenn überhaupt etwas “versteckt” werden soll.

Einziger Unterschied:

- struct: standardmässig wird nichts versteckt
- class : standardmässig wird alles versteckt

Datenkapselung: public / private

```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Anwendungscode:

```
rational r;  
r.n = 1;      // error: n is private  
r.d = 2;      // error: d is private  
int i = r.n;  // error: n is private
```

Datenkapselung: public / private

```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Gute Nachricht: `r.d = 0` aus Versehen geht nicht mehr

Anwendungscode:

```
rational r;  
r.n = 1;      // error: n is private  
r.d = 2;      // error: d is private  
int i = r.n;  // error: n is private
```

Datenkapselung: public / private

```
class rational {  
    int n;  
    int d; // INV: d != 0  
};
```

Gute Nachricht: `r.d = 0` aus Versehen geht nicht mehr

Schlechte Nachricht: wir können gar nichts mehr machen ...

Anwendungscode:

```
rational r;  
r.n = 1;      // error: n is private  
r.d = 2;      // error: d is private  
int i = r.n;  // error: n is private
```

Memberfunktionen: Deklaration

```
class rational {  
public:  
    // POST: return value is the numerator of this instance  
    int numerator () const {  
        return n;  
    }  
    // POST: return value is the denominator of this instance  
    int denominator () const {  
        return d;  
    }  
private:  
    int n;  
    int d; // INV: d!= 0  
};
```

Memberfunktionen: Deklaration

öffentlich Bereich

```
class rational {  
public:  
    // POST: return value is the numerator of this instance  
    int numerator () const {  
        return n;  
    }  
    // POST: return value is the denominator of this instance  
    int denominator () const {  
        return d;  
    }  
private:  
    int n;  
    int d; // INV: d!= 0  
};
```

Memberfunktionen: Deklaration

öffentlich Bereich

```
class rational {  
public:  
    // POST: return value is the numerator of this instance  
    int numerator () const {  
        return n;  
    }  
    // POST: return value is the denominator of this instance  
    int denominator () const {  
        return d;  
    }  
private:  
    int n;  
    int d; // INV: d!= 0  
};
```

Memberfunktion

Memberfunktionen: Deklaration

öffentlich Bereich

```
class rational {  
public:  
    // POST: return value is the numerator of this instance  
    int numerator () const {  
        return n;  
    }  
    // POST: return value is the denominator of this instance  
    int denominator () const {  
        return d;  
    }  
private:  
    int n;  
    int d; // INV: d!= 0  
};
```

Memberfunktion

Memberfunktionen haben Zugriff auf private Daten

Memberfunktionen: Aufruf

```
// Definition des Typs
class rational {
    ...
};
...
// Variable des Typs
rational r;
int n = r.numerator();    // Zaehler
int d = r.denominator();  // Nenner
```

Member-Zugriff



const und Memberfunktionen

```
class rational {  
public:  
    int numerator () const  
    { return n; }  
    void set_numerator (int N)  
    { n = N;}  
    ...  
}
```

```
rational x;  
x.set_numerator(10); // ok;  
const rational y = x;  
int n = y.numerator(); // ok;  
y.set_numerator(10); // error;
```

Das `const` an einer Memberfunktion liefert das Versprechen, dass eine Instanz nicht über diese Funktion verändert wird.

`const` Objekte dürfen nur `const` Memberfunktionen aufrufen!

Memberfunktionen: Definition

```
// POST: returns numerator of this instance  
int numerator () const  
{  
    return n;  
}
```

Memberfunktionen: Definition ???

```
// POST: returns numerator of this instance
int numerator () const
{
    return n;
}
```

Memberfunktionen: Definition

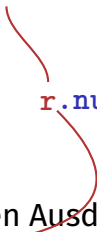
```
// POST: returns numerator of this instance
int numerator () const
{
    return n;                r.numerator()
}
```

- Eine Memberfunktion wird *für* einen Ausdruck der Klasse aufgerufen.

Memberfunktionen: Definition

```
// POST: returns numerator of this instance
int numerator () const
{
    return n;
}
```

r.numerator()



- Eine Memberfunktion wird *für* einen Ausdruck der Klasse aufgerufen. In der Funktion: ~~this~~ ist der Name dieses *impliziten Arguments*.

Memberfunktionen: Definition

```
// POST: returns numerator of this instance
int numerator () const
{
    return n;                r.numerator()
}
```

- Eine Memberfunktion wird *für* einen Ausdruck der Klasse aufgerufen. In der Funktion: `this` ist der Name dieses *impliziten Arguments*.
- Das *const* bezieht sich auf die Instanz `this`

Memberfunktionen: Definition

```
// POST: returns numerator of this instance
int numerator () const
{
    return n;
}
```

Diagram: A red curved arrow points from the variable `n` in the `return` statement to the text `numerator of this instance`. Another red curved arrow points from the text `r.numerator()` to the `numerator` property access in the same text.

- Eine Memberfunktion wird *für* einen Ausdruck der Klasse aufgerufen. In der Funktion: `this` ist der Name dieses *impliziten Arguments*.
- Das `const` bezieht sich auf die Instanz `this`
- `n` ist Abkürzung für `this->n` (genaue Erklärung von „->“ nächste Woche)

This rational vs. dieser Bruch

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return n;  
    }  
};  
  
rational r;  
...  
std::cout << r.numerator();
```


This rational vs. dieser Bruch

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return this->n;  
    }  
};  
  
rational r;  
...  
std::cout << r.numerator();
```

This rational vs. dieser Bruch

So wäre es in etwa ...

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return this->n;  
    }  
};  
  
rational r;  
...  
std::cout << r.numerator();
```

This rational vs. dieser Bruch

So wäre es in etwa ...

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return this->n;  
    }  
};  
  
rational r;  
...  
std::cout << r.numerator();
```

... ohne Memberfunktionen

```
struct bruch {  
    int n;  
    ...  
};  
  
int numerator (const bruch& dieser)  
{  
    return dieser.n;  
}  
  
bruch r;  
..  
std::cout << numerator(r);
```

Initialisierung? Konstruktoren!

```
class rational
{
public:
    rational (int num, int den)
        : n (num), d (den)
    {
        assert (den != 0);
    }
    ...
};
...
rational r = rational(2,3); // r = 2/3
```

Initialisierung? Konstruktoren!

```
class rational
{
public:
    rational (int num, int den)
        : n (num), d (den) ← Initialisierung der
                               Membervariablen
    {
        assert (den != 0); ← Funktionsrumpf.
    }
    ...
};
...
rational r = rational(2,3); // r = 2/3
```

Der Default-Konstruktor

```
class rational
{
public:
    ...
    rational () ← Leere Argumentliste
        : n (0), d (1)
    {}
    ...
};
...
rational r;    // r = 0
// Abkürzung für rational r = rational();
```

Der Default-Konstruktor

```
class rational
{
public:
    ...
    rational () ← Leere Argumentliste
        : n (0), d (1)
    {}
    ...
};
...
rational r;    // r = 0
// Abkürzung für rational r = rational();
```

⇒ Es gibt keine uninitialisierten Variablen vom Typ rational mehr!

Alternative: Default-Konstruktor löschen

```
class rational
{
public:
    ...
    rational () = delete;
    ...
};
...
rational r;  // error: use of deleted function 'rational::rational()'

```

⇒ Es gibt keine uninitialisierten Variablen vom Typ rational mehr!

Initialisierung “rational = int”?

```
class rational
{
public:
    rational (int num)
        : n (num), d (1)
    {}
    ...
};
...
rational r = rational(2); // Explizite Initialisierung mit 2
rational s = 2; // Implizite Konversion
```

Initialisierung “rational = int”?

```
class rational
{
public:
    rational (int num)
        : n (num), d (1)
    {}    ← Leerer Funktionsrumpf
    ...
};
...
rational r = rational(2); // Explizite Initialisierung mit 2
rational s = 2; // Implizite Konversion
```

Benutzerdefinierte Konversionen

sind definiert durch Konstruktoren mit genau *einem* Argument

```
rational (int num) ← Benutzerdefinierte Konversion von int  
    : n (num), d (1) nach rational. Damit wird int zu einem  
    {} Typ, dessen Werte nach rational kon-  
        vertierbar sind.
```

```
rational r = 2; // implizite Konversion
```

Member-Definition: In-Class

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return n;  
    }  
    ....  
};
```

- Keine Trennung zwischen Deklaration und Definition (schlecht für Bibliotheken)

Member-Definition: In-Class vs. Out-of-Class

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const  
    {  
        return n;  
    }  
    ....  
};
```

- Keine Trennung zwischen Deklaration und Definition (schlecht für Bibliotheken)

```
class rational {  
    int n;  
    ...  
public:  
    int numerator () const;  
    ...  
};  
  
int rational::numerator () const  
{  
    return n;  
}
```

- So geht's auch.

19. Container und Iteratoren

Container, Mengen, Iteratoren, const-Iteratoren, Algorithmen

Vektoren sind Container

- Abstrakt gesehen ist ein Vektor
 1. Eine (geordnete) Ansammlung von Elementen
 2. Plus Operationen auf dieser Ansammlung
- In C++ heissen `vector<T>` und ähnliche „Ansammlungs“-Datenstrukturen *Container*
- In manchen Sprachen, z.B. Java, *Collections* genannt

Container-Eigenschaften

- Jeder Container hat bestimmte *charakteristische Eigenschaften*
- Ein Vektor `std::vector<T>` z.B. die folgenden:

Container-Eigenschaften

- Jeder Container hat bestimmte *charakteristische Eigenschaften*
- Ein Vektor `std::vector<T>` z.B. die folgenden:
 - Effizienter, index-basierter Zugriff (`v[i]`)
 - Effiziente Speichernutzung: Nur die Elemente selbst belegen Platz (plus Elementezähler)

Container-Eigenschaften

- Jeder Container hat bestimmte *charakteristische Eigenschaften*
- Ein Vektor `std::vector<T>` z.B. die folgenden:
 - Effizienter, index-basierter Zugriff (`v[i]`)
 - Effiziente Speichernutzung: Nur die Elemente selbst belegen Platz (plus Elementezähler)
 - Einfügen/Entfernen an beliebigem Index ist potenziell ineffizient
 - Suchen eines bestimmten Elements ist potenziell ineffizient

Container-Eigenschaften

- Jeder Container hat bestimmte *charakteristische Eigenschaften*
- Ein Vektor `std::vector<T>` z.B. die folgenden:
 - Effizienter, index-basierter Zugriff (`v[i]`)
 - Effiziente Speichernutzung: Nur die Elemente selbst belegen Platz (plus Elementezähler)
 - Einfügen/Entfernen an beliebigem Index ist potenziell ineffizient
 - Suchen eines bestimmten Elements ist potenziell ineffizient
 - Kann Elemente mehrfach enthalten
 - Elemente sind in Einfügereihenfolge enthalten (geordnet aber unsortiert)

Container in C++

- Fast jede Anwendung erfordert die Verwaltung und Manipulation von beliebig vielen Datensätzen

Container in C++

- Fast jede Anwendung erfordert die Verwaltung und Manipulation von beliebig vielen Datensätzen
- Aber mit unterschiedlichen Anforderungen (z.B. Elemente nur hinten anhängen, fast nie entfernen, oft suchen, ...)

Container in C++

- Fast jede Anwendung erfordert die Verwaltung und Manipulation von beliebig vielen Datensätzen
- Aber mit unterschiedlichen Anforderungen (z.B. Elemente nur hinten anhängen, fast nie entfernen, oft suchen, ...)
- Deswegen enthält die Standardbibliothek von C++ diverse Container mit unterschiedlichen Eigenschaften, siehe <https://en.cppreference.com/w/cpp/container>

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

```
std::list<int> l = {1, 2, 3};
```

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

```
std::list<int> l = {1, 2, 3};  
  
// add 4 to the back of the list  
l.push_back(4); // l = {1, 2, 3, 4}
```

Beispiel-Container: `std::list<T>`

- wie ein Vektor, eine geordnete Liste von Elementen
- Eigenschaften:
 - effizientes Einfügen/Entfernen an beliebiger Stelle
 - kein Random Access
 - ineffizientere Speichernutzung als Vektor

```
std::list<int> l = {1, 2, 3};  
  
// add 4 to the back of the list  
l.push_back(4); // l = {1, 2, 3, 4}  
  
// add 0 to the front of the list  
l.push_front(0); // l = {0, 1, 2, 3, 4}
```

Beispiel-Container: `std::unordered_set<T>`

- Eine *mathematische Menge* ist eine ungeordnete, duplikatfreie Zusammenfassung von Elementen:

$$\{1, 2, 1\} = \{1, 2\} = \{2, 1\}$$

- In C++: `std::unordered_set<T>`
- Eigenschaften:
 - Kann kein Element doppelt enthalten
 - Elemente haben keine bestimmte Reihenfolge

Beispiel-Container: `std::unordered_set<T>`

- Eine *mathematische Menge* ist eine ungeordnete, duplikatfreie Zusammenfassung von Elementen:

$$\{1, 2, 1\} = \{1, 2\} = \{2, 1\}$$

- In C++: `std::unordered_set<T>`
- Eigenschaften:
 - Kann kein Element doppelt enthalten
 - Elemente haben keine bestimmte Reihenfolge
 - Kein indexbasierter Zugriff (`s[i]` nicht definiert)

Beispiel-Container: `std::unordered_set<T>`

- Eine *mathematische Menge* ist eine ungeordnete, duplikatfreie Zusammenfassung von Elementen:

$$\{1, 2, 1\} = \{1, 2\} = \{2, 1\}$$

- In C++: `std::unordered_set<T>`
- Eigenschaften:
 - Kann kein Element doppelt enthalten
 - Elemente haben keine bestimmte Reihenfolge
 - Kein indexbasierter Zugriff (`s[i]` nicht definiert)
 - Effiziente „Element enthalten?“-Prüfung
 - Effizientes Einfügen und Löschen von Elementen

Beispiel-Container: `std::unordered_set<T>`

- Eine *mathematische Menge* ist eine ungeordnete, duplikatfreie Zusammenfassung von Elementen:

$$\{1, 2, 1\} = \{1, 2\} = \{2, 1\}$$

- In C++: `std::unordered_set<T>`
- Eigenschaften:
 - Kann kein Element doppelt enthalten
 - Elemente haben keine bestimmte Reihenfolge
 - Kein indexbasierter Zugriff (`s[i]` nicht definiert)
 - Effiziente „Element enthalten?“-Prüfung
 - Effizientes Einfügen und Löschen von Elementen
- Randbemerkung: Implementiert als Hash-Tabelle

Anwendungsbeispiel `std::unordered_set<T>`

Problem:

- Gegeben eine Sequenz an Paaren (*Name*, *Prozente*) von Code-Expert-Submissions ...

```
// Input: file submissions.txt
Friedrich 90
Sasse 70
Schwerhoff 10
Lehner 20
Schwerhoff 11
Sasse 60
```

Anwendungsbeispiel `std::unordered_set<T>`

Problem:

- Gegeben eine Sequenz an Paaren (*Name*, *Prozente*) von Code-Expert-Submissions ...

```
// Input: file submissions.txt
Friedrich 90
Sasse 70
Schwerhoff 10
Lehner 20
Schwerhoff 11
Sasse 60
```

- ... bestimme die Abgebenden, die mindestens 50% erzielt haben

```
// Output
Friedrich Sasse
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt"); ← Öffne submissions.txt  
std::unordered_set<std::string> names;
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

```
std::cout << "Unique submitters: "  
           << names << '\n';
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");
std::unordered_set<std::string> names; ← Namen-Menge, initial leer

std::string name;
unsigned int score;

while (in >> name >> score) {
    if (50 <= score)
        names.insert(name);
}

std::cout << "Unique submitters: "
          << names << '\n';
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");  
std::unordered_set<std::string> names;
```

```
std::string name;  
unsigned int score;
```



Paar (*Name*, *Punkte*)

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

```
std::cout << "Unique submitters: "  
           << names << '\n';
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");  
std::unordered_set<std::string> names;
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

← Nächstes Paar einlesen

```
std::cout << "Unique submitters: "  
           << names << '\n';
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");  
std::unordered_set<std::string> names;
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

Namen merken falls Punkte
ausreichen



```
std::cout << "Unique submitters: "  
           << names << '\n';
```

Anwendungsbeispiel `std::unordered_set<T>`

```
std::ifstream in("submissions.txt");  
std::unordered_set<std::string> names;
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

```
std::cout << "Unique submitters: "  
          << names << '\n';
```



Gemerkte Namen ausgeben

Beispiel-Container: `std::set<T>`

- Fast gleich wie `std::unordered_set<T>`, aber die Elemente sind *geordnet*

$$\{1, 2, 1\} = \{1, 2\} \neq \{2, 1\}$$

Beispiel-Container: `std::set<T>`

- Fast gleich wie `std::unordered_set<T>`, aber die Elemente sind *geordnet*

$$\{1, 2, 1\} = \{1, 2\} \neq \{2, 1\}$$

- Elemente suchen, einfügen und löschen weiterhin effizient (besser als bei `std::vector<T>`), aber weniger effizient als bei `std::unordered_set<T>`
- Denn das Beibehalten der Ordnung zieht etwas Aufwand nach sich

Beispiel-Container: `std::set<T>`

- Fast gleich wie `std::unordered_set<T>`, aber die Elemente sind *geordnet*

$$\{1, 2, 1\} = \{1, 2\} \neq \{2, 1\}$$

- Elemente suchen, einfügen und löschen weiterhin effizient (besser als bei `std::vector<T>`), aber weniger effizient als bei `std::unordered_set<T>`
- Denn das Beibehalten der Ordnung zieht etwas Aufwand nach sich
- Randbemerkung: Implementiert als Rot-Schwarz-Baum

Anwendungsbeispiel `std::set<T>`

```
std::ifstream in("submissions.txt");
```

```
std::set<std::string> names; ← set Statt unordered_set ...
```

```
std::string name;
```

```
unsigned int score;
```

```
while (in >> name >> score) {
```

```
    if (50 <= score)
```

```
        names.insert(name);
```

```
}
```

```
std::cout << "Unique submitters: "
```

```
    << names << '\n';
```

Anwendungsbeispiel `std::set<T>`

```
std::ifstream in("submissions.txt");  
std::set<std::string> names;
```

```
std::string name;  
unsigned int score;
```

```
while (in >> name >> score) {  
    if (50 <= score)  
        names.insert(name);  
}
```

```
std::cout << "Unique submitters: "  
          << names << '\n';
```

← ... und die Ausgabe erfolgt
alphabetisch sortiert

Iteratoren

- viele Operationen (z.B. Ausgabe eines Containers) erfordern Iteration über Container
- $\Rightarrow$ Jeder C++-Container implementiert seinen eigenen *Iterator*

Iteratoren

- viele Operationen (z.B. Ausgabe eines Containers) erfordern Iteration über Container
- $\Rightarrow$ Jeder C++-Container implementiert seinen eigenen *Iterator*
- Gegeben ein Container `c`:
 - `it = c.begin()`: Iterator aufs erste Element
 - `it = c.end()`: Iterator *hinters* letzte Element
 - `*it`: Zugriff aufs aktuelle Element (Dereferenzoperator)
 - `++it`: Iterator um ein Element verschieben
- Iteratoren sind quasi gepimpte Zeiger (siehe nächste Woche!)

Beispiel: Iteration über `std::vector`

it ist ein zu `std::vector<int>` passender Iterator

```
std::vector<int> v = {1, 2, 3};  
  
for (std::vector<int>::iterator it = v.begin();  
    it != v.end();  
    ++it) {  
  
    *it = -*it;  
}
```


Beispiel: Iteration über `std::vector`

```
std::vector<int> v = {1, 2, 3};
```

it zeigt initial aufs erste Element

```
for (std::vector<int>::iterator it = v.begin(),  
     it != v.end();  
     ++it) {  
  
    *it = -*it;  
}
```

Beispiel: Iteration über `std::vector`

```
std::vector<int> v = {1, 2, 3};
```

```
for (std::vector<int>::iterator it = v.begin();
```

```
    it != v.end();
```

```
    ++it) {
```

```
    *it = -*it;
```

```
}
```

← Abbruch falls it Ende erreicht hat

Beispiel: Iteration über `std::vector`

```
std::vector<int> v = {1, 2, 3};
```

```
for (std::vector<int>::iterator it = v.begin();
```

```
    it != v.end();
```

```
    ++it) {
```

it elementweise vorwärtssetzen

```
    *it = -*it;
```

```
}
```

Beispiel: Iteration über `std::vector`

```
std::vector<int> v = {1, 2, 3};
```

```
for (std::vector<int>::iterator it = v.begin();  
     it != v.end();  
     ++it) {
```

```
    *it = -*it; }  
}
```

Aktuelles Element negieren ($e \rightarrow -e$)

Beispiel: Iteration über `std::vector`

Zur Erinnerung: Type-Aliasie können genutzt werden um oft verwendete Typnamen abzukürzen

```
using ivit = std::vector<int>::iterator; // int-vector iterator

for (ivit it = v.begin();
    ...
```

Beispiel: Iteration über `std::vector`

Zur Erinnerung: Type-Aliasie können genutzt werden um oft verwendete Typnamen abzukürzen

```
using ivit = std::vector<int>::iterator; // int-vector iterator

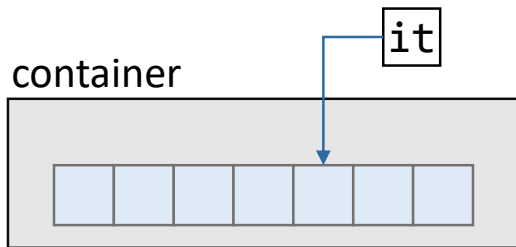
for (ivit it = v.begin();
    ...
```

alternativ können wir `auto` verwenden

```
for (auto it = v.begin();
    ...
```

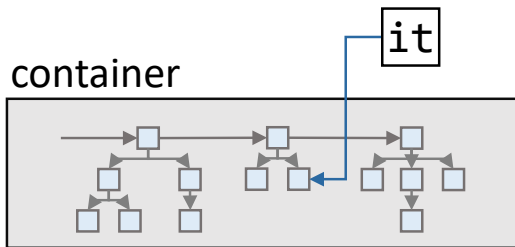
Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung



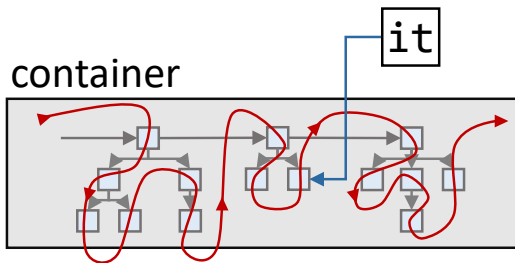
Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung



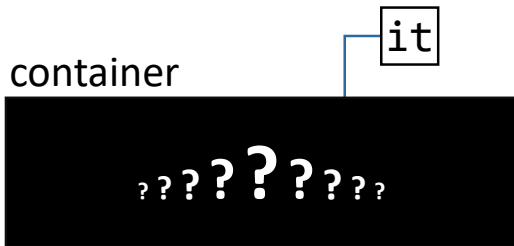
Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung
- Iterator weiss, wie man die Elemente „seines“ Containers abläuft



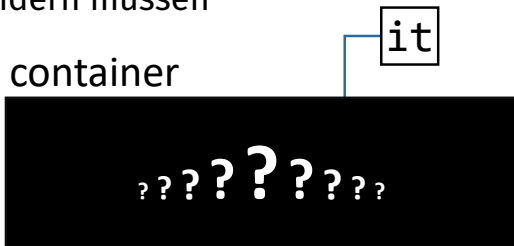
Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung
- Iterator weiss, wie man die Elemente „seines“ Containers abläuft
- Nutzer müssen und sollen interne Details nicht kennen



Iteratoren

- Iteratoren ermöglichen Zugriff auf verschiedene Container auf *uniforme* Weise: `*it`, `++it`, etc.
- Nutzer bleiben unabhängig von der Container-Implementierung
- Iterator weiss, wie man die Elemente „seines“ Containers abläuft
- Nutzer müssen und sollen interne Details nicht kennen
- $\Rightarrow$ Containerimplementierung kann geändert werden, ohne dass Nutzer Code ändern müssen



Negieren als Funktion

Wie zuvor: Übergabe eines *Arbeitsbereichs* (-intervalls)

```
void neg(std::vector<int>::iterator begin,  
        std::vector<int>::iterator end) {
```

← Elemente im Intervall
[begin, end) negieren

```
    for (std::vector<int>::iterator it = begin;  
         it != end;  
         ++it) {  
  
        *it = -*it;  
    }  
}
```

Negieren als Funktion

Wie zuvor: Übergabe eines *Arbeitsbereichs* (-intervalls)

```
void neg(std::vector<int>::iterator begin;  
        std::vector<int>::iterator end);
```

```
// in main():
```

```
std::vector<int> v = {1, 2, 3};
```

```
neg(v.begin(), v.begin() + (v.size() / 2)); ← Erste Hälfte negieren
```

Algorithmen-Bibliothek in C++

- Die C++-Standardbibliothek enthält viele nützliche Algorithmen (Funktionen), die auf durch Iteratoren bestimmten Intervallen [*Anfang*, *Ende*) arbeiten
- Zum Beispiel `find`, `fill` and `sort`; siehe auch <https://en.cppreference.com/w/cpp/algorithm>

Algorithmen-Bibliothek in C++

- Die C++-Standardbibliothek enthält viele nützliche Algorithmen (Funktionen), die auf durch Iteratoren bestimmten Intervallen [*Anfang*, *Ende*) arbeiten
- Zum Beispiel `find`, `fill` and `sort`; siehe auch <https://en.cppreference.com/w/cpp/algorithm>
- Dank Iteratoren können diese ≥ 100 (!) Algorithmen auf beliebigen* Containern ausgeführt werden: Den 17 (!) Standardcontainern von C++, unsere eigenen Containers, etc.
- Gäbe es diesen uniformen Zugriff auf Containerelemente nicht, müsste *sehr* viel Code dupliziert werden

Nicht jeder Algorithmus kann auf jedem Container aufgerufen werden. Z.B. kann ein `std::unordered_set` nicht sortiert werden.

Bereichsbasierte for-Schleife

- Sequenzielle Iteration über einen Container (hier: `std::list<int>`):

```
std::list<int> l = {0, 0, 0};  
for (list<int>::iterator it = l.begin(); it != l.end(); ++it)  
    std::cout << *it; // prints 000
```

- Kann alternativ auch wie folgt geschrieben werden:

```
for (int e : l) std::cout << e; // prints 000
```

Wird vom Compiler zu Iterator-basierter Schleife übersetzt.

Bereichsbasierte for-Schleife

- Sequenzielle Iteration über einen Container (hier: `std::list<int>`):

```
std::list<int> l = {0, 0, 0};  
for (list<int>::iterator it = l.begin(); it != l.end(); ++it)  
    std::cout << *it; // prints 000
```

- Kann alternativ auch wie folgt geschrieben werden:

```
for (int e : l) std::cout << e; // prints 000
```

Wird vom Compiler zu Iterator-basierter Schleife übersetzt.

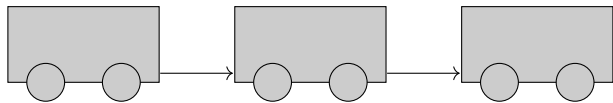
- Modifizierender Zugriff ist auch möglich:

```
for (int& e : l) e += 3; // modifies  
for (int e : l) std::cout << e; // prints 333
```

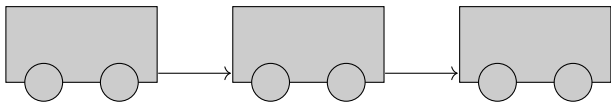
20. Dynamischer Speicher und Zeiger

Adressen und Zeiger, Const-Zeiger, **new**-Ausdruck

Wagons

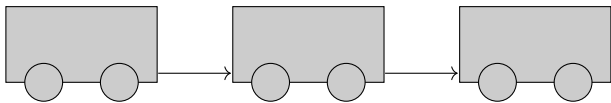


Wagons



Ziel: beliebig viele Wagons aneinanderhängen

Wagons



Ziel: beliebig viele Wagons aneinanderhängen

Idee: Objekte vom Typ `wagon`, die sich den nächsten Wagon speichern

Wagons: 1. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon next;  
};
```

Wagons: 1. Versuch

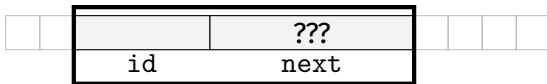
```
struct wagon {  
    unsigned int id;  
    wagon next;  
};
```

field 'next' has incomplete type 'wagon'

Wagons: 1. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon next;  
};
```

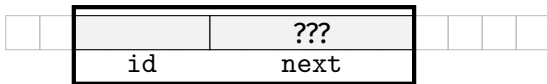
field 'next' has incomplete type 'wagon'



Wagons: 1. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon next;  
};
```

field 'next' has incomplete type 'wagon'



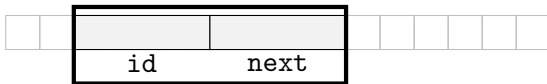
Struct kann keine Membervariable des gleichen Typs haben (unendliche Rekursion)

Wagons: 2. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```

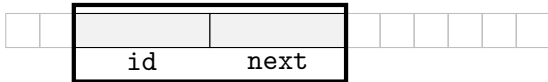
Wagons: 2. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```



Wagons: 2. Versuch

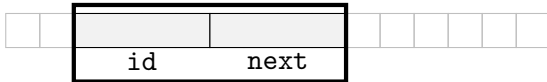
```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```



```
wagon w = wagon(1);
```

Wagons: 2. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```

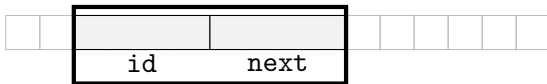


```
wagon w = wagon(1);
```

uninitialized reference member in 'struct wagon&'

Wagons: 2. Versuch

```
struct wagon {  
    unsigned int id;  
    wagon& next;  
    wagon(unsigned int id) : id(id) {}  
};
```



```
wagon w = wagon(1);
```

uninitialized reference member in 'struct wagon&'

Referenzen müssen bei Deklaration initialisiert werden

Zeiger (Pointer)

Zeiger (Pointer) sind wie Referenzen, die auch ins Nichts zeigen dürfen!

Zeiger (Pointer)

Zeiger (Pointer) sind wie Referenzen, die auch ins Nichts zeigen dürfen!

wagon&: Referenz auf ein Wagon

Zeiger (Pointer)

Zeiger (Pointer) sind wie Referenzen, die auch ins Nichts zeigen dürfen!

wagon&: Referenz auf ein Wagon

wagon*: Pointer auf ein Wagon

Zeiger (Pointer) dürfen nach ihrer Erstellung verändert werden.

Zeiger-Typen

T* Zeiger-Typ für zugrunde liegenden Typ T

Ein Ausdruck vom Typ T* heisst *Zeiger (auf T)*

```
int* p = ...; // Zeiger auf einen int  
std::string* q = ...; // Zeiger auf einen std::string
```

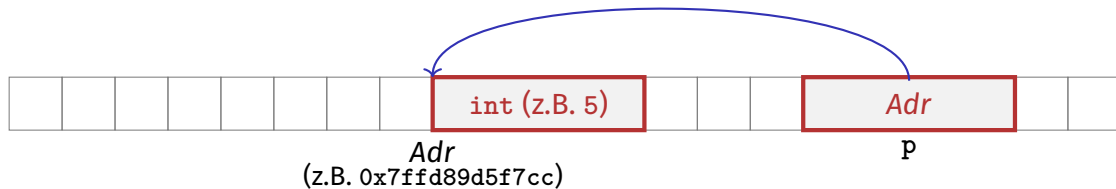
Zeiger-Typen

T* Zeiger-Typ für zugrunde liegenden Typ T

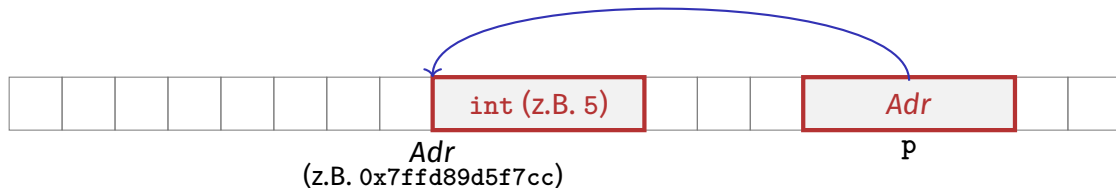
Wo ein T* draufsteht, muss auch ein T drin sein

```
int* p = ...;  
std::string* q = p; // compiler error!
```

Zeiger-Typen

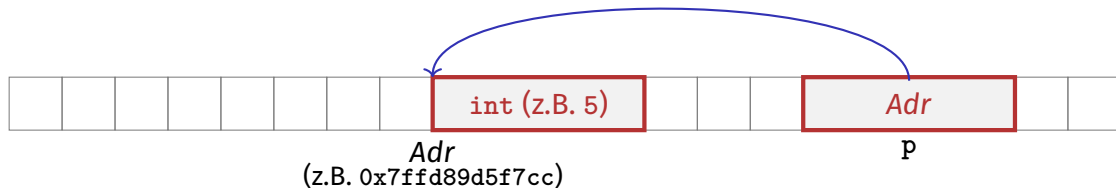


Zeiger-Typen



```
int* p = ...;  
std::cout << p; // z.B. 0x7ffd89d5f7cc
```

Zeiger-Typen

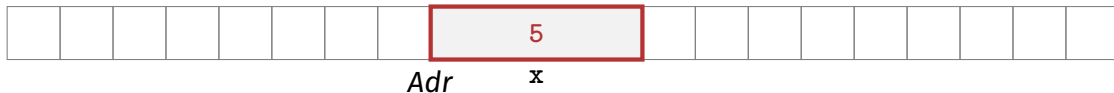


```
int* p = ...;  
std::cout << p; // z.B. 0x7ffd89d5f7cc
```

Wert eines Zeigers auf T ist die Adresse eines Objektes vom Typ T

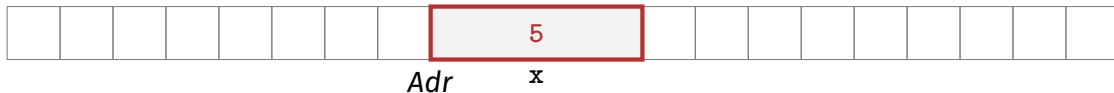
Adressoperator

```
int x = 5; // x initialisiert mit 5
```



Adressoperator

```
int x = 5; // x initialisiert mit 5  
    &x
```



Adressoperator

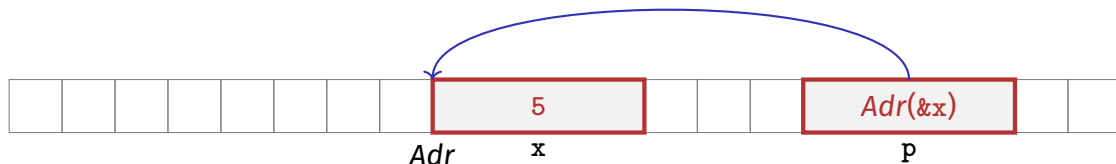
&expr

expr: L-Wert vom Typ *T*

- Wert des Ausdrucks *&expr*: Die Adresse des Objekts (L-Wertes) *expr*

Adressoperator

```
int x = 5; // x initialisiert mit 5  
int* p = &x; // p initialisiert mit der Adresse von x
```



Adressoperator

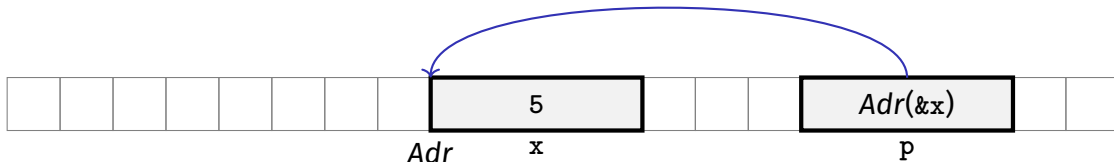
&expr

expr: L-Wert vom Typ *T*

- **Wert** des Ausdrucks *&expr*: Die *Adresse* des Objekts (L-Wertes) *expr*
- **Typ** des Ausdrucks: Ein Zeiger *T** (vom Typ *T*)

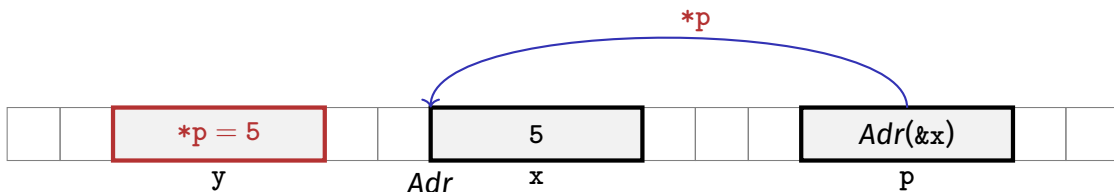
Dereferenzoperator

```
int x = 5;  
int* p = &x; // p = Adresse von x
```



Dereferenzoperator

```
int x = 5;  
int* p = &x; // p = Adresse von x  
int y = *p;  // y = 5
```



Dereferenzoperator

Dereferenzoperator *

**expr*

expr: R-Wert vom Typ T^*

Dereferenzoperator

Dereferenzoperator *

**expr*

expr: R-Wert vom Typ T^*

- **Wert** des Ausdrucks: Der Wert des Objekts an der durch *expr* bestimmten Adresse

Dereferenzoperator

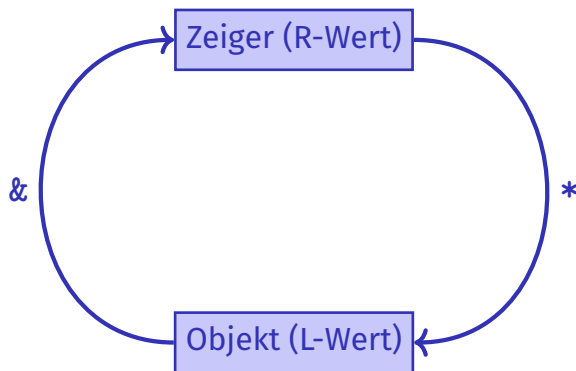
Dereferenzoperator *

**expr*

expr: R-Wert vom Typ T^*

- **Wert** des Ausdrucks: Der Wert des Objekts an der durch *expr* bestimmten Adresse
- **Typ** des Ausdrucks: T

Adress- und Dereferenzoperator



Die Deklaration

```
T* p;      // p ist vom Typ „Zeiger auf T“
```


Die Deklaration

```
T* p;      // p ist vom Typ „Zeiger auf T“
```

kann gelesen werden als

```
T *p;      // *p ist vom Typ T
```

Null-Zeiger

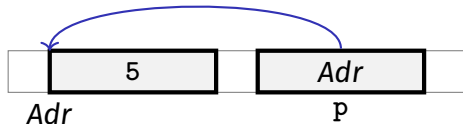
- Spezieller Zeigerwert, der angibt, dass (noch) auf kein Objekt gezeigt wird
- Repräsentiert durch das Literal `nullptr` (konvertierbar nach `T*`)

```
int* p = nullptr;
```

- Kann nicht dereferenziert werden (undefiniertes Verhalten!)
⇒ in Praxis oft Laufzeitfehler
- Dient der Vermeidung undefinierten Verhaltens

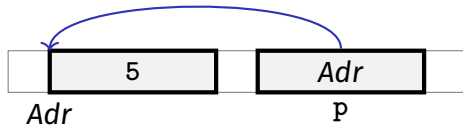
```
int* p; // Zugriff auf p ist undefiniertes Verhalten  
int* q = nullptr; // q hingegen zeigt explizit ins Nichts
```

Zeiger-Visualisierungen



- Bisher: Technische präzise Visualisierung *mit Indirektion*
- In p steckt eine Adresse, erst $*p$ gibt uns den Wert/das Objekt, auf den p konzeptionell zeigt

Zeiger-Visualisierungen



- Bisher: Technische präzise Visualisierung *mit Indirektion*
- In p steckt eine Adresse, erst $*p$ gibt uns den Wert/das Objekt, auf den p konzeptionell zeigt

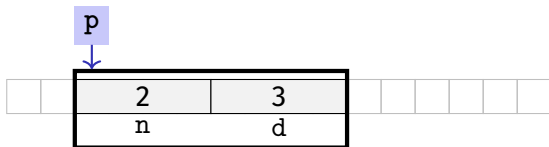


- Neu: Vereinfachte Visualisierung *ohne Indirektion*
- Adresse ist i.d.R. nur „Mittel zum Zweck“; relevant ist, worauf p konzeptionell zeigt

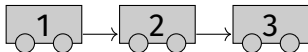
Zeiger auf Klasse

```
struct rational
{
    int n, d;
    rational(int num, int den) : n(num), d(den) {}
};
```

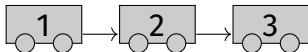
```
rational r = rational(2, 3);
rational* p = &r;
std::cout << (*p).n << (*p).d; // std::cout << p->n << p->d;
```



Wagons: Fortschritt

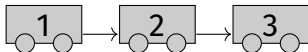


Wagons: Fortschritt



```
struct wagon {  
    unsigned int id;  
    wagon* next;  
    wagon(unsigned int id) : id(id), next(nullptr) {}  
};
```

Wagons: Fortschritt



```
struct wagon {  
    unsigned int id;  
    wagon* next;  
    wagon(unsigned int id) : id(id), next(nullptr) {}  
};
```

```
wagon w1 = wagon(1);  
wagon w2 = wagon(2);  
wagon w3 = wagon(3);  
w1.next = &w2;  
w2.next = &w3;  
std::cout << w1.next->next->id; // outputs 3
```


Funktionen mit/ohne Effekt

- Zeiger können, wie auch Referenzen, für Funktionen mit Effekt verwendet werden.

```
void set_next(wagon* w1, wagon* w2) {  
    w1->next = w2;  
}
```

Funktionen mit/ohne Effekt

- Zeiger können, wie auch Referenzen, für Funktionen mit Effekt verwendet werden.

```
void set_next(wagon* w1, wagon* w2) {  
    w1->next = w2;  
}
```

- Aber viele Funktionen verändern die Daten nicht, sie lesen sie nur
⇒ Verwendung von `const`

Funktionen mit/ohne Effekt

- Zeiger können, wie auch Referenzen, für Funktionen mit Effekt verwendet werden.

```
void set_next(wagon* w1, wagon* w2) {  
    w1->next = w2;  
}
```

- Aber viele Funktionen verändern die Daten nicht, sie lesen sie nur
⇒ Verwendung von const

```
void print(const wagon* w) {  
    std::cout << "Wagon " << w->id;  
}
```

Positionierung von Const

`const T` ist äquivalent zu `T const` (und kann auch so geschrieben werden):

```
const int zero = ...  ⇔  int const zero = ...  
const int& nil = ...  ⇔  int const& nil = ...
```

Positionierung von Const

`const T` ist äquivalent zu `T const` (und kann auch so geschrieben werden):

```
const int zero = ...   ⇔   int const zero = ...  
const int& nil = ...   ⇔   int const& nil = ...
```

Beide Schreibweisen werden in der Praxis genutzt

Const und Zeiger

Lies Deklaration von rechts nach links

```
int const p1;
```

p1 ist konstante Ganzzahl

Const und Zeiger

Lies Deklaration von rechts nach links

```
int const p1;
```

p1 ist konstante Ganzzahl

```
int const* p2;
```

p2 ist Zeiger auf konstante Ganzzahl
(read-only Pointer auf Ganzzahl)

Const und Zeiger

Lies Deklaration von rechts nach links

```
int const p1;
```

p1 ist konstante Ganzzahl

```
int const* p2;
```

p2 ist Zeiger auf konstante Ganzzahl
(read-only Pointer auf Ganzzahl)

```
int* const p3;
```

p3 ist konstanter Zeiger auf Ganzzahl
(nicht-veränderbarer Pointer auf eine Ganzzahl)

Const und Zeiger

Lies Deklaration von rechts nach links

<code>int const p1;</code>	p1 ist konstante Ganzzahl
<code>int const* p2;</code>	p2 ist Zeiger auf konstante Ganzzahl (read-only Pointer auf Ganzzahl)
<code>int* const p3;</code>	p3 ist konstanter Zeiger auf Ganzzahl (nicht-veränderbarer Pointer auf eine Ganzzahl)
<code>int const* const p4;</code>	p4 ist konstanter Zeiger auf konstante Ganzzahl (nicht-veränderbarer read-only Pointer auf eine Ganzzahl)

Beliebig viele Wagons

mit n Variablen vom Typ `wagon` können wir nur n Wagons haben...

Beliebig viele Wagons

mit n Variablen vom Typ `wagon` können wir nur n Wagons haben...

Wie können wir zur Laufzeit mehr Wagons erstellen?

Beliebig viele Wagons

mit n Variablen vom Typ `wagon` können wir nur n Wagons haben...

Wie können wir zur Laufzeit mehr Wagons erstellen?

⇒ `new`-Ausdruck!

Beliebig viele Wagons

mit n Variablen vom Typ `wagon` können wir nur n Wagons haben...

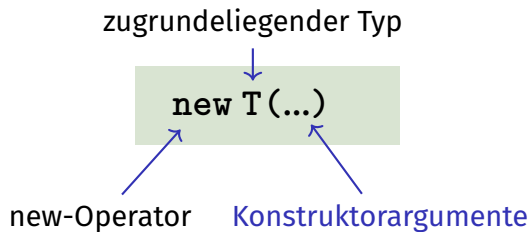
Wie können wir zur Laufzeit mehr Wagons erstellen?

⇒ `new`-Ausdruck!

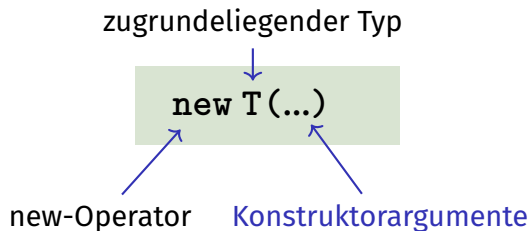
```
new wagon(4);
```

erstellt zur Laufzeit ein neues Objekt vom Typ `wagon` mit id 4

new-Ausdruck

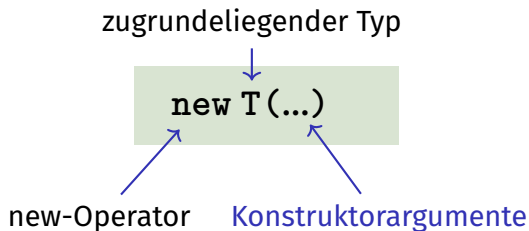


new-Ausdruck



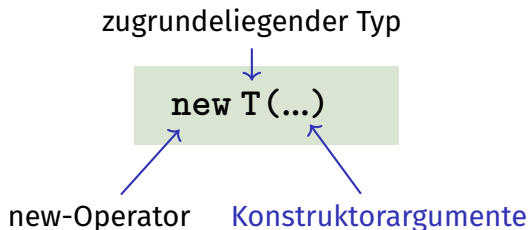
- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...

new-Ausdruck



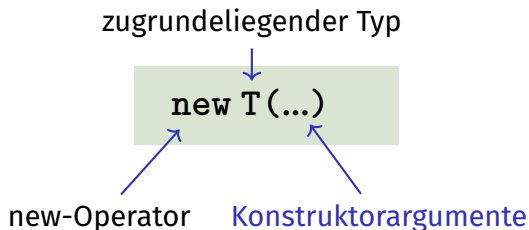
- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert

new-Ausdruck



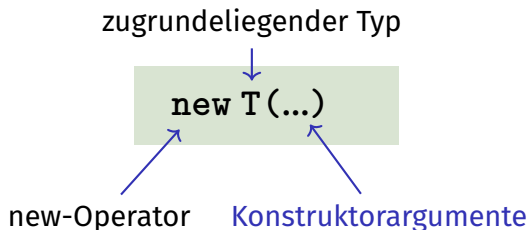
- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert:** Adresse des neuen `T`-Objekts, **Typ:** Zeiger `T*`

new-Ausdruck



- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert:** Adresse des neuen `T`-Objekts, **Typ:** Zeiger `T*`
- Objekt „lebt“ bis es explizit gelöscht wird
 - Mittels `new` allozierter Speicher wird *nicht* automatisch **dealloziert** (= freigegeben), sobald der Pointer auf ihn out of scope geht

new-Ausdruck



- **Effekt:** Speicher für ein neues Objekt vom Typ `T` wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert:** Adresse des neuen `T`-Objekts, **Typ:** Zeiger `T*`
- Objekt „lebt“ bis es explizit gelöscht wird
 - Mittels `new` allozierter Speicher wird *nicht* automatisch **dealloziert** (= freigegeben), sobald der Pointer auf ihn out of scope geht
 - Zu jedem `new` gehört ein `delete`, das den Speicher explizit freigibt
→ nächste Woche

Beliebig viele Wagons: (partielle) Lösung

```
unsigned int n; // number of wagons
std::cin >> n;
assert(n >= 1);

wagon first = wagon(1);
wagon* last = &first;
for (unsigned int i = 2; i <= n; ++i) {
    wagon* new_wagon = new wagon(i);
    last->next = new_wagon;
    last = new_wagon;
}

std::cout << first.next->id;
std::cout << first.next->next->id;
```

Beliebig viele Wagons: (partielle) Lösung

```
unsigned int n; // number of wagons
std::cin >> n;
assert(n >= 1);

wagon first = wagon(1);
wagon* last = &first;
for (unsigned int i = 2; i <= n; ++i) {
    wagon* new_wagon = new wagon(i);
    last->next = new_wagon;
    last = new_wagon;
}

std::cout << first.next->id; // outputs 2
std::cout << first.next->next->id;
```

Beliebig viele Wagons: (partielle) Lösung

```
unsigned int n; // number of wagons
std::cin >> n;
assert(n >= 1);

wagon first = wagon(1);
wagon* last = &first;
for (unsigned int i = 2; i <= n; ++i) {
    wagon* new_wagon = new wagon(i);
    last->next = new_wagon;
    last = new_wagon;
}

std::cout << first.next->id; // outputs 2
std::cout << first.next->next->id; // outputs 3
```

21. Dynamische Datenstrukturen I

Verkettete Liste `std::list<int>` (vereinfacht)

Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff



Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff
- Jedes Element zeigt auf seinen Nachfolger



Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff
- Jedes Element zeigt auf seinen Nachfolger



Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff
- Jedes Element zeigt auf seinen Nachfolger
- Einfügen und Löschen beliebiger Elemente ist einfach



Anderes Speicherlayout: Verkettete Liste

- Kein zusammenhängender Speicherbereich und kein wahlfreier Zugriff
- Jedes Element zeigt auf seinen Nachfolger
- Einfügen und Löschen beliebiger Elemente ist einfach



⇒ `std::list<int>` kann als verkettete Liste realisiert werden

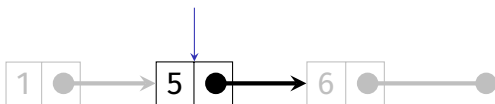
Verkettete Liste: Zoom



```
struct lnode {  
    int value;  
    lnode* next;  
  
    lnode(int v, lnode* n): value(v), next(n) {} // Constructor  
};
```

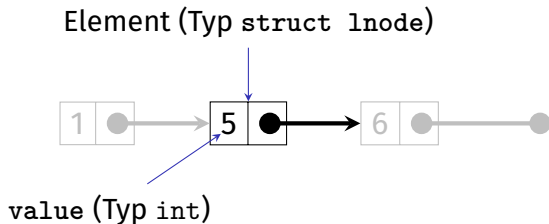
Verkettete Liste: Zoom

Element (Typ struct lnode)



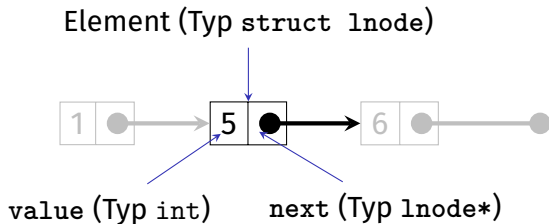
```
struct lnode {  
    int value;  
    lnode* next;  
  
    lnode(int v, lnode* n): value(v), next(n) {} // Constructor  
};
```

Verkettete Liste: Zoom



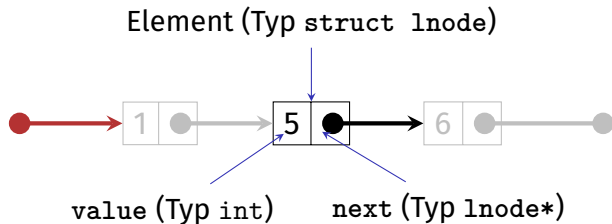
```
struct lnode {  
    int value;  
    lnode* next;  
  
    lnode(int v, lnode* n): value(v), next(n) {} // Constructor  
};
```

Verkettete Liste: Zoom



```
struct lnode {  
    int value;  
    lnode* next;  
  
    lnode(int v, lnode* n): value(v), next(n) {} // Constructor  
};
```


Liste = Zeiger aufs erste Element



```
class our_list {  
    lnode* head;  
public:  
    our_list(unsigned int size);  
    unsigned int size() const;  
    ...  
};
```

Funktion `our_list::print()`

```
struct lnode {  
    int value;  
    lnode* next;  
    ...  
};
```

```
void our_list::print() const {  
    for (lnode* n = head;  Zeiger auf erstes Element  
        n != nullptr;  
        n = n->next)  
    {  
        std::cout << n->value << ' ' ;  
    }  
}
```

Funktion `our_list::print()`

```
struct lnode {  
    int value;  
    lnode* next;  
    ...  
};
```

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  n = n->next)  
    {  
        std::cout << n->value << ' ';  
    }  
}
```

Abbruch falls Ende erreicht

Funktion `our_list::print()`

```
struct lnode {  
    int value;  
    lnode* next;  
    ...  
};
```

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next) {  
        std::cout << n->value << ' ';  
    }  
}
```

← Zeiger elementweise voranschieben

Funktion `our_list::print()`

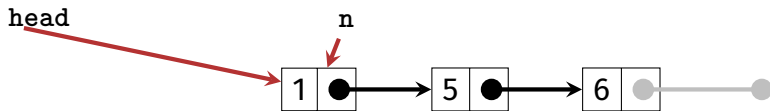
```
struct lnode {  
    int value;  
    lnode* next;  
    ...  
};
```

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' ';  
    }  
}
```

← Aktuelles Element ausgeben

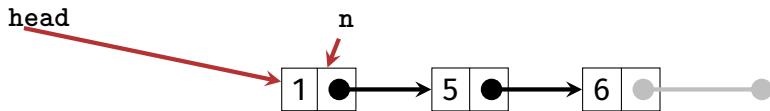
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' ';  
    }  
}
```



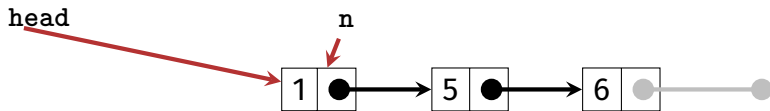
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' ';  
    }  
}
```



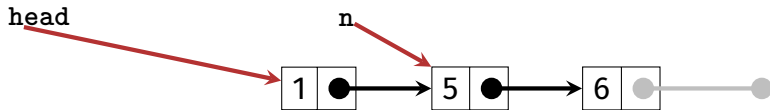
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1  
    }  
}
```



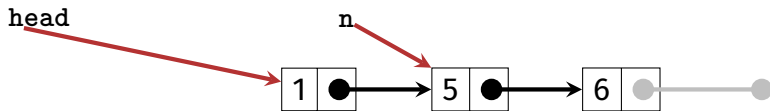
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1  
    }  
}
```



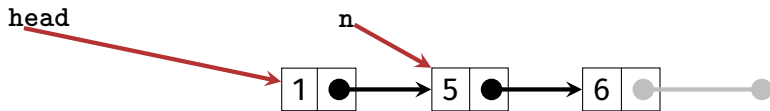
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1  
    }  
}
```



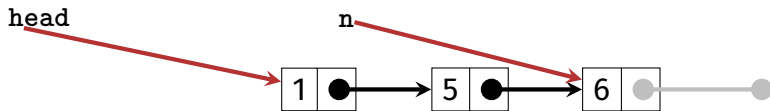
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5  
    }  
}
```



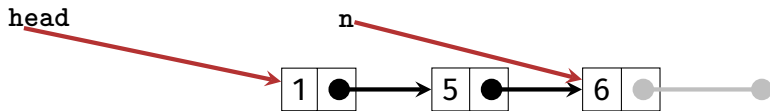
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5  
    }  
}
```



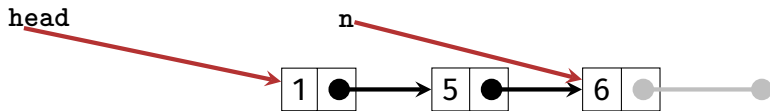
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5  
    }  
}
```



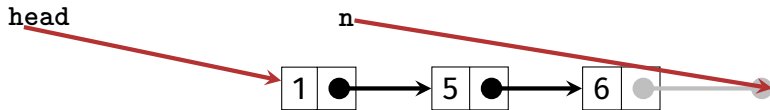
Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5 6  
    }  
}
```



Funktion `our_list::print()`

```
void our_list::print() const {  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
    {  
        std::cout << n->value << ' '; // 1 5 6  
    }  
}
```



Funktion `our_list::operator[]`

Zugriff auf i -tes Element ähnlich implementiert wie `print()`:

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  Zeiger auf erstes Element  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```


Funktion `our_list::operator[]`

Zugriff auf i -tes Element ähnlich implementiert wie `print()`:

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```



Bis zum i -ten voranschreiten

Funktion `our_list::operator[]`

Zugriff auf i -tes Element ähnlich implementiert wie `print()`:

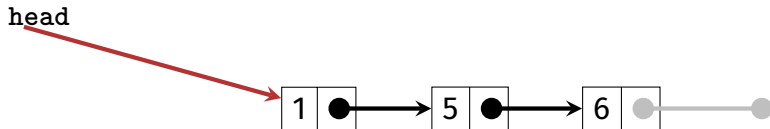
```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```

← `i`-tes Element zurückgeben

Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

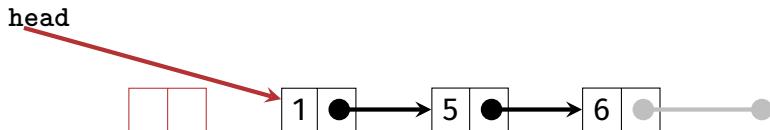
```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

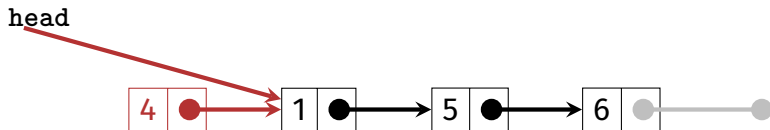
```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

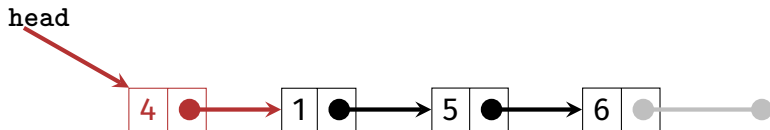
```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

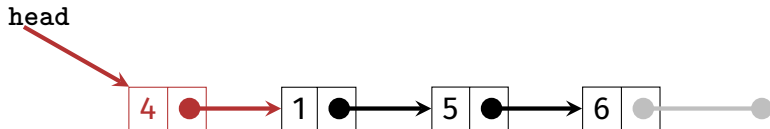
```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Funktion `our_list::push_front()`

Vorteil `our_list`: Elemente am Anfang anfügen ist sehr einfach:

```
void our_list::push_front(int e) {  
    head =  
        new lnode(e, head);  
}
```



Achtung: Wäre der neue `lnode` nicht *dynamisch* alloziert, dann würde er am Ende von `push_front` sofort wieder gelöscht (= Speicher dealloziert)

Funktion `our_list::our_list()`

Konstruktor kann mittels `push_front()` implementiert werden:

```
our_list::our_list(unsigned int size) {  
    head = nullptr;  head zeigt zunächst ins Nichts  
  
    for (unsigned int i = 0; i < size; ++i)  
        push_front(0);  
}
```


Funktion `our_list::our_list()`

Konstruktor kann mittels `push_front()` implementiert werden:

```
our_list::our_list(unsigned int size) {  
    head = nullptr;  
  
    for (unsigned int i = 0; i < size; ++i)  
        push_front(0);  
}
```

← size mal 0 vorne
anfügen

Funktion `our_list::our_list()`

Konstruktor kann mittels `push_front()` implementiert werden:

```
our_list::our_list(unsigned int size) {  
    head = nullptr;  
  
    for (unsigned int i = 0; i < size; ++i)  
        push_front(0);  
}
```

Anwendungsbeispiel:

```
our_list v = our_list(3);  
v.print(std::cout); // 0 0 0
```

Funktion `our_list::push_back()` v1

Einfach, aber ineffizient: Verkettete Liste bis ans Ende traversieren und neues Element anhängen.

```
void our_list::push_back(int e) {  
    lnode* n = head;     
  
    for (; n->next != nullptr; n = n->next);  
  
    n->next =  
        new lnode(e, nullptr);  
}
```

Funktion `our_list::push_back()` v1

Einfach, aber ineffizient: Verkettete Liste bis ans Ende traversieren und neues Element anhängen.

```
void our_list::push_back(int e) {  
    lnode* n = head;  
  
    for (; n->next != nullptr; n = n->next);  
  
    n->next =  
        new lnode(e, nullptr);  
}
```

... und bis zum letzten
Element gehen



Funktion `our_list::push_back()` v1

Einfach, aber ineffizient: Verkettete Liste bis ans Ende traversieren und neues Element anhängen.

```
void our_list::push_back(int e) {  
    lnode* n = head;  
  
    for (; n->next != nullptr; n = n->next);  
  
    n->next =  
        new lnode(e, nullptr)  
}
```

Neues Element an bisher
letztes anhängen

Funktion `our_list::push_back()` v1

Einfach, aber ineffizient: Verkettete Liste bis ans Ende traversieren und neues Element anhängen.

```
void our_list::push_back(int e) {  
    lnode* n = head;  
  
    for (; n->next != nullptr; n = n->next);  
  
    n->next =  
        new lnode(e, nullptr);  
}
```

Was passiert, wenn die Liste leer ist? 🤔

Funktion `our_list::push_back()` v1

Was passiert, wenn die Liste leer ist? → Dann ist `head == nullptr` ...

```
void our_list::push_back(int e) {  
    lnode* n = head;  
  
    for (; n->next != nullptr; n = n->next);  
    ...  
}
```

... und wir dereferenzieren `nullptr` → Undefined Behavior!

Funktion `our_list::push_back()` v2

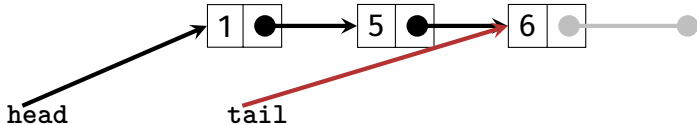
Finale Version, ohne vorheriges Undefined Behavior:

```
void our_list::push_back(int e) {  
    if (head == nullptr) { // Case 1: empty list  
        head = new lnode(e, nullptr);  
    } else { // Case 2: non-empty list (code as before)  
        lnode* n = head;  
        for (; n->next != nullptr; n = n->next);  
  
        n->next = new lnode(e, nullptr);  
    }  
}
```


Funktion `our_list::push_back()`

■ Effizienter, aber auch etwas komplexer:

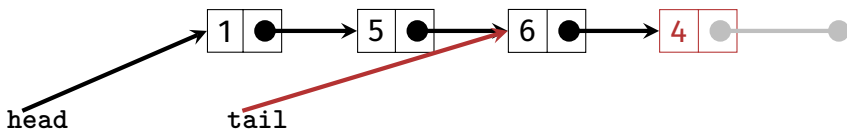
1. Zweiter Zeiger, der auf das letzte Element zeigt: `tail`



Funktion `our_list::push_back()`

■ Effizienter, aber auch etwas komplexer:

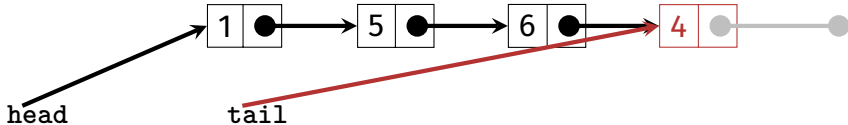
1. Zweiter Zeiger, der auf das letzte Element zeigt: `tail`
2. Mittels dieses Zeigers kann direkt am Ende angehängt werden



Funktion `our_list::push_back()`

■ Effizienter, aber auch etwas komplexer:

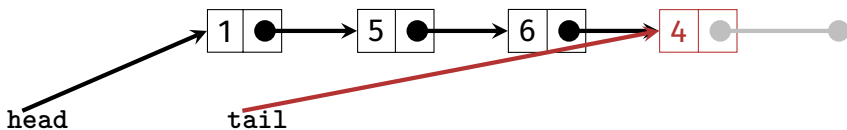
1. Zweiter Zeiger, der auf das letzte Element zeigt: `tail`
2. Mittels dieses Zeigers kann direkt am Ende angehängt werden



Funktion `our_list::push_back()`

■ Effizienter, aber auch etwas komplexer:

1. Zweiter Zeiger, der auf das letzte Element zeigt: `tail`
2. Mittels dieses Zeigers kann direkt am Ende angehängt werden



- ## ■ Aber:
- Verschiedene Grenzfälle, z.B. Vektor noch leer, müssen beachtet werden

Funktion `our_list::size()`

Einfach, aber ineffizient: Grösse durch abzählen *berechnen*

```
unsigned int our_list::size() const {  
    unsigned int c = 0;  
  
    for (_____  
        _____;  
        _____)  
    {  
        _____;  
    }  
  
    return c;  
}
```

Funktion `our_list::size()`

Einfach, aber ineffizient: Grösse durch Abzählen *berechnen*

```
unsigned int our_list::size() const {  
    unsigned int c = 0;  Zähler initial 0  
  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
        ++c;  
  
    return c;  
}
```

Funktion `our_list::size()`

Einfach, aber ineffizient: Grösse durch Abzählen *berechnen*

```
unsigned int our_list::size() const {  
    unsigned int c = 0;  
  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
        ++c;  
  
    return c;  
}
```



Länge der Kette abzählen

Funktion `our_list::size()`

Einfach, aber ineffizient: Grösse durch Abzählen *berechnen*

```
unsigned int our_list::size() const {  
    unsigned int c = 0;  
  
    for (lnode* n = head;  
         n != nullptr;  
         n = n->next)  
        ++c;  
  
    return c;   
}
```

Zähler zurückgeben

Funktion `our_list::size()`

Effizienter, aber etwas komplexer: Grösse als Membervariable *nachhalten*

1. Membervariable `unsigned int count` zur Klasse `our_list` hinzufügen

Funktion `our_list::size()`

Effizienter, aber etwas komplexer: Grösse als Membervariable *nachhalten*

1. Membervariable `unsigned int count` zur Klasse `our_list` hinzufügen
2. `count` muss nun bei *jeder* Operation, die die Grösse des Vektors verändert (z.B. `push_front`), aktualisiert werden

Ein Iterator für `our_list`

Wir brauchen:

1. Einen `our_list`-spezifischen Iterator mit mindestens folgender Funktionalität:
 - Zugriff aktuelles Element: `operator*`
 - Iterator vorwärtssetzen: `operator++`
 - Ende-Erreicht-Prüfung: `operator!=` (oder `operator==`)

Ein Iterator für `our_list`

Wir brauchen:

1. Einen `our_list`-spezifischen Iterator mit mindestens folgender Funktionalität:
 - Zugriff aktuelles Element: `operator*`
 - Iterator vorwärtssetzen: `operator++`
 - Ende-Erreicht-Prüfung: `operator!=` (oder `operator==`)
2. Memberfunktionen `begin()` und `end()` für `our_list` um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten

Iterator our_list::iterator (Schritt 1/2)

```
class our_list {  
    ...  
public:  
    class iterator {  
        ...  
    };  
  
    ...  
}
```

- Der Iterator gehört zu unserer Liste, daher ist `iterator` eine öffentliche *innere Klasse* von `our_list`

Iterator our_list::iterator (Schritt 1/2)

```
class our_list {  
    ...  
public:  
    class iterator {  
        ...  
    };  
  
    ...  
}
```

- Der Iterator gehört zu unserer Liste, daher ist `iterator` eine öffentliche *innere Klasse* von `our_list`
- Instanzen unseres Iterators sind vom Typ `our_list::iterator`

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;   
  
public:  
    iterator(lnode* n);  
    iterator& operator++();  
    int& operator*() const;  
    bool operator!=(const iterator& other) const;  
};
```

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;  
  
public:  
    iterator(lnode* n);  
    iterator& operator++();  
    int& operator*() const;  
    bool operator!=(const iterator& other) const;  
};
```

Erzeuge Iterator auf bestimmtes Element

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;  
  
public:  
    iterator(lnode* n);  
    iterator& operator++(); ← Iterator ein Element vorwärtssetzen  
    int& operator*() const;  
    bool operator!=(const iterator& other) const;  
};
```

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;  
  
public:  
    iterator(lnode* n);  
    iterator& operator++();  
    int& operator*() const; ← Zugriff auf aktuelles Element  
    bool operator!=(const iterator& other) const;  
};
```

Iterator our_list::iterator (Schritt 1/2)

```
class iterator {  
    lnode* node;  
  
public:  
    iterator(lnode* n);  
    iterator& operator++();  
    int& operator*() const;  
    bool operator!=(const iterator& other) const;  
};
```



Vergleich mit anderem Iterator

Iterator our_list::iterator (Schritt 1/2)

```
// Constructor
our_list::iterator::iterator(lnode* n): node(n) {}

// Pre-increment
our_list::iterator& our_list::iterator::operator++() {
    assert(node != nullptr);

    node = node->next;

    return *this;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Constructor
our_list::iterator::iterator(lnode* n): node(n) {}

// Pre-increment
our_list::iterator& our_list::iterator::operator++() {
    assert(node != nullptr);

    node = node->next;

    return *this;
}
```

Iterator initial auf n zeigen lassen

Iterator our_list::iterator (Schritt 1/2)

```
// Constructor
our_list::iterator::iterator(lnode* n): node(n) {}

// Pre-increment
our_list::iterator& our_list::iterator::operator++() {
    assert(node != nullptr);

    node = node->next;

    return *this;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Constructor
our_list::iterator::iterator(lnode* n): node(n) {}

// Pre-increment
our_list::iterator& our_list::iterator::operator++() {
    assert(node != nullptr);

    node = node->next; ← Iterator ein Element vorwärtssetzen

    return *this;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Constructor
our_list::iterator::iterator(lnode* n): node(n) {}

// Pre-increment
our_list::iterator& our_list::iterator::operator++() {
    assert(node != nullptr);

    node = node->next;

    return *this;
}
```

Referenz auf verschobenen Iterator zurückgeben

Iterator our_list::iterator (Schritt 1/2)

```
// Element access
int& our_list::iterator::operator*() const {
    return node->value;
}

// Comparison: when are two iterators not equal?
bool our_list::iterator::operator!=(
    const our_list::iterator& other) const
{
    return node != other.node;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Element access
int& our_list::iterator::operator*() const {
    return node->value; ← Zugriff auf aktuelles Element
}

// Comparison: when are two iterators not equal?
bool our_list::iterator::operator!=(
    const our_list::iterator& other) const
{
    return node != other.node;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Element access
int& our_list::iterator::operator*() const {
    return node->value;
}

// Comparison: when are two iterators not equal?
bool our_list::iterator::operator!=(
    const our_list::iterator& other) const
{
    return node != other.node;
}
```

Iterator our_list::iterator (Schritt 1/2)

```
// Element access
int& our_list::iterator::operator*() const {
    return node->value;
}

// Comparison: when are two iterators not equal?
bool our_list::iterator::operator!=(
    const our_list::iterator& other) const
{
    return node != other.node; ←
```

this Iterator verschieden von other falls sie
auf unterschiedliche Elemente zeigen

Ein Iterator für `our_list` (Wiederholung)

Wir brauchen:

1. Einen `our_list`-spezifischen Iterator mit mindestens folgender Funktionalität:
 - Zugriff aktuelles Element: `operator*`
 - Iterator vorwärtssetzen: `operator++`
 - Ende-Erreicht-Prüfung: `operator!=` (oder `operator==`)
2. Memberfunktionen `begin()` und `end()` für `our_list` um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten



Iterator our_list::iterator (Schritt 2/2)

```
class our_list {  
    ...  
public:  
    class iterator {...};  
  
    iterator begin();  
    iterator end();  
  
    ...  
}
```

our_list braucht Memberfunktionen um Iteratoren *auf den Anfang* bzw. *hinter das Ende* des Vektors herausgeben zu können

Iterator our_list::iterator (Schritt 2/2)

```
our_list::iterator our_list::begin() {  
    return our_list::iterator(head);  
}
```

Iterator auf erstes Vektorelement

```
our_list::iterator our_list::end() {  
    return our_list::iterator(nullptr);  
}
```

Iterator our_list::iterator (Schritt 2/2)

```
our_list::iterator our_list::begin() {  
    return our_list::iterator(head);  
}
```

```
our_list::iterator our_list::end() {  
    return our_list::iterator(nullptr);  
}
```



Iterator hinter letztes Vektorelement

Random Access mit `our_list::operator[]`?

Zugriff auf i -tes Element

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```

Random Access mit `our_list::operator[]`?

Zugriff auf i -tes Element

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```

muss durch die i ersten Elemente iterieren...

Random Access mit `our_list::operator[]`?

Zugriff auf i -tes Element

```
int& our_list::operator[](unsigned int i) {  
    lnode* n = head;  
  
    for (int j = 0; j < i; ++j)  
        n = n->next;  
  
    return n->value;  
}
```

muss durch die i ersten Elemente iterieren...

Wie erhalten wir Random Access?

22. Dynamische Datenstrukturen II und Speicherverwaltung

Problem

Letzte Woche: Dynamische Datenstruktur `our_list`

Haben dynamischen Speicher angelegt, aber nicht wieder freigegeben.
Insbesondere: keine Funktionen zum Entfernen von Elementen aus `our_list`.

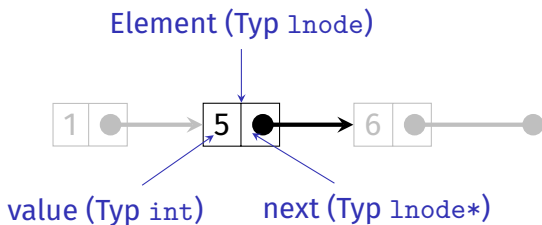
Heute: Korrektes Speichermanagement mit `new` und `delete` mit Hilfe der (einfacheren) dynamischen Datenstruktur `stack`.



Ziel: Stapelklasse mit Speichermanagement

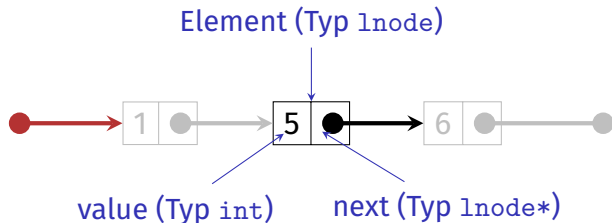
```
class stack {  
public:  
    // post: Element auf den Stapel legen  
    void push(int value);  
    // pre: Stack nicht leer  
    // post: Entfernt oberstes Element vom Stapel  
    void pop();  
    // pre: Stack nicht leer  
    // post: Gibt Wert des obersten Elementes zurück  
    int top() const;  
    // post: gibt zurück, ob Stack leer ist  
    bool empty() const;  
    ...  
};
```

Erinnerung: Verkettete Liste



```
struct lnode {  
    int value;  
    lnode* next;  
    // constructor  
    lnode(int v, lnode* n) : value(v), next(n) {}  
};
```

Stapel = Zeiger aufs oberste Element



```
class stack {  
public:  
    void push(int value);  
    ...  
private:  
    lnode* topn;  
};
```


Leerer Stapel

```
class stack {  
public:  
    stack() : topn(nullptr) {} // default constructor  
  
    void push(int value);  
    void pop();  
    int top() const;  
    bool empty() const;  
  
private:  
    lnode* topn;  
}
```

empty(), top()

// POST: Returns true if stack is empty, false otherwise

```
bool stack::empty() const {
```

```
    _____  
}
```

// PRE: Stack must be non-empty

// POST: Returns the stack's top value

```
int stack::top() const {
```

```
    _____  
    _____  
}
```

empty(), top()

```
// POST: Returns true if stack is empty, false otherwise
bool stack::empty() const {
    return topn == nullptr;
}

// PRE: Stack must be non-empty
// POST: Returns the stack's top value
int stack::top() const {
    _____
    _____
}
```

empty(), top()

```
// POST: Returns true if stack is empty, false otherwise
bool stack::empty() const {
    return topn == nullptr;
}

// PRE: Stack must be non-empty
// POST: Returns the stack's top value
int stack::top() const {
    assert(!empty());
    

---


}
```

empty(), top()

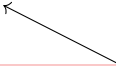
```
// POST: Returns true if stack is empty, false otherwise
bool stack::empty() const {
    return topn == nullptr;
}

// PRE: Stack must be non-empty
// POST: Returns the stack's top value
int stack::top() const {
    assert(!empty());
    return topn->value;
}
```

empty(), top()

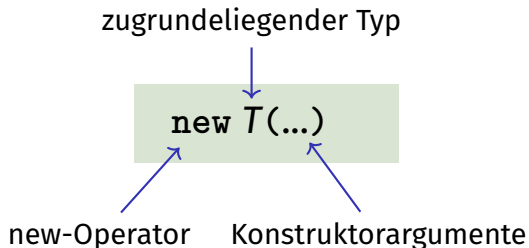
```
// POST: Returns true if stack is empty, false otherwise
bool stack::empty() const {
    return topn == nullptr;
}

// PRE: Stack must be non-empty
// POST: Returns the stack's top value
int stack::top() const {
    assert(!empty());
    return topn->value;
}
```



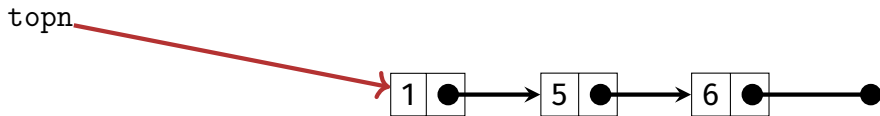
Erinnerung: Abkürzung für (*topn).value

Erinnerung: der `new`-Ausdruck



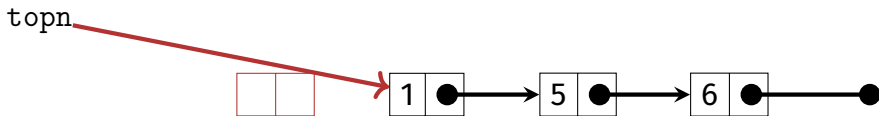
- **Effekt**: Speicher für ein neues Objekt vom Typ T wird alloziert ...
- ...und mit Hilfe des passenden Konstruktors initialisiert
- **Wert**: Adresse des neuen T -Objekt, **Typ**: Zeiger T^* !

```
void stack::push(int value) {  
    topn = new lnode(value, topn);  
}
```



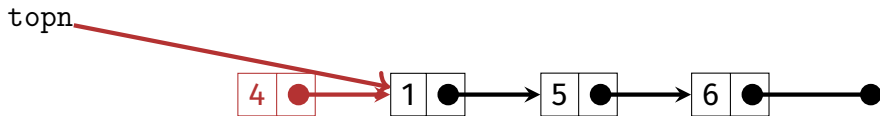
- **Effekt:** Neues Objekt vom Typ T wird im Speicher angelegt ...

```
void stack::push(int value) {  
    topn = new lnode(value, topn);  
}
```



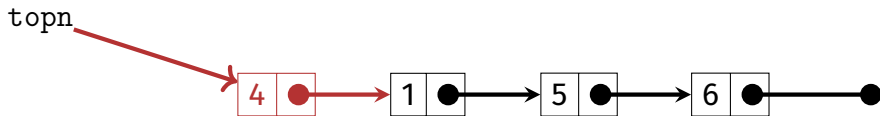
- **Effekt:** Neues Objekt vom Typ T wird im Speicher angelegt ...
- ...und mit Hilfe des passenden Konstruktors initialisiert.

```
void stack::push(int value) {  
    topn = new lnode(value, topn);  
}
```



- **Effekt:** Neues Objekt vom Typ T wird im Speicher angelegt ...
- ...und mit Hilfe des passenden Konstruktors initialisiert.
- **Wert:** Adresse des neuen Objekts

```
void stack::push(int value) {  
    topn = new lnode(value, topn);  
}
```

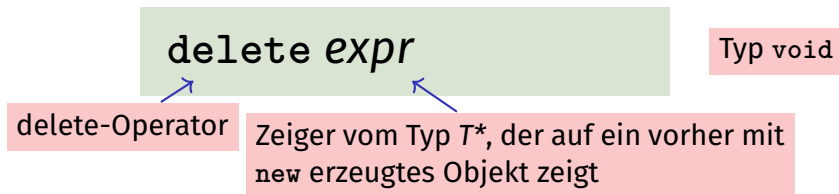


Der delete-Ausdruck

Objekte, die mit `new` erzeugt worden sind, haben dynamische Speicherdauer: sie „leben“, bis sie *explizit gelöscht* werden.

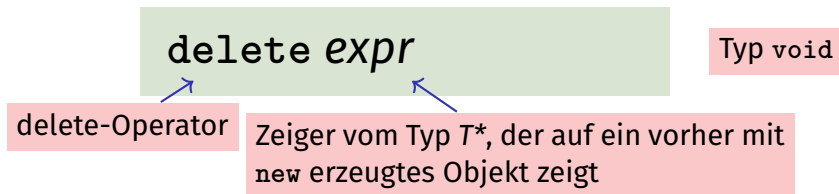
Der delete-Ausdruck

Objekte, die mit `new` erzeugt worden sind, haben dynamische Speicherdauer: sie „leben“, bis sie *explizit gelöscht* werden.



Der delete-Ausdruck

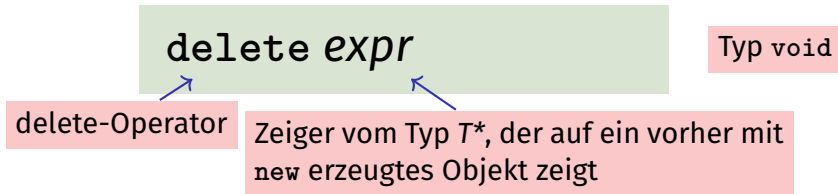
Objekte, die mit `new` erzeugt worden sind, haben dynamische Speicherdauer: sie „leben“, bis sie *explizit gelöscht* werden.



- **Effekt:** Objekt wird dekonstruiert (Erklärung folgt)
... und Speicher wird freigegeben.

Der delete-Ausdruck

Objekte, die mit `new` erzeugt worden sind, haben dynamische Speicherdauer: sie „leben“, bis sie *explizit gelöscht* werden.



- **Effekt:** Objekt wird dekonstruiert (Erklärung folgt)
... und Speicher wird freigegeben.

Achtung: `delete` setzt den Zeiger *nicht* auf `nullptr`

Wer geboren wird, muss sterben...

Richtlinie „Dynamischer Speicher“

Zu jedem `new` gibt es ein passendes `delete`!

Wer geboren wird, muss sterben...

Richtlinie „Dynamischer Speicher“

Zu jedem `new` gibt es ein passendes `delete`!

Nichtbeachtung führt zu *Speicherlecks*:

Wer geboren wird, muss sterben...

Richtlinie „Dynamischer Speicher“

Zu jedem `new` gibt es ein passendes `delete`!

Nichtbeachtung führt zu *Speicherlecks*:

- Ungenutzte Objekte, die den Speicher blockieren...

Wer geboren wird, muss sterben...

Richtlinie „Dynamischer Speicher“

Zu jedem `new` gibt es ein passendes `delete`!

Nichtbeachtung führt zu *Speicherlecks*:

- Ungenutzte Objekte, die den Speicher blockieren...
- ...bis er irgendwann voll ist (**heap overflow**)

Aufpassen mit `new` und `delete`!

```
rational* t = new rational;  
rational* s = t;  
delete s;  
int nominator = t->denominator();
```

Aufpassen mit `new` und `delete`!

```
rational* t = new rational; ←  
rational* s = t;  
delete s;  
int nominator = t->denominator();
```

Speicher für t wird angelegt

Aufpassen mit `new` und `delete`!

```
rational* t = new rational;  Speicher für t wird angelegt  
rational* s = t;  Auch andere Zeiger können auf das Objekt zeigen..  
delete s;  
int nominator = t->denominator();
```

Aufpassen mit `new` und `delete`!

```
rational* t = new rational; ← Speicher für t wird angelegt  
rational* s = t; ← Auch andere Zeiger können auf das Objekt zeigen..  
delete s; ← ... und zur Freigabe verwendet werden.  
int nominator = t->denominator();
```

Aufpassen mit `new` und `delete`!

```
rational* t = new rational; ← Speicher für t wird angelegt  
rational* s = t; ← Auch andere Zeiger können auf das Objekt zeigen..  
delete s; ← ... und zur Freigabe verwendet werden.  
int nominator = (*t).denominator(); ← Fehler: Speicher freigegeben!
```

Dereferenzieren eines „dangling pointers“

- Zeiger auf freigegebene Objekte: hängende Zeiger (*dangling pointers*)

Aufpassen mit `new` und `delete`!

```
rational* t = new rational; ← Speicher für t wird angelegt  
rational* s = t; ← Auch andere Zeiger können auf das Objekt zeigen..  
delete s; ← ... und zur Freigabe verwendet werden.  
int nominator = (*t).denominator(); ← Fehler: Speicher freigegeben!
```

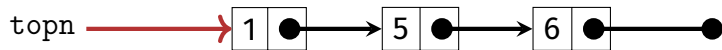
↑

Dereferenzieren eines „dangling pointers“

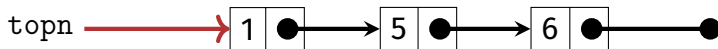
- Zeiger auf freigegebene Objekte: hängende Zeiger (*dangling pointers*)
- Mehrfache Freigabe eines Objektes mit `delete` ist ein ähnlicher schwerer Fehler.

Weiter mit dem Stapel:

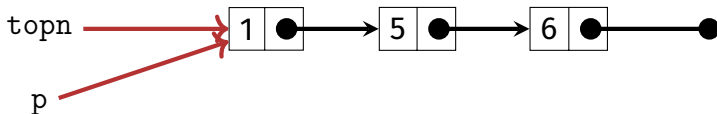
pop()



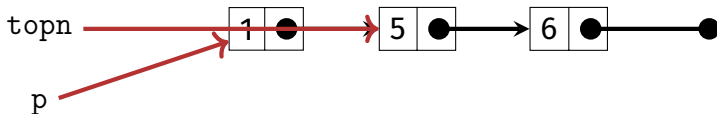
```
void stack::pop() {  
    assert(!empty());  
    lnode* p = topn;  
    topn = topn->next;  
    delete p;  
}
```



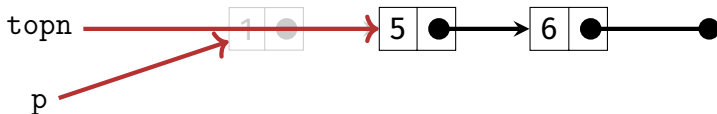
```
void stack::pop() {  
    assert(!empty());  
    lnode* p = topn;  
    topn = topn->next;  
    delete p;  
}
```



```
void stack::pop() {  
    assert(!empty());  
    lnode* p = topn;  
    topn = topn->next;  
    delete p;  
}
```

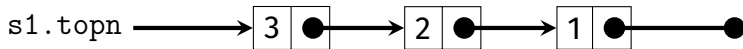


```
void stack::pop() {  
    assert(!empty());  
    lnode* p = topn;  
    topn = topn->next;  
    delete p;  
}
```



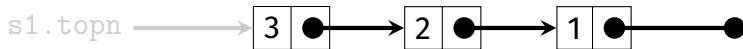
Zombie-Elemente

```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



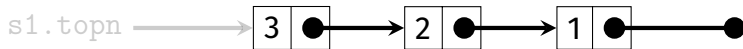
Zombie-Elemente

```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



Zombie-Elemente

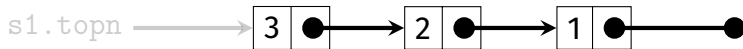
```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



- `s1` nach dem Codeblock nicht mehr zugreifbar (Stapel ist „gestorben“) ...

Zombie-Elemente

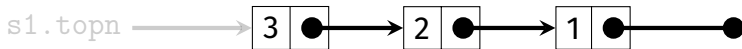
```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



- s1 nach dem Codeblock nicht mehr zugreifbar (Stapel ist „gestorben“) ...
- ...aber die drei *Elemente* (1node-Instanzen) des Stapels leben weiter
→ Speicherleck!

Zombie-Elemente

```
{  
    stack s1; // lokale Variable  
    s1.push(1);  
    s1.push(2);  
    s1.push(3);  
    std::cout << s1; // 3 2 1  
}
```



- `s1` nach dem Codeblock nicht mehr zugreifbar (Stapel ist „gestorben“) ...
- ...aber die drei *Elemente* (`1node`-Instanzen) des Stapels leben weiter
→ Speicherleck!
- Sie sollten zusammen mit `s1` dealloziert werden

Der Destruktor

- Der Destruktor einer Klasse T ist die eindeutige Memberfunktion mit Deklaration

$\sim T ();$

- Wird automatisch aufgerufen, wenn die Speicherdauer eines Klassenobjekts vom Typ T endet – z.B. bei Aufruf von `delete` auf einem Objekt vom Typ T^* oder wenn der Gültigkeitsbereich eines Objektes vom Typ T endet.

Der Destruktor

- Der Destruktor einer Klasse T ist die eindeutige Memberfunktion mit Deklaration

$\sim T ();$

- Wird automatisch aufgerufen, wenn die Speicherdauer eines Klassenobjekts vom Typ T endet – z.B. bei Aufruf von `delete` auf einem Objekt vom Typ T^* oder wenn der Gültigkeitsbereich eines Objektes vom Typ T endet.
- Falls kein Destruktor deklariert ist, so wird er automatisch erzeugt und ruft die Destruktoren für die Membervariablen auf (Zeiger `topn`, kein Effekt – Grund für Zombie-Elemente)

Mit dem Destruktor klappt's!

```
// POST: the dynamic memory of *this is deleted
stack::~~stack() {
    while (topn != nullptr) {
        lnode* t = topn;
        topn = t->next;
        delete t;
    }
}
```

Mit dem Destruktor klappt's!

```
// POST: the dynamic memory of *this is deleted
stack::~~stack() {
    while (topn != nullptr) {
        lnode* t = topn;
        topn = t->next;
        delete t;
    }
}
```

- löscht automatisch alle Stapелеlemente, wenn der Stapel ungültig wird

Mit dem Destruktor klappt's!

```
// POST: the dynamic memory of *this is deleted
stack::~~stack() {
    while (topn != nullptr) {
        lnode* t = topn;
        topn = t->next;
        delete t;
    }
}
```

- löscht automatisch alle Stapелеlemente, wenn der Stapel ungültig wird
- Unsere Stapel-Klasse scheint jetzt die Richtlinie „Dynamischer Speicher“ zu befolgen (?)

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n";
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n";
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n";
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n";
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n";
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n";
```

```
s2.pop();
```


Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1
```

```
s2.pop();
```

Stapel fertig?

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1
```

```
s2.pop(); // Oops, Programmabsturz!
```

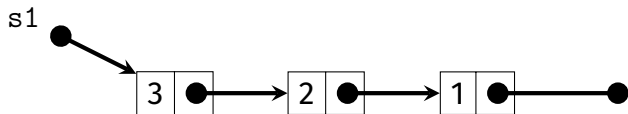
```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1
```

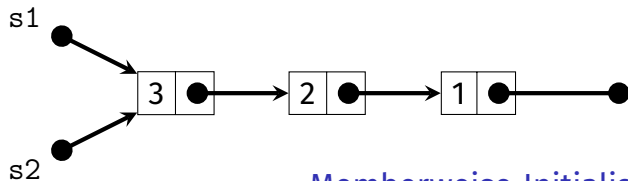
```
s2.pop(); // Oops, Programmabsturz!
```

Was ist hier schiefgegangen?



```
...  
stack s2 = s1;  
std::cout << s2 << "\n";  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

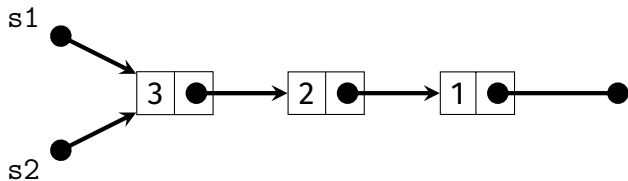
Was ist hier schiefgegangen?



Memberweise Initialisierung: kopiert nur
den topn-Zeiger

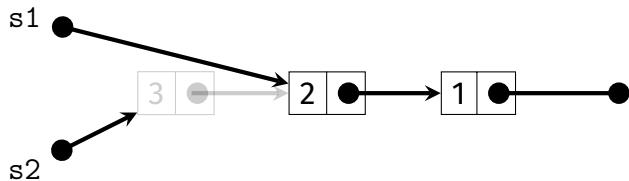
```
...  
stack s2 = s1; ←  
std::cout << s2 << "\n";  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

Was ist hier schiefgegangen?



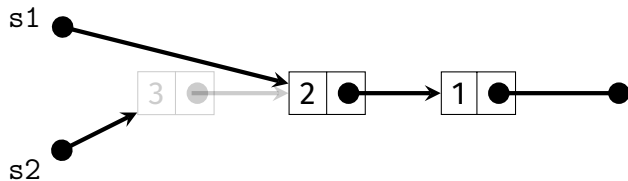
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

Was ist hier schiefgegangen?



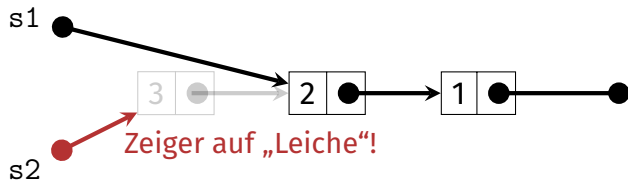
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```


Was ist hier schiefgegangen?



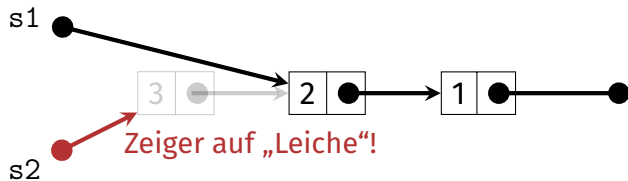
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop();
```

Was ist hier schiefgegangen?



```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop();
```

Was ist hier schiefgegangen?



```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop(); // Oops, Programmabsturz!
```

Das eigentliche Problem

Schon das geht schief:

```
{  
    stack s1;  
    s1.push(1);  
    stack s2 = s1;  
}
```

Beim Verlassen des Gültigkeitsbereiches werden beide Stacks aufgeräumt (dekonstruiert). Aber beide Stacks versuchen dieselben Daten zu löschen, denn sie haben Zugriff auf denselben Zeiger.

Lösungsmöglichkeiten

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).
oder
2. Wir erlauben keine "Kopien".

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. Wir erlauben keine "Kopien".

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

Lösungsmöglichkeiten

1. **Wir erstellen eine echte Kopie aller Daten (deep copy).**

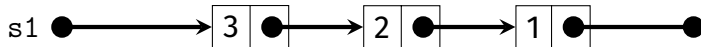
oder

2. Wir erlauben keine "Kopien".

oder

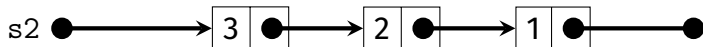
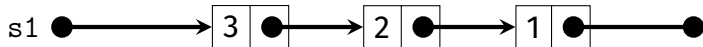
3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

1. Wir erstellen eine echte Kopie!



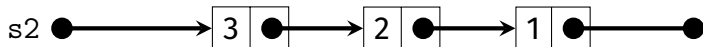
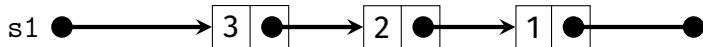
```
...  
stack s2 = s1;  
std::cout << s2 << "\n";  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



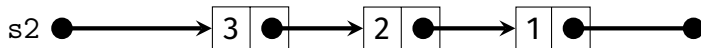
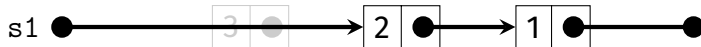
```
...  
stack s2 = s1;  
std::cout << s2 << "\n";  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



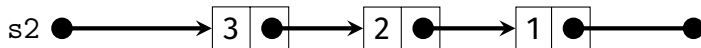
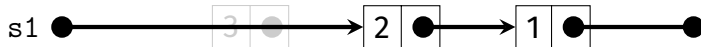
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



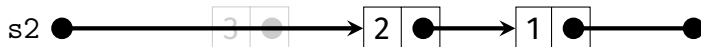
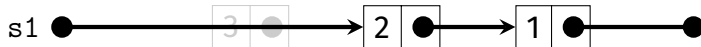
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n";  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



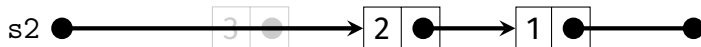
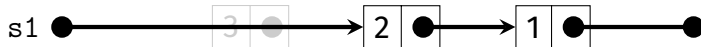
```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop();
```

1. Wir erstellen eine echte Kopie!



```
...  
stack s2 = s1;  
std::cout << s2 << "\n"; // 3 2 1  
  
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
  
s2.pop(); // ok
```


Der Copy-Konstruktor

- Der Copy-Konstruktor einer Klasse T ist der eindeutige Konstruktor mit Deklaration

$T(\text{const } T\& x);$

- wird automatisch aufgerufen, wenn Werte vom Typ T mit Werten vom Typ T *initialisiert* werden

$T\ x = t; \quad (t \text{ vom Typ } T)$

$T\ x(t);$

Der Copy-Konstruktor

- Der Copy-Konstruktor einer Klasse T ist der eindeutige Konstruktor mit Deklaration

$T(\text{const } T\& x);$

- wird automatisch aufgerufen, wenn Werte vom Typ T mit Werten vom Typ T *initialisiert* werden

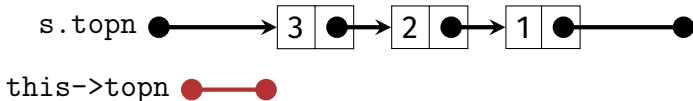
$T\ x = t;$ (t vom Typ T)

$T\ x(t);$

- Falls kein Copy-Konstruktor deklariert ist, so wird er automatisch erzeugt (und initialisiert memberweise – Grund für obiges Problem)

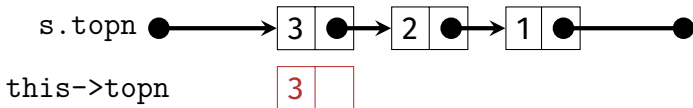
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



Mit dem Copy-Konstruktor klappt's!

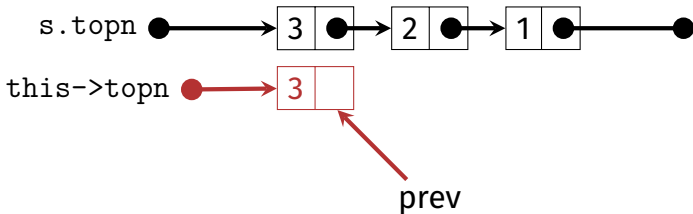
```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



prev

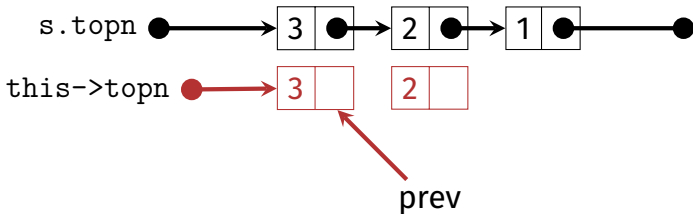
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



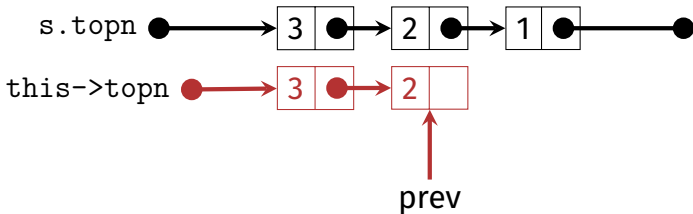
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



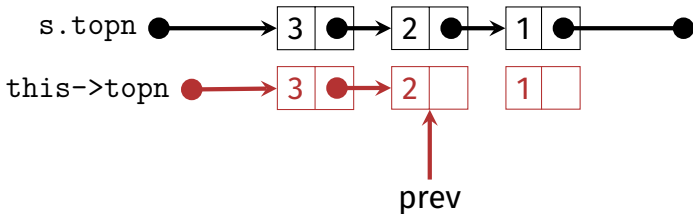
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



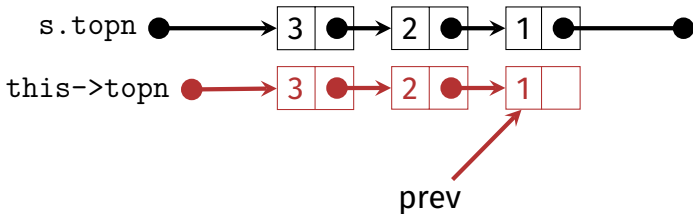
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



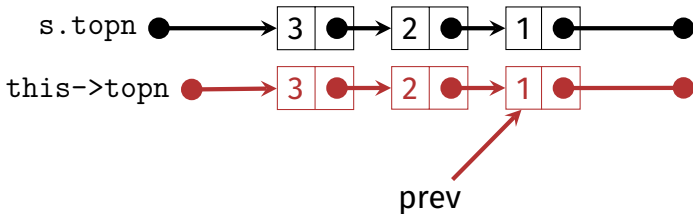
Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



Mit dem Copy-Konstruktor klappt's!

```
// POST: *this is initialized with a copy of s
stack::stack(const stack& s) : topn (nullptr) {
    if (s.topn == nullptr) return;
    topn = new lnode(s.topn->value, nullptr);
    lnode* prev = topn;
    for (lnode* n = s.topn->next; n != nullptr; n = n->next) {
        lnode* copy = new lnode(n->value, nullptr);
        prev->next = copy;
        prev = copy;
    }
}
```



Initialisierung $\neq$ Zuweisung!

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2 = s1; // Initialisierung
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
s2.pop(); // ok: Copy-Konstruktor!
```

Initialisierung $\neq$ Zuweisung!

```
stack s1;  
s1.push(1);  
s1.push(2);  
s1.push(3);  
std::cout << s1 << "\n"; // 3 2 1
```

```
stack s2;  
s2 = s1; // Zuweisung
```

```
s1.pop();  
std::cout << s1 << "\n"; // 2 1  
s2.pop(); // Oops, Programmabsturz!
```

Der Zuweisungsoperator

- Überladen von `operator=` als Memberfunktion
- Wie Copy-Konstruktor ohne Initialisierer, aber zusätzlich

Der Zuweisungsoperator

- Überladen von `operator=` als Memberfunktion
- Wie Copy-Konstruktor ohne Initialisierer, aber zusätzlich
 - Prüfen auf Selbstzuweisung (`s1 = s1`), die keinen Effekt haben sollten

Der Zuweisungsoperator

- Überladen von `operator=` als Memberfunktion
- Wie Copy-Konstruktor ohne Initialisierer, aber zusätzlich
 - Prüfen auf Selbstzuweisung (`s1 = s1`), die keinen Effekt haben sollten
 - Freigeben des Speichers des bisherigen (jetzt alten) Inhalts

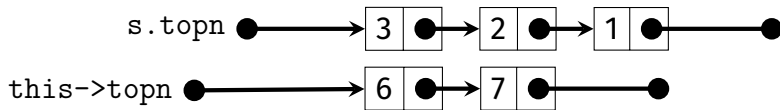
Der Zuweisungsoperator

- Überladen von `operator=` als Memberfunktion
- Wie Copy-Konstruktor ohne Initialisierer, aber zusätzlich
 - Prüfen auf Selbstzuweisung (`s1 = s1`), die keinen Effekt haben sollten
 - Freigeben des Speichers des bisherigen (jetzt alten) Inhalts
- Falls kein Zuweisungsoperator deklariert ist, so wird er automatisch erzeugt (und weist memberweise zu – Grund für obiges Problem)

Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){

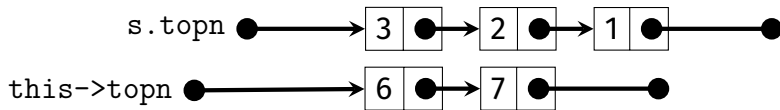
}
}
```



Mit dem Zuweisungsoperator klappt's!

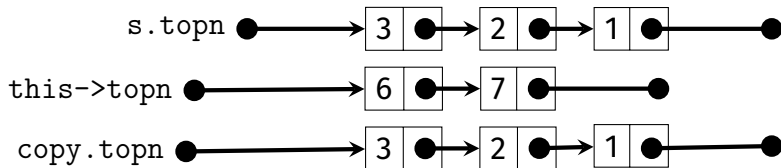
```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung

    }
}
```



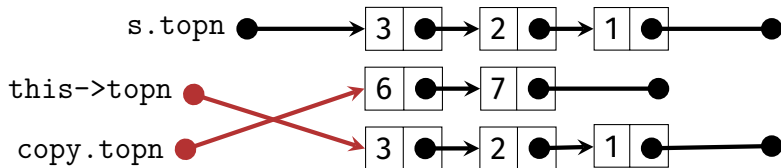
Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
    }
}
```



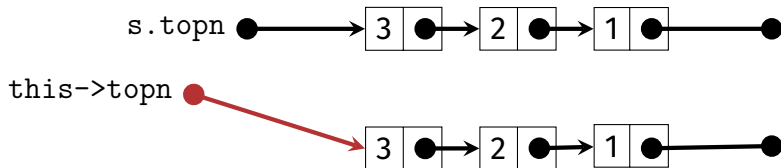
Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
        std::swap(topn, copy.topn); // copy hat nun den Müll!
    }
}
```



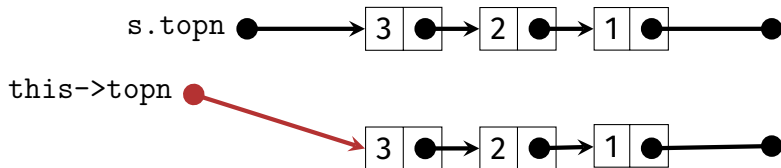
Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
        std::swap(topn, copy.topn); // copy hat nun den Müll!
    } // copy wird aufgeräumt -> Dekonstruktion
}
```



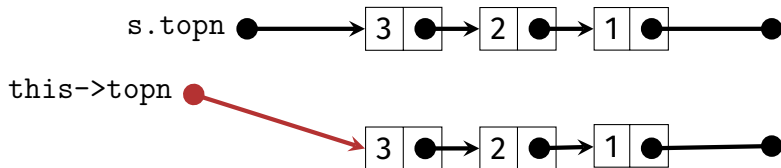
Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
        std::swap(topn, copy.topn); // copy hat nun den Müll!
    } // copy wird aufgeräumt -> Dekonstruktion
    return *this; // Rückgabe als L-Wert (Konvention)
}
```



Mit dem Zuweisungsoperator klappt's!

```
// POST: *this (left operand) becomes a copy of s (right operand)
stack& stack::operator=(const stack& s){
    if (this != &s) { // keine Selbstzuweisung
        stack copy = s; // Kopierkonstruktor
        std::swap(topn, copy.topn); // copy hat nun den Müll!
    } // copy wird aufgeräumt -> Dekonstruktion
    return *this; // Rückgabe als L-Wert (Konvention)
}
```



Copy & Swap-Idiom: Coooooler Trick! 😊

Fertig

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s); // copy constructor  
    stack& operator=(const stack& s); // assignment operator  
  
    void push(int value);  
    void pop();  
    int top() const;  
    bool empty() const;  
private:  
    lnode* topn;  
};
```


Dreierregel (Rule of Three)

Typ, der dynamischen Speicher verwaltet (z.B. unsere Klasse für Stapel), braucht:

- Konstruktoren
- Destruktor
- Copy-Konstruktor
- Zuweisungsoperator

Dreierregel (Rule of Three)

Typ, der dynamischen Speicher verwaltet (z.B. unsere Klasse für Stapel), braucht:

- Konstruktoren
 - Destruktor
 - Copy-Konstruktor
 - Zuweisungsoperator
- } Dreierregel: definiert eine Klasse eines davon, so muss sie auch die anderen zwei definieren!

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. Wir erlauben keine "Kopien".

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. **Wir erlauben keine "Kopien".**

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

2. Stapel ohne Kopien!

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s)=delete; // no copy constructor  
    stack& operator=(const stack& s)=delete; // no assignment operator  
    ...  
};
```

2. Stapel ohne Kopien!

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s)=delete; // no copy constructor  
    stack& operator=(const stack& s)=delete; // no assignment operator  
    ...  
};
```

```
stack s1;  
stack s2;  
s2 = s1 // compiler error: use of deleted function
```

2. Stapel ohne Kopien!

```
class stack {  
public:  
    stack(); // constructor  
    ~stack(); // destructor  
    stack(const stack& s)=delete; // no copy constructor  
    stack& operator=(const stack& s)=delete; // no assignment operator  
    ...  
};
```

```
stack s1;  
stack s2;  
s2 = s1 // compiler error: use of deleted function
```

```
stack s1;  
stack s3 = s1; // compiler error: use of deleted function
```

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. Wir erlauben keine "Kopien".

oder

3. Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).

Lösungsmöglichkeiten

1. Wir erstellen eine echte Kopie aller Daten (deep copy).

oder

2. Wir erlauben keine "Kopien".

oder

3. **Wir lösen obiges Problem der gemeinsam genutzten Datenstruktur (hier nicht weiter diskutiert).**

Smart Pointers

Smart Pointers sind Pointers mit zusätzlicher Funktionalität, die das Memory Management für uns übernehmen.

Smart Pointers

Smart Pointers sind Pointers mit zusätzlicher Funktionalität, die das Memory Management für uns übernehmen.

`std::shared_ptr<T>`

zählt die Anzahl Zeiger, die auf ein Objekt verweisen. Löscht Objekt nur wenn diese Anzahl auf 0 zurückfällt.

Smart Pointers

Smart Pointers sind Pointers mit zusätzlicher Funktionalität, die das Memory Management für uns übernehmen.

`std::shared_ptr<T>`

zählt die Anzahl Zeiger, die auf ein Objekt verweisen. Löscht Objekt nur wenn diese Anzahl auf 0 zurückfällt.

`std::unique_ptr<T>`

Verbietet, dass mehrere Zeiger auf ein Objekt zeigen können.

Shared Pointers: Beispiel

■ Shared Pointer erstellen

```
// create shared pointer with std::make_shared<T>
// std::make_shared<T> takes same arguments as constructor of T
#include <memory>
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);
```

Shared Pointers: Beispiel

■ Shared Pointer erstellen

```
// create shared pointer with std::make_shared<T>
// std::make_shared<T> takes same arguments as constructor of T
#include <memory>
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);
```

■ Handling des Shared Pointers wie mit normalem Pointer

```
// dereferencing
rational r = *sp1;
// assignment
std::shared_ptr<rational> sp2 = sp1;
// member access
std::cout << sp2->n << " / " << sp2->d; // 3 / 4
```

Shared Pointers: Beispiel

- Es wird mitgezählt, wie viele Shared Pointer auf das Objekt zeigen

```
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);  
std::shared_ptr<rational> sp2 = sp1;  
std::cout << sp1.use_count(); // 2
```

Shared Pointers: Beispiel

- Es wird mitgezählt, wie viele Shared Pointer auf das Objekt zeigen

```
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);  
std::shared_ptr<rational> sp2 = sp1;  
std::cout << sp1.use_count(); // 2
```

- Wenn ein Shared Pointer out of scope geht, wird Counter dekrementiert

```
{  
    std::shared_ptr<rational> sp3 = sp2;  
    std::cout << sp1.use_count(); // 3  
} // sp3 goes out of scope here; counter is decreased  
sp1 = nullptr; // explicitly remove pointer sp1 to object  
std::cout << sp2.use_count(); // 1
```


Shared Pointers: Beispiel

- Es wird mitgezählt, wie viele Shared Pointer auf das Objekt zeigen

```
std::shared_ptr<rational> sp1 = std::make_shared<rational>(3, 4);  
std::shared_ptr<rational> sp2 = sp1;  
std::cout << sp1.use_count(); // 2
```

- Wenn ein Shared Pointer out of scope geht, wird Counter dekrementiert

```
{  
    std::shared_ptr<rational> sp3 = sp2;  
    std::cout << sp1.use_count(); // 3  
} // sp3 goes out of scope here; counter is decreased  
sp1 = nullptr; // explicitly remove pointer sp1 to object  
std::cout << sp2.use_count(); // 1
```

- wenn Counter 0 wird, wird Objekt destruiert und Speicher freigegeben

Anwendungsbeispiel: Werkstatt



Arbeiter arbeiten in Werkstatt.
Sie benutzen regelmässig gewisse Werkzeuge und *teilen* sich diese.
Ungebrauchte Werkzeuge sollen automatisch zurückgegeben werden

Werkstatt: 1. Versuch

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
struct worker {  
    std::map<std::string, tool> tools; // all tools a worker is using  
    void use(tool t); // acquire  
    tool offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```

Werkstatt: 1. Versuch

```
void worker::use(tool t) {  
    tools[t.name] = t;  
}  
tool worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool borrow(std::string tool_name) {  
    return tool(tool_name);  
}
```

Werkstatt: 1. Versuch

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

Werkstatt: 1. Versuch

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned

hammer gets returned

hammer gets returned

hammer gets returned

Werkstatt: 1. Versuch

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned

hammer gets returned

hammer gets returned

hammer gets returned

Jeder Worker hat seine eigene Kopie des Werkzeugs: kein Teilen!
(Noch schlimmer, wir haben hier 4 Kopien des Hammers!)

Werkstatt: Versuch mit Zeiger

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
using tool_ptr = tool*;
```

```
struct worker {  
    std::map<std::string, tool_ptr> tools; // all tools a worker is using  
    void use(tool_ptr t); // acquire  
    tool_ptr offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```


Werkstatt: Versuch mit Zeiger

```
void worker::use(tool_ptr t) {  
    tools[t->name] = t;  
}  
tool_ptr worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool_ptr borrow(std::string tool_name) {  
    return new tool(tool_name);  
}
```

Werkstatt: Versuch mit Zeiger

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

Werkstatt: Versuch mit Zeiger

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

Wer baut Speicher für Werkzeuge ab, wenn sie nicht mehr in Gebrauch sind?

Werkstatt: Versuch mit Zeiger

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

Wer baut Speicher für Werkzeuge ab, wenn sie nicht mehr in Gebrauch sind?

Memory Leak!

Werkstatt: Lösung mit Shared Pointers

```
struct tool {  
    std::string name;  
    tool(std::string name) : name(name) {}  
    ~tool() {std::cout << name << " gets returned!\n";}  
};
```

```
using tool_ptr = std::shared_ptr<tool>;
```

```
struct worker {  
    std::map<std::string, tool_ptr> tools; // all tools a worker is using  
    void use(tool_ptr t); // acquire  
    tool_ptr offer(std::string tool_name); // share  
    void release(std::string tool_name); // release  
};
```

Werkstatt: Lösung mit Shared Pointers

```
void worker::use(tool_ptr t) {  
    tools[t->name] = t;  
}  
tool_ptr worker::offer(std::string tool_name) {  
    return tools[tool_name];  
}  
void worker::release(std::string tool_name) {  
    tools.erase(tool_name);  
}  
  
tool_ptr borrow(std::string tool_name) {  
    return std::make_shared<tool>(tool_name);  
}
```

Werkstatt: Lösung mit Shared Pointers

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned!

Werkstatt: Lösung mit Shared Pointers

```
worker w1;  
worker w2;  
w1.use(borrow("hammer"));  
w2.use(w1.offer("hammer"));  
w2.release("hammer");
```

hammer gets returned!

Sobald der letzte Shared Pointer auf ein Werkzeug out of scope geht, wird der Speicher (genau einmal!) abgebaut.

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

■ Handling des Unique Pointers wie mit normalem Pointer

```
rational r = *up;  
std::cout << up->n << " / " << up->d; // 3 / 4
```

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

■ Handling des Unique Pointers wie mit normalem Pointer

```
rational r = *up;  
std::cout << up->n << " / " << up->d; // 3 / 4
```

■ Zuweisung nicht erlaubt...

```
std::unique_ptr<rational> x = up; // compiler error
```

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

■ Handling des Unique Pointers wie mit normalem Pointer

```
rational r = *up;  
std::cout << up->n << " / " << up->d; // 3 / 4
```

■ Zuweisung nicht erlaubt...

```
std::unique_ptr<rational> x = up; // compiler error
```

■ ...nur in Kombination mit Übertragen des Besitzes

```
std::unique_ptr<rational> x = std::move(up) // transfer ownership
```

Unique Pointers: Beispiel

■ Unique Pointer erstellen

```
std::unique_ptr<rational> up = std::make_unique<rational>(3, 4);
```

■ Handling des Unique Pointers wie mit normalem Pointer

```
rational r = *up;  
std::cout << up->n << " / " << up->d; // 3 / 4
```

■ Zuweisung nicht erlaubt...

```
std::unique_ptr<rational> x = up; // compiler error
```

■ ...nur in Kombination mit Übertragen des Besitzes

```
std::unique_ptr<rational> x = std::move(up) // transfer ownership
```

Unique Pointer ist alleiniger Besitzer des Objekts, auf das er zeigt;
Objekt wird zerstört, wenn der Besitzer out of scope geht.

Anwendungsbeispiel: Hotdesk-Büro



Arbeiter beziehen bei Ankunft Gegenstände wie Schreibtisch und Computer.
Sie teilen sie nicht.
Unbenutzte sollen automatisch freigegeben werden.

Erste Versuche

```
struct item {
    std::string name;
    item(std::string name) : name(name) {};
    ~item() {std::cout << name << " gets returned!\n";}
};

struct worker {
    std::map<std::string, item> items; // all items a worker is using
    void use(item t); // acquire
    item release(std::string item_name); // release
};
```

Erste Versuche

```
struct item {
    std::string name;
    item(std::string name) : name(name) {};
    ~item() {std::cout << name << " gets returned!\n";}
};

struct worker {
    std::map<std::string, item> items; // all items a worker is using
    void use(item t); // acquire
    item release(std::string item_name); // release
};
```

■ ohne Pointer

Erste Versuche

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}   
};  
  
struct worker {  
    std::map<std::string, item> items; // all items a worker is using  
    void use(item t); // acquire  
    item release(std::string item_name); // release  
};
```

- ohne Pointer $\Rightarrow$ Kein Teilen der Gegenstände: Kopien werden erstellt!

Erste Versuche

```
struct item {
    std::string name;
    item(std::string name) : name(name) {};
    ~item() {std::cout << name << " gets returned!\n";}
};

struct worker {
    std::map<std::string, item*> items; // all items a worker is using
    void use(item* t); // acquire
    item* release(std::string item_name); // release
};
```

- ohne Pointer $\Rightarrow$ Kein Teilen der Gegenstände: Kopien werden erstellt!
- mit Raw Pointer

Erste Versuche

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}   
};  
  
struct worker {  
    std::map<std::string, item*> items; // all items a worker is using  
    void use(item* t); // acquire  
    item* release(std::string item_name); // release  
};
```

- ohne Pointer $\Rightarrow$ Kein Teilen der Gegenstände: Kopien werden erstellt!
- mit Raw Pointer $\Rightarrow$ Gegenstände werden nie zurückgegeben

Versuch mit Shared Pointers

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}  
};
```

```
using item_ptr = std::shared_ptr<item>;
```

```
struct worker {  
    std::map<std::string, item_ptr> items; // all items a worker is using  
    void use(item_ptr t); // acquire  
    item_ptr release(std::string item_name); // release  
};
```

Versuch mit Shared Pointers

```
void worker::use(item_ptr t){  
    items[t->name] = t;  
}
```

```
item_ptr worker::release(std::string item_name) {  
    item_ptr t = items[item_name];  
    items.erase(item_name);  
    return t;  
}
```

```
item_ptr claim(std::string item_name) {  
    return std::make_shared<item>(item_name);  
}
```

Versuch mit Shared Pointers

```
worker w1;  
worker w2;  
auto desk = claim("desk");  
w1.use(desk);  
w2.use(desk);
```

Versuch mit Shared Pointers

```
worker w1;  
worker w2;  
auto desk = claim("desk");  
w1.use(desk);  
w2.use(desk);
```

desk gets returned!

Versuch mit Shared Pointers

```
worker w1;  
worker w2;  
auto desk = claim("desk");  
w1.use(desk);  
w2.use(desk);
```

desk gets returned!

Schreibtisch wird zwar korrekt freigegeben, aber mehrere Arbeiter können sich einen Schreibtisch teilen.

Hotdesk-Büro: Lösung mit Unique Pointers

```
struct item {  
    std::string name;  
    item(std::string name) : name(name) {};  
    ~item() {std::cout << name << " gets returned!\n";}  
};
```

```
using item_ptr = std::unique_ptr<item>;
```

```
struct worker {  
    std::map<std::string, item_ptr> items; // all items a worker is using  
    void use(item_ptr t); // acquire  
    item_ptr release(std::string item_name); // release  
};
```

Hotdesk-Büro: Lösung mit Unique Pointers

```
void worker::use(item_ptr t){  
    items[t->name] = std::move(t);  
}
```

```
item_ptr worker::release(std::string item_name) {  
    item_ptr t = std::move(items[item_name]);  
    items.erase(item_name);  
    return t;  
}
```

```
item_ptr claim(std::string item_name) {  
    return std::make_unique<item>(item_name);  
}
```

Hotdesk-Büro: Lösung mit Unique Pointers

```
worker w1;  
worker w2;  
w1.use(claim("desk"));  
w2.use(w1.release("desk"));
```

Hotdesk-Büro: Lösung mit Unique Pointers

```
worker w1;  
worker w2;  
w1.use(claim("desk"));  
w2.use(w1.release("desk"));
```

desk gets returned!

Hotdesk-Büro: Lösung mit Unique Pointers

```
worker w1;  
worker w2;  
w1.use(claim("desk"));  
w2.use(w1.release("desk"));
```

desk gets returned!

Sobald der Unique Pointer auf ein Objekt (der alleinige Besitzer eines Objekts) out of scope geht, wird der Speicher (genau einmal!) abgebaut.

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet
- Problem 2: Speicher nicht sofort frei, erst nach folgendem GC Lauf

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet
- Problem 2: Speicher nicht sofort frei, erst nach folgendem GC Lauf

Datenstrukturen mit geteilten Objekten brauchen Verwaltung

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet
- Problem 2: Speicher nicht sofort frei, erst nach folgendem GC Lauf

Datenstrukturen mit geteilten Objekten brauchen Verwaltung

- Beispiel: ein Knoten wird in zwei stacks benutzt – beim Ändern des Knotenwert verändern sich beide Stacks

Speicherverwaltung unabhängig von C++

Garbage Collector (GC): in anderen Programmiersprachen übernimmt der GC das Entfernen alter Daten

- Problem 1: nicht voraussagbare Latenz wenn gestartet
- Problem 2: Speicher nicht sofort frei, erst nach folgendem GC Lauf

Datenstrukturen mit geteilten Objekten brauchen Verwaltung

- Beispiel: ein Knoten wird in zwei stacks benutzt – beim Ändern des Knotenwert verändern sich beide Stacks

Andere Sprachen haben also auch Speicherverwaltung, verstecken diese nur (soweit möglich). Bessere Kontrolle in C++!

23. Dynamische Datenstrukturen III

Wiederholung: `vector<T>`

- Kann mit beliebiger Grösse `n` initialisiert werden

Wiederholung: `vector<T>`

- Kann mit beliebiger Grösse `n` initialisiert werden
- Unterstützt diverse Operationen:

```
e = v[i];           // Get element
v[i] = e;           // Set element
l = v.size();        // Get size
v.push_back(e);      // Append element
...
```

Wiederholung: `vector<T>`

- Kann mit beliebiger Grösse `n` initialisiert werden
- Unterstützt diverse Operationen:

```
e = v[i];           // Get element
v[i] = e;           // Set element
l = v.size();        // Get size
v.push_back(e);      // Append element
...
```

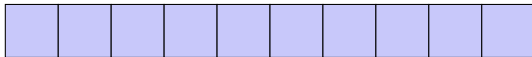
- Ein Vektor ist eine *dynamische Datenstruktur*, deren Grösse sich zur Laufzeit ändern kann

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Vektoren im Speicher

Bereits bekannt: Ein Vektor belegt einen *zusammenhängenden* Speicherbereich



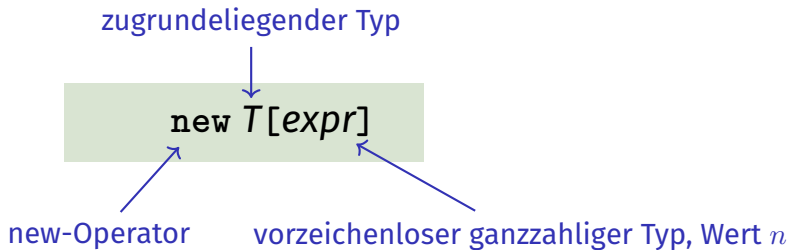
Vektoren im Speicher

Bereits bekannt: Ein Vektor belegt einen *zusammenhängenden* Speicherbereich

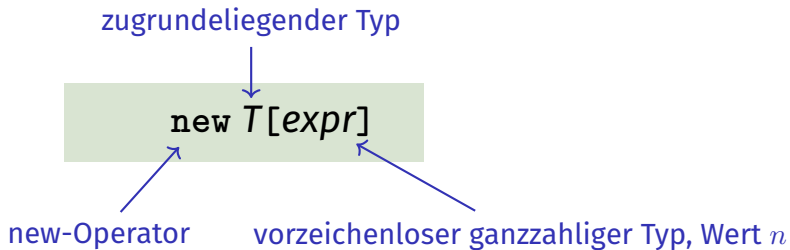


Frage: Wie *alloziert* (reserviert) man einen Speicherblock *beliebiger* Grösse zur Laufzeit, also *dynamisch*?

Der `new`-Ausdruck für Arrays



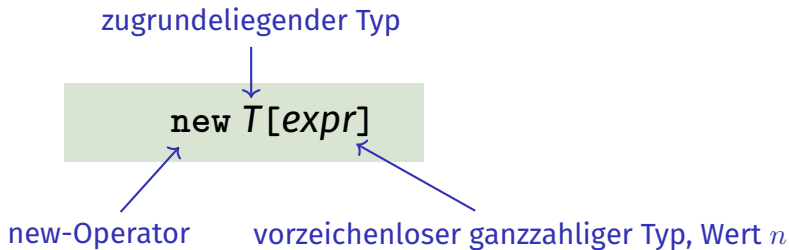
Der `new`-Ausdruck für Arrays



- **Effekt:** Neuer zusammenhängender Bereich im Speicher für n Elemente vom Typ T wird alloziert



Der `new`-Ausdruck für Arrays



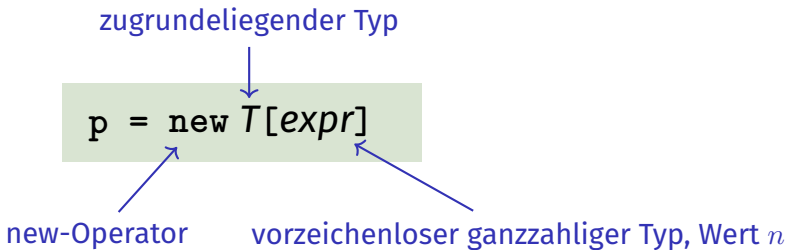
- **Effekt:** Neuer zusammenhängender Bereich im Speicher für n Elemente vom Typ T wird alloziert



- Dieser Speicherbereich wird **Array** (der Länge n) genannt

Der `new`-Ausdruck für Arrays

zugrundeliegender Typ



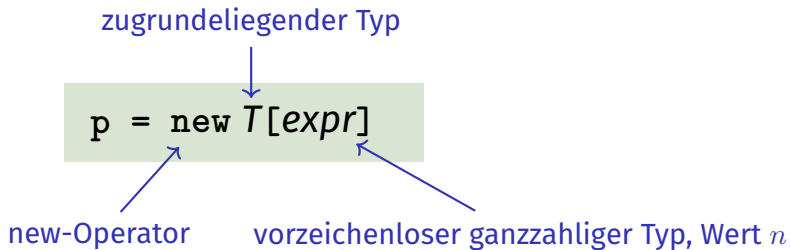
```
p = new T[expr]
```

The diagram shows the expression `p = new T[expr]` inside a light green rectangular box. Three blue arrows point from external text labels to parts of the expression: one from 'zugrundeliegender Typ' to the `T`, one from 'new-Operator' to the `new` keyword, and one from 'vorzeichenloser ganzzahliger Typ, Wert *n*' to the `expr` inside the brackets.

new-Operator

vorzeichenloser ganzzahliger Typ, Wert *n*

Der new-Ausdruck für Arrays



- **Wert:** Die Startadresse des Speicherbereichs



- **Typ:** Ein Zeiger T^*

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```



Frage: Wie zeigt man auf hintere Elemente?

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```



Frage: Wie zeigt man auf hintere Elemente? → mittels *Zeiger-Arithmetik*:

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```



Frage: Wie zeigt man auf hintere Elemente? → mittels *Zeiger-Arithmetik*:

■ p gibt die Adresse des *ersten* Array-Elements, *p dessen Wert

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```



Frage: Wie zeigt man auf hintere Elemente? → mittels *Zeiger-Arithmetik*:

- p gibt die Adresse des *ersten* Array-Elements, $*p$ dessen Wert
- $*(p + i)$ gibt den Wert des *i-ten* Array-Elements, für $0 \leq i < n$

Zeiger-Arithmetik: Zeiger plus int

```
T* p = new T[n]; // p points to first array element
```

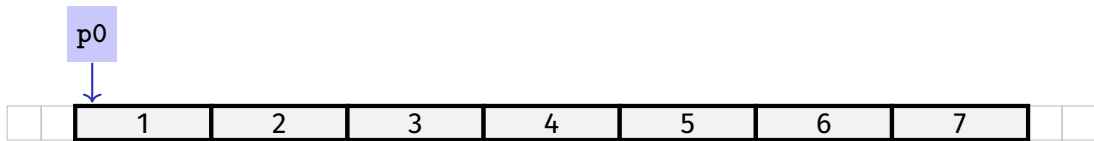


Frage: Wie zeigt man auf hintere Elemente? → mittels *Zeiger-Arithmetik*:

- p gibt die Adresse des *ersten* Array-Elements, $*p$ dessen Wert
- $*(p + i)$ gibt den Wert des *i-ten* Array-Elements, für $0 \leq i < n$
- $*p$ ist äquivalent zu $*(p + 0)$

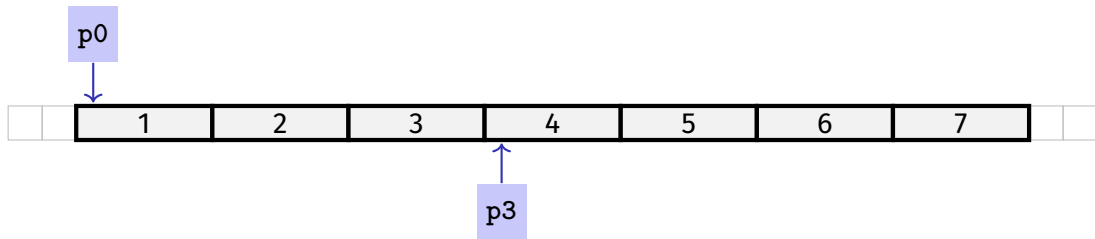
Zeiger-Arithmetik: Zeiger plus int

```
int* p0 = new int[7]{1,2,3,4,5,6,7};    // p0 points to 1st element  
// p3 points to 4th element  
// set value of 6th element to 600  
// output 6th element's value (i.e., 600)
```



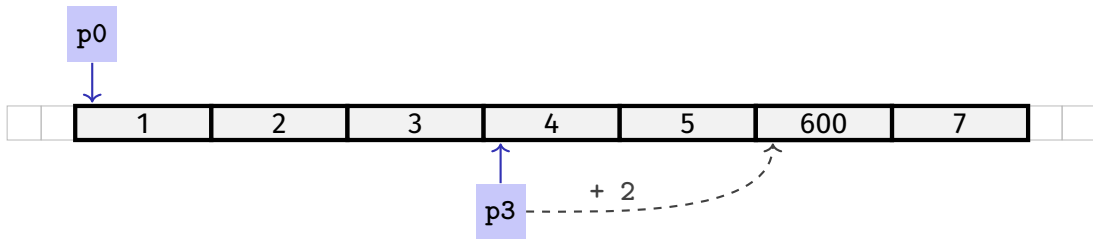
Zeiger-Arithmetik: Zeiger plus int

```
int* p0 = new int[7]{1,2,3,4,5,6,7};    // p0 points to 1st element
int* p3 = p0 + 3;    // p3 points to 4th element
                        // set value of 6th element to 600
                        // output 6th element's value (i.e., 600)
```



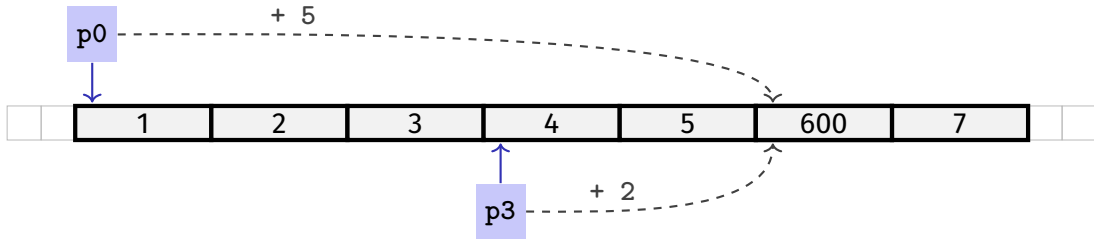
Zeiger-Arithmetik: Zeiger plus int

```
int* p0 = new int[7]{1,2,3,4,5,6,7};    // p0 points to 1st element
int* p3 = p0 + 3;    // p3 points to 4th element
*(p3 + 2) = 600;    // set value of 6th element to 600
std::cout << *(p0 + 5);    // output 6th element's value (i.e., 600)
```



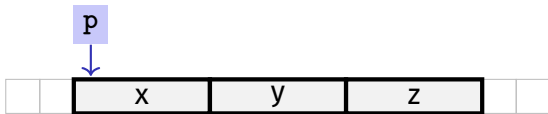
Zeiger-Arithmetik: Zeiger plus int

```
int* p0 = new int[7]{1,2,3,4,5,6,7};    // p0 points to 1st element
int* p3 = p0 + 3;    // p3 points to 4th element
*(p3 + 2) = 600;    // set value of 6th element to 600
std::cout << *(p0 + 5);    // output 6th element's value (i.e., 600)
```



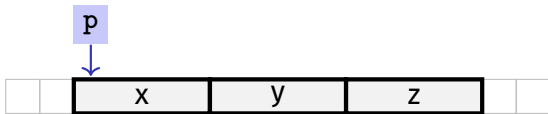
Sequenzielle Zeiger-Iteration

```
char* p = new char[3]{'x', 'y', 'z'};
```



Sequenzielle Zeiger-Iteration

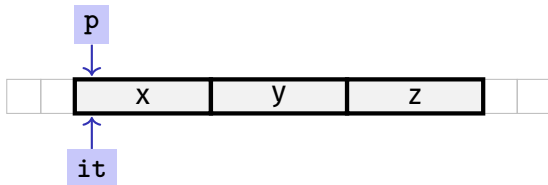
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' '  
}
```

Sequenzielle Zeiger-Iteration

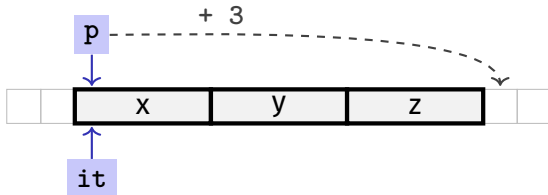
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p; ← it zeigt aufs erste Element  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';<br>}
```

Sequenzielle Zeiger-Iteration

```
char* p = new char[3]{'x', 'y', 'z'};
```



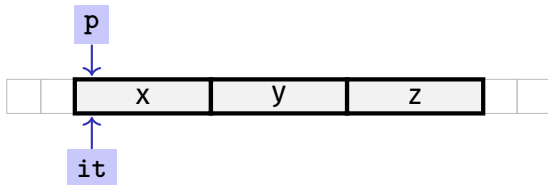
```
for (char* it = p;  
    it != p + 3;   
    ++it) {
```

Abbruch falls Ende erreicht

```
    std::cout << *it << ' ';<br>}</pre>
```

Sequenzielle Zeiger-Iteration

```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {
```

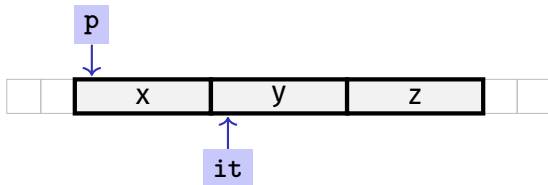
```
    std::cout << *it << ' ';
```

```
}
```

← Aktuelles Element ausgeben: 'x'

Sequenzielle Zeiger-Iteration

```
char* p = new char[3]{'x', 'y', 'z'};
```



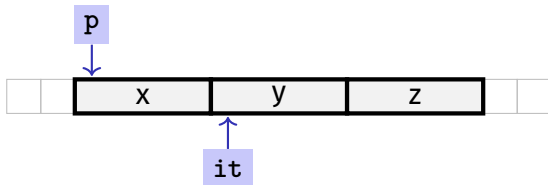
```
for (char* it = p;  
    it != p + 3;  
    ++it) {
```

Zeiger elementweise voranschieben

```
    std::cout << *it << ' ';    // x  
}
```

Sequenzielle Zeiger-Iteration

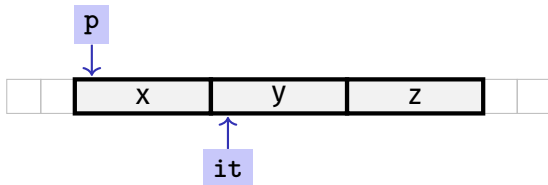
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x  
}
```

Sequenzielle Zeiger-Iteration

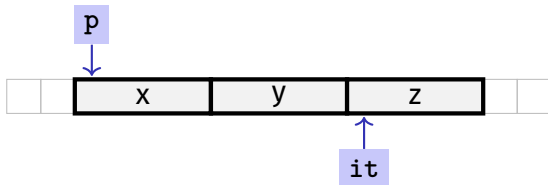
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y  
}
```


Sequenzielle Zeiger-Iteration

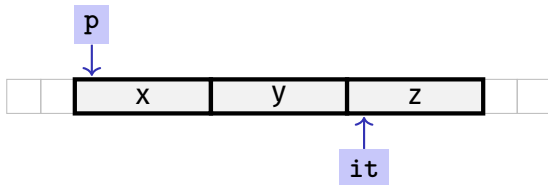
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y  
}
```

Sequenzielle Zeiger-Iteration

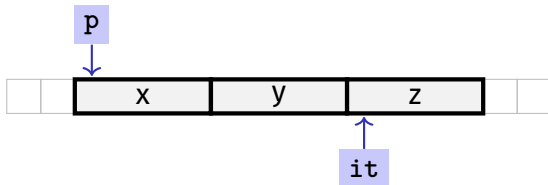
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y  
}
```

Sequenzielle Zeiger-Iteration

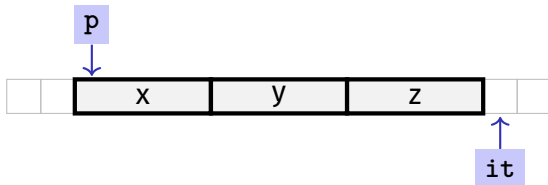
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y z  
}
```

Sequenzielle Zeiger-Iteration

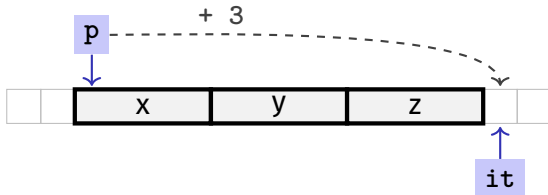
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y z  
}
```

Sequenzielle Zeiger-Iteration

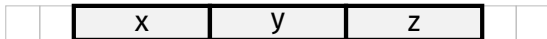
```
char* p = new char[3]{'x', 'y', 'z'};
```



```
for (char* it = p;  
    it != p + 3;  
    ++it) {  
  
    std::cout << *it << ' ';    // x y z  
}
```

Wahlfreier Zugriff auf Arrays

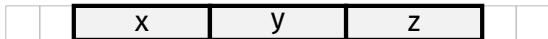
```
char* p = new char[3]{'x', 'y', 'z'};
```



- Der Ausdruck $*(p + i)$
- kann auch geschrieben werden als $p[i]$

Wahlfreier Zugriff auf Arrays

```
char* p = new char[3]{'x', 'y', 'z'};
```



- Der Ausdruck `*(p + i)`
- kann auch geschrieben werden als `p[i]`
- z.B. `p[1] == *(p + 1) == 'y'`

Wahlfreier Zugriff auf Arrays

Iteration über ein Array mittels Indizes und *wahlfreiem Zugriff*:

```
char* p = new char[3]{'x', 'y', 'z'};

for (int i = 0; i < 3; ++i)
    std::cout << p[i] << ' ';
```


Wahlfreier Zugriff auf Arrays

Iteration über ein Array mittels Indizes und *wahlfreiem Zugriff*:

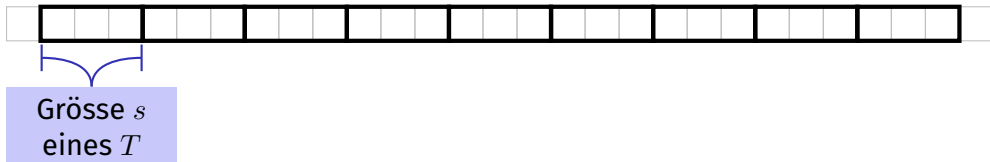
```
char* p = new char[3]{'x', 'y', 'z'};

for (int i = 0; i < 3; ++i)
    std::cout << p[i] << ' ';
```

Aber: Dies ist weniger *effizient* als der vorher gezeigte *sequenzielle* Zugriff mittels Zeiger-Iteration

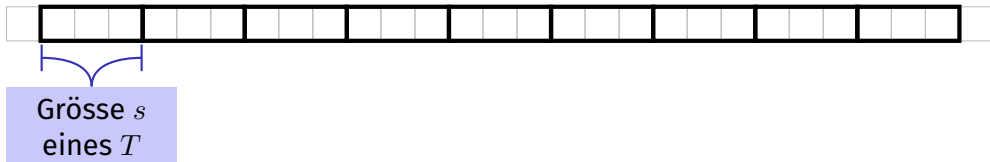
Wahlfreier Zugriff auf Arrays

```
 $T^* \text{ p} = \text{new } T[n];$ 
```



Wahlfreier Zugriff auf Arrays

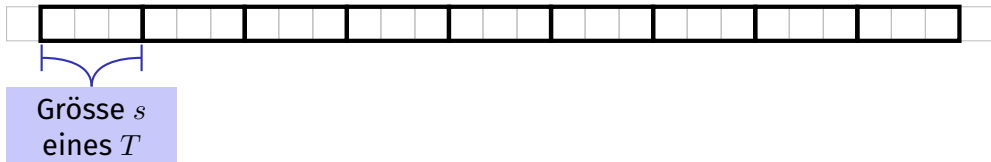
```
T* p = new T[n];
```



- Zugriff $p[i]$, also $*(p + i)$, „kostet“ Berechnung $p + i \cdot s$

Wahlfreier Zugriff auf Arrays

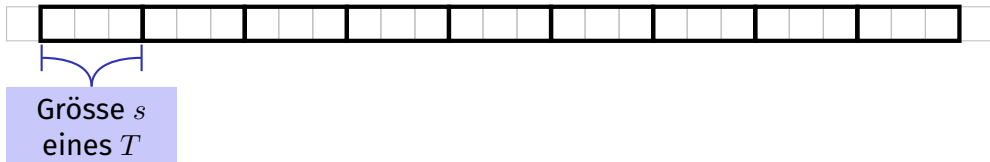
```
T* p = new T[n];
```



- Zugriff $p[i]$, also $*(p + i)$, „kostet“ Berechnung $p + i \cdot s$
- Iteration mittels *wahlfreiem Zugriff* ($p[0], p[1], \dots$) kostet eine Addition und eine Multiplikation pro Zugriff

Wahlfreier Zugriff auf Arrays

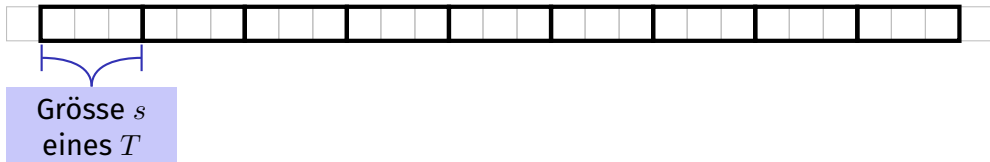
```
T* p = new T[n];
```



- Zugriff $p[i]$, also $*(p + i)$, „kostet“ Berechnung $p + i \cdot s$
- Iteration mittels *wahlfreiem Zugriff* ($p[0], p[1], \dots$) kostet eine Addition und eine Multiplikation pro Zugriff
- Iteration mittels *sequentielltem Zugriff* ($++p, ++p, \dots$) kostet nur eine Addition pro Zugriff

Wahlfreier Zugriff auf Arrays

```
T* p = new T[n];
```



- Zugriff $p[i]$, also $*(p + i)$, „kostet“ Berechnung $p + i \cdot s$
- Iteration mittels *wahlfreiem Zugriff* ($p[0], p[1], \dots$) kostet eine Addition und eine Multiplikation pro Zugriff
- Iteration mittels *sequenziellem Zugriff* ($++p, ++p, \dots$) kostet nur eine Addition pro Zugriff
- Sequenzieller Zugriff ist daher für Iterationen zu bevorzugen

Ein Buch lesen ...mit wahlfreiem Zugriff

Wahlfreier Zugriff

- öffne Buch auf S.1
- klappe Buch zu
- öffne Buch auf S.2-3
- klappe Buch zu
- öffne Buch auf S.4-5
- klappe Buch zu
-

Wahlfreier Zugriff

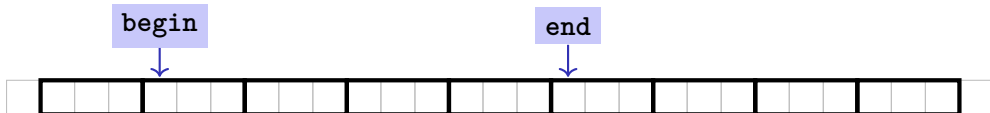
- öffne Buch auf S.1
- klappe Buch zu
- öffne Buch auf S.2-3
- klappe Buch zu
- öffne Buch auf S.4-5
- klappe Buch zu
-

Sequenzieller Zugriff

- öffne Buch auf S.1
- blättere um
- blättere um
- blättere um
- blättere um
- blättere um
- ...

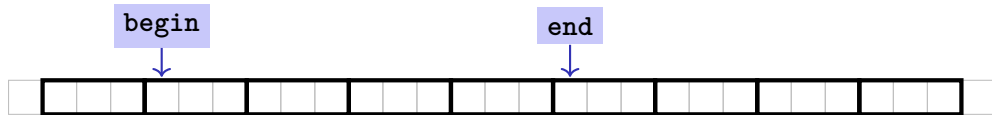
Arrays in Funktionen

Konvention in C++: Übergabe eines Arrays (oder eines Array-Ausschnitts) mit zwei Zeigern



Arrays in Funktionen

Konvention in C++: Übergabe eines Arrays (oder eines Array-Ausschnitts) mit zwei Zeigern



- **begin**: Zeiger auf das erste Element
- **end**: Zeiger *hinter* das letzte Element

Arrays in Funktionen

Konvention in C++: Übergabe eines Arrays (oder eines Array-Ausschnitts) mit zwei Zeigern



- `begin`: Zeiger auf das erste Element
- `end`: Zeiger *hinter* das letzte Element
- `[begin, end)` bezeichnet die Elemente des Array-Ausschnitts

Arrays in Funktionen

Konvention in C++: Übergabe eines Arrays (oder eines Array-Ausschnitts) mit zwei Zeigern



- `begin`: Zeiger auf das erste Element
- `end`: Zeiger *hinter* das letzte Element
- `[begin, end)` bezeichnet die Elemente des Array-Ausschnitts
- `[begin, end)` ist leer, wenn `begin == end`
- `[begin, end)` muss ein *gültiger Bereich* sein, d.h. ein echter (evtl. leerer) Array-Ausschnitt

Arrays in (mutierenden) Funktionen: fill

```
// PRE: [begin, end) ist ein gültiger Bereich  
// POST: Jedes Element in [begin, end) wurde auf value gesetzt  
void fill(int* begin, int* end, int value) {  
    for (int* p = begin; p != end; ++p)  
        *p = value;  
}
```

Arrays in (mutierenden) Funktionen: fill

```
// PRE: [begin, end) ist ein gültiger Bereich
// POST: Jedes Element in [begin, end) wurde auf value gesetzt
void fill(int* begin, int* end, int value) {
    for (int* p = begin; p != end; ++p)
        *p = value;
}
```

```
int* p = new int[5];
fill(p, p+5, 1); // Array bei p wird zu {1, 1, 1, 1, 1}
```

Array-basierter Vektor

■ Vektoren ... da war doch was 🤔

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Array-basierter Vektor

- Vektoren ... da war doch was 🤔
- Nun wissen wir, wie man Speicherblöcke beliebiger Grösse allozieren kann ...

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Array-basierter Vektor

- Vektoren ... da war doch was 🤔
- Nun wissen wir, wie man Speicherblöcke beliebiger Grösse allozieren kann ...
- ...und können einen Vektor, auf einem solchen Speicherblock aufbauend, implementieren

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Array-basierter Vektor

- Vektoren ... da war doch was 🤔
- Nun wissen wir, wie man Speicherblöcke beliebiger Grösse allozieren kann ...
- ...und können einen Vektor, auf einem solchen Speicherblock aufbauend, implementieren
- `our_vector` – ein Array-basierter Vektor für `int`-Elemente

Unser eigener Vektor!

Wir implementieren unseren eigenen Vektor: `vec<int>`

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements;   
    unsigned int count;  
  
};
```

Zeiger auf erstes Element

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count;   
};
```

← Anzahl Elemente

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size);  
    unsigned int size() const;  
    int& operator[](int i);  
    void print(std::ostream& sink) const;  
};
```



Konstruktor

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; ←  
    int& operator[](int i);  
    void print(std::ostream& sink) const;  
};
```

Grösse des Vektors

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); ← Elementzugriff  
    void print(std::ostream& sink) const;  
};
```

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; ←  
};
```

Elemente ausgeben

Array-basierter Vektor `our_vector`: Klassensignatur

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

Konstruktor `our_vector::our_vector()`

Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) {

    elements = new int[size];
}
```

Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) {

    elements = new int[size];
}
```

Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) ← {
    elements = new int[size];
}
```

Grösse speichern

Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) {

    elements = new int[size];
}
```

← Speicher allozieren

Konstruktor `our_vector::our_vector()`

```
our_vector::our_vector(unsigned int size)
    : count(size) {

    elements = new int[size];
}
```

Achtung: Vektor wurde nicht initialisiert (mit einem Standardwert, z.B. 0)

Funktion `our_vector::size()`

Funktion `our_vector::size()`

```
int our_vector::size() const {  
    return count;  
}
```

Funktion `our_vector::size()`

```
int our_vector::size() const ← {  
    return count;  
}
```

Verändert den Vektor nicht

Funktion `our_vector::size()`

```
int our_vector::size() const {  
    return count;   
}
```

Grösse zurückgeben

Funktion `our_vector::size()`

```
int our_vector::size() const {  
    return count;  
}
```

Anwendungsbeispiel:

```
our_vector v = our_vector(7);  
assert(v.size() == 7); // ok
```

Funktion `our_vector::operator[]`

Funktion `our_vector::operator[]`

```
int& our_vector::operator[](int i) {  
    return elements[i];  
}
```

Funktion `our_vector::operator[]`

```
int& our_vector::operator[](int i) {  
    return elements[i];  
}
```

← i-tes Element zurückgeben

Funktion `our_vector::operator[]`

```
int& our_vector::operator[](int i) {  
    return elements[i];  
}
```

Elementzugriff mit Indexüberprüfung:

```
int& our_vector::at(int i) {  
    assert(0 <= i && i < count);  
    return elements[i];  
}
```


Funktion `our_vector::operator[]`

```
int& our_vector::operator[](int i) {  
    return elements[i];  
}
```

Elementzugriff mit Indexüberprüfung:

```
int& our_vector::at(int i) {  
    assert(0 <= i && i < count);  
    return elements[i];  
}
```

Anwendungsbeispiel:

```
our_vector v = our_vector(7);  
v[6] = 0;  
std::cout << v[6]; // Outputs 0
```

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;  
         p != elements + count;  
         ++p)  
    {  
        sink << *p << ' '  
    }  
}
```

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;  Zeiger auf erstes Element  
        p != elements + count;  
        ++p)  
    {  
        sink << *p << ' ' ;  
    }  
}
```

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;  
         p != elements + count;   
         ++p)  
    {  
        sink << *p << ' ';  
    }  
}
```

Abbruch falls hinter
letztem Element

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;  
         p != elements + count;  
         ++p) {  
        sink << *p << ' ';  
    }  
}
```

← Zeiger elementweise voranschieben

Funktion `our_vector::print()`

Elemente mittels sequenziellem Zugriff ausgeben:

```
void our_vector::print(std::ostream& sink) const {  
    for (int* p = elements;  
         p != elements + count;  
         ++p)  
    {  
        sink << *p << ' ';  
    }  
}
```



Aktuelles Element ausgeben

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
_____ operator<<(_______ sink,  
                    _____ vec) {  
    vec.print(sink);  
    return _____;  
}
```

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
std::ostream& operator<<(std::ostream& sink,  
                        const our_vector& vec) {  
    vec.print(sink);  
    return sink;  
}
```

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
std::ostream& operator<<(std::ostream& sink,  
                        const our_vector& vec) {  
    vec.print(sink);  
    return sink;  
}
```

Beobachtungen:

- Konstante Referenz auf `vec`, da unverändert

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
std::ostream& operator<<(std::ostream& sink,  
                        const our_vector& vec) {  
    vec.print(sink);  
    return sink;  
}
```

Beobachtungen:

- Konstante Referenz auf `vec`, da unverändert
- Aber nicht auf `sink`: Elemente ausgeben gleich Veränderung

Funktion `our_vector::print()`

Abschluss: Ausgabeoperator überladen:

```
std::ostream& operator<<(std::ostream& sink,  
                        const our_vector& vec) {  
    vec.print(sink);  
    return sink;  
}
```

Beobachtungen:

- Konstante Referenz auf `vec`, da unverändert
- Aber nicht auf `sink`: Elemente ausgeben gleich Veränderung
- `sink` zurückgeben, um verkettete Ausgaben zu ermöglichen, z.B.
`std::cout << v << '\n';`

Ein Iterator für `our_vector`

Ein Iterator für `our_vector`

Klasse mit

- `operator*`
- `operator++`
- `operator!=`

Ein Iterator für `our_vector`

Klasse mit

- `operator*`
- `operator++`
- `operator!=`

Im `our_vector` haben wir bereits einen Pointer (`elements`) vom Typ `int*` mit allen Funktionalitäten, die ein Iterator benötigt (Pointer sind Iteratoren).

Ein Iterator für `our_vector`

Klasse mit

- `operator*`
- `operator++`
- `operator!=`

Im `our_vector` haben wir bereits einen Pointer (`elements`) vom Typ `int*` mit allen Funktionalitäten, die ein Iterator benötigt (Pointer sind Iteratoren).

Wir brauchen nur noch Memberfunktionen `begin()` und `end()` im `our_vector`, um einen Iterator auf den Anfang bzw. hinter das Ende zu erhalten

Iterator für our_vector

```
class our_vector {
    int* elements;
    unsigned int count;
public:
    using iterator = int*; // our_vector::iterator stands for int*
    ...
    iterator begin() {
        return elements;
    }
    iterator our_vector::end() {
        return elements + count;
    }
    ...
}
```

Iterator für our_vector

```
class our_vector {  
    int* elements;  
    unsigned int count;  
public:  
    using iterator = int*; // our_vector::iterator stands for int*  
    ...  
    iterator begin() {  
        return elements;   
    }  
    iterator    
        return elements + count;  
    }  
    ...  
}
```

Iterator für our_vector

```
class our_vector {  
    int* elements;  
    unsigned int count;  
public:  
    using iterator = int*; // our_vector::iterator stands for int*  
    ...  
    iterator begin() {  
        return elements;  
    }  
    iterator our_vector::end() {  
        return elements + count;   
    }  
    ...  
}
```

Iterator hinter letztes Vektorelement

our_vector Iterator Verwendung

Funktionalität, die auf Iteratoren beruht, kann auf unserem Vektor verwendet werden

```
our_vector v;  
...  
for(int& i: v){ // iterator used implicitly  
    i += 3;  
}  
int max = *std::max_element(v.begin(), v.end());  
...
```

Was passiert hier?

```
class our_vector {  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

Was passiert hier?

```
class our_vector {  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

```
for (unsigned int i = 0; i < 1000000; ++i) {  
    our_vector v = our_vector(1000000);  
}
```

Was passiert hier?

```
class our_vector {  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

```
for (unsigned int i = 0; i < 1000000; ++i) {  
    our_vector v = our_vector(1000000);  
}
```

Vektoren gehen out of scope und werden abgebaut (inklusive der Zeiger auf die Elemente)

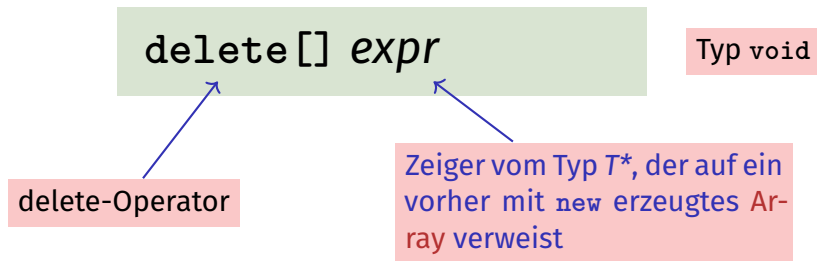
Was passiert hier?

```
class our_vector {  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public:  
    our_vector(unsigned int size); // Constructor  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elements  
};
```

```
for (unsigned int i = 0; i < 1000000; ++i) {  
    our_vector v = our_vector(1000000);  
}
```

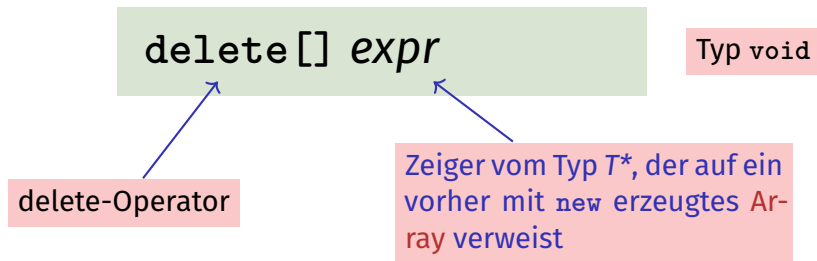
Vektoren gehen out of scope und werden abgebaut (inklusive der Zeiger auf die Elemente), aber Elemente (jeweils eine Million viele) leben weiter...

Der delete-Ausdruck für Arrays



- **Effekt:** Array wird gelöscht, Speicher wird wieder freigegeben

Der delete-Ausdruck für Arrays



- **Effekt:** Array wird gelöscht, Speicher wird wieder freigegeben

Achtung: `delete[]` setzt den Zeiger *nicht* auf `nullptr`

Vektor: delete[] im Destruktor

- zum new[] im Konstruktor...

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = new int[size];  
}
```

Vektor: delete[] im Destruktor

- zum new[] im Konstruktor...

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = new int[size];  
}
```

- ...braucht es ein delete[] im Destruktor

```
our_vector::~~our_vector(){  
    delete[] elements;  
}
```

Was passiert hier?

```
{  
    our_vector v = our_vector(10);  
    our_vector w = v;  
}
```

Was passiert hier?

```
{  
    our_vector v = our_vector(10);  
    our_vector w = v;  
}
```

⇒ Elemente des Vektors werden zweimal abgebaut...

```
{  
    our_vector v = our_vector(10);  
    our_vector w;  
    w = v;  
}
```

Was passiert hier?

```
{  
    our_vector v = our_vector(10);  
    our_vector w = v;  
}
```

⇒ Elemente des Vektors werden zweimal abgebaut...

```
{  
    our_vector v = our_vector(10);  
    our_vector w;  
    w = v;  
}
```

⇒ hier ebenso...

Dreierregel (Rule of Three)...

```
class our_vector {  
    // Private (internal) state:  
    int* elements; // Pointer to first element  
    unsigned int count; // Number of elements  
public: // Public interface:  
    our_vector(unsigned int size); // Constructor  
    our_vector(const our_vector& v); // Copy constructor  
    ~our_vector(); // Destructor  
    operator=(const our_vector& v); // Assignment operator  
    unsigned int size() const; // Size of vector  
    int& operator[](int i); // Access an element  
    void print(std::ostream& sink) const; // Output elems.  
}
```

Kopierkonstruktor und Zuweisungsoperator

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = new int[count];  
    for (unsigned int i = 0; i < count; ++i) {  
        elements[i] = v.elements[i];  
    }  
}
```

```
our_vector& our_vector::operator=(const our_vector& v) {  
    if(this != &v) { // no self assignment  
        auto copy = v;  
        std::swap(elements, copy.elements);  
        std::swap(count, copy.count);  
    }  
    return *this;  
}
```

Alternative: Smart Pointer

Alternative: Smart Pointer

- Member: `std::unique_ptr<int[]>` statt `int* elements`

```
class our_vector {  
    std::unique_ptr<int[]> elements;  
    ...  
};
```

Alternative: Smart Pointer

- Member: `std::unique_ptr<int[]>` statt `int* elements`

```
class our_vector {  
    std::unique_ptr<int[]> elements;  
    ...  
};
```

- Konstruktor: `std::make_unique<int[]>(size)` statt `new int[size]`

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = std::make_unique<int[]>(size);  
}
```

Alternative: Smart Pointer

- Member: `std::unique_ptr<int []>` statt `int* elements`

```
class our_vector {  
    std::unique_ptr<int []> elements;  
    ...  
};
```

- Konstruktor: `std::make_unique<int []>(size)` statt `new int[size]`

```
our_vector::our_vector(unsigned int size) : count(size) {  
    elements = std::make_unique<int []>(size);  
}
```

- kein `delete []` nötig im Destruktor (automatisch erzeugter Destruktor genügt)

Alternative: Smart Pointer

- Kopier-Konstruktor: `std::make_unique` statt `new[]`

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = std::make_unique<int[]>(count);  
    ...  
}
```

Alternative: Smart Pointer

- Kopier-Konstruktor: `std::make_unique` statt `new[]`

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = std::make_unique<int[]>(count);  
    ...  
}
```

- Zuweisungsoperator mit Copy & Swap-Idiom wie gehabt

Alternative: Smart Pointer

- Kopier-Konstruktor: `std::make_unique` statt `new[]`

```
our_vector::our_vector(const our_vector& v): count(v.count) {  
    elements = std::make_unique<int[]>(count);  
    ...  
}
```

- Zuweisungsoperator mit Copy & Swap-Idiom wie gehabt
- Raw Pointer extrahieren für Iterator mit `elements.get()`

```
our_vector::iterator our_vector::begin() {  
    return our_vector::iterator(elements.get());  
}  
  
our_vector::iterator our_vector::end() {  
    return our_vector::iterator(elements.get() + count);  
}
```

Elemente hinzufügen und löschen?

```
class our_vector {  
    ...  
    void push_front(int e)      // Prepend e to vector  
    void push_back(int e)      // Append e to vector  
    void remove(unsigned int i) // Cut out ith element  
    ...  
}
```

Elemente hinzufügen und löschen?

```
class our_vector {  
    ...  
    void push_front(int e)      // Prepend e to vector  
    void push_back(int e)      // Append e to vector  
    void remove(unsigned int i) // Cut out ith element  
    ...  
}
```

Gemeinsamkeit: Diese Operationen müssen die *Grösse* des Vektors verändern

Arrays vergrössern/verkleinern

Ein allozierter Speicherblock (z.B. `new int [3]`) kann nicht nachträglich vergrössert/verkleinert werden

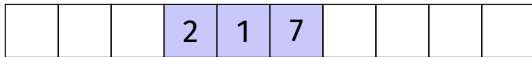
Arrays vergrössern/verkleinern

Ein allozierter Speicherblock (z.B. `new int [3]`) kann nicht nachträglich vergrössert/verkleinert werden

2	1	7
---	---	---

Arrays vergrössern/verkleinern

Ein allozierter Speicherblock (z.B. `new int [3]`) kann nicht nachträglich vergrössert/verkleinert werden

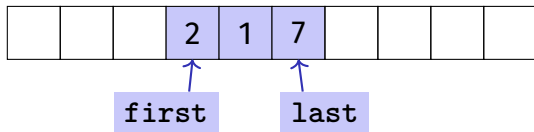


Möglichkeit:

- Mehr Speicher als initial nötig allozieren

Arrays vergrössern/verkleinern

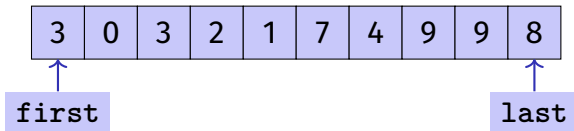
Ein allozierter Speicherblock (z.B. `new int [3]`) kann nicht nachträglich vergrössert/verkleinert werden



Möglichkeit:

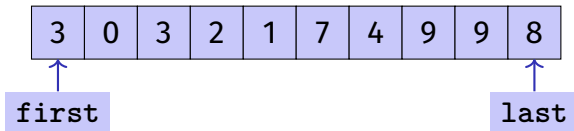
- Mehr Speicher als initial nötig allozieren
- Befüllen aus der Mitte heraus, mittels Zeigern auf erstes und letztes Element

Arrays vergrössern/verkleinern



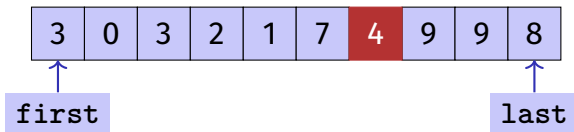
- Aber irgendwann sind alle Plätze belegt

Arrays vergrössern/verkleinern



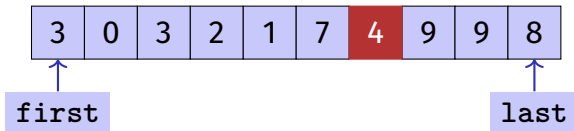
- Aber irgendwann sind alle Plätze belegt
- Dann nötig: Grösseren Speicherblock allozieren und Daten umkopieren

Arrays vergrössern/verkleinern

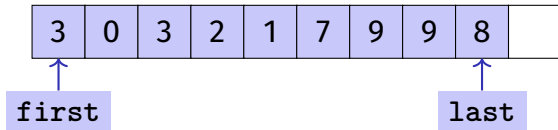


Elemente löschen erfordert verschieben (via kopieren) aller vorhergehenden oder nachfolgenden Elemente

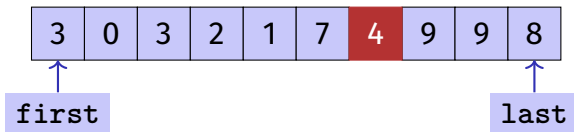
Arrays vergrössern/verkleinern



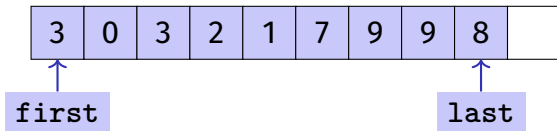
Elemente löschen erfordert verschieben (via kopieren) aller vorhergehenden oder nachfolgenden Elemente



Arrays vergrössern/verkleinern



Elemente löschen erfordert verschieben (via kopieren) aller vorhergehenden oder nachfolgenden Elemente



Ähnlich: Einfügen an beliebiger Position

Effizienz: Arrays vs. Verkettete Listen

- Speicher: Unser `our_vector` belegt ungefähr n ints (Vektorgrösse n), unser `our_list` ungefähr $3n$ ints (ein Zeiger belegt i.d.R. 8 Byte)

Effizienz: Arrays vs. Verkettete Listen

- Speicher: Unser `our_vector` belegt ungefähr n ints (Vektorgrösse n), unser `our_list` ungefähr $3n$ ints (ein Zeiger belegt i.d.R. 8 Byte)
- Laufzeit (mit `our_vector = std::vector`, `our_list = std::list`):

prepending (insert at front) [100,000x]:

- ▶ `our_vector`: 310 ms
- ▶ `our_list`: 8 ms

appending (insert at back) [100,000x]:

- ▶ `our_vector`: 1 ms
- ▶ `our_list`: 9 ms

removing first [100,000x]:

- ▶ `our_vector`: 329 ms
- ▶ `our_list`: 4 ms

removing last [100,000x]:

- ▶ `our_vector`: 0 ms
- ▶ `our_list`: 4 ms

removing randomly [10,000x]:

- ▶ `our_vector`: 1 ms
- ▶ `our_list`: 93 ms

inserting randomly [10,000x]:

- ▶ `our_vector`: 6 ms
- ▶ `our_list`: 334 ms

batch inserting & removing randomly (600 elements) [6,000x]:

- ▶ `our_vector`: 803 ms
- ▶ `our_list`: 282 ms

fully iterate sequentially (5000 elements) [5,000x]:

- ▶ `our_vector`: 279 ms
- ▶ `our_list`: 410 ms

24. Subtyping, Polymorphie und Vererbung

Die nachfolgenden Folien dieses Kapitels sind **nicht klausurrelevant**
Ausdrucksbäume, Aufgabenteilung und Modularisierung, Typhierarchien,
virtuelle Funktionen, dynamische Bindung, Code-Wiederverwendung,
Konzepte der objektorientierten Programmierung

Kurzvorstellung: Ausdrucksbäume

- Ziel: Arithmetische Ausdrücke repräsentieren, z.B.

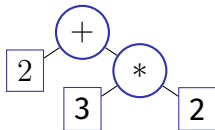
$$2 + 3 * 2$$

Kurzvorstellung: Ausdrucksbäume

- Ziel: Arithmetische Ausdrücke repräsentieren, z.B.

$$2 + 3 * 2$$

- Arithmetische Ausdrücke bilden eine *Baumstruktur*

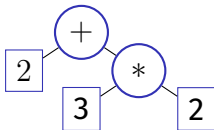


Kurzvorstellung: Ausdrucksbäume

- Ziel: Arithmetische Ausdrücke repräsentieren, z.B.

$$2 + 3 * 2$$

- Arithmetische Ausdrücke bilden eine *Baumstruktur*



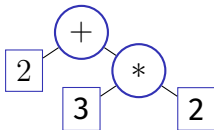
- Ausdrucksbäume bestehen aus *unterschiedlichen* Knoten:

Kurzvorstellung: Ausdrucksbäume

- Ziel: Arithmetische Ausdrücke repräsentieren, z.B.

$$2 + 3 * 2$$

- Arithmetische Ausdrücke bilden eine *Baumstruktur*



- Ausdrucksbäume bestehen aus *unterschiedlichen* Knoten: Literale (z.B. 2), binäre Operatoren (z.B. +), unäre Operatoren (z.B. $\sqrt{}$), Funktionsanwendungen (z.B. $\cos$), etc.

Ein einziger Knotentyp

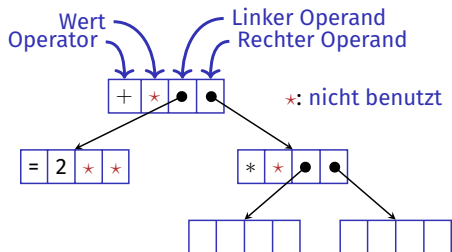
Ausdrucksbäume implementiert mittels *eines einzigen* Knotentyps:

```
struct tnode {  
    char op; // Operator ('=' for literals)  
    double val; // Literal's value  
    tnode* left; // Left child (or nullptr)  
    tnode* right; // ...  
    ...  
};
```

Ein einziger Knotentyp

Ausdrucksbäume implementiert mittels *eines einzigen* Knotentyps:

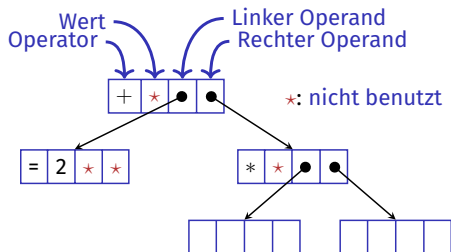
```
struct tnode {  
    char op; // Operator ('=' for literals)  
    double val; // Literal's value  
    tnode* left; // Left child (or nullptr)  
    tnode* right; // ...  
    ...  
};
```



Ein einziger Knotentyp

Ausdrucksbäume implementiert mittels *eines einzigen* Knotentyps:

```
struct tnode {  
    char op; // Operator ('=' for literals)  
    double val; // Literal's value  
    tnode* left; // Left child (or nullptr)  
    tnode* right; // ...  
    ...  
};
```



Beobachtung: `tnode` ist die „Summe“ aller benötigten Knoten (Konstanten, Addition, ...) $\Rightarrow$ Speicherverschwendung, unelegant...

Nachteile

...und jede Funktion muss diese „Summe“ wieder „auseinander nehmen“,
z.B.:

Nachteile

...und jede Funktion muss diese „Summe“ wieder „auseinander nehmen“, z.B.:

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val; // n is a constant  
    double l = 0;  
    if (n->left) l = eval(n->left); // n is not a unary operator  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r; // n is an addition node  
        case '*': return l*r; // ...  
        ...  
    }
```


Nachteile

...und jede Funktion muss diese „Summe“ wieder „auseinander nehmen“, z.B.:

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val; // n is a constant  
    double l = 0;  
    if (n->left) l = eval(n->left); // n is not a unary operator  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r; // n is an addition node  
        case '*': return l*r; // ...  
        ...  
    }
```

⇒ Umständlich und somit fehleranfällig

Nachteile

```
struct tnode {  
    char op;  
    double val;  
    tnode* left;  
    tnode* right;  
    ...  
};
```

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val;  
    double l = 0;  
    if (n->left) l = eval(n->left);  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r;  
        case '*': return l*r;  
        ...  
    }
```

Nachteile

```
struct tnode {  
    char op;  
    double val;  
    tnode* left;  
    tnode* right;  
    ...  
};
```

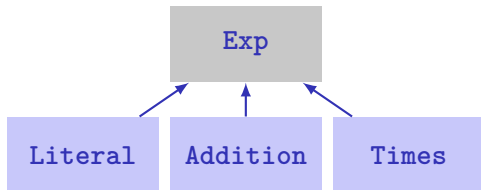
```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val;  
    double l = 0;  
    if (n->left) l = eval(n->left);  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l+r;  
        case '*': return l*r;  
        ...  
    }
```

Dieser Code ist nicht *modular* – das ändern wir heute!

Neue Konzepte heute

1. Subtyping

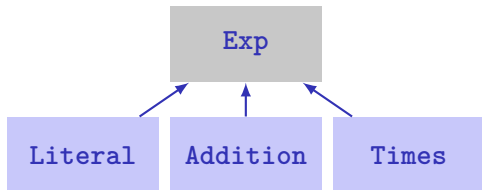
- Typhierarchie: `Exp` repräsentiert allgemeine Ausdrücke, `Literal` etc. sind konkrete Ausdrücke



Neue Konzepte heute

1. Subtyping

- Typhierarchie: `Exp` repräsentiert allgemeine Ausdrücke, `Literal` etc. sind konkrete Ausdrücke
- Jedes `Literal` etc. ist auch ein `Exp` (Subtyp-Beziehung)

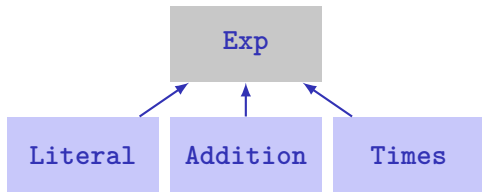


Neue Konzepte heute

1. Subtyping

- Typhierarchie: `Exp` repräsentiert allgemeine Ausdrücke, `Literal` etc. sind konkrete Ausdrücke
- Jedes `Literal` etc. ist auch ein `Exp` (Subtyp-Beziehung)
- Deswegen kann ein `Literal` etc. überall dort genutzt werden, wo ein `Exp` erwartet wird:

```
Exp* e = new Literal(132);
```



2. Polymorphie und dynamische Bindung

- Eine Variable vom *statischen* Typ `Exp` kann Ausdrücke mit unterschiedlichen *dynamischen* Typen „beherbergen“:

```
Exp* e = new Literal(2); // e is the literal 2  
e = new Addition(e, e); // e is the addition 2 + 2
```

2. Polymorphie und dynamische Bindung

- Eine Variable vom *statischen* Typ `Exp` kann Ausdrücke mit unterschiedlichen *dynamischen* Typen „beherbergen“:

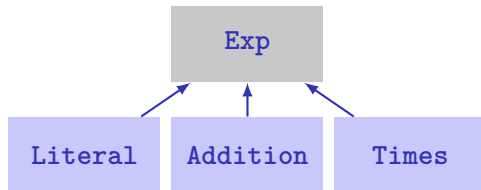
```
Exp* e = new Literal(2); // e is the literal 2  
e = new Addition(e, e); // e is the addition 2 + 2
```

- Ausgeführt werden die Memberfunktionen des *dynamischen* Typs:

```
Exp* e = new Literal(2);  
std::cout << e->eval(); // 2  
  
e = new Addition(e, e);  
std::cout << e->eval(); // 4
```


3. Vererbung

- Manche Funktionalität ist für mehrere Mitglieder der Typhierarchie gleich

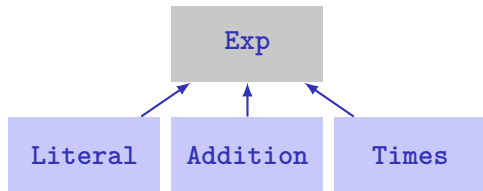


Neue Konzepte heute

3. Vererbung

- Manche Funktionalität ist für mehrere Mitglieder der Typhierarchie gleich
- Z.B. die Berechnung der Grösse (Verschachtelungstiefe) binärer Ausdrücke (Addition, Times):

$$1 + \text{size}(\text{left operand}) + \text{size}(\text{right operand})$$



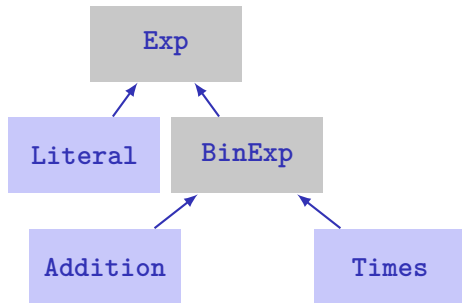
Neue Konzepte heute

3. Vererbung

- Manche Funktionalität ist für mehrere Mitglieder der Typhierarchie gleich
- Z.B. die Berechnung der Grösse (Verschachtelungstiefe) binärer Ausdrücke (Addition, Times):

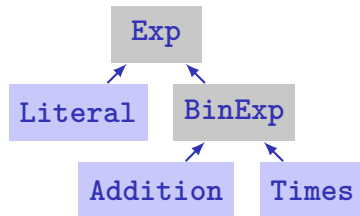
$$1 + \text{size}(\text{left operand}) + \text{size}(\text{right operand})$$

⇒ Funktionalität einmal implementieren und dann an Subtypen *vererben*



Vorteile

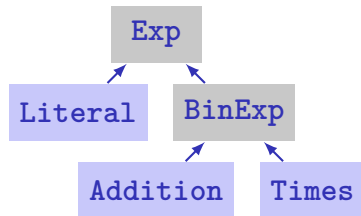
- Subtyping, Polymorphie und dynamische Bindung ermöglichen *Modularisierung durch Spezialisierung*



```
Exp* e = new Literal(2);  
std::cout << e->eval();  
  
e = new Addition(e, e);  
std::cout << e->eval();
```

Vorteile

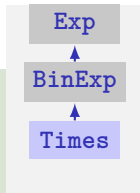
- Subtyping, Polymorphie und dynamische Bindung ermöglichen *Modularisierung durch Spezialisierung*
- Vererbung erlaubt gemeinsamen Code trotz Modularisierung
⇒ *Codeduplikation vermeiden*



```
Exp* e = new Literal(2);  
std::cout << e->eval();  
  
e = new Addition(e, e);  
std::cout << e->eval();
```

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



Anmerkung: Wir konzentrieren uns heute auf die neuen Konzepte (Subtyping, ...) und ignorieren den davon unabhängigen Aspekt der Kapselung (class, private vs. public Membervariablen)

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



■ BinExp ist eine von Exp *abgeleitete Klasse*¹

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



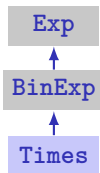
- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



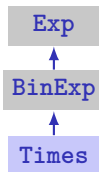
- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp
- BinExp *erbt* von Exp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



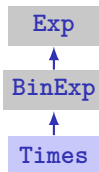
- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp
- BinExp *erbt* von Exp
- Die Vererbung von Exp zu BinExp ist *öffentlich* (public), daher ist BinExp ein *Subtyp* von Exp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



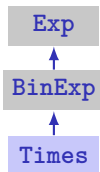
- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp
- BinExp *erbt* von Exp
- Die Vererbung von Exp zu BinExp ist *öffentlich* (public), daher ist BinExp ein *Subtyp* von Exp
- Analog: Times und BinExp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Syntax und Terminologie

```
struct Exp {  
    ...  
}  
  
struct BinExp : public Exp {  
    ...  
}  
  
struct Times : public BinExp {  
    ...  
}
```



- BinExp ist eine von Exp *abgeleitete Klasse*¹
- Exp ist die *Basisklasse*² von BinExp
- BinExp *erbt* von Exp
- Die Vererbung von Exp zu BinExp ist *öffentlich* (public), daher ist BinExp ein *Subtyp* von Exp
- Analog: Times und BinExp
- Subtyprelation ist transitiv: Times ist ebenfalls ein Subtyp von Exp

¹Subklasse, Kindklasse

²Superklasse, Elternklasse

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```



Aktiviert dynamische Bindung

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

Erzwingt Implementierung durch
abgeleitete Klassen ...

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

... das macht Exp zu einer *abstrakten* Klasse

Erzwingt Implementierung durch abgeleitete Klassen ...

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

```
struct Literal : public Exp {  
    double val;  
  
    Literal(double v);  
    int size() const;  
    double eval() const;  
};
```

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

```
struct Literal : public Exp { ← Literal erbt von Exp ...  
    double val;  
  
    Literal(double v);  
    int size() const;  
    double eval() const;  
};
```

Abstrakte Klasse Exp und konkrete Klasse Literal

```
struct Exp {  
    virtual int size() const = 0;  
    virtual double eval() const = 0;  
};
```

```
struct Literal : public Exp {  
    double val;
```

← Literal erbt von Exp ...

```
    Literal(double v);  
    int size() const;  
    double eval() const;  
};
```

← ...ist aber ansonsten eine ganz normale Klasse

Literal: Implementierung

```
Literal::Literal(double v): val(v) {}
```

Literal: Implementierung

```
Literal::Literal(double v): val(v) {}
```

```
int Literal::size() const {  
    return 1;  
}
```

Literal: Implementierung

```
Literal::Literal(double v): val(v) {}
```

```
int Literal::size() const {  
    return 1;  
}
```

```
double Literal::eval() const {  
    return this->val;  
}
```

Subtyping: Ein Literal ist ein Ausdruck

Ein Zeiger auf einen Subtyp kann überall dort verwendet werden, wo ein Zeiger auf einen Supertyp gefordert ist:

```
Literal* lit = new Literal(5);
```


Subtyping: Ein Literal ist ein Ausdruck

Ein Zeiger auf einen Subtyp kann überall dort verwendet werden, wo ein Zeiger auf einen Supertyp gefordert ist:

```
Literal* lit = new Literal(5);  
Exp* e = lit; // OK: Literal is a subtype of Exp
```

Subtyping: Ein Literal ist ein Ausdruck

Ein Zeiger auf einen Subtyp kann überall dort verwendet werden, wo ein Zeiger auf einen Supertyp gefordert ist:

```
Literal* lit = new Literal(5);  
Exp* e = lit; // OK: Literal is a subtype of Exp
```

Aber nicht umgekehrt:

```
Exp* e = ...  
Literal* lit = e; // ERROR: Exp is not a subtype of Literal
```

Polymorphie: Ein Literal verhält sich wie ein Literal

```
struct Exp {  
    ...  
    virtual double eval();  
};  
  
double Literal::eval() {  
    return this->val;  
}
```

```
Exp* e = new Literal(3);  
std::cout << e->eval(); // 3
```

Polymorphie: Ein Literal verhält sich wie ein Literal

```
struct Exp {  
    ...  
    virtual double eval();  
};  
  
double Literal::eval() {  
    return this->val;  
}
```

```
Exp* e = new Literal(3);  
std::cout << e->eval(); // 3
```

- *Virtuelle* Memberfunktionen: der *dynamische* Typ (hier: `Literal`) bestimmt die auszuführenden Memberfunktionen
⇒ *dynamische Bindung*

Polymorphie: Ein Literal verhält sich wie ein Literal

```
struct Exp {  
    ...  
    virtual double eval();  
};  
  
double Literal::eval() {  
    return this->val;  
}
```

```
Exp* e = new Literal(3);  
std::cout << e->eval(); // 3
```

- *Virtuelle* Memberfunktionen: der *dynamische* Typ (hier: `Literal`) bestimmt die auszuführenden Memberfunktionen
⇒ *dynamische Bindung*
- Ohne `virtual` bestimmt der *statische* Typ (hier: `Exp`) die auszuführende Funktion

Polymorphie: Ein Literal verhält sich wie ein Literal

```
struct Exp {  
    ...  
    virtual double eval();  
};  
  
double Literal::eval() {  
    return this->val;  
}
```

```
Exp* e = new Literal(3);  
std::cout << e->eval(); // 3
```

- *Virtuelle* Memberfunktionen: der *dynamische* Typ (hier: `Literal`) bestimmt die auszuführenden Memberfunktionen
⇒ *dynamische Bindung*
- Ohne `virtual` bestimmt der *statische* Typ (hier: `Exp`) die auszuführende Funktion
- Wir vertiefen das nicht weiter

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```


Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Addition::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Addition::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Times::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Addition::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Times::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```



Aufgabenteilung

Weitere Ausdrücke: Addition und Times

```
struct Addition : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Addition::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```

```
struct Times : public Exp {  
    Exp* left; // left operand  
    Exp* right; // right operand  
    ...  
};
```

```
int Times::size() const {  
    return 1 + left->size()  
           + right->size();  
}
```



Aufgabenteilung



Codeduplizierung

Gemeinsamkeiten auslagern ...: BinExp

```
struct BinExp : public Exp {  
    Exp* left;  
    Exp* right;  
  
    BinExp(Exp* l, Exp* r);  
    int size() const;  
};
```

```
BinExp::BinExp(Exp* l, Exp* r): left(l), right(r) {}
```

Gemeinsamkeiten auslagern ...: BinExp

```
struct BinExp : public Exp {  
    Exp* left;  
    Exp* right;  
  
    BinExp(Exp* l, Exp* r);  
    int size() const;  
};
```

```
BinExp::BinExp(Exp* l, Exp* r): left(l), right(r) {}
```

```
int BinExp::size() const {  
    return 1 + this->left->size() + this->right->size();  
}
```

Bemerkung: BinExp implementiert eval nicht und ist daher, genau wie Exp, eine abstrakte Klasse

...Gemeinsamkeiten erben: Addition

```
struct Addition : public BinExp {  
    Addition(Exp* l, Exp* r);  
    double eval() const;  
};
```

...Gemeinsamkeiten erben: Addition

```
struct Addition : public BinExp {  
    Addition(Exp* l, Exp* r);  
    double eval() const;  
};
```

← Addition erbt Membervariablen
(left, right) und Funktionen
(size) von BinExp

...Gemeinsamkeiten erben: Addition

```
struct Addition : public BinExp {  
    Addition(Exp* l, Exp* r);  
    double eval() const;  
};
```

```
Addition::Addition(Exp* l, Exp* r): BinExp(l, r) {}
```



Aufruf des *Superkonstruktors*
(Konstruktor von BinExp) zwecks
Initialisierung der Membervari-
ablen *left* und *right*

...Gemeinsamkeiten erben: Addition

```
struct Addition : public BinExp {  
    Addition(Exp* l, Exp* r);  
    double eval() const;  
};
```

```
Addition::Addition(Exp* l, Exp* r): BinExp(l, r) {}
```

```
double Addition::eval() const {  
    return  
        this->left->eval() +  
        this->right->eval();  
}
```

...Gemeinsamkeiten erben: Times

```
struct Times : public BinExp {  
    Times(Exp* l, Exp* r);  
    double eval() const;  
};
```

```
Times::Times(Exp* l, Exp* r): BinExp(l, r) {}
```

```
double Times::eval() const {  
    return  
        this->left->eval() *  
        this->right->eval();  
}
```

Beobachtung: `Additon::eval()` und `Times::eval()` sind sich sehr ähnlich und könnten ebenfalls zusammengelegt werden. Das dafür notwendige Konzept der *funktionalen Programmierung* geht jedoch über diesen Kurs hinaus.

Weitere Ausdrücke und Operationen

- Weitere Ausdrücke, als von `Exp` abgeleitete Klassen, sind möglich, z.B. $-$, $/$, $\sqrt{}$, $\cos$, $\log$

Weitere Ausdrücke und Operationen

- Weitere Ausdrücke, als von `Exp` abgeleitete Klassen, sind möglich, z.B. $-$, $/$, $\sqrt{}$, $\cos$, $\log$
- Eine ehemalige Bonusaufgabe (Teil der heutigen Vorlesungsbeispiele auf Code Expert) veranschaulicht, was alles möglich ist: Variablen, trigonometrische Funktionen, Parsing, Pretty-Printing, numerische Vereinfachungen, symbolische Ableitungen, ...

Mission: Monolithisch → modular ✓

```
struct tnode {  
    char op;  
    double val;  
    tnode* left;  
    tnode* right;  
    ...  
}
```

```
double eval(const tnode* n) {  
    if (n->op == '=') return n->val;  
    double l = 0;  
    if (n->left != 0) l = eval(n->left);  
    double r = eval(n->right);  
    switch(n->op) {  
        case '+': return l + r;  
        case '*': return l * r;  
        case '-': return l - r;  
        case '/': return l / r;  
        default:  
            // unknown operator  
            assert (false);  
    }  
}
```

```
int size (const tnode* n) const { ... }  
...
```

```
struct Literal : public Exp {  
    double val;  
    ...  
    double eval() const {  
        return val;  
    }  
};
```

```
struct Addition : public Exp {  
    ...  
    double eval() const {  
        return left->eval() + right->eval();  
    }  
};
```

```
struct Times : public Exp {  
    ...  
    double eval() const {  
        return left->eval() * right->eval();  
    }  
};
```

+

```
struct Cos : public Exp {  
    ...  
    double eval() const {  
        return std::cos(argument->eval());  
    }  
};
```

Es gibt noch so viel mehr ...

Nicht gezeigt/besprochen:

- Private Vererbung (`class B : public A`)
- Subtyping und Polymorphie ohne Zeiger
- Nicht-virtuelle Memberfunktionen und statische Bindung
(~~`virtual`~~ `double eval()`)
- Überschreiben geerbter Memberfunktionen und Aufrufen der überschriebenen Implementierung
- Mehrfachvererbung (multiple inheritance)
- ...

Gute Neuigkeit: Das heutige Kapitel (24. Subtyping, Polymorphie und Vererbung) ist *nicht* prüfungsrelevant

Objektorientierte Programmierung

Im letzten Kursdrittel wurden einige Konzepte der *objektorientierten Programmierung* vorgestellt, die auf den kommenden Folien noch einmal kurz zusammengefasst werden.

Kapselung (Wochen 10-13):

- Verbergen der Implementierungsdetails von Typen (privater Bereich) vor Benutzern
- Definition einer Schnittstelle (öffentlicher Bereich) zum kontrollierten Zugriff auf Werte und Funktionalität
- Ermöglicht das Sicherstellen von Invarianten, sowie den Austausch von Implementierungen ohne Anpassungen von Benutzercode

Objektorientierte Programmierung

Subtyping (Woche 14):

- Typhierarchien mit Super- und Subtypen können angelegt werden um Verwandtschaftbeziehungen sowie Abstraktionen und Spezialisierungen zu modellieren
- Ein Subtyp unterstützen mindestens die Funktionalität, die auch der Supertyp unterstützt – i.d.R. aber mehr, d.h. Subtypen erweitern die Schnittstelle (den öffentlichen Bereich) ihrer Supertypen
- Daher können Subtypen überall dort eingesetzt werden, wo Supertypen verlangt sind ...
- ...und Funktionen, die auf abstrakteren Typen (Supertypen) operieren können, können auch auf spezialisierteren Typen (Subtypen) operieren
- Die in Woche 7 vorgestellten Streams bilden eine solche Typhierarchie: `ostream` ist der abstrakte Supertyp, `ofstream` etc. sind spezialisierte Subtypen

Polymorphie und *dynamische Bindung* (Woche 14):

- Ein Zeiger vom statischen Typ T_1 kann zur Laufzeit auf Objekte vom (dynamischen) Typ T_2 zeigen, falls T_2 ein Subtyp von T_1 ist
- Wird eine virtuelle Memberfunktion von einem solchen Zeiger aus aufgerufen, so entscheidet der dynamische Typ darüber, welche Funktion ausgeführt wird
- D.h.: Trotz gleichem statischen Typ kann beim Zugriff auf eine gemeinsame Schnittstelle (Memberfunktionen) eines solchen Zeigers ein anderes Verhalten auftreten
- Zusammen mit Subtyping ermöglicht es dies, neue konkrete Typen (Streams, Ausdrücke, ...) zu einem bestehenden System hinzuzufügen, ohne dieses abändern zu müssen

Vererbung (Woche 14):

- Abgeleitete Klassen erben die Funktionalität, d.h. die Implementierungen von Memberfunktionen, ihrer Elternklassen
- Dies ermöglicht es, gemeinsam genutzten Code wiederverwenden zu können und vermeidet so Codeduplikation
- Geerbte Implementierungen können auch überschrieben werden, um zu erreichen, dass eine abgeleitete Klasse sich anders verhält als ihre Elternklasse (im Kurs nicht gezeigt)

25. Zusammenfassung

Zweck und Format

Nennung der wichtigsten Stichwörter zu den Kapiteln. Checkliste: „kann ich mit jedem Begriff etwas anfangen?“

- Ⓜ Motivation: Motivierendes Beispiel zum Kapitel
- Ⓚ Konzepte: Konzepte, die nicht von der Implementation (Sprache) C++abhängen
- Ⓢ Sprachlich (C++): alles was mit der gewählten Sprache zusammenhängt
- Ⓑ Beispiele: genannte Beispiele der Vorlesung

Kapitelüberblick

- 1. Einführung
- 2. Erstes C++ Programm
- 3. Ganze Zahlen
- 4. Wahrheitswerte
- 5./6. Kontrollanweisungen
- 7./8. Fließkommazahlen
- 9./10. Funktionen
- 11. Vektoren
- 12. Referenztypen
- 13. Texte und Strings
- 14. Mehrdimensionale Vektoren
- 15./16. Rekursion
- 17./18. Custom Data Types
- 19. Container, Iteratoren und Algorithmen
- 20. Dynamischer Speicher und Zeiger
- 21.-23. Dynamische Datenstrukturen und Speicherverwaltung
- 24. Subtyping, Polymorphie und Vererbung

1. Einführung

Ⓜ

- Euklidischer Algorithmus

Ⓚ

- Algorithmus, Turingmaschine, Programmiersprachen, Kompilation, Syntax und Semantik
- Werte und Effekte, (Fundamental)typen, Literale, Variablen, Bezeichner, Objekte, Ausdrücke, Operatoren, Anweisungen

Ⓢ

- Include-Direktiven `#include <iostream>`
- Hauptfunktion `int main(){...}`
- Kommentare, Layout `// Kommentar`
- Typen, Variablen, L-Wert `a` , R-Wert `a+b`
- Ausdrucksanweisung `b=b*b;` , Deklarationsanweisung `int a;`, Rückgabeeanweisung `return 0;`

2. Erstes C++ Programm

(M)

- Einfache Berechnung von a^8

(K)

- Programmiersprachen-Elemente

(S)

- Kommentare und Layout
- Include-Direktive
- main-Funktion
- Werte, Effekte
- Typen, Funktionalität
- Literale, Variablen, Konstanten
- Bezeichner, Namen
- Ausdrücke
- L- und R-Werte
- Operatoren
- Anweisungen

(B)

- Erstes Programm

3. Ganze Zahlen

- Ⓜ ■ Celsius to Fahrenheit
- Ⓚ ■ Assoziativität und Präzedenz, Stelligkeit
 - Ausdrucksbäume, Auswertungsreihenfolge
 - Arithmetische Operatoren
 - Binärzahldarstellung, Hexadezimale Zahlen, Wertebereich
 - Zahlendarstellung mit Vorzeichen, Zweierkomplement
- Ⓢ ■ Arithmetische Operatoren `9 * celsius / 5 + 32`
 - Inkrement / Dekrement `expr++`
 - Arithmetische Zuweisungen `expr1 += expr2`
 - Konversion `int` $\leftrightarrow$ `unsigned int`
- Ⓑ ■ Celsius to Fahrenheit, Ersatzwiderstand

4. Wahrheitswerte

Ⓚ

- Boole'sche Funktionen, Vollständigkeit
- DeMorgan'sche Regeln

Ⓢ

- Der Typ `bool`
- Logische Operationen `a && !b`
- Relationale Operationen `x < y`
- Präzedenzen `7 + x < y && y != 3 * z`
- Kurzschlussauswertung `x != 0 && z / x > y`
- Die `assert`-Anweisung, `#include <cassert>`

Ⓑ

- Div-Mod Identität.

5./6. Kontrollanweisungen

(M)

- Linearer Kontrollfluss vs. interessante Programme, Spaghetti-Code

(K)

- Auswahlanweisungen, Iterationsanweisungen
- (Vermeidung von) Endlosschleifen, Halteproblem
- Sichtbarkeits- und Gültigkeitsbereich, Automatische Speicherdauer
- Äquivalenz von Iterationsanweisungen

(S)

- if Anweisungen `if (a % 2 == 0) {..}`
- for Anweisungen `for (unsigned int i = 1; i <= n; ++i) ...`
- while und do-Anweisungen `while (n > 1) {...}`
- Blöcke, Sprunganweisungen `if (a < 0) continue;`
- Switch Anweisung `switch(grade) {case 6: }`

(B)

- Summenberechnung (Gauss), Primzahltest, Collatz-Folge, Fibonacci Zahlen, Taschenrechner, Notenausgabe

7./8. Fließkommazahlen

Ⓜ

- Richtig Rechnen: Celsius / Fahrenheit

Ⓚ

- Fixkomma- vs. Fließkommazahldarstellung
- (Löcher im) Wertebereich
- Rechnen mit Fließkommazahlen, Umrechnung
- Fließkommazahlensysteme, Normalisierung, IEEE Standard 754
- *Richtlinien für das Rechnen mit Fließkommazahlen*

Ⓢ

- Typen `float`, `double`
- Fließkommaliterale `1.23e-7f`

Ⓑ

- Celsius/Fahrenheit, Euler, Harmonische Zahlen

9./10. Funktionen

(M)

- Potenzberechnung

(K)

- Kapselung von Funktionalität
- Funktionen, formale Argumente, Aufrufargumente
- Gültigkeitsbereich, Vorwärts-Deklaration
- Prozedurales Programmieren, Modularisierung, Getrennte Übersetzung
- *Stepwise Refinement*

(S)

- Funktionsdeklaration, -definition `double pow(double b, int e){ ... }`
- Funktionsaufruf `pow (2.0, -2)`
- Der typ `void`

(B)

- Potenzberechnung, perfekte Zahlen, Minimum, Kalender

11. Vektoren

Ⓜ

- Iteration über Daten: Sieb des Eratosthenes

Ⓚ

- Vektoren, Speicherlayout, Wahlfreier Zugriff
- (Fehlende) Grenzenprüfung
- Vektoren

Ⓢ

- Vektor Typen `std::vector<int> a {4,3,5,2,1};`

Ⓑ

- Sieb des Eratosthenes

12. Referenztypen

(M)

- Funktion Swap

(K)

- Werte-/ Referenzsemantik, Pass by Value / Pass by Reference, Return by Reference
- Lebensdauer von Objekten / Temporäre Objekte
- Konstanten

(S)

- Referenztyp `int& a`
- Call by Reference und Return by Reference `int& increment (int& i)`
- Const-Richtlinie, Const-Referenzen, Referenzrichtlinie

(B)

- Swap, Inkrement

13 Zeichen und Strings

- ① ■ Caesar Code
- ② ■ Zeichen: ASCII, UTF8, Texte, Strings
- ③ ■ Zeichen und Texte, der Typ char `char c = 'a';`, Konversion nach `int`
- ④ ■ Caesar-Code

14. Vektoren

- Matrizen, Tabellen
- Mehrdimensionale Vektoren
- Vektoren von Vektoren
- auto
- Summe des unteren Dreiecks

15./16. Rekursion

- Ⓜ ■ Rekursive math. Funktionen, Das n-Queen Problem, Kommandozeilenrechner
- Ⓚ ■ Rekursion
 - Aufrufstapel, Gedächtnis der Rekursion
 - Korrektheit, Terminierung,
 - Rekursion vs. Iteration
 - Decrease and Conquer vs. Divide and Conquer
- Ⓑ ■ Fakultät, GGT, Sudoku-Löser, Taschenrechner, Vektorsumme

17./18. Custom Data Types

(M)

- Datentyp Rationale Zahlen selber bauen

(K)

- Heterogene Datenstruktur
- Funktions- und Operator-Overloading
- Datenkapselung, Konstruktion, Mitgliedsfunktionen

(S)

- Struct Definition `struct rational {int n; int d;};`
 - Mitgliedszugriff `result.n = a.n * b.d + a.d * b.n;`
 - Initialisierung und Zuweisung,
 - Überladen von Funktionen `pow(2)` vs. `pow(3,3);`, Überladen von Operatoren
 - Klassen `class rational { ... };`
 - Zugriffssteuerung `public: / private:`
 - Mitgliedsfunktionen `int rational::denominator () const`
 - Das implizite Argument der Memberfunktionen
- (B)
- rationale Zahlen, komplexe Zahlen

19. Container, Iteratoren und Algorithmen

- (M) Vektoren sind Container
- (K) Iterieren mit Zeigern
 - Container und Iteratoren
 - Algorithmen
- (S) Iteratoren `std::vector<int>::iterator`
 - Algorithmen der Standardbibliothek `std::fill (a, a+5, 1);`
 - Einen Iterator implementieren
 - Iteratoren und `const`
- (B) Ausgeben eines Vektors, einer Menge

20. Dynamischer Speicher und Zeiger

- Unsere eigene Datenstruktur
- Allokation, Zeiger-Typen, Verkettete Liste, Allokation, Deallokation, Dynamischer Datentyp
- Die `new` Anweisung
- Zeiger `int* x;`, Nullzeiger `nullptr.`
- Adress-, Dereferenzoperator `int *ip = &i; int j = *ip;`
- Zeiger und Const `const int *a;`
- Verkettete Liste, Stack

21.-23. Dynamische Datenstrukturen und Speicherverwaltung

(M)

- Stack
- Verkettete Liste
- Vektor

(K)

- Richtlinie „Dynamischer Speicher“
- Gemeinsamer Zeiger-Zugriff
- Dynamischer Datentyp

(S)

- `new` und `delete`
- Desktruktor `stack::~~stack()`
- Kopierkonstruktor `stack::stack(const stack& s)`
- Zuweisungsoperator `stack& stack::operator=(const stack& s)`
- Dreierregel

(B)

- Stack, Verkettete Liste, Vektor

24. Subtyping, Polymorphie und Vererbung

- ① ■ Erweitern und Verallgemeinern von Ausdrucksbäumen
- ② ■ Subtyping
 - Polymorphie und dynamische Bindung
 - Vererbung
- ③ ■ Basisklasse `struct Exp{}`
 - Abgeleitete Klasse `struct BinExp: public Exp{}`
 - Abstrakte Klasse `struct Exp{virtual int size() const = 0...}`
 - Polymorphie `virtual double eval()`
- ④ ■ Ausdrucksknoten und Erweiterungen

Ende

Ende der Vorlesung.